

Backend Software Architecture — using — **Golang**

Microservices, distributed systems, and
cloud-native technologies



Bharat Chandra Baddepudi



Backend Software Architecture

— using —

Golang

Microservices, distributed systems, and
cloud-native technologies



Bharat Chandra Baddepudi



Backend Software Architecture using Golang

*Microservices, distributed systems,
and
cloud-native technologies*

Bharat Chandra Baddepudi



www.bpbonline.com

First Edition 2025

Copyright © BPB Publications, India

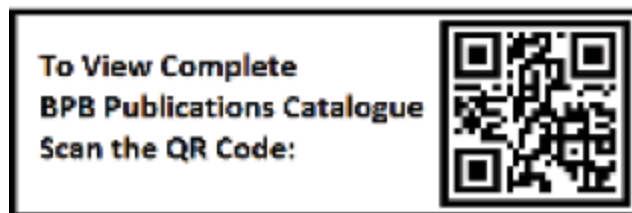
ISBN: 978-93-65893-557

All Rights Reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means or stored in a database or retrieval system, without the prior written permission of the publisher with the exception to the program listings which may be entered, stored and executed in a computer system, but they can not be reproduced by the means of publication, photocopy, recording, or by any electronic and mechanical means.

LIMITS OF LIABILITY AND DISCLAIMER OF WARRANTY

The information contained in this book is true to correct and the best of author's and publisher's knowledge. The author has made every effort to ensure the accuracy of these publications, but publisher cannot be held responsible for any loss or damage arising from any information in this book.

All trademarks referred to in the book are acknowledged as properties of their respective owners but BPB Publications cannot guarantee the accuracy of this information.



www.bpbonline.com

OceanofPDF.com

Dedicated to

My mummy and daddy

[OceanofPDF.com](https://oceanofpdf.com)

About the Author

Bharat Chandra Baddepudi is a software engineer with over 20 years of enterprise software research and development experience. Bharat holds a Bachelor's degree in Computer Science and Engineering from the Indian Institute of Technology, Madras, and a Master's degree in Computer Science from the University of Texas at Austin. He has worked at large software companies as well as been in founder roles at technology startups, building expertise in various layers of the software stack. Oracle was his first dive into the software industry, where he worked on the database kernel internals and later went on to deliver the Exadata Flash Cache solutions. After that he designed and developed a document database in the Hadoop ecosystem at MapR Inc. As the founding engineer at Yugabyte next, he first contributed to the consensus layer of its distributed Postgres database engine. Taking on broader impact end to end user features, he was instrumental in providing the design and solutions for server elasticity in the cloud along with related data migration needs. Currently he is working on large scale distributed systems and document databases at LinkedIn.

He holds several patents in backend architectures, storage solutions, and cloud domains and has given talks in technical meetups along with writing blogs. He is passionate about building large-scale distributed systems that provide the backbone to various web applications and related use cases. His awareness of technological advances allows him to contribute to state-of-the-art and emerging roadmaps.

OceanofPDF.com

About the Reviewer

Flávio Costa is a seasoned Software and Solutions Architect with over 24 years of experience across industries in Brazil, Portugal, and the USA. Proficient in programming languages such as JavaScript, TypeScript, C++, C# and Golang, he has a proven track record in designing and implementing scalable systems using microservices, clean architecture, and domain-driven design. Flávio has led critical projects in logistics, banking, fintech, and IoT, working extensively with cloud platforms like Azure, GCP, and AWS to deliver robust and resilient solutions.

Currently, Flávio dedicates his efforts to advancing his expertise in P2P networks and blockchain technologies using Golang. With a passion for solving complex challenges and staying at the forefront of innovation, he combines his technical skills with a deep understanding of decentralized systems to contribute meaningfully to modern software development.

[OceanofPDF.com](https://oceanofpdf.com)

Acknowledgement

I would like to express my sincere gratitude to all those who contributed to the completion of this book.

First and foremost, I extend my heartfelt appreciation to my whole family for their unwavering support and encouragement throughout this journey. The ability to give me space along with great understanding from my wife and the constant love, mischief and affection from my two kids helped push through some of the chapters. Their encouragement has been a constant source of motivation. Thank you for your invaluable support.

I am immensely grateful to BPB Publications for their guidance and expertise in bringing this book to fruition. For a first time author, their support and assistance were invaluable in navigating the complexities of the writing and publishing process.

I would also like to acknowledge the reviewers, technical experts, and editors who provided valuable feedback and contributed to the refinement of this manuscript in a timely manner. Your insights and suggestions have significantly enhanced the quality of the book.

Last but not least, I want to express my gratitude to the readers who have shown interest in our book. Your support and encouragement have been deeply appreciated.

Thanks to everyone who has played a part directly or indirectly in making this book a reality.

OceanofPDF.com

Preface

Modern technologies and software applications cater to a wide range of real world solutions that have steadily enhanced the speed and quality of life in the past half century. Such Web2.0, and more recently, AI applications, need solid backend system support to run the service logic. These backends have evolved over the past few decades, with advances along the aspects of programming, infrastructure, and distributed systems, as well as deployment tooling. There are fundamental requirements for building a resilient and scalable backend solution that can handle ever-growing user workloads as well as storage needs, which we will explore in this book.

Some of the challenges and trade offs in building such systems using a relatively new programming language, Golang, are also explored in this book. It has become one of the go to languages for building backend systems faster and with cleaner concurrency, cross-compiling, and high speed features. We will also delve into service creation and faster development using a range of Go Frameworks such as Gin, Echo, Beego, and FastHTTP to support highly performant business applications. Microservices and cloud services are also becoming a key compute component in building backends, which are essential for continuous and iterative deployment and software management.

After that, we will get to understand the data modeling, storage, messaging, and processing angles of the equation. Keeping the service running and maintainable is part of the operational side that needs attention once the development phase is completed for any given feature. Security should be top of mind for any production system across all these data and compute components.

This book can be used as a step-by-step guide to achieve vital expertise to design, build, implement, deploy, and maintain large-scale backend

systems. Each of the core concepts is covered with example use cases and code snippets to help along in the journey as per the chapters below.

Chapter 1: Backend Systems Components - First, we provide an overview of the various aspects that are essential to building backend systems. As a professional software engineer a good understanding of these components and tradeoffs will help deliver robust solutions for any given problem. Right from picking a programming language and code management to cloud deployment and monitoring there are essential and well understood mechanisms to achieve high quality service.

Chapter 2: Golang Overview - One of the newer languages developed specifically for web services and cloud era applications is Golang, also known as Go. Once we cover the basics of Go, there is code organization patterns that ensure an easy way to reuse code as well as deliver the complete software bundle. Along with delivering functionality, the robustness of the algorithms and code quality has to be ensured for making the service runtime as smooth as possible and reduce regressions or failures.

Chapter 3: Web Frameworks - This chapter builds a vital competency to design and implement simple deployability, inter-platform support, integrated error checking, web server creation, and other development on the server using a range of Go Frameworks and libraries. We will be checking Gin, Echo, Beego, and Fasthttp trade offs to strengthen the server-side to support the scalability and high-speed performance of business applications.

Chapter 4: Microservices - This chapter dives into the world of deploying the backend services that are built using Go programs. We look at the history of deployments and how kubernetes has emerged as the leader for fine-grained service orchestration and deployment aspects that it handles automatically. We end this chapter with some possible drawbacks of microservices that need attention.

Chapter 5: Distributed Systems Overview - This chapter covers all the important ingredients that a backend developer needs to understand to help cook up a robust and resilient distributed system. Since modern services just cannot run on a single computer or server, we describe how distributed

setups can help overcome that restriction and the common pitfalls that need to be avoided in a such distributed solutions.

Chapter 6: Cross Service APIs - Services depending on each other need a mechanism to transfer information for making progress and delivering the right functionality of the application to the end user. We discuss some of the standard options such as REST, gPRC and GraphQL that are used in most current enterprise solutions using Go.

Chapter 7: Data Modeling - Data is an integral part of services that need to store and return related information for user application or offline processing. We discuss various persistence stores along with a tradeoff among traditional relational databases and streaming services and the newer general NewSQL/DistributedSQL options. We conclude the chapter with mechanisms that can be used in Go to connect to such data stores.

Chapter 8: Scalability, Availability and Other-ilities - This chapter digs deeper into the various aspects of setting up a large-scale distributed backend service to be available and adapt to changing workloads. We identify what are common bottlenecks at various layers of architecture and how to prevent them with the right tooling and mechanisms. We will also look into how to monitor and detect any anomalies and keep the system usable along with support from Go for these aspects.

Chapter 9: Containerization - This is an extension to Chapter 4 where we discussed microservices components. Here we quickly take a deep dive into how containers are created and deployed on top of Golang built software. Kubernetes is used as the vehicle for orchestration across such creation and updation across all the service containers. We conclude with how container APIs work in Golang.

Chapter 10: Code, CI/CD and Cloud - Once the basic aspects of the server functionality are in place, we cover the advanced processes of the software lifecycle, which make the code more robust and easy to deploy to on-premises server infrastructure or onto public clouds. There are some common development and monitoring tools that work with web scale services which we will explore as options in this chapter. We cover these as part of the DevOps and CI/CD processes and conclude with Golang cloud support.

Chapter 11: Securing Your Server - Server security is a crucial aspect in any software system to prevent data breaches or misuse of information and other malware. We discuss all the security concepts first and then look into well documented security holes that are tracked by the OWASP organization and how to prevent such vulnerabilities using mechanisms like encryption and Go library support.

Chapter 12: Upgrades and Maintenance - Version changes are inevitable in an enterprise software system, as these are integral aspects of **SDLC (Software Development Life Cycle)**. We provide some standard practices and options for rolling out such service upgrade and maintenance changes. Readers can also gain practical experience using the Golang examples in this chapter.

Chapter 13: Summary and Conclusion - Once we have all the technical information needed about the components for building large scale backend services with Golang, we will delve into the main aspects of growing and sustaining teams needed for such projects. The essential soft skills as well as do's and don't's for successful career growth are also outlined. Readers will get options for planning for the unknown as well as for next steps that can help make an impact in the software industry.

OceanofPDF.com

Code Bundle and Coloured Images

Please follow the link to download the
Code Bundle and the *Coloured Images* of the book:

<https://rebrand.ly/4f119e>

The code bundle for the book is also hosted on GitHub at <https://github.com/bpbpublications/Backend-Software-Architecture-using-Golang>. In case there's an update to the code, it will be updated on the existing GitHub repository.

We have code bundles from our rich catalogue of books and videos available at <https://github.com/bpbpublications>. Check them out!

Errata

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

errata@bpbonline.com

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

Did you know that BPB offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.bpbonline.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at :

business@bpbonline.com for more details.

At www.bpbonline.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on BPB books and eBooks.

Piracy

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at business@bpbonline.com with a link to the material.

If you are interested in becoming an author

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit www.bpbonline.com. We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit www.bpbonline.com.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



OceanofPDF.com

Table of Contents

1. Backend Systems Components

Introduction

Structure

Objectives

Server-side programming

Early days: 1950s-1960s

Mainframe era: 1960s-1970s

Client-server architecture: 1980s-1990s

Web/Internet era: 1990s-2000s

Modern web development: 2010s-2020s

Choosing a language

Source code version control

Repository

Commits

Branches

Merging

Pull requests

Conflict resolution

Git

Subversion

Mercurial

Perforce

Data modeling

Entities

Relationships

Security

Application programming interfaces

Simple Object Access Protocol APIs

Remote Procedure Calls APIs

WebSocket APIs

REST APIs

GraphQL

System testing

Unit testing

Integration testing

Acceptance testing

Regression testing

Performance testing

Security testing

Compatibility testing

Deployment and monitoring

Verification

Post-deployment tasks

Monitoring and maintenance

Scaling

Cloud services

Conclusion

2. Golang Overview

Introduction

Structure

Objectives

Golang installation

Data structures

Arrays

Slices

Maps

Structs

Interfaces and functions

Interface definition

Implementing interfaces

Using an interface

Empty interface

Type assertion

Type switch

Unit testing

Mock based testing

Concurrency

Goroutines

Channels

Buffered channels

Select statement

sync package

Mutex/sync.Mutex

Wait group/sync.WaitGroup

RWMutex /sync.RWMutex

Cond/sync.Cond

Object relational mapping

GORM

SQLBoiler

Ent

Go modules

Using Go modules

Software delivery

Version management

Benefits of versioning

Continuous integration and continuous deployment

Containerization

Deployment

Push Docker images

Launching the application

Monitoring and logging

Conclusion

3. Web Frameworks

Introduction

Structure

Objectives

Overview of web frameworks

Gin

Getting started with Gin

Echo

Getting started with Echo

Beego

Getting started with Beego

Revel

Getting started with Revel

FastHTTP

Getting started with FastHTTP

Conclusion

4. Microservices

Introduction

Structure

Objectives

History

Monolithic architecture

Advantages

Drawbacks

Service-oriented architecture

Advantages

Disadvantages

Microservices

Advantages

Disadvantages

Best practices

Service-oriented architecture versus microservices

Virtual machines

Advantages

Disadvantages

Containers

Virtual machines versus containers

Kubernetes architecture

Master nodes

Worker nodes

Networking

Storage

Custom components

Conclusion

5. Distributed Systems Overview

Introduction

Structure

Objectives

History

Components

- Computing nodes*
- Network infrastructure*
- Messaging*
- Data storage and management*
- Coordination and synchronization*
- Security*
- Scalability and load balancing*
- Fault tolerance and reliability*
- Service discovery*
- Monitoring and tracking*

Distributed system fallacies

- Fixing the fallacies*

Concurrency

- Mutual exclusion*
- Atomicity*
- Isolation*
- Deadlock detection and avoidance*
- Synchronization*
- Transactional support*
- Non-blocking operations*

Data consistency

Fault tolerance

- Replication*
- Erasur coding*
- Checkpointing and rollback*
- Consensus algorithms*
- Health monitoring*
- Failover and self-healing mechanisms*

Load balancing

- Load balancing algorithms*

CAP theorem

PACELC

Managing distributed systems

Monitoring and logging

Configuration management

Security management

Scalability management

Service management

Performance optimization

Tools and technologies

Golang for distributed systems

Conclusion

6. Cross Service APIs

Introduction

Structure

Objectives

History

Early days: 1950s-1960s

Time-sharing systems: 1960s-1970s

UNIX pipes: 1970s

Distributed networking: 1980s

Parallel and distributed computing: 1990s

Modern IPC mechanisms: 2000s-present

Microservice mechanisms: 2010s-present

Representational State Transfer

Origins: 1990s

Early adoption: 2000s

Mainstream adoption: 2010s

Current trends: 2020s

REST standards

Resource identification

HTTP methods

HTTP status codes

Payload format

Statelessness

Versioning

Security

Documentation

Hypermedia as the engine of application state

More considerations

REST using Golang

Accessing the API

Google remote procedure calls

Origins and development at Google: 2000s

Launch of gRPC: 2010-2015

Community adoption and growth: 2016-2018

Advanced features: 2019-2020

Widespread adoption: 2020-present

gRPC using Golang

Install prerequisites

Define protocol buffers

Go code from the .proto file

Server implementation

Emulate client calls

Run the server and client

GraphQL

Origins at Facebook: 2012-2015

Community growth: 2015-2016

Ecosystem expansion: 2017-2018

Advancements and enterprise adoption: 2018-2021

Latest innovations: 2021-present

GraphQL using Golang

Advanced features

Comparing REST, gRPC and GraphQL

Protocol and data format

Performance

Development and tooling

Compatibility and interoperability

Streaming and real-time communication

Error handling

Security

Types of use cases

Summary

Conclusion

7. Data Modeling

Introduction

Structure

Objectives

History

Data entity relationship

Entity-relationship diagram

Benefits of using ERDs

Using data modeling

NoSQL data modeling

Document stores

Key-value stores

Column-family stores

Graph databases

Takeaways

Golang database connectors

PostgreSQL

MySQL

MongoDB

Redis

Neo4j

Golang data modeling support

Golang object-relational mapping

Ent

SQLX

GQLGen

Data streaming solutions

Define the event schema

Event and processing times

Design for schema evolution

Use of metadata

Aggregation and state management

Apache Kafka

Apache Pulsar

Redis Streams

Apache Flink

Conclusion

8. Scalability, Availability and Other-ilities

Introduction

Structure

Objectives

Overview

Availability

Scalability

Horizontal and vertical scaling

Load balancing

Caching

Asynchronous processing

Microservices

Elasticity

Database scalability

Monitoring and analytics

Reliability

Redundancy

Fault tolerance

Monitoring and alerting

Error handling and logging

Data integrity

Software management

Security

Documentation

Monitoring

Performance monitoring

Health monitoring

Security monitoring

Network monitoring

Alerting

Visualization

Logging and tracing

Maintainability

Modular design

Code quality

Documentation

Automated testing

Version control

Continuous integration and continuous deployment

Knowledge sharing

Usability

- User-centered design*
- User interface design*
- Information architecture*
- Accessibility*
- Responsiveness*
- Error handling*
- Documentation*
- Workflow design*
- Localization*

Golang-ilities

Conclusion

9. Containerization

Introduction

Structure

Objectives

Creating containers

- Creating a Go application*

- Creating a Dockerfile*

- Building the Docker image*

- Running the Docker container*

- Application testing*

- Docker image registry*

- Deploying the container*

Deploying containers

- Prerequisites*

- Creating a deployment YAML file*

- Creating a service YAML file*

- Deploying to Kubernetes*

- Verifying the deployment*

- Accessing the application*

Container management tools

Horizontal Pod Autoscaler

Cluster Autoscaler

Rolling updates

Canary deployments

Rollout management

Helm

Kustomize

Prometheus and Grafana

Service mesh

Container security

Container image security

Container runtime security

Network security

Secrets management

Role-based access control

Pod Security Standards

Admission controllers

Service accounts

Encryption and certificates

Auditing

Health checks

Liveness probe

Readiness probe

Startup probe

Customized checks

Event-based checks

Container APIs in Golang

Docker APIs

Kubernetes API

Open Container Initiative APIs

Conclusion

10. Code, CI/CD and Cloud

Introduction

Structure

Objectives

Code quality

Readable code

Follow coding standards

Use standard library

Testing

Documentation

Code reviews

Refactoring

Performance optimization

Versioning and dependencies

DevOps

Continuous integration

Continuous delivery

Infrastructure as code

Configuration management

Continuous monitoring

Security and compliance

Collaboration

Release management

Observability

Incident management

Continuous integration/continuous deployment

Day 2 operations

Public cloud services

Service types

Benefits of public cloud

Sample use cases

Major cloud providers

Challenges

Golang for cloud

Cloud APIs

Microservices orchestration

Pub/Sub messaging

Serverless computing

Cloud deployments

Logging and monitoring

Security and compliance

Conclusion

11. Securing Your Server

Introduction

Structure

Objectives

Security concepts

Open Web Application Security Project

OWASP Top 10

OWASP projects

Community and guidance

Data leaks

Detecting data leaks

Preventing data leaks

Authentication

Detecting authentication issues

Preventing authentication issues

Authorization

Detecting authorization issues

Preventing authorization issues

Transport Layer Security

Security measures

Golang software security

Conclusion

12. Upgrades and Maintenance

Introduction

Structure

Objectives

Day 2 operations

Monitoring

Software upgrades

Scaling

Backup and disaster recovery

Security management

Performance optimization

Configuration management

Compliance and auditing

Upgrades overview

Bug fixes

Security enhancements

Performance improvements

New functionality

Infrastructure changes

Data migration

End of life management

Upgrades models

Monolithic upgrades

Rolling upgrades

Phased rollout

Canary releases
Blue-green deployment
Feature flags
Hot fixes

Golang support

Graceful shutdowns and restarts
Hot reloading
Blue-green deployment in Go
Canary deployment and feature flags in Go
Rolling deployments with Kubernetes
Database schema migrations
Versioning and compatibility

Conclusion

13. Summary and Conclusion

Introduction
Structure
Objectives
Software team building
Twenty-one lessons
Soft skills
Next steps
Conclusion

Index

CHAPTER 1

Backend Systems Components

Introduction

On our journey, we will first delve into an overview of the essential concepts and considerations needed for software engineers to build backend systems. Software engineering itself is defined notably along these two ways:

The systematic application of scientific and technological knowledge, methods, and experience to the design, implementation, testing, and documentation of software.

—The Bureau of Labor Statistics

The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.

—IEEE Standard Glossary of Software Engineering Terminology

As a professional software engineer, a good understanding of the components and trade-offs for the above aspects of software development will help deliver robust solutions for any given problem. From picking a programming language and code management, to cloud deployment and monitoring, we will cover the essential building blocks of complex backend systems.

Structure

The chapter covers the following topics:

- Server-side programming
- Source code version control
- Data modeling
- Security
- Application programming interfaces
- System testing
- Deployment and monitoring
- Cloud services

Objectives

The main goal of this chapter is to provide an overview of the different aspects of building complex backend systems. We will delve into various stages of the software lifecycle along with the tools used as part of the development process and describe how those pieces are joined together. The following figure is the pictorial depiction of some of the main ingredients that contribute to a backend system:

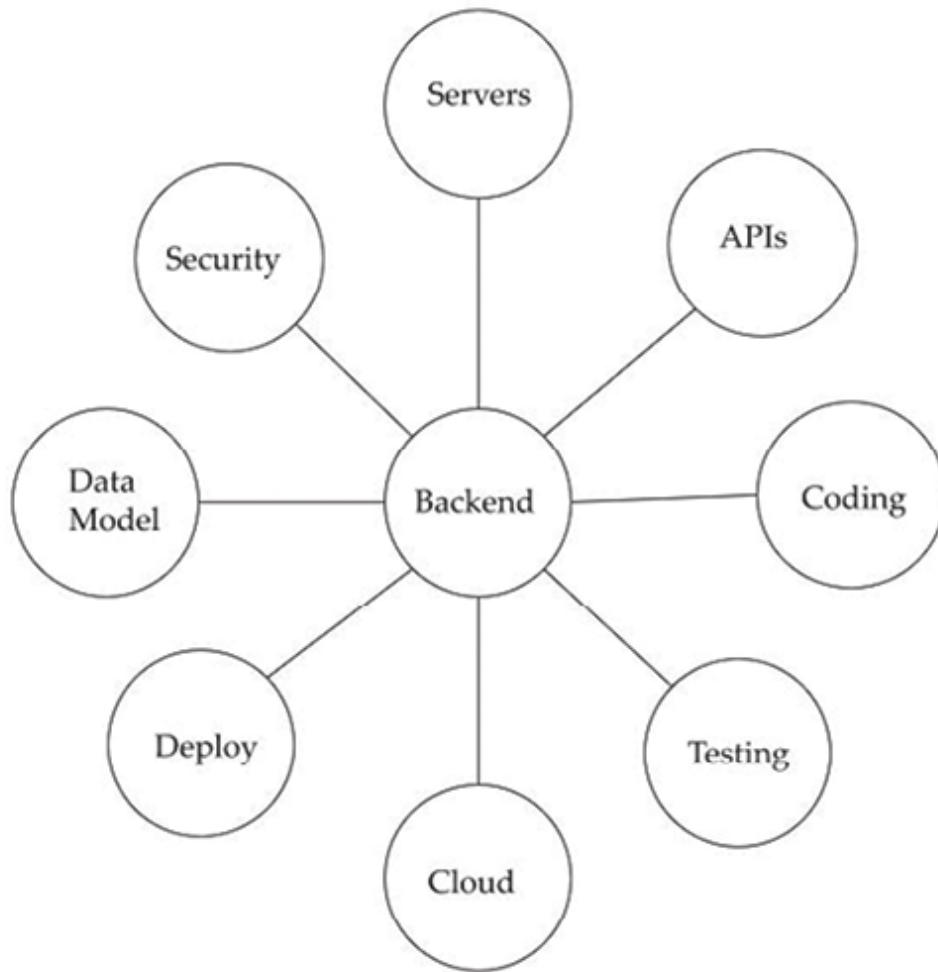


Figure 1.1: Components of backend software systems

By the end of this chapter, you will understand the various components and their interactions, which are needed to build a robust backend software system.

Server-side programming

A server is a computer system or software that provides functionality for other programs or devices, known as **clients**. In backend development, servers handle requests from clients and execute the necessary logic to fulfill those requests. They are part of the backend infrastructure that is needed to deploy any software system.

Backend programming languages are used to build the server-side logic and functionality of web applications. Backend software systems are primarily built using server-side programming languages such as Java, Go, C++, Ruby, etc. These languages are responsible for handling requests from clients, processing related data known to the server, and generating and returning the responses. This includes internal tasks such as database operations, user authentication, server-side processing, and communication with other dependent and downstream services.

The history of programming languages is closely tied to the evolution of computing and the internet. This section discusses a brief history of some key milestones and developments in this area.

Early days: 1950s-1960s

Right after the initial innovations in theoretical computing machines and invent of devices that can crunch numbers and calculate following a sequence of instructions, there was a need for developing some formal languages to program them.

- The earliest programming languages, such as Assembly and Fortran, were primarily used for low-level system programming and scientific computing.
- **Common Business-Oriented Language (COBOL)**, developed in the late 1950s, became widely used for business and administrative applications, including backend systems.

Mainframe era: 1960s-1970s

Mainframes were large-scale machines with greater reliability that processed large amounts of data for missing critical applications such as finance and healthcare.

- As mainframe computers became more prevalent, programming languages like COBOL and PL/I were commonly used for backend development, handling tasks such as data processing and transaction management.
- *IBM's System/360* mainframe architecture played a significant role in shaping backend computing during this period.

Client-server architecture: 1980s-1990s

In this era, data management and the beginning of practical use cases of distributed systems as well as networking boosted relational database usage and growth in inter-connected machines.

- With the rise of client-server architectures, languages like C, C++, and Pascal were used for backend development, often in conjunction with database systems such as *Oracle* and *IBM DB2*.
- Middleware technologies like **Common Object Request Broker Architecture (CORBA)** and **Distributed Component Object Model (DCOM)** facilitated communication between distributed systems.

Web/Internet era: 1990s-2000s

The advent of the **World Wide Web (WWW)** led to the emergence of new backend programming languages and technologies to allow efficient web languages as well as different operating systems.

- Java, released by *Sun Microsystems* in 1995, gained popularity for backend development due to its platform independence and robustness.
- *Microsoft* introduced **Active Server Pages (ASP)**, later replaced by ASP.NET, for server-side web development using languages like VBScript and C#.
- Python, with frameworks like Django and Flask, and Ruby, with Ruby on Rails, also gained traction for web backend development during this period.

Modern web development: 2010s-2020s

Once the internet was more widely available and understood, and with advances in hardware speed and power, there were significant areas of improvement in **software as a service (SaaS)**. Different tool sets were developed for building cloud-based apps and mobile services, with large-scale analytics on unstructured data.

- JavaScript, traditionally used for frontend development, gained popularity for backend development with the rise of Node.js, enabling full-stack JavaScript development.

- The popularity of microservices architecture led to the adoption of backend languages and frameworks optimized for scalability and modularity, such as Go and Elixir with the Phoenix framework.
- The proliferation of cloud computing services, data science and containerization technologies further influenced backend development practices, with languages like Python, Go, and Rust gaining popularity for cloud-native applications.

Overall, the history of backend programming languages reflects the evolution of computing paradigms, from mainframes to the web and beyond, driven by advancements in technology and changes in application requirements. Understanding these concepts is crucial for backend developers to build robust, scalable, and secure backend systems and web applications. Different languages and frameworks may prioritize these concepts differently, so it is important for developers to choose the right tools for their specific project requirements and constraints as discussed below.

Choosing a language

Choosing a backend programming language in the current software industry involves balancing several trade-offs based on the needs of the application being developed, the team's background, and the infrastructure in use. Here are the key trade-offs to consider:

- **Performance versus development:** Languages like C++, Rust, and Go offer high runtime performance, making them suitable for applications with heavy computational loads or real-time requirements. However, these languages may have a steeper learning curve and require more boilerplate code. On the other hand, Python and Ruby are typically easier to write and read, with shorter development cycles. This is ideal for rapid prototyping or startups but may not match the type safety and raw performance of lower-level languages.
- **Concurrency and scalability:** Java and Go are designed with concurrency in mind, offering threading and concurrency models for handling many simultaneous connections, which is ideal for I/O-bound and network-heavy and highly scalable applications. Python and Ruby

can handle concurrency but may require third-party libraries or asynchronous programming models that can be complex to manage.

- **Ecosystem:** JavaScript and Python have vast ecosystems with packages for almost every purpose, making it easy to integrate third-party tools, **application programming interface (API)**, and services. Java and C# have robust ecosystems in enterprise environments, with mature libraries, frameworks, and security features ideal for large-scale applications. However, they may be more limited in emerging tech areas like data science or machine learning compared to Python.
- **Integration with infrastructure:** Java and C# integrate well with legacy enterprise systems, making them ideal choices if the new application will interact with older, established systems. Python and JavaScript are more commonly used for modern, cloud-based, and serverless architectures.
- **Long-term maintenance:** Static languages like Java, Go and Rust tend to lead to more maintainable codebases over time due to type safety and strict syntax. Dynamic languages like Python and JavaScript allow for rapid changes but may accumulate technical debt if not carefully managed, especially in larger codebases.

Newer languages tend to have smaller developer talent pool, and some languages might need licenses. Each backend programming language has its advantages and trade-offs, so the best choice depends on balancing the above factors according to your project's specific goals and constraints. We will dig into one of the newer Web 2.0 programming languages of Golang and use it to demonstrate how to put building backend systems into practice.

Source code version control

Having discussed program development, source code version control is the next key component any developer should be closely familiar with to deliver backend system features efficiently. Also known as a **version control system (VCS)** or revision control, it is a critical component of modern software development that records changes to files over time so that one can recall specific versions later and track the history of code paths.

Understanding branching strategies, pull requests, and code review processes is important for managing backend codebases. It is an essential tool for collaborative software development and for managing changes to codebases over time across contributors. This section discusses an overview of how it works and some commonly used VCSs.

Repository

A repository, or repo, is a central location where the history of changes to your codebase is stored. It typically contains all the files that make up your project along with metadata about those files. GitHub is a cloud-based platform where you can store, share, and work together with others to write code.

Commits

A commit is a snapshot of the code at a specific point in time. When you make changes to your code, you commit those changes to the repository along with a message describing what was changed.

Branches

Branches are separate lines of development that diverge from the main line of development (often called the **master** or **main branch**). They allow multiple developers to work on different features or fixes simultaneously without interfering with each other. Branches can be created, merged, and deleted as needed.

Merging

Merging is the process of combining changes from one branch into another. This is typically done when a feature is complete and ready to be integrated into the main codebase.

Pull requests

In many VCS, such as Git, developers use pull requests to propose changes to the main codebase. Pull requests allow other developers to review the proposed changes before they are merged into the main branch.

Conflict resolution

Sometimes, when merging changes from one branch into another, conflicts may arise if the same file has been modified in different ways on different branches. Resolving these conflicts involves manually reconciling the differences between the conflicting changes while retaining the expected logic.

Let us take a look at some commonly used VCS.

Git

Git is one of the most popular distributed VCS. It is widely used in both open-source and commercial projects due to its speed, flexibility, and powerful branching and merging capabilities. We have provided an overview of the steps needed to get the locally developed and tested code into a hosted repository master branch so that it can be accessed by other users of that repo. An example of such a hosted service is <https://github.com/>.

Figure 1.2 shows the usual steps in getting local code merged into a remote repository, along with sample Git commands:

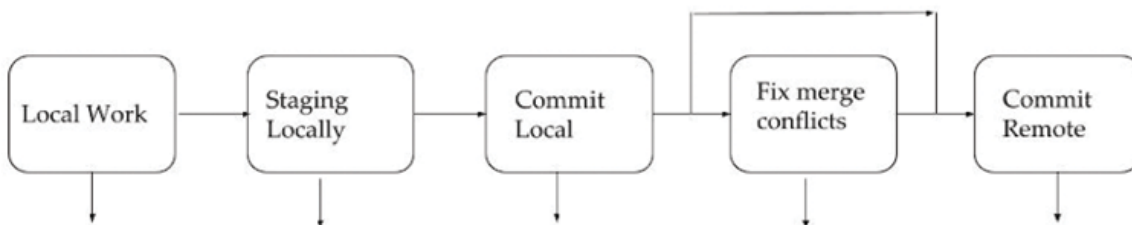


Figure 1.2: Common code version control steps

Subversion

Subversion (SVN) is a centralized VCS that was popular before the widespread adoption of distributed systems like Git. It is still used in some organizations, particularly those with legacy codebases.

Mercurial

Mercurial is another distributed VCS similar to Git. It offers many of the same features and capabilities but has a different underlying architecture.

Perforce

Perforce is a centralized VCS commonly used in enterprise settings, particularly for managing large codebases and binary files.

Using VCSs such as Git allows developers to collaborate effectively, track changes, and maintain codebase integrity.

Data modeling

Backend systems often involve storing and retrieving data from some form of persistent storage. Data modeling involves designing the structure of such data to be stored, while storage mechanisms determine how and where the data is stored efficiently and securely.

Let us consider a simple data modeling example for an e-commerce website that sells some products and has customers order these products. We will focus on modeling the data for the products, customers, and order entities.

Entities

The following three are the main types of the elements in this sample data model:

- **Product:** This entity represents the products available for sale on the e-commerce platform. Each product has attributes such as **ProductID**, **Name**, **Description**, **Price**, **Quantity**, etc.
- **Customer:** This entity represents the customers who visit the e-commerce platform. Each customer has attributes such as **CustomerID**, **Name**, **Email**, **Address**, etc.
- **Order:** This entity represents the orders placed by customers. Each order has attributes such as **OrderID**, **CustomerID** (to link the order to the customer who placed it), **OrderDate**, **TotalAmount**, etc.

Relationships

This aspect describes the associations among the elements of this data model:

- **Product-order relationship:** Each order can consist of multiple products, and each product can be part of multiple orders. This is a many-to-many relationship. To represent this, we can introduce a junction table called **OrderDetails**. This table will have attributes such as **OrderID** and **ProductID**, indicating which products are included in each order.
- **Customer-order relationship:** Each customer can place multiple orders, but each order is placed by only one customer. This is a one-to-many relationship. The **CustomerID** attribute in the Order entity establishes this relationship.

Let us take a look at an example schema:

Product (ProductID, Name, Description, Price, Quantity)

Customer (CustomerID, Name, Email, Address)

Order (OrderID, CustomerID, OrderDate, TotalAmount)

OrderDetails (OrderID, ProductID, Quantity)

Product:

(ProductID, Name, Description, Price, Quantity)

(1, "Laptop", "High-performance computer", 1000, 50)

(2, "Smartphone", "Latest smartphone model", 800, 100)

(3, "Headphones", "Noise-canceling headphones", 200, 75)

Customer:

(CustomerID, Name, Email, Address)

(1, "John Doe", "john@example.com", "123 Main St, City, Country")

(2, "Jane Smith", "jane@example.com", "456 Elm St, City, Country")

Order:

(OrderID, CustomerID, OrderDate, TotalAmount)

(101, 1, "2024-04-08", 1200)

(102, 2, "2024-04-08", 1000)

Order details:

(OrderID, ProductID, Quantity)

(101, 1, 1) # John ordered 1 Laptop

(101, 3, 2) # John ordered 2 Headphones

(102, 2, 2) # Jane ordered 2 Smartphones

This is a simple example of data modeling for an e-commerce system. It defines the entities (Product, Customer, Order) and their attributes, as well as the relationships between them. [Figure 1.3](#) presents the visual summary of this data model:

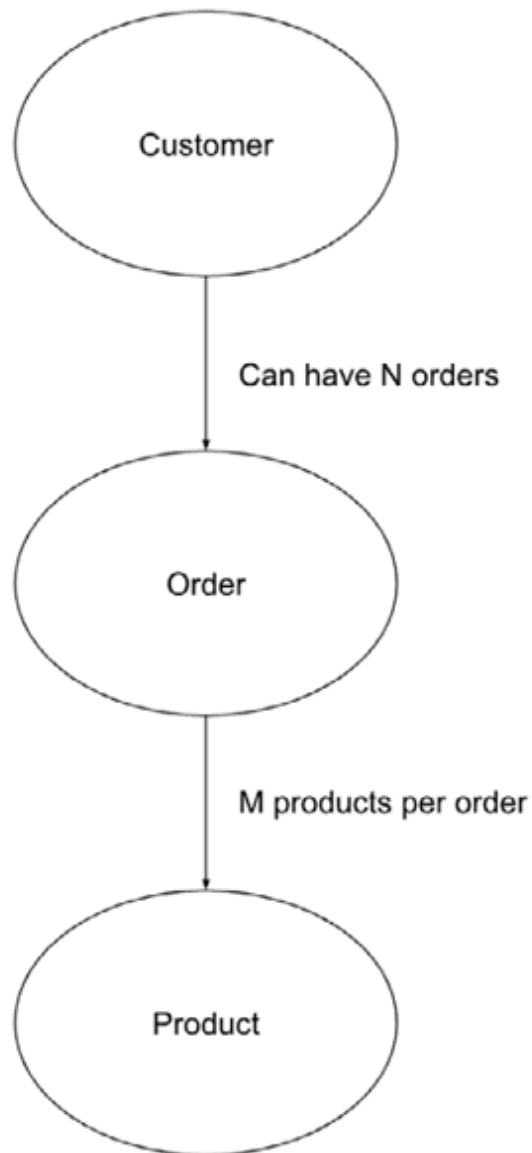


Figure 1.3: Customer, Order, and Product relational data model

This model can be expanded and optimized based on specific requirements and considerations such as performance, scalability, and data integrity constraints.

Databases usually provide the best mechanisms for creating, modifying/deleting, and retrieving stored data. Popular examples include Oracle, MySQL, PostgreSQL, MongoDB, and distributed SQL like YugabyteDB and CockroachDB. Understanding database concepts such as relational databases (SQL), NoSQL databases, data modeling, querying, indexing, and transactions is crucial to providing value to the user. We will cover this in more detail in *Chapter 7, Data Modeling*.

Some backend frameworks also provide pre-built components and tools to streamline the development of backend systems.

Security

Security is a critical concern in backend development. Implementing security measures to protect the backend system from unauthorized access, data breaches, **denial of service (DoS)** attacks, and other security threats is essential. It involves protecting data and services by safeguarding against common security threats such as SQL injection attacks, **distributed DoS (DDoS)**, and **Cross-Site Scripting (XSS)**. This includes practices such as authentication, authorization, encryption, and secure communication protocols (for example, HTTPS).

Authentication verifies users identities, ensuring they are who they claim to be. Authorization determines what actions users are allowed to perform within the system once they are authenticated. Backend languages often provide mechanisms for user authentication and authorization. Common techniques for authentication include **JSON Web Tokens (JWT)**, OAuth, and session-based authentication.

So, backend developers must be mindful of security concerns such as data validation and input sanitization. In a future chapter, we will cover more details of the vulnerabilities and solutions to prevent them.

Application programming interfaces

Backend services commonly build and publish APIs that enable communication between different software systems or components. Such services expose APIs at known **endpoints** (such as URL and a server port) that allow clients to communicate with that backend server.

API architecture is usually explained in terms of client and server. The application sending the request is called the **client**, and the application sending the response is called the **server**. APIs enable clients to be in one service that can interact with backend servers in a structured and efficient manner.

There are a few different ways that APIs can work depending on when and why they were created.

Simple Object Access Protocol APIs

These APIs use **Simple Object Access Protocol (SOAP)**. Client and server exchange messages using XML. This is a cumbersome and less flexible API that was more popular in the past.

Remote Procedure Calls APIs

These APIs are called **Remote Procedure Calls (RPC)**. The client completes a function (or procedure) on the server, and the server sends the output back to the client. One common usage is via the *Google* published standard of gRPC, which uses protocol buffers for request and response definitions.

WebSocket APIs

WebSocket API is another modern web API development that uses JSON objects to pass data. It supports two-way communication between clients and the server. The server can send callback messages to connected clients, making it more efficient than the REST API.

REST APIs

These are the most popular and flexible APIs found on the web today. The client sends requests to the server as data. The server uses this client input to start internal functions and returns output data back to the client. We will see its advantage below and use this as the default mode for building backend service communication networks.

REST APIs provide a standardized way for clients to interact with backend systems over HTTP, using methods such as GET, POST, PUT, DELETE, etc. APIs define how different software components should interact. The key benefit of this is that it is a stateless mode of communication, so there is no overhead for the server to store any information about each such request or its progress. These also provide options to securely connect using authentication.

GraphQL

GraphQL is a query language for APIs developed by *Facebook* that allows clients to request only the data they need and nothing more. Instead of multiple endpoints like in REST, there is only one endpoint in GraphQL, and clients can specify their data requirements in the query. It is particularly useful for complex and rapidly evolving applications where the data requirements of clients may vary widely.

Choosing between REST and GraphQL often depends on factors like the complexity of your application, the specific requirements of your clients, and your team's familiarity with each technology. REST might be more suitable for simple, resource-oriented APIs, while GraphQL shines in scenarios where flexibility and efficiency in data fetching are preferred.

System testing

Testing ensures that the backend system functions as expected and meets its requirements. Writing comprehensive unit tests, integration tests, and end-to-end tests helps ensure the correctness/quality, reliability, and functionality of a product and prevents breaking changes (also known as **regressions**) when new code is added down the line. There are various types of testing, each serving a specific purpose in the software development life cycle. We will be exploring these types in this section.

Unit testing

This involves testing individual units or components of the software to ensure they function correctly in isolation. Automated testing frameworks are often used for this purpose.

Integration testing

This type of testing ensures that different software components or modules work together as expected. It focuses on testing the interactions between integrated units.

Acceptance testing

Also known as **user acceptance testing (UAT)**, this type of testing involves verifying that the software meets the user's requirements and is ready for deployment. It is typically performed by end-users or stakeholders, potentially as a proof of concept for ensuring all desired requirements have been met. The user-friendliness and intuitiveness of the software interface can be covered. It involves observing real users as they interact with the software to identify usability issues and gather feedback for improvement.

Regression testing

Regression testing ensures that recent software changes or updates have not adversely affected existing features or functionalities. It involves re-running previously conducted tests to validate that no new defects have been introduced.

Performance testing

Performance testing evaluates the responsiveness, stability, and scalability of the software under various workload conditions. It includes load testing, stress testing, and scalability testing to identify performance bottlenecks and ensure optimal system behavior. Usually, this involves tracking metrics such as latency per operation and throughput of the system along with operating systems metrics such as memory consumed, CPU used, I/O rate, and network speed.

Security testing

This type of testing assesses the security of the software by identifying vulnerabilities and weaknesses that could be exploited by malicious users. It includes techniques such as penetration testing, vulnerability scanning, security code and compliance reviews.

Compatibility testing

Compatibility testing ensures that the software functions correctly across different platforms, devices, operating systems, and web browsers. It verifies that the software is compatible with its intended usage environment.

These are just a few examples of the many types of testing that can be employed as part of the software development life cycle to ensure the quality and reliability of the product as it evolves. Each type of testing serves a specific purpose and contributes to delivering a high-quality software solution.

Next, we will see how such tested code is deployed and monitored on production servers.

Deployment and monitoring

Deployment involves releasing the software as a service to production environments, often using **continuous integration/continuous deployment (CI/CD)** pipelines. The integration process maps to the code/feature merge steps along with generating a version to identify the current state of the software. The deployment process corresponds to getting the desired (normally latest) version of the software running on backend servers. The continuous aspect of CI/CD relates to not having to schedule a special window for stopping existing service processes in most cases.

The deployment process is usually automated using various tools that proceed according to some pre-configured deployment plan. These might involve transferring files to servers, updating configuration settings, switching over to the desired version in the runtime configuration, and restarting processes to run the configured version of services. Think of this

process as a train always leaving the station, and once your software is ready to board, it gets onto the train and gets scheduled.

Verification

After deployment, one would need to verify that the software is functioning correctly in the target environment. This may involve running automated tests, manual checks using spot requests, and monitoring operating system and service logs for errors/warnings.

Post-deployment tasks

The next logical step is to complete post-deployment tasks, such as updating documentation, notifying users or stakeholders, and performing any necessary cleanup or maintenance tasks. Documenting backend system components, including APIs, codebase, configuration, and deployment processes, is essential for facilitating collaboration, onboarding new developers, and ensuring maintainability.

Monitoring and maintenance

Monitoring involves observing the system's performance, availability, and other metrics to identify issues and optimize performance proactively. A designated site reliability team or the developers can themselves continuously monitor the deployed software to ensure it remains operational and performant. This involves monitoring system and service level metrics, tracking security vulnerabilities, and applying patches or updates as needed.

Some or most of these monitoring is automated by sending over the metrics to graphs and setting up automated alerts to notify personnel even when there is a discrepancy in the values observed.

Monitoring and identifying performance bottlenecks or regressions in backend systems via metrics and graphs can significantly help improve system efficiency and service responsiveness. This involves profiling and optimizing the code logic and other internal algorithms, such as minimizing database queries and leveraging caching mechanisms.

Another option for deployment is to use systems for longer end-to-end testing, referred to as **canary-based deployments**. In these, a new version of the software can be tested in parallel with the previous version on real-world traffic before upgrading the whole service to the latest version.

Implementing logging and monitoring mechanisms also allows developers to track system behavior, diagnose issues, and optimize performance as well. This involves logging relevant events, setting up monitoring tools such as graphs and profilers, and analyzing end-to-end system metrics. Logging involves recording events and activities within the backend system for analysis, debugging, and auditing purposes.

Scaling

Based on the monitoring, if the software needs to handle increased load or user demand, one can scale the deployment by adding more resources, such as servers or cloud instances, and adjusting configurations as necessary. Two options for this correspond to vertical or horizontal scaling.

Vertical scaling, also known as **scale-up**, implies increasing the resources of one or more of the servers (or replacing them with bigger machines). This could mean adding more powerful CPUs, increasing the assigned CPU count, or increasing the memory available to the service. Horizontal scaling, also known as **scale-out**, corresponds to increasing the number of servers (virtual machines) on which the service runs and allowing user traffic access to the extra servers.

Cloud services

Once the code is tested and we have seen what deployment entails, we now cover where the services can be deployed. For the past decade or so, some of the backend infrastructure components have moved to a service offering model. The evolution of the cloud has helped reduce the overhead of maintaining data centers and various infrastructure-related services being provided by cloud services. **Amazon Web Services (AWS)**, one of the first cloud service providers, maintains a growing set of off-the-shelf services that can be used to quickly and flexibly build scalable and resilient

applications. Similar options are present on other public cloud services, such as *Microsoft Azure* and *Google Cloud*.

Such services can further reduce the amount of time developers spend on infrastructure and platform-related components. They become robust and readily available pieces that are pluggable via APIs into the service backend. Think of cloud services as selling the components for building appliances—it is up to the solutions architect or developer to put them in the right place within their application backend framework to build a TV or a car.

Note: That the above options can be used to create a hybrid solution, where the services are deployed across clouds and also among self-managed data centers. This option can be used when the application or data has certain geographic or security requirements.

We will explore the details of different tools available on cloud platforms that could help increase the velocity of backend development in Chapter 10, CI/CD and Cloud.

Conclusion

In this chapter, we discussed various aspects of building a robust backend system and got an overview of existing technologies that help us achieve this goal. In the upcoming chapters, we will build upon some of these aspects to gain hands-on experience creating and deploying backend solutions.

As part of that sequence, in the next chapter, we are going to learn about one of the newer languages that have quickly become the industry standard for backend development, especially for microservice architectures and cloud-native systems.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 2

Golang Overview

Introduction

Building the backend systems starts with creating services that provide functionality for any use case. The core mechanism for performing this process is programming.

Programming is the art of telling another human being what one wants the computer to do.

— Donald Ervin Knuth

At some level, programming is an art centered around logic and a disciplined approach to building solutions using the right tools with coherent design paradigms. The programming language tool of choice for creating the application or backend depends highly on the engineer, their team and on the target market in some instances. In the current state of the software industry, Python and Java are highly used for historical reasons, though newer and more sophisticated languages such as Rust and Golang are gaining ground.

One of the newer languages developed specifically for Web 2.0 and cloud era backend services and applications is Golang, also known as just **Go**. We will pick that as the protagonist for building modern software.

Let us delve into the details of its syntax and semantics and why those make it a good tool for our journey. Once the basics of Go are covered, there are

also code organization patterns that make it really easy to reuse code and deliver the complete software bundle. Along with delivering on time, quality must be ensured to make the process as smooth as possible and reduce regressions or failures.

Structure

The chapter covers the following topics:

- Golang installation
- Data structures
- Interfaces and functions
- Concurrency
- Object relational mapping
- Go modules
- Software delivery

Objectives

This chapter takes us into the multiple facets of the Golang programming language. The goal by the end of this chapter is to understand the various data structures, how they managed via functions and can be safely synchronized as needed across execution contexts.

We will also delve into common tools for connecting a service written in Golang to datastores needed for backend systems storage and retrieval. The last portion of the chapter will provide steps on how to bundle the software and its dependencies for rolling out to servers for deployment.

By the end of this chapter, the reader should be able to grasp the main aspects of Go that help build software systems faster and cleaner.

Golang installation

The first call to action is to download and install the latest version of Golang for the **operating system (OS)** of your choice. Since MacOS is

common for backend development, we have provided sample steps based on that OS below:

1. **Download Go package:** Visit the official Go download page <https://go.dev/doc/install> to get the latest version for MacOS. Download the **.pkg** installer file for macOS (it will be something like **go1.23.4.darwin-amd64.pkg**).
2. **Install Go:** Double-click the downloaded **.pkg** file to launch the installer. Follow the on-screen instructions to complete the installation. By default, Go language binaries are installed in **/usr/local/go** location.
3. **Environment variables:** Open or create a profile file, such as **.zshrc** (for Zsh) or **.bash_profile** (for Bash), in your home directory. Add the following lines to set up your Go related environment paths:

```
export PATH=$PATH:/usr/local/go/bin
```

```
export GOPATH=$HOME/go_code
```

```
export PATH=$PATH:$GOPATH/bin
```

All the go files we create in this chapter can be created in the **~/go_code** directory to use the above setup. Save the file using one of the following to reload the terminal configuration:

```
source ~/.zshrc      # For Zsh
```

```
source ~/.bash_profile # For Bash
```

4. **Verify:** Run the following command to check the installed Go version and you should see the expected installed go version as the one downloaded in *Step 1*:

```
go version
```

Data structures

Go is a statically typed, compiled programming language designed by *Google* engineers. Go has a straightforward syntax and comes with a standard library that includes support for common data structures such as slices, maps, and channels. Let us now discuss some of the common data structures in Go and their use cases.

Arrays

An array is a fixed-size data structure that holds elements of a specific type. Once an array is initialized with a size, its length cannot change. Below we show how to create and initialize such array types.

```
1. package main
2.
3. import "fmt"
4.
5. func main() {
6.     // Declare an array of length 5 containing integers
7.     var myArray [5]int
8.
9.     // Initialize array values
10.    myArray[0] = 10
11.    myArray[1] = 20
12.    myArray[2] = 30
13.
14.    fmt.Println("Array: ", myArray)
15. }
```

Note that the size of the array above is 5, but only three of the array indices are initialized, so by default, Go will leave the last two values as zero.

This structure can be useful for small sets of related data that do not need to change once created. It is rare to use this in a very general-purpose Go program. Instead, slices are more commonly used for getting reference semantics, as we will see in the next section.

Slices

A slice is a dynamically sized, flexible view of an array. It can grow and shrink, and it is more commonly used than arrays as the slice is just a reference to the array, and there are no values copied over, so it is more efficient. A sample of operations possible on a slice is shown in the

following snippet:

```
1. package main
2.
3. import "fmt"
4.
5. func main() {
6.     // Create a slice of integers
7.     slice := []int{10, 20, 30, 40}
8.
9.     // Append an element to the slice
10.    slice = append(slice, 50)
11.
12.    fmt.Println("Slice:", slice)
13.
14.    // Create a sub-slice from the slice
15.    subSlice := slice[1:3]
16.    fmt.Println("Sub-slice:", subSlice)
17. }
```

One can immediately notice that the type of **slice** variable is auto derived using the `:=` option to create the object, implicitly also declaring it along with initializing it at *line 7*. At *line 10* it can just be reassigned using `=`.

Maps

A map is an unordered collection of key-value pairs. It is similar to dictionaries in other languages. Maps provide constant time complexity for most insert and lookup operations. The following code sample shows steps to create, get, and update map data structures:

```
1. package main
2.
3. import "fmt"
```

```

4.
5. func main() {
6.     // Declare a map with string keys and integer values
7.     myMap := make(map[string]int)
8.
9.     // Add key-value pairs to the map
10.    myMap["a"] = 1
11.    myMap["b"] = 2
12.    myMap["c"] = 3
13.
14.    fmt.Println("Map:", myMap)
15.
16.    // Access values using keys
17.    fmt.Println("Value for key 'b':", myMap["b"])
18.
19.    // Update the value for a key
20.    myMap["b"] = 22
21. }

```

Note: That there is no explicit set data type in Go, but usual standard is to use a map with the values as generic interface{} to emulate a set for the unique keys.

Structs

Structs are used to define custom data types in Go. They group related data together and provide a way to create user-defined data types. A struct can be made of native types fields or contain other structs as fields as well. The following snippet shows data struct definition, initialization and update on the value for the fields:

```

1. package main
2.
3. import "fmt"
4.
5. // Define a struct with some fields

```

```

6. type Person struct {
7.     Name string
8.     Age  int
9. }
10.
11. func main() {
12.     // Initialize a struct
13.     person := Person {
14.         Name: "Alice",
15.         Age: 30,
16.     }
17.
18.     // Access the struct fields
19.     fmt.Println("Name:", person.Name)
20.     fmt.Println("Age:", person.Age)
21.
22.     // Modify struct fields
23.     person.Age = 31
24.     fmt.Println("Updated age:", person.Age)
25. }

```

These are some of the most common data structures in Go. Any experienced programmer from the Java/C++ world, with a keen observation, can note that there are no explicit constructors or destructors in Golang structs. One can just get a default zero-value when a variable is created. Depending on the type of the variable, Go assigns it a default—for all numeric types, the default value is 0; for Boolean, it is false; and for strings, it is an empty string (""), and for slices, it is the empty initial state, ready to be appended.

One can get more advanced data structures like linked lists, heaps, trees, and even bloom filters by using existing libraries or building their own as a side project. You can begin writing efficient Go programs by understanding how to use these basic data structures.

Interfaces and functions

Interfaces are a key feature of the Go programming language. They provide a way to define behavior through a set of function signatures without specifying how that behavior should be implemented. Interfaces allow for polymorphism and decoupling in Go code, enabling you to write flexible and reusable code. We will discuss the key concepts of Go interfaces further.

Interface definition

An interface is a set of functions showing expected parameters and return value types (aka signatures). It is defined using the **type** keyword followed by the interface name and a list of method signatures. Refer to the following example:

```
1. type Shape interface {  
2.     Area() float64  
3.     Perimeter() float64  
4. }
```

In this example, **Shape** type objects need to define these two matching functions.

Implementing interfaces

A struct type implements an interface just by providing methods with the exact signatures specified in the interface. Go does not require explicit declarations to say a type implements an interface; as long as the methods match, the type implicitly implements the interface.

Note: There could be other methods in the implementing class as well.

```
1. type Circle struct {  
2.     radius float64  
3. }  
4.  
5. func (c Circle) Area() float64 {
```

```

6.  return 3.14 * c.radius * c.radius
7. }
8.
9. func (c Circle) Perimeter() float64 {
10.  return 2 * 3.14 * c.radius
11. }
12.
13. // Circle now implicitly implements the Shape interface.
14.
15. // There can be other functions as well for this struct.
16. func (c Circle) Color(input string) string {
17.  return "light" + input
18. }

```

Note that functions are defined for a given struct itself, there is no concept of a class Java/C++ class in Go. So, using an interface and a struct, anyone familiar with object-oriented design principles can use them in Go as well.

Using an interface

Once a type implements an interface, you can use that type anywhere the interface is expected. This allows for writing functions that operate on different types as long as they implement the required interface.

In the following example, the **Circle** object is being used where **Shape** type is expected:

```

1. func printShapeDetails(s Shape) {
2.     fmt.Printf("Area: %.2f, Perimeter: %.2f\n",
3.         s.Area(), s.Perimeter())
4. }
5.
6. func main() {
7.     c := Circle{radius: 5}

```

```
8. // The function works with any type that implements the
9. // two functions in Shape interface
10. printShapeDetails(c)
11. }
```

Empty interface

The empty interface (**interface{}**) is a special interface that does not specify any methods. It can hold values of any type. It is useful for writing functions that accept any type of value.

So, any value type can be passed in as a parameter to the **printValue** function, as shown:

```
1. func printValue(value interface{}) {
2.     fmt.Println("Value:", value)
3. }
4.
5. func main() {
6.     printValue(42)    // Prints integer
7.     printValue("Hello") // Prints string
8.     printValue(3.14) // Prints float
9. }
```

Type assertion

Type assertion allows you to retrieve the underlying concrete type from an interface. Use the syntax **value, ok := interfaceVariable.(Type)** to perform a safe type assertion.

This sample code checks if the type of variable **x** is an integer or not:

```
1. func main() {
2.     var x interface{} = 42
3.     if value, ok := x.(int); ok {
4.         fmt.Println("The value is an integer:", value)
```



```

5.     } else {
6.         fmt.Println("The value is not an integer")
7.     }
8. }

```

Type switch

A type switch allows you to perform different operations based on the underlying type of an interface value. Use the syntax **switch v := interfaceVariable.(type)** to perform a type switch.

The type of the passed in interface is dynamically derived below. Then the corresponding action (print in this case) can be performed based on that type, as shown:

```

1. func printType(value interface{}) {
2.     switch v := value.(type) {
3.     case int:
4.         fmt.Println("This is an integer:", v)
5.     case string:
6.         fmt.Println("This is a string:", v)
7.     default:
8.         fmt.Println("Unknown type, expected integer or string")
9.     }
10. }
11.
12. func main() {
13.     printType(42)           // Output: This is an integer: 42
14.     printType("Hello")     // Output: This is a string: Hello
15.     printType(3.14)        // Output: Unknown type
16. }

```

Go interfaces provide a way to design flexible and reusable functions and code that can work with different types as long as they satisfy the method

signatures defined in the interface. This helps in writing modular and decoupled code, making your programs easier to maintain and extend.

Unit testing

Unit testing might be incorrectly considered optional as a trade-off with getting software deployed. This would, in fact, have a high probability of causing failures and might require reverting changes, adding operational overhead.

Hence, the first line of defense is unit testing the code that is in some function using some data structures, as discussed in earlier sections. Test functions should be treated as first-class functions and given the same priority in coding standards as code functions, including testing various normal paths as well as error cases of production code. Here are some standard ways to unit test in Go.

In Go, unit tests are typically placed in separate test files with the naming pattern of `<code_file_name>_test.go` and are usually within the same package as the code being tested. These test files can use the natively provided **testing** package to assert that the code behaves as expected.

Since testing (third-party) dependencies are not always possible or needed, mocks are usually preferred. Mocks are objects that simulate the behavior of real objects, allowing you to isolate the code being tested from its dependencies. By using mocks, you can focus on testing the functionality of a particular function without external dependencies affecting the test results.

Basic tests

Tests use functions with names that start with **Test** and take a single argument of type `*testing.T`. The following is an example of how to test a function **Add** that adds two integers:

```
1. package main
2.
3. import «testing»
4.
5. func TestAdd(t *testing.T) {
```

```

6.  result := Add(2, 3)
7.  if result != 5 {
8.      t.Errorf("Expected 5, but got %d", result)
9.  }
10. }

```

Triggering **go test** from the command line runs all the tests when run at the top-level directory of the project. One can also run the pattern **go test go test <package_name> -run <test_name>** for a single test.

Note: That in IDE (ex., IntelliJ), one can run a single unit test to save time. Use the testing package to assert conditions within tests using methods such as `t.Errorf()`, `t.Fatalf()`, and `t.Logf()`.

Instead of testing only a single input/output value, one can also add in table-driven tests, where test cases are organized in a table (slice of structs) or as maps of inputs and outputs that can be looped over. This allows you to test multiple scenarios within a single test function.

The following is a table test performing addition on first two elements of each row and confirming the result is the third element, in a for loop over all rows of the table:

```

1. func TestAdd(t *testing.T) {
2.     tests := []struct {a, b, want int}{
3.         {2, 3, 5},
4.         {4, 5, 9},
5.         {10, 20, 30},
6.     }
7.
8.     for _, tt := range tests {
9.         got := Add(tt.a, tt.b)
10.        if got != tt.want {
11.            t.Errorf("Add(%d, %d) = %d; want %d",
12.                tt.a, tt.b, got, tt.want)
13.        }

```

```
14.     }
15. }
16. }
```

Mock based testing

Mocks are essential for isolating the code being tested from external dependencies, such as databases, APIs, or other services. Manually creating a mock of dependencies by implementing the interface and providing methods that simulate the behavior of the real object might be cumbersome and need regular updates, so it would not be the best use of time for testing the current in-progress feature logic.

Hence using third-party libraries such as **gomock** or **testify/mock** to generate mocks based on interfaces is the standard way to test these downstream functions. These libraries provide tools to create mocks, set up output or error expectations, and verify interactions:

- First install **gomock**:
 - `go install github.com/golang/mock/mockgen@latest`
- Create a mock from an interface:

Let us use a sample interface as below in **mock_service.go**

```
1. package main
2.
3. type MyService interface {
4.     DoSomething() (int, error)
5. }
6.
7. type MockService struct{}
8.
9. func (ms *MockService) DoSomething() (int, error) {
10.     return 45, nil
11. }
12.
```

```
13. func main() {
```

```
14. }
```

Run the following to create the mock for this interface

```
1. mockgen -source=mock_service.go -  
   destination=mock_test_gen.go -package=main . MyService
```

- In the **test** file, use the mock generated in **mock_test_gen.go**, as shown below:

```
1. package main
```

```
2.
```

```
3. import (
```

```
4.     "testing"
```

```
5.     "github.com/golang/mock/gomock"
```

```
6. )
```

```
7.
```

```
8. func TestMyFunction(t *testing.T) {
```

```
9.     ctrl := gomock.NewController(t)
```

```
10.    defer ctrl.Finish()
```

```
11.
```

```
12.    mockService := NewMockMyService(ctrl)
```

```
13.    expected := 44
```

```
14.    mockService.EXPECT().DoSomething().Return(expected, nil)
```

```
15.
```

```
16.    result, err := myFunction(mockService)
```

```
17.    if err != nil {
```

```
18.        t.Fatalf("Unexpected error: %v", err)
```

```
19.    }
```

```
20.    if result != 44 {
```

```
21.        t.Errorf("Expected %d, but got %d", expected, result)
```

```
22.    }
```

```
23. }
```

By combining unit testing and mocking, you can write tests that effectively verify the behavior of your code while maintaining separation from external dependencies. This approach leads to more robust and maintainable software.

Concurrency

Go has robust concurrency support built into the language. Concurrency is one of Go's standout features and is made possible through goroutines, channels, and the **sync** package. These tools enable developers to write concurrent and parallel programs efficiently while providing shared data safety.

Goroutines

A goroutine is a lightweight, independently executing function or method. It allows you to run multiple functions simultaneously without the need to manually create OS level threads.

Every go routine is of the format **go <function call>**. The following is a function called **printMessage**, which is called an asynchronous goroutine from the main thread and prints other messages inline synchronously:

```
1. package main
2.
3. import (
4.     "fmt"
5.     "time"
6. )
7.
8. func main() {
9.     // Launch a new goroutine
10.    go printMessage("Hello from goroutine!")
11.
12.    fmt.Println("Hello from main!")
13.}
```

```
14. // Launch another goroutine
15. go printMessage("Hello from second goroutine!")
16.
17. // Wait for the goroutines to finish
18. time.Sleep(1 * time.Second)
19. }
20.
21. func printMessage(message string) {
22.     fmt.Println(message)
23. }
```

Note: The 1-second sleep is a random time, and there is no guarantee that either of the Go routines will complete it in that time, nor is there a guarantee on the order of the print output. In a future section, we will see how it can be guaranteed using the Golang native sync library.

Channels

Channels are used for communication between goroutines. They provide a way for goroutines to send and receive data. Channels are strongly typed, so a channel created for a specific data type can only transfer data of that type. The following is channel for sending and receiving integer types. A Go routine sends the value, and the main thread receives that value into a local variable:

```
1. package main
2.
3. import "fmt"
4.
5. func main() {
6.     // Create a channel for integers
7.     ch := make(chan int)
8.
9.     // Launch a goroutine that sends a value to the channel
10.    go func() {
```

```
11.     ch <- 44 // Send a value to the channel
12. }()
13.
14. // Receive the value from the channel
15. value := <-ch
16. fmt.Println("Received value:", value)
17. }
```

Buffered channels

Channels can be buffered to store multiple values. When a channel is buffered, goroutines sending and receiving data can operate asynchronously, reducing the chance of blocking. Compared to the non-buffered mode above, this mode is preferred when the sender and receiver can be loosely coupled and do not need strict ordering for each synchronized send over the channel.

The following example shows multiple values can be sent on to a channel with chosen capacity. For the values sent, they can be stored in different receive variables:

```
1. package main
2.
3. import "fmt"
4.
5. func main() {
6.     // Create a buffered channel with capacity 2
7.     ch := make(chan int, 2)
8.
9.     // Send values to the channel
10.    ch <- 11
11.    ch <- 22
12.
13.    // Receive values from the channel
14.    value1 := <-ch
```



```
15. value2 := <-ch
16.
17. fmt.Println("Received values:", value1, value2)
18. }
```

Select statement

The select statement allows you to wait on multiple channel operations simultaneously. It is useful for handling multiple channels or timeouts.

The following sample code shows two channels being filled from two go routines, but receiver is waiting for either using the select option as can be extrapolated. One can choose to handle only one channel value in such cases:

```
1. package main
2.
3. import (
4.     "fmt"
5.     "time"
6. )
7.
8. func main() {
9.     ch1 := make(chan string)
10.    ch2 := make(chan string)
11.
12.    go func() {
13.        time.Sleep(2 * time.Second)
14.        ch1 <- "Channel 1"
15.    }()
16.
17.    go func() {
18.        time.Sleep(1 * time.Second)
```

```

19.     ch2 <- "Channel 2"
20. }()
21.
22. select {
23.     case msg := <-ch1:
24.         fmt.Println("Received from channel 1:", msg)
25.     case msg := <-ch2:
26.         fmt.Println("Received from channel 2:", msg)
27. }
28. }

```

In this case, since channel 2 is filled sooner (due to smaller sleep time), that path of the **select** statement will be exercised.

sync package

Go provides the **sync** package for synchronization primitives like mutexes, wait groups, and condition variables. These provide an easy way to create and wait for sub-routines to complete and guarantee correctness for the expected behavior across parallel executions. It is essential for writing concurrent Go programs that involve multiple goroutines working on shared data or resources.

Let us take a look at the key synchronization primitives provided by the **sync** package and their usage.

Mutex/sync.Mutex

A **mutex** provides mutual exclusion across threads/goroutines when accessing shared resources. It allows only one goroutine at a time to execute a critical section of code, such as accessing shared data structures. One can use **Lock()** to acquire the mutex and **Unlock()** to release it.

The shared variable **count** is being accessed in three different Go routine calls below, and the **increment** function uses a mutex to lock the entry for access. This ensures each call exclusively uses the value after the previous

parallel call is completed, and this ensures the value of **count** is exactly three at the end of the program. Refer to the following:

```
1. package main
2.
3. import (
4.     "fmt"
5.     "sync"
6. )
7.
8. var count int
9. var mu sync.Mutex
10.
11. func increment() {
12.     mu.Lock()
13.     defer mu.Unlock()
14.     count++
15. }
16.
17. func main() {
18.     var wg sync.WaitGroup
19.     wg.Add(3)
20.
21.     // Start three goroutines
22.     go func() {
23.         increment()
24.         wg.Done()
25.     }()
26.     go func() {
27.         increment()
28.         wg.Done()
29.     }()
```

```

30. go func() {
31.     increment()
32.     wg.Done()
33. }()
34.
35. wg.Wait()
36. fmt.Println("Final count:", count) // Should be 3.
37. }

```

Note: In the function `increment` we use the keyword `defer` with the `Unlock` call. This is a clean, in-built option to declare an action only once at the end of function completion. Even if the function has multiple completion points (even on error returns), the `defer` will execute and cleanup, thus reducing orphaned entities holding up resources (lock in this case).

Wait group/sync.WaitGroup

A **wait group** is used to wait for a collection of goroutines to finish execution. Use **Add(n)** to add **n** to the wait group, **Done()** to indicate a goroutine has finished its work, and **Wait()** to block until all goroutines have been completed.

The following code snippet uses three worker goroutines. The wait group count is added for each worker creation, and when each worker is done, it notifies the worker group logic that it is completed. The **wg.Wait** function in the main thread exits when the initially added count matches the done count (three in this case):

```

1. package main
2.
3. import (
4.     "fmt"
5.     "sync"
6. )
7.
8. func worker(id int, wg *sync.WaitGroup) {
9.     defer wg.Done()

```

```

10.  fmt.Printf("Worker %d starting\n", id)
11.  // Simulate work
12.  fmt.Printf("Worker %d done\n", id)
13. }
14.
15. func main() {
16.     var wg sync.WaitGroup
17.
18.     // Launch three goroutines
19.     for i := 1; i <= 3; i++ {
20.         wg.Add(1)
21.         go worker(i, &wg)
22.     }
23.
24.     wg.Wait() // Waits for all workers to finish
25.     fmt.Println("All worker goroutines completed»)
26. }

```

RWMutex /sync.RWMutex

An **RWMutex** is a reader-writer mutex that provides a way to safely coordinate access to shared data between readers and writers. Use **RLock()** and **RUnlock()** for reading access, and **Lock()** and **Unlock()** for writing access. RWMutex provides better throughput for reads compared to using only a Mutex in all access cases.

This provides all readers with access to the data at the same time but mutually excludes writes at the same time, thus improving read throughput and performance for readers. The **readData** function can progress even when the write function is running, so readers can complete it faster. This helps in the case of applications that are read-heavy, like cached reads. For example, 80% of accesses are reads, and 20% are writes. In the following code, we have show sample reader and writer go routines using RWMutex:

```

1. package main

```

```
2.
3. import (
4.     "fmt"
5.     "sync"
6. )
7.
8. var data int
9. var rwMutex sync.RWMutex
10.
11. func readData() {
12.     rwMutex.RLock()
13.     defer rwMutex.RUnlock()
14.     fmt.Println("Read data:", data)
15. }
16.
17. func writeData(newData int) {
18.     rwMutex.Lock()
19.     defer rwMutex.Unlock()
20.     data = newData
21.     fmt.Println("Wrote data:", data)
22. }
23.
24. func main() {
25.     var wg sync.WaitGroup
26.     wg.Add(2)
27.
28.     go func() {
29.         writeData(44)
30.         wg.Done()
31.     }()
32.
```

```

33. go func() {
34.     readData()
35.     wg.Done()
36. }()
37.
38. wg.Wait()
39. }

```

Cond/sync.Cond

A **cond** variable is a condition variable that can be used for signaling between goroutines. Use **NewCond()** to create a new cond with an associated locker (e.g., a mutex or RWMutex). Use **Wait()** to wait for a condition and **Signal()** or **Broadcast()** to notify one or all waiting goroutines.

In this sample code, the condition to wait is while the **counter** value is **0** in **waitForIncrement** Go routine thread. When the increment is completed in the other Go routine, that will signal a single waiter, which wakes up the first thread to make progress:

```

1. package main
2.
3. import (
4.     "fmt"
5.     "time"
6.     "sync"
7. )
8.
9. var cond = sync.NewCond(&sync.Mutex{})
10. var counter int
11. var wg sync.WaitGroup
12.
13. func increment() {
14.     defer wg.Done()

```

```

15. cond.L.Lock()
16. defer cond.L.Unlock()
17. counter++
18. fmt.Println("Incremented counter:", counter)
19. cond.Signal() // Signal one waiting goroutine
20. }
21.
22. func waitForIncrement() {
23.     defer wg.Done()
24.     cond.L.Lock()
25.     defer cond.L.Unlock()
26.     for counter == 0 {
27.         cond.Wait() // Wait for counter to be incremented
28.     }
29.     fmt.Println("Counter is now:", counter)
30. }
31.
32. func main() {
33.     wg.Add(1)
34.     go increment()
35.     wg.Add(1)
36.     go waitForIncrement()
37.
38.     // Allow for goroutines to complete
39.     wg.Wait()
40. }

```

Even though **increment** is called before in main, it will wait for the signal after the **counter** is incremented in the other go routine.

The **sync** package provides the foundational synchronization primitives needed to write safe and efficient concurrent programs in Go. By using

these tools, you can control access to shared data and resources between goroutines and avoid race conditions and other concurrency issues.

All these tools allow you to effectively work with concurrency in Go, enabling you to handle multiple tasks simultaneously and create efficient and responsive services which can handle large scale user workloads.

Object relational mapping

One of the major aspects of the backend system is the ability to handle data via access to database technologies. Go provides various libraries for interacting with such databases, including **object relational mapping (ORM)** libraries. ORMs simplify the process of mapping Go structs to database tables and allow developers to work with data using Go data types instead of writing raw SQL queries. In this section, we will discuss some of the popular ORM libraries written in Go.

GORM

GORM is the most widely used ORM library in Go. It supports most popular databases, such as MySQL, PostgreSQL, SQLite, and SQL Server, and provides a high-level API for working with databases using Go structs.

The following example shows how a connection is set up to a database programmatically and how **Create, Read, Update, and Delete (CRUD)** operations can be performed on the data in it:

```
1. package main
2.
3. import (
4.     "fmt"
5.     "gorm.io/driver/sqlite"
6.     "gorm.io/gorm"
7. )
8.
9. // Define a Go struct that maps to a database table
10. type User struct {
```

```
11. ID uint `gorm:"primaryKey"`
12. Name string `gorm:"size:100"`
13. Age int
14. }
15.
16. func main() {
17.     // Connect to a SQLite database (or any other
18.     // supported database)
19.     db, err := gorm.Open(sqlite.Open("test.db"), &gorm.Config{})
20.     if err != nil {
21.         fmt.Println("Error connecting to the database:", err)
22.         return
23.     }
24.
25.     // Auto migrate the User struct to be used for creating a
26.     // corresponding database table.
27.     db.AutoMigrate(&User{})
28.
29.     // Create a new user
30.     user := User{Name: "Alice", Age: 30}
31.     db.Create(&user)
32.
33.     // Query for the user
34.     var queriedUser User
35.     db.First(&queriedUser, user.ID)
36.     fmt.Println("Queried user:", queriedUser)
37.
38.     // Update the user
39.     db.Model(&queriedUser).Update("Age", 31)
40.
41.     // Delete the user
```

```
42. db.Delete(&queriedUser)
43. }
```

SQLBoiler

SQLBoiler is an ORM that provides code generation for connecting to a database. It reads your database schema and generates Go structs, allowing you to write queries using Go code.

SQLBoiler requires some setup steps and code generation, so here we have briefly described the process:

1. Install SQLBoiler and configure it to connect with your database.
2. Generate the Go structs and ORM code using SQLBoiler.
3. Use the generated code to interact with your database.

Ent

Ent is another modern ORM that allows you to define your data schema using Go code and provides a code generation tool to create Go structs, methods, and queries based on the schema. Here is the overview of the steps:

1. Install Ent and set up your project to use it.
2. Define your data schema using Go code (e.g., defining fields and edges).
3. Use the Ent code generator to generate the necessary Go code.
4. Use the generated Go code to interact with your database.

These are just a few examples of popular Go ORM libraries. Each library has its own advantages and disadvantages, so you can choose the one that best fits your needs and coding style. Most of these libraries also support complex querying, eager loading, and relationships between entities, making them powerful tools for working with databases in Go.

While ORMs can improve productivity, especially in smaller projects, their downsides become more apparent in larger, performance-critical, or highly complex applications. Let us also summarize some of the cons of using ORMs in general:

- **Performance overhead:** ORMs often add significant overhead compared to raw SQL queries, as they translate code into SQL, execute the query, and map the results back to objects. This can lead to slower database interactions and loss of scalability, especially for complex queries in high-traffic or performance-critical applications.
- **Loss of SQL power:** SQL is an evolved and powerful language designed for complex data querying, aggregation, and manipulation. ORMs usually support only a subset of SQL's capabilities, making it difficult to execute advanced queries without falling back to raw SQL. If one needs transaction support, custom joins, advanced filtering, window functions, or complex subqueries, there may be a need to write raw SQL.
- **Increased maintenance costs:** ORMs abstract away the SQL layer, but this abstraction can sometimes increase the complexity of debugging and troubleshooting issues. It can be challenging to understand which part of the ORM maps to which SQL queries that are being generated. Identifying performance bottlenecks may require deeper knowledge of the ORM and its query generation process, causing increased learning curves.
- **ORM lock-in:** Using an ORM often leads to code that is tightly coupled to the ORM's framework and conventions, which can make it challenging to switch to a different ORM or revert to raw SQL. Migrating away from an ORM can be costly, as it may require rewriting a significant portion of the data access layer as well as hitting cross-database SQL compatibility.

Go modules

Most complex software engineering projects rely on standard libraries and dependencies that have already been published. Go modules is a dependency management system introduced in Go 1.11 that helps manage such dependencies in Go projects. It allows you to specify, download, and update external dependencies in a controlled manner.

This is the main option for reusing (public and permitted) functionality from other packages so that new projects can be quickly built on top of such

existing building blocks. Go modules also enable you to work with multiple versions of a package and maintain the reproducibility of builds.

Let us look at the key concepts of Go modules and define the concepts involved:

- **Modules:** A module is a collection of Go packages stored in a file tree. It is defined by a **go.mod** file that contains information about the module's dependencies and version.
- **go.mod file:** This file is created in the root directory of your project and contains metadata about the module, such as its name and dependencies.
- **go.sum file:** This file contains checksums of the dependencies used in the project to ensure integrity and reproducibility of builds. For any new or updated dependency, the checksum for the downloaded module is updated in the **go.sum** file. This checksum is used to cross check the validity of that module version for each build.

Using Go modules

Let us discuss the steps to create and use the Go modules that are needed for a project:

- **Initialize a module:** To initialize a module in a Go project, navigate to the project directory and run the shell command:
1. `go mod init <module-name>`
<module-name> is the name of your module, usually a URL or path.
- **Add dependencies:** To add a dependency on another package, you can use the **go get** shell command:
1. `go get <dependency>`
<dependency> is the module path (e.g., **github.com/some/package**). Similarly, if your project is in a public GitHub repository, others can pull it in as a dependency using the package's name (For example, **package name** in the code snippets in earlier sections of this chapter).
- **Update dependencies:** To update a specific dependency, use the **go get** command with the package and the desired version:

```
1. go get <dependency>@<version>
```

For example:

```
1. go get github.com/some/package@v1.2.3
```

After adding the dependency, you will notice that the **go.mod** and **go.sum** files are updated with the information about the package.

- **Pinning dependencies:** One can specify the following in the **go.mod** file to pin a version of a desired package (to allow no indirect dependencies overwriting for example):

```
1. replace github.com/ext/pkg => github.com/ext/pkg v3.4.5
```

- **Removing dependencies:** To remove a dependency, you can use:

```
1. go mod tidy
```

The **go mod tidy** command helps clean up your **go.mod** file and **go.sum** file by removing unused dependencies and adding any missing ones.

- **Using the dependency:** In your Go code, you can import and use the added package as follows:

```
1. package main
2.
3. import "github.com/some/package@v1.2.3"
4.
5. func main() {
6.     package.printInfo("Hello to new package!")
7. }
```

- **Building and running with modules:** To build or run a Go project using modules, just use the regular Go commands (**go build**, **go run**) in the project's root directory. This will produce the actual binary that will be used to run your business logic after deployment.
- **Vendor directory:** If you want to use a vendor directory to manage dependencies locally when not connected to the internet, you can enable it using:

```
1. go mod vendor
```

By following all the above steps, you can effectively manage dependencies in your Go projects using Go modules.

Software delivery

In Go, software delivery encompasses a range of practices and tools that facilitate the build, testing, and deployment of Go applications. The goal is to ensure the application is delivered efficiently and with high quality.

Following are the essential steps and aspects to consider in the process of Go software delivery:

- **Building Go applications:** The first step involves building the binaries after ensuring the testing and quality are in place.
- **Go build:** Use the **go build** command to compile your Go code into an executable. This command takes care of downloading dependencies, building the application, and linking it to produce the final executable binary file:

```
1. go build -o myapp main.go
```

- **Cross-compilation:** Go supports cross-compilation, allowing you to build executables for different operating platforms and architectures. This is useful in the case when the developer or build environment is using a different OS compared to the deployment servers that will run the software. Use the **GOOS** and **GOARCH** environment variables to specify the target server platform and architecture:

```
1. GOOS=linux GOARCH=amd64 go build -o myapp main.go
```

- **Testing Go applications:** There are different native tools for testing and ensuring a good quality of coverage for the software being rolled out.
- **Go test:** Use the **go test** command to run all the unit tests for your Go packages. As discussed before, this command discovers and runs tests based on the test files (named in ***_test.go** format) in your project to ensure all clear on core unit testing front on the release candidate version before deployment.

```
1. go test ./...
```

- **Coverage:** Go provides a built-in coverage tool to measure test coverage. Use the **-cover** flag with **go test** to generate a coverage report. Having a high threshold value for the percentage of lines (like 95+%) covered by unit testing is a good way of ensuring high quality as well as reducing chances of breaking existing functionalities:

```
1. go test -cover ./...
```

- **Code quality and static analysis:** There are quite a few existing native and third-party tools that help maintain code quality in terms of ensuring basic aspects of formatting along with incorrect variable initialization as well as overly complicated, unreadable or long functions.
- **Golint:** Use built-in **golint** to check your code for style guide violations and coding best practices:

```
1. golint ./...
```

One of the best guides to follow Golang industry standard is the *Google style guide*. Another option is to follow https://go.dev/doc/effective_go to read up the options for your code concerns.

- **Go vet:** Use language provided **go vet** to analyze your code for potential errors, such as incorrect formatting, misuse of variables, and type mismatches.

```
1. go vet ./...
```

- **SonarQube:** SonarQube is an open-core product for static code analysis, with additional features offered in commercial editions. It can be integrated with **continuous integration/continuous development (CI/CD)** pipelines and offers online reports on duplicated code, coding standards, unit tests, code coverage, code complexity, comments, bugs, and security recommendations.
- **Integration with IDEs and editors:** Most modern IDEs and text editors (e.g., Visual Studio Code, IntelliJ, GoLand, Atom, Sublime Text) provide built-in static code analysis support or plugins for linting Go code. These integrations can provide real-time feedback on your code as you write, helping you catch issues early, even before

compiling the code.

Version management

Software versioning is a practice used to assign unique identifiers to different versions of software to help track and manage changes over time. Independent of the programming language or deployment servers being used, the concepts of versioning and industry standard release management strategies ensure smooth deployment processes. Versioning can be achieved by tagging a version number to the commits in GitHub and using a release candidate with the desired set of features to create a fully tested new release version each time for deployment.

Semantic versioning (also known as **semvar**) is a widely used system for versioning software. It consists of three parts assigned to a version number: major, minor, and patch fields with dot separating each one. Here is the semantics of each field:

- **Major:** Increment the major version when one makes any incompatible public API changes.
- **Minor:** Increment the minor version when one adds brand new functionality or does changes in a backward-compatible manner.
- **Patch:** Increment the patch version when you make backward-compatible bug fixes.

For example: Version 2.3.4 might represent the fourth patch release of the third minor release of the second major version.

Benefits of versioning

These are some of the benefits of tracking release versions in general:

- Versioning helps track changes in the codebase over time, making it easier to understand the history of the software.
- Proper versioning helps manage dependencies between different components or projects.
- By following semantic versioning, you can ensure backward compatibility and prevent breaking changes.
- Version numbers can help communicate the maturity, status and

stability of software releases to end users.

By following recommended versioning practices, software engineers can effectively manage and track changes in software projects, improve collaboration with users, and maintain a consistent and reliable source of truth for the service's history.

Continuous integration and continuous deployment

Since automation is the game's name for efficiency, integrating your Go applications with CI/CD pipelines to automate building, testing, and deploying your code is highly recommended. These can be configured to automatically take version increments and release branch management into their setup. Popular CI/CD platforms include GitHub Actions, GitLab CI/CD, Jenkins, and CircleCI. Set up automated tests to run whenever code is committed or pushed to your repository. These tools also allow setting up linters and code coverage tools to ensure code quality for each feature or fix commit.

Containerization

Once the code is deemed to be of good quality and passes all the tests, the next step is to prepare the current snapshot of the software for the servers. Using Docker to containerize your Go applications is a standard way of making them portable and easier to deploy. Create a Docker file to specify how to build and run your application, as shown:

1. FROM golang:latest as builder
2. WORKDIR /app
3. COPY ..
4. RUN go build -o myapp
- 5.
6. FROM alpine:latest
7. WORKDIR /app
8. COPY --from=builder /app/myapp /app/
9. CMD ["/app/myapp"]

The above file creates an Alpine Linux-based image, copies your binary, and sets up the command to run it once binaries are deployed to the servers.

Deployment

Once the application binaries and dependencies are containerized, those Go applications can be deployed as standalone executables or using containers in Kubernetes clusters, as described above. The following are the standard deployment options for your service:

- **Infrastructure as a service (IaaS):** Deploy your Go service to **virtual machines (VMs)** on cloud platforms. Using cloud-provided tools, manage the VMs' lifecycle yourself.
- **Platform as a service (PaaS):** Deploy your application using a PaaS solution (e.g., *Google App Engine*, *Heroku*) that manages the underlying infrastructure.
- **Container orchestration:** Deploy your Go service using Kubernetes or other container orchestration platforms for better management and scalability. We will delve into this more in [Chapter 9, Containerization](#).

Push Docker images

If you are using Docker, push your images to a container registry (e.g., *Docker Hub*, *AWS ECR*, *Google Container Registry*) for deployment:

1. `docker build -t myapp:latest .`
2. `docker tag myapp:latest myregistry.com/myapp:latest`
3. `docker push myregistry.com/myapp:latest`

Launching the application

Launch the application in the cloud, depending on your cloud provider and strategy. For example, use `kubectl` to deploy to a Kubernetes cluster, or use the cloud provider's CLI or web console UI to deploy the service. We will investigate this and related tools in [Chapter 9, Containerization](#).

Monitoring and logging

Integrating monitoring and logging solutions to keep track of your application's performance and health will help figure out issues due to increased load or some other metrics of interest.

Common tools for monitoring service metrics include Prometheus and Grafana. One can also set up monitoring using cloud-native tools like *AWS CloudWatch* and *Google Cloud Monitoring*, which provide similar functionality.

Implementing centralized logging solutions like **Elasticsearch, Logstash, and Kibana (ELK)** stack or cloud-specific solutions to aggregate and analyze logs can help track any exceptions with the code error handling that must be fixed.

These practices and tools provide a strong foundation for efficient software delivery in Go. In summary, deploying your Go service to the cloud or on-premises involves building, configuring, running, and monitoring your application. Choose the deployment strategy that best suits your use case and leverage cloud-native tools to ensure efficient, scalable, and secure deployment. By following these strategies, you can achieve fast, reliable, and iterative development and deployment cycles for your Go applications.

Conclusion

In this chapter, we examined the key concepts of Golang and how they can be used effectively to build highly parallelizable and scalable software. The software engineering team also has the responsibility of thorough unit testing, which goes hand in hand with building such projects.

Operationalizing these changes onto production servers requires a disciplined approach to rolling out and tracking them. Most of these concepts are programming language agnostic and provide an overview of coordinating such events on a regular cadence to achieve automation for regular feature delivery for user needs.

In the next chapter, we are going to learn about higher-level web frameworks that are available in Go.

CHAPTER 3

Web Frameworks

Introduction

Building on the previous chapter on Golang details, this chapter adds vital competencies to designing and implementing a web server. This includes various aspects like middleware development, routing primitives, inter-platform support, and integrated error checking on the server using a range of Go Frameworks and libraries.

If I have seen further, it is by standing on the shoulders of giants.

—Sir Isaac Newton

Reusing existing components to build faster and further in software engineering is a well-known paradigm. In the world of web development, there are many such frameworks that can be used. Django for Python, Rails for Ruby, Spring Boot for Java, and Play for Scala are some of the notable frameworks in those worlds.

For Golang, we will be discussing Gin, Echo, Beego, Revel, and FastHTTP frameworks along with the trade-offs among them and how they can help faster development as well as strengthen the server side to support the scalability and high-speed performance of business applications.

Structure

The chapter covers the following topics:

- Overview of web frameworks
- Gin
- Echo
- Beego
- Revel
- FastHTTP

Objectives

This chapter teaches the reader how to build web applications quickly using Golang by using frameworks that allow faster development iteration.

First, we will start off with an overview of why frameworks are preferred and what specific advantages they provide in the software development lifecycle. Then, the chapter will provide details on how to set them up along with sample code on how to use these frameworks inside your service.

By the end of this chapter, you will be able to understand the most commonly used web frameworks that are available and the pros and cons of each one.

Overview of web frameworks

First, we get an overview of why Golang web frameworks have become popular. They provide quicker and easier REST A creation options for web service development and using any web framework can be beneficial for a variety of reasons, as detailed below:

- **Faster development:** Web frameworks often provide pre-built components and tools that allow you to quickly create and launch applications. Features such as routing, middleware built-ins, request parsing, and response formatting are commonly included, saving you time during development.
- **Consistency and structure:** Frameworks provide a consistent structure for organizing your code, making your project more maintainable and

easier to understand for future enhancements. They often follow established design patterns and best practices, helping the developer to write cleaner and more organized code.

- **Built-in features:** Most web frameworks come with built-in features such as routing, templating, session management, and more. These features reduce the need for external libraries or custom implementations for common tasks in cross-service interactions. This allows server-side development to reduce the overhead of code bloat for non-business logic-related aspects.
- **Community aspect:** Popular web frameworks usually have active developer communities and extensive documentation, making it easier to find support, tutorials, and best practices. You can leverage the experiences of other developers and access a wealth of knowledge and resources. It also provides a platform for giving back to the community if any enhancements could help more users.
- **Middleware and extensibility:** Frameworks often provide middleware support, allowing you to add functionality such as authentication, logging, and error handling easily. They also offer extensibility through plugins, allowing you to add additional features and customize the framework as needed.
- **Performance and scalability:** Web frameworks are optimized for performance and often include support for efficient request handling and asynchronous processing. They can handle common performance-related tasks, such as caching and load balancing, which can help your application scale more effectively and provide better throughput and latencies.
- **Security:** Web frameworks often include built-in security features such as CSRF protection, input validation, and session management. These features help protect your application from common security vulnerabilities.
- **Cross-platform support:** Some frameworks support multiple platforms and environments, enabling you to build cross-platform applications more easily. This includes build on an **operating system**

(OS) familiar to the developer but getting the ability to deploy it seamlessly onto another OS such as MacOS, Windows or Linux distributions.

While web frameworks offer many benefits, there are some cases where you might opt not to use one. For example, for very simple or specific applications that prefer fine-grained control or have performance constraints, you may prefer to build your application without a framework. Additionally, using a framework introduces dependencies, so if you prefer a lighter, more customized solution, you may consider not using a framework. It might also add the need to track version dependency changes on third-party software bundles.

Ultimately, choosing a web framework should be based on your project's specific needs and development goals. When deciding, consider your desired deployment timeline, application complexity, and long-term maintenance costs.

In the next few sections, we will examine some of the most popular web frameworks and their varied functionality and approaches to helping you build faster and more securely.

Gin

The Gin web framework is one of the most popular and widely used web frameworks in the Go ecosystem. It is known for its speed, ease of use, and simplicity. In this section, we will provide an overview of the Gin framework, including its features and how to get started. First, some of the advantages of using this framework are listed below:

- **High performance:** Gin is known for its fast performance and efficient request handling. It uses HTTP router optimizations to achieve low latency and fast response times.
- **Simple API:** Gin offers a simple and intuitive API for building web applications and RESTful APIs. The framework is designed to be easy to learn and use.
- **Middleware support:** Gin supports the use of middleware for handling tasks such as authentication, logging, and request processing. You can

define middleware functions that can be applied globally or on a per-route basis.

- **JSON and XML handling:** Gin makes it easy to handle JSON and XML data in requests and responses. It provides built-in functions for parsing JSON and XML payloads and rendering responses in these formats.
- **Error handling:** The framework provides convenient ways to handle errors, including custom error responses. It supports exception handling and graceful recovery from panics.
- **Routing:** Gin offers flexible and expressive routing capabilities, allowing you to define routes for different HTTP methods and URL patterns. Routes can be grouped and organized into different routers for better modularity.
- **Template rendering:** Gin supports rendering HTML templates using popular templating engines such as HTML and Jade. It allows you to integrate templates into your web application easily.
- **WebSocket support:** Gin can work with directly web sockets, allowing you to create real-time web applications.
- **Documentation:** Gin has comprehensive documentation at <https://gin-gonic.com/docs/> and a large, active community, making it easy to find resources and support.

Getting started with Gin

Following are some of the sample steps to get setup with the Gin package and usage patterns:

- **Install:** To install Gin, you can use the Go package manager (**go get**):

1. `go get -u github.com/gin-gonic/gin`

Here is a simple example of how to create a web server using Gin:

1. `package main`

- 2.

3. `import (`

4. `"net/http"`

```

5.  "github.com/gin-gonic/gin"
6. )
7.
8. func main() {
9.     r := gin.Default()
10.    r.GET("/hello", func(c *gin.Context) {
11.        c.String(http.StatusOK, "Hello, Gin World!")
12.    })
13.    r.Run() // Listen and serve requests on 0.0.0.0:8080
14. }

```

In this example, we import the **Gin** package, create a new router, define a route **"/hello"** for your service with a handler function, and start the server on the default HTTP port **8080**.

- **Routing:** You can also define routes for different HTTP methods:

```

1. r.GET("/get", func(c *gin.Context) {
2.     c.String(http.StatusOK, "GET request")
3. })
4.
5. r.POST("/post", func(c *gin.Context) {
6.     c.String(http.StatusOK, "POST request")
7. })

```

- **Middleware:** Middleware functions, which are usually to perform certain actions for all endpoints, can be defined globally or on a per-route basis, as shown:

```

1. func loggingMiddleware(c *gin.Context) {
2.     // Do some logging here
3.     c.Next()
4. }
5.
6. r.Use(loggingMiddleware) // Apply middleware globally

```

7. `r.Run()`

- **Response rendering:** Gin can render responses in various formats, such as JSON, XML, HTML, or plain text. Refer to the following:

```
1. r.GET("/json", func(c *gin.Context) {  
2.     data := gin.H{"message": "Hello, World!",}  
3.     c.JSON(http.StatusOK, data)  
4. })  
5.  
6. r.GET("/html", func(c *gin.Context) {  
7.     c.HTML(http.StatusOK, "template.html",  
8.         gin.H{"title": "Hello, World!", })  
9. })
```

In this example, the first output is JSON format, and the second is html format rendering.

- **Template rendering:** Gin supports template rendering using common template engines. One can just load the template files from a known location into the context as follows:

```
1. r.LoadHTMLGlob("templates/*")
```

Gin is a great choice for building web applications and RESTful APIs in Go due to its simplicity, performance, and comprehensive features. It is suitable for a variety of projects but mostly for small to medium-scale applications. For readers with Python and Django backgrounds, the Gin framework would be most familiar with Go. To learn more about Gin, you can visit the gin documentation page, <https://github.com/gin-gonic/gin>, and explore its features in detail.

Echo

The Echo web framework is popular and powerful in the Go ecosystem. It is known for its simplicity, speed, minimalistic design, and better usability in building microservices. Here is an overview of the Echo framework, including its features and how to get started:

- **High performance:** Echo is optimized for performance and speed,

using efficient request handling and routing to provide low latency and fast response times.

- **Intuitive API:** Echo provides a straightforward and easy-to-use API for building web applications and RESTful APIs. The framework emphasizes simplicity and productivity.
- **Middleware support:** Echo supports using middleware for handling tasks such as authentication, logging, and request processing. Middleware functions can be applied globally, per group, or per route.
- **Routing:** Echo offers flexible and expressive routing capabilities. It allows you to define routes for different HTTP methods and URL patterns. It supports route grouping and parameterized routes.
- **JSON and XML handling:** Echo provides built-in support for parsing JSON and XML data in requests and responses. It makes it easy to handle and render JSON and XML data.
- **Error handling:** Echo offers convenient handling of errors, including custom error responses. You can define error handlers for different status codes.
- **Static file serving:** Echo can serve static files directly from a directory, making it easy to create web applications with static assets.
- **WebSocket support:** Echo supports WebSocket connections, enabling you to build real-time web applications. That provides bi-directional communication between the client and servers and allows for quick updates from the server based on client inputs.
- **Documentation:** Echo has extensive documentation and an active community, providing support and resources for developers.

Getting started with Echo

Following are some of the sample steps to get set with the Echo package and usage patterns:

- **Installation:** To install Echo, you can use the Go package manager (**go get**):

1. `go get -u github.com/labstack/echo/v4`

Here is a simple example of how to create a web server using Echo:

```
1. package main
2.
3. import (
4.     "github.com/labstack/echo/v4"
5.     "net/http"
6. )
7.
8. func main() {
9.     e := echo.New()
10.    e.GET("/hello", func(c echo.Context) error {
11.        return c.String(http.StatusOK, "Hello, World!")
12.    })
13.    e.Start(":8080") // Start the server on port 8080
14. }
```

- **Routing:** You can define routes for different HTTP methods just like with Gin:

```
1. e.GET("/get", func(c echo.Context) error {
2.     return c.String(http.StatusOK, "GET request")
3. })
4.
5. e.POST("/post", func(c echo.Context) error {
6.     return c.String(http.StatusOK, "POST request")
7. })
```

- **Middleware:** Middleware functions can be defined globally, per group, or per route:

```
1. func loggingMiddleware(next echo.HandlerFunc) echo.HandlerFunc {
2.     return func(c echo.Context) error {
3.         // Do some logging here
4.         return next(c)
```

```

5. }
6. }
7.
8. e.Use(loggingMiddleware) // Apply middleware globally

```

- **Response rendering:** Echo supports rendering responses in various formats such as JSON, XML, and plain text:

```

1. e.GET("/json", func(c echo.Context) error {
2.     data := map[string]interface{}{
3.         "message": "Hello, World!",
4.     }
5.     return c.JSON(http.StatusOK, data)
6. })
7.
8. e.GET("/xml", func(c echo.Context) error {
9.     data := map[string]interface{}{
10.        "message": "Hello, World!",
11.    }
12.    return c.XML(http.StatusOK, data)
13. })

```

- **Static file serving:** Echo can serve static files directly from a directory:

```

1. e.Static("/static", "path/to/static/files")

```

- **Error handling:** You can define custom error handlers for different status codes:

```

1. e.HTTPErrorHandler = func(err error, c echo.Context) {
2.     c.JSON(http.StatusInternalServerError,
3.         map[string]string{"error": err.Error()})
4. }

```

Echo is a great choice for building web applications and RESTful APIs in Go due to its simplicity, performance, and comprehensive features. It is suitable for a variety of projects, from small to large-scale applications. To

learn more about Echo, you can visit the official documentation at <https://echo.labstack.com> and explore its features in more detail.

Beego

Beego is a full-stack web application framework for the Go programming language. It provides a range of built-in features and tools that simplify web development, including an **object-relational mapping (ORM)** system, a router, a template engine, and other utilities such as caching, logging, and session management. This makes Beego a versatile choice for building robust and scalable web applications in Go. Here is an overview of Beego, including its features and how to get started:

- **Full-stack framework:** Beego offers a comprehensive set of features, including routing, ORM, templating, caching, session management, and more. This allows you to develop web applications using a single framework without relying heavily on external libraries.
- **Routing:** Beego provides flexible and expressive routing capabilities, allowing you to define routes for different HTTP methods and URL patterns. It supports RESTful routing, regular expression routing, and automatic route generation.
- **Built-in ORM:** Beego includes a built-in ORM system that provides an easy way to interact with databases using Go structs. The ORM supports various databases such as MySQL, PostgreSQL, SQLite, and others.
- **Template engine:** Beego supports templating using its built-in template engine, similar to the standard Go template engine. It also supports other templating engines, such as Jade (Pug) and others.
- **Error handling:** Beego provides a structured approach to error handling, including custom error pages and error codes.
- **Middleware:** Beego supports middleware functions that can be applied globally or per route. This allows you to easily handle common tasks such as authentication, logging, and request processing.
- **Caching:** The framework includes a caching system that supports various caching backends such as in-memory, file, Redis, and others.

This can help improve performance by caching data and reducing the load on databases and other resources.

- **Session management:** Beego provides built-in session management that supports various storage backends such as memory, file, Redis, and others. This makes it easy to manage user sessions and maintain state across requests.
- **Logging:** The framework includes a built-in logging system that supports various log levels and output destinations.
- **Documentation:** Beego has comprehensive documentation and an active community, providing support and resources for developers.

Getting started with Beego

Following are some of the sample steps to get set with the **Beego** package and usage patterns:

- **Installation:** To install Beego, you can use the Go package manager (**go get**):

```
1. go get -u github.com/beego/beego/v2
```

Here is a simple example of how to create a web server using Beego:

```
1. package main
2.
3. import (
4.     "github.com/beego/beego/v2/server/web"
5.     "github.com/beego/beego/v2/server/web/context"
6. )
7.
8. func main() {
9.     web.Get("/hello", func(ctx *context.Context) {
10.         ctx.WriteString("Hello, World!")
11.     })
12.     web.Run() // Start the server on default port 8080
13. }
```


- **Routing:** Beego allows you to define routes for different HTTP methods using functions such as **web.Get()**, **web.Post()**, etc. It also supports route parameters and regular expressions for more complex routes.
- **ORM:** The built-in ORM system allows you to define Go structs as database models and perform database operations using an intuitive API:

```

1. type User struct {
2.   ID int `orm:"auto"`
3.   Name string
4. }
5.
6. func main() {
7.   orm.RegisterModel(new(User))
8.   orm.RunSyncdb("default", false, true)
9. }

```

- **Template rendering:** Beego supports rendering responses using HTML templates:

```

1. web.Get("/template", func(ctx *context.Context) {
2.   ctx.Output.Body([]byte("<h1>Hello, World!</h1>"))
3. })

```

- **Error handling:** Beego offers custom error handling, allowing you to define error pages and responses for different status codes. For example, this code allows the creation of a special handler for a **404** error:

```

1. package main
2.
3. import (
4.   "net/http"
5.   "html/template"
6.   "github.com/beego/beego/v2/server/web"

```

```

7.
8. )
9.
10. func page_not_found(rw http.ResponseWriter, r *http.Request) {
11.     t, _ := template.ParseFiles(
12.         web.BConfig.WebConfig.ViewsPath + "/404.html")
13.     data := make(map[string]interface{})
14.     data["content"] = "page not found"
15.     t.Execute(rw, data)
16. }
17.
18. func main() {
19.     web.ErrorHandler("404", page_not_found)
20.     web.Router("/", &web.Controller{})
21.     web.Run()
22. }
23.

```

Beego is a comprehensive web framework for Go that provides a wide range of features and tools for building web applications and RESTful APIs. It is suitable for projects of various sizes and complexities, offering developers ease of use and productivity. To learn more about Beego, you can visit the official documentation site at <https://beego.me/docs/> and explore its features in more detail.

Revel

Revel is a full-stack web application framework for the Go programming language. It offers a variety of built-in features to simplify web development, including routing, templating, ORM, caching, and more. Revel aims to provide a structured and efficient approach to building web applications in Go. Here is an overview of Revel, including its features and how to get started:

- **Full-stack framework:** Revel provides a comprehensive set of

features, including routing, templating, ORM, caching, and more. This full-stack approach allows you to build web applications using a single framework without relying heavily on external libraries.

- **Routing:** Revel offers flexible routing capabilities that allow you to define routes for different HTTP methods and URL patterns. It supports RESTful routing and route parameters.
- **Built-in ORM:** Revel includes a built-in ORM system that provides an easy way to interact with databases using Go structs. The ORM supports various databases such as MySQL, PostgreSQL, SQLite, and more.
- **Template engine:** Revel provides built-in support for template rendering using the standard Go templating system. Templates can be organized into different directories and reused across the application.
- **Error handling:** Revel includes a structured approach to error handling, allowing you to define custom error responses and pages. It offers an easy way to handle different HTTP status codes.
- **Session management:** Revel includes built-in support for session management, allowing you to maintain user sessions across requests. It supports various session storage backends.
- **Caching:** The framework includes built-in support for caching, which can improve application performance. It supports various caching backends, such as in-memory and file-based caching.
- **Logging:** Revel provides a built-in logging system with support for different log levels and output destinations. This helps you monitor application behavior and diagnose issues.
- **Testing support:** Revel includes built-in support for testing your application, making it easier to write and run tests.
- **Documentation:** Revel has detailed documentation and a supportive community, providing resources and support for developers.

Getting started with Revel

Following are some of the sample steps to get set with the Revel package and usage patterns:

1. **Installation:** To install Revel, you can use the Go package manager (**go get**):

1. `go get -u github.com/revel/revel`
2. `go get -u github.com/revel/cmd/revel`

2. **Creating a new Revel application:** You can create a new Revel application using the Revel command-line tool:

1. `revel new myapp`
2. `cd myapp`

3. **Running a Revel application:** To run your Revel application, use the **revel run** command:

1. `revel run myapp`

4. **Routing:** Revel provides a **conf/routes** file where you can define your application's routes:

1. `GET / App.Index`
2. `GET /hello App.Hello`
3. `POST /update App.Update`

5. **Controllers:** Controllers in Revel are defined using structs with methods representing different actions:

1. package controllers
- 2.
3. `import (`
4. `"github.com/revel/revel"`
5. `)`
- 6.
7. `type App struct {`
8. `*revel.Controller`
9. `}`
- 10.
11. `func (c App) Index() revel.Result {`
12. `return c.Render()`
13. `}`

14.

```
15. func (c App) Hello() revel.Result {  
16.     return c.RenderText("Hello, World!")  
17. }
```

6. **Template rendering:** Revel supports rendering responses using HTML templates in the **views** directory:

```
1. views/  
2.   └─ App  
3.     └─ index.html
```

7. **Error handling:** Revel provides a **conf/errors** directory where you can define custom error pages for different HTTP status codes.

Revel is a full-stack web framework for Go that offers a variety of features for building robust web applications and RESTful APIs. It emphasizes structure, testing support, and productivity. Revel is suitable for projects that require a comprehensive framework with built-in functionality and not many configuration requirements. To learn more about Revel, visit the official website at <https://revel.github.io/> and explore its documentation and community.

FastHTTP

FastHTTP is an efficient and high-performance HTTP server framework for Go. It is known for its speed and low memory usage. FastHTTP provides a low-level API for building HTTP servers and is ideal for developers looking for high-performance and lightweight solutions.

Although it is not as full-featured as other web frameworks like Gin or Echo, it is often used as a foundation for other high-performance frameworks such as Revel. Here is an overview of FastHTTP, including its features and how to get started:

- **High performance:** FastHTTP is optimized for speed and performance, with efficient request handling and low memory overhead. It is one of the fastest HTTP frameworks available for Go.
- **Low-level API:** FastHTTP provides a low-level API that gives

developers fine-grained control over request and response handling. This low-level access allows for greater flexibility and optimization opportunities.

- **Request handling:** FastHTTP offers efficient request handling, supporting large requests and multipart form data. It provides direct access to request and response buffers for fast processing.
- **Asynchronous processing:** The framework supports asynchronous request processing, efficiently handling concurrent requests.
- **WebSocket support:** FastHTTP includes built-in support for WebSockets, which allows you to build real-time web applications.
- **Security:** FastHTTP includes built-in support for secure request handling and protections against common security vulnerabilities.
- **File serving:** The framework can serve static files directly from a specified directory.
- **Middleware:** While FastHTTP itself does not provide high-level middleware support, it can be integrated with other frameworks or libraries to add middleware functionality.

Getting started with FastHTTP

Following are some of the sample steps to get set up with the **FastHTTP** package and related usage patterns:

1. **Installation:** To install FastHTTP, you can use the Go package manager (**go get**):

```
2. go get -u github.com/valyala/fasthttp
```

Here is a simple example of how to create an HTTP server using FastHTTP:

```
1. package main
2.
3. import "github.com/valyala/fasthttp"
4.
5. func requestHandler(ctx *fasthttp.RequestCtx) {
6.     ctx.WriteString("Hello, World!")
```

```

7. }
8.
9. func main() {
10.     fasthttp.ListenAndServe(":8080", requestHandler)
11. }

```

2. **Request handling:** In FastHTTP, the **requestHandler** function takes a single argument, a **fasthttp.RequestCtx** instance, which provides access to the request and response buffers. You can use methods on the **RequestCtx** to read request data and write response data.

3. **File serving:** You can serve static files using the **fasthttp.FS** method:

```

1. fs := &fasthttp.FS{
2.     Root:         "static", // Path to static files
3.     GenerateIndexPages: true,
4. }
5. fasthttp.ListenAndServe(":8080", fs.NewRequestHandler())

```

4. **WebSocket support:** FastHTTP supports WebSockets, allowing you to build real-time applications:

```

1. import (
2.     "github.com/valyala/fasthttp"
3.     "github.com/valyala/fasthttp/websocket"
4. )
5.
6. func websocketHandler(ctx *fasthttp.RequestCtx) {
7.     err := websocket.Upgrade(ctx, func(conn *websocket.Conn) {
8.         // Handle WebSocket connection
9.     })
10.    if err != nil {
11.        ctx.Error(err.Error(), fasthttp.StatusInternalServerError)
12.    }
13. }
14.

```

```

15. func main() {
16.     fasthttp.ListenAndServe(":8080", websocketHandler)
17. }

```

FastHTTP is an excellent choice for developers who need a high-performance HTTP server framework in Go. While it may require more effort to build complex web applications compared to higher-level frameworks, its speed and efficiency make it suitable for performance-critical projects. To learn more about FastHTTP, you can visit the GitHub repository at <https://github.com/valyala/fasthttp> for documentation and examples.

Conclusion

In this chapter, we dug through different frameworks to ease web application development via more streamlined routing and inter-service communications.

We have summarized the key features of these frameworks in the following table:

Feature	Gin	Echo	Beego	Revel	FastHTTP
Main use case	High-performance API and web applications	High-performance REST APIs and web applications	Full-stack web applications	Full-stack web applications	High-performance HTTP server
Performance	High (lightweight, fast router)	High (optimized router, low memory)	Medium	Medium	Very High (optimized for speed)
Ease of use	Simple and easy-to-learn API	Simple and intuitive API	Full featured but more complex	More opinionated and structured	Low-level API, requires more setup
Routing	Fast, lightweight router	Fast, customizable router	RESTful routing, controller-based	Built-in RESTful routing	Basic routing, primarily for simple requests

Feature	Gin	Echo	Beego	Revel	FastHTTP
Built-in ORM	No	No	Yes	No	No
Middleware support	Yes (via handlers)	Yes (flexible support)	Yes (via filters)	Yes (via filters)	Limited (external libraries)
Error handling	Basic error handling	Centralized error handler	Custom error pages and handling	Custom error handling	Requires manual setup
Logging	Basic logging support	Built-in logging	Built-in logging	Built-in logging	Limited logging (external setup)
Community and documentation	Large community, well-documented	Large community, well-documented	Good community and documentation	Moderate documentation, smaller community	Moderate documentation, smaller community
Best for	Lightweight APIs, small to medium applications	REST APIs, real-time applications, microservices	Full-stack web applications	Full-stack applications, opinionated setup	Low-level, high-performance web servers

Table 3.1 : Comparison of the various features across frameworks

The following is a table of relevant open-source GitHub metrics that can be compared to provide insights into the usage and community involvement for each of these frameworks:

Framework	Stars	Forks	Watching
Gin	75.8K	7.8K	1.4K
Echo	28.6K	2.2K	527
Beego	30.9K	5.6K	1.2K
Revel	13.1K	1.4K	519
FastHTTP	21.1K	1.7K	391

Table 3.2 : Comparison of the various GitHub metrics as of May 2024

Interestingly, all of these are under an *MIT software license*. Such solutions will evolve, and they can be experimented with and upgraded as needed as the application's needs evolve.

In the next chapter, we will learn about microservice evolution and best practices for service deployment.

OceanofPDF.com

CHAPTER 4

Microservices

Introduction

From the previous chapters, we gained knowledge on building server-side systems using the Golang and web frameworks. In this chapter, we dive into the world of bringing such software, independent of the programming language, to life by deploying those services. We will look at the history of deployments and review the current set of options for running the software efficiently. We will highlight how Kubernetes has emerged as the leader for service orchestration and discuss aspects of deployment management automation it provides.

The whole is greater than the sum of its parts.

—Aristotle

The golden rule: can you make a change to a service and deploy it by itself without changing anything else?

—Sam Newman, *Building Microservices: Designing Fine-Grained Systems*

One of the major requirements in the software industry has evolved to be the ability to deliver software updates on an increasingly regular cadence with the least service interference and without having the need for manual steps and processes overhead. This need for automation and quick iterations for the delivery of each software version has molded the advances in service deployment. Splitting the application into independent entities

provides greater agility and flexibility for the overall deployment process.

Structure

To get the learnings from the past and to understand the current state of the deployment world, this chapter covers the following topics:

- History
- Monolithic architecture
- Service-oriented architecture
- Microservices
- Service-oriented architecture versus microservices
- Virtual machines
- Containers
- Virtual machines versus containers
- Kubernetes architecture

Objectives

The main goal of the chapter is to introduce the most common options currently available for software deployment and service delivery for any organization.

The reader will gain perspective on how automation and delivering software using industry-standard platforms can reduce the organizational burden of maintaining service management. This allows the development and operational teams to iterate quickly on new user requests as well as internal improvements that make it to production.

By the end of this chapter, the reader will be able to understand the trade-offs for using microservices and how to handle real world scale and run a robust application.

History

The history of software deployment is a fascinating journey that has evolved alongside advancements in technology and computing. Let us take a brief overview of it:

- **Manual installation:** Between 1950 and 1990 software distribution and deployment involved physical media like punch cards, tapes, floppy disks, or CDs. Users would receive such devices containing current version of the software and had to manually install the software on their computers, following installation instructions and documentation provided by vendors.
- **Early automation:** As software became more complex and widespread, developers began creating installation scripts to automate the deployment process on their local machines. These scripts helped streamline installations and reduce human error. The scripts themselves would have to evolve to meet the changing needs of the vendor software and have added some toil to the process.

The concept of package managers also emerged to simplify the process of installing, updating, and managing software on a system. Package managers like **Advanced Package Tool (APT)** for Debian-based systems and **Yellowdog Updater, Modified (YUM)** for *Red Hat* based systems became popular in the Linux world.

- **Client-server architecture:** From the early 1980s, client-server architecture emerged with the rise of networking technologies. Applications were deployed on centralized servers, and clients accessed them remotely over a network. Deployment involved installing and configuring server software on dedicated hardware. Administrators could also automate the deployment of related software over the **local area network (LAN)** to multiple client machines from a central server, reducing the overhead of individual installations.
- **Web-based deployment:** The advent of the internet led to the proliferation of web-based applications in the late 1990s. Software vendors began offering applications as web services accessible through web browsers. Deployment involved running web servers and hosting applications on servers accessible via the internet as an **Internet Protocol (IP)** address or DNS name.

These applications were called **software as a service (SaaS)** and eliminated the need for users to install software locally, either on the machine or in large data centers. This model gained popularity due to its accessibility and reduced need for deployment on the user/client side.

- **Virtualization:** The early 2000s saw the dawn of virtualization technologies like VMware, Xen, and KVM, which gained popularity for server consolidation, resource optimization, and infrastructure flexibility. **Virtual machines (VMs)** allowed multiple virtualized **operating systems (OS)** to run on a single physical server, streamlining software deployment and management.
- **Containers:** After the 2010s, containerization technologies like Docker, Kubernetes, and Containerd projects further revolutionized software deployment by providing lightweight, portable, and scalable packaging for applications and their dependencies. Containers encapsulated applications in self-contained units, enabling consistent deployment across different environments, and were more lightweight and flexible compared to VMs.

Overall, virtualization technologies like VMware and containerization platforms like Docker steered the software deployment into abstracting away applications from the underlying hardware and OS aspects. This allowed for resource utilization across applications and simplified deployment across different environments.

- **Continuous integration/continuous deployment:** From the mid 2010s, **continuous integration/continuous deployment (CI/CD)** practices automated the process of building, testing, and deploying software, enabling faster and more reliable software releases. Tools like Jenkins, GitLab CI/CD, and Travis CI facilitated the implementation of CI/CD pipelines for automating software deployment workflows.
- **Infrastructure as code:** The late 2010s started the era of **infrastructure as code (IaC)**, with tools like Terraform, Ansible, and Chef allowing developers to manage infrastructure configurations programmatically, treating the configuration set up as code-like steps. This approach enabled consistent and repeatable software deployments

across different environments using version-controlled configuration files.

- **Serverless computing:** Serverless computing abstracted away infrastructure management starting mid 2010s, allowing developers to focus solely on writing and deploying code without managing servers or containers. Platforms like AWS Lambda and Azure Functions offered event-driven, auto-scaling execution environments for running code snippets or functions, making it easier to deploy and scale applications without managing servers locally.
- **Microservices architecture:** Microservices architecture decomposed applications into small, independently deployable services, each with its own runtime environment and data storage. This approach that evolved in the late 2010s enabled faster development, improved scalability, and better resilience compared to monolithic architectures. With this architecture, each service can be updated and deployed independently without affecting other services.
- **Edge computing:** Since the late 2010s, edge computing has emerged as a deployment paradigm that brings computation and data storage closer to the location where they are needed, reducing latency and bandwidth usage. Edge computing platforms like AWS IoT Greengrass and Azure IoT Edge enable deploying and managing applications at the edge of the network.

These timelines provide a broad overview of how software deployment mechanisms have evolved over time, driven by technological advancements, changing business requirements, and shifts in computing paradigms.

Over the next few sections, we will explore the main approaches still used in the industry and provide a holistic approach to choosing the right option for service deployment.

Monolithic architecture

Monolithic architecture refers to a **software design approach** where the entire application is built as a single, unified unit. This usually implies a

single GitHub or equivalent code repository, mostly written in a single programming language. This would encapsulate all business functionality into one software version for build and rollout to production systems.

Advantages

Here are some advantages of monolithic services:

- **Simplicity:** Monolithic architectures are relatively simple to develop, deploy, and manage compared to distributed architectures like microservices. There is only one codebase to maintain, and developers can work on different application parts without worrying too much about inter-service communication or deployment dependencies.
- **Ease of development:** Developing a monolithic application is often faster and more straightforward than building a distributed system. Developers can focus on implementing features and functionality without the complexity of managing multiple services, APIs, and data stores.
- **Common technology stack:** In a monolithic architecture, all application components share the same technology stack and dependencies. This simplifies development, testing, and debugging, as developers do not need to worry about compatibility or integration challenges between different technologies.
- **Simplified deployment:** Deploying a monolithic application involves deploying a single artifact (for example, a WAR file or executable) to a server or container. This simplicity makes it easier to manage deployment processes, rollbacks, and versioning than deploying multiple services in a distributed environment.
- **Performance:** Monolithic architectures can offer better performance than distributed architectures in certain scenarios, especially when communication overhead between services is minimal. In-memory function calls and direct database access within a monolithic application can be more efficient than remote API calls between microservices.
- **Debugging and testing:** Debugging and testing monolithic

applications are generally easier because all components are tightly integrated and share the same runtime environment. Developers can debug the entire application in a single process and write comprehensive end-to-end tests without extensive mocking or stubbing.

- **Deployment flexibility:** Monolithic applications can be deployed in various environments, including traditional data centers, virtual machines, or cloud platforms. This flexibility allows organizations to choose deployment options that best suit their infrastructure requirements and operational preferences.
- **Resource efficiency:** For small to medium sized applications or projects with limited scalability requirements, a monolithic architecture may offer sufficient resource efficiency without the overhead of managing multiple services and infrastructure components.

While monolithic architectures offer simplicity and ease of development, they may not be suitable for all use cases, especially for large-scale, complex applications with high scalability, availability, and maintainability requirements. Organizations should carefully evaluate their architectural choices based on factors such as project scope, team expertise, scalability needs, and long-term growth plans.

Drawbacks

While monolithic architectures have their advantages, they also come with several challenges, especially if the requirement is to build a modern web application. Let us take a look at them:

- **Limited scalability:** Monolithic applications are typically deployed as a single unit, making it challenging to scale individual components independently. Scaling requires replicating the entire application stack, which can lead to inefficiencies and increased costs, especially if certain components experience higher demand than others.
- **Difficulty in CD:** CI/CD practices are more challenging to implement in monolithic architectures. Since the entire application is deployed as a single unit, any change requires redeploying the entire application,

which can slow down the release cycle. This might also imply taking downtime and a potential increase in the risk of deployment failures and the need for rollbacks.

- **Technology lockstep:** In monolithic architectures, all application components share the same technology stack and dependencies. This tight coupling makes adopting new technologies or upgrading existing ones difficult without affecting the entire application. It can also limit developers flexibility in choosing the best tools for each component.
- **Complexity and maintenance:** Monolithic applications tend to become complex over time as new features are added. This complexity can make the codebase difficult to understand, maintain, build, and debug. As a result, developers may spend more time navigating and refactoring the codebase for new features, which can slow down development velocity.

Deploying changes to a monolithic application carries a higher risk compared to microservices or other distributed architectures. A single bug or error in one component can potentially bring down the entire application, causing widespread disruption and downtime.

- **Resource consumption:** Monolithic architectures often consume more resources than necessary, especially during periods of low demand. Since all components are deployed together, resources cannot be allocated dynamically based on the workload of individual components, leading to inefficient resource utilization for entities like CPU, memory, and network bandwidth.
- **Team scalability:** Large monolithic codebases can be challenging for development teams to work on collaboratively. As the codebase grows, it becomes increasingly difficult for multiple teams to work on different parts of the application simultaneously without stepping on each other's toes or introducing conflicts.
- **Vendor lock-in:** Monolithic architectures may lead to vendor lock-in, as the entire application is built around a specific set of technologies and infrastructure. Switching to alternative solutions or migrating to different vendors can be prohibitively difficult and costly.

To address these drawbacks, many organizations are adopting alternative architectural patterns such as microservices, serverless computing, or containerization, which offer greater scalability, flexibility, and maintainability.

Service-oriented architecture

Before we travel to the microservices destination, let us take a pit stop at its predecessor that took aim at some of the above disadvantages with a monolithic approach. **Service-oriented architecture (SOA)** is an architectural approach that structures software applications as a collection of loosely coupled, interoperable services. These services could be independently developed, deployed, and consumed, and they communicate with each other over a network using proposed standard protocols.

Advantages

SOA aims to create modular, reusable components that can be combined to build complex systems. Here are the advantages of this approach:

- **Modularity:** SOA promotes modularity by breaking down complex systems into smaller, independently deployable services. This makes it easier to develop, maintain, and scale applications, as changes to one service have minimal impact on others.
- **Reusability:** Services in SOA are designed to be reusable across multiple applications and business processes. This reduces redundancy and promotes consistency, as developers can leverage existing services rather than reinventing the wheel.
- **Interoperability:** SOA emphasizes standardized protocols and data formats for communication between services. This enables interoperability between different systems and technologies, allowing services to be integrated seamlessly regardless of the underlying platform.
- **Scalability:** SOA allows individual services to be scaled independently based on demand. This flexibility enables organizations to allocate resources more efficiently and handle fluctuations in workload.

effectively.

- **Flexibility and agility:** SOA enables organizations to adapt to changing business requirements more easily by composing and recomposing services to meet new needs. This agility is particularly valuable in dynamic and competitive business environments.

Disadvantages

Now, we can see the flip side of the SOA coin, which shows the side effects of the approaches. They are as follows:

- **Complexity:** Implementing SOA requires careful planning and design to ensure proper service identification, definition, and clearly defining integration points across services. Managing many services and their interactions can introduce complexity, especially as the system grows in size and complexity.
- **Performance overhead:** Communication between services in SOA typically involves network calls, which can introduce latency and overhead compared to in-process method calls in monolithic architectures. This overhead can impact system performance, especially in distributed environments.
- **Service discovery:** As the number of services grows, managing service discovery, versioning, and dependency management can become challenging. Without proper governance and tooling, it can be difficult to track service dependencies and ensure compatibility between different versions.
- **Security and governance:** SOA introduces new security challenges, such as authentication, authorization, and data integrity, especially in distributed environments where services are exposed over the network. Proper security measures and governance frameworks must be implemented to mitigate these risks.
- **Vendor lock-in:** Adopting proprietary service technologies or platforms can lead to vendor lock-in, making it difficult and costly to migrate to alternative solutions in the future. Organizations should carefully evaluate vendor dependencies and consider adopting open

standards to mitigate this risk.

Overall, while SOA offers many benefits in terms of modularity, reusability, and interoperability, it also comes with challenges related to complexity, performance, governance, and security. Successful adoption of SOA requires careful consideration of these factors and a well-defined architectural strategy.

Microservices

Now, we can study microservices and see how the next set of challenges from SOA were overcome. Microservices is also an architectural style that structures an application as a collection of small, autonomous services modeled around a business domain. Each microservice is designed to perform a specific function and communicates with other services through well-defined APIs.

Here is an overview of the main aspects of microservices:

- **Granularity of services:** Microservices decompose applications into independent services that can be developed, deployed, and scaled individually. Each service corresponds to a specific business function and operates within its own process.
- **Independently deployable:** Each microservice can be deployed independently of other services, allowing for more agile and continuous deployment processes. This independence facilitates rapid development and operational iteration and reduces the risk of affecting the entire system when changes are made.
- **Decentralized data management:** Microservices favor decentralized data management, meaning each service manages its own database or data store. This contrasts with monolithic architectures and SOA, where a single database is shared across the entire application.
- **Technology diversity:** Teams can choose the best technology stack for their specific service without being constrained by a common framework or language used in other parts of the application. This allows for greater flexibility and optimization for each service's requirements.

- **Domain-driven design:** Microservices are often modeled around business domains, enabling better alignment with business processes and improving the scalability and maintainability across all the services of the system.
- **Resilience and fault tolerance:** Microservices architectures are designed to be resilient to failures. If one service fails, it does not necessarily bring down the entire system. Patterns like circuit breakers and service discovery are used to enhance fault tolerance.

A sample overall system view of a sample application with three interacting microservices (at different software version) is depicted in [Figure 4.1](#):

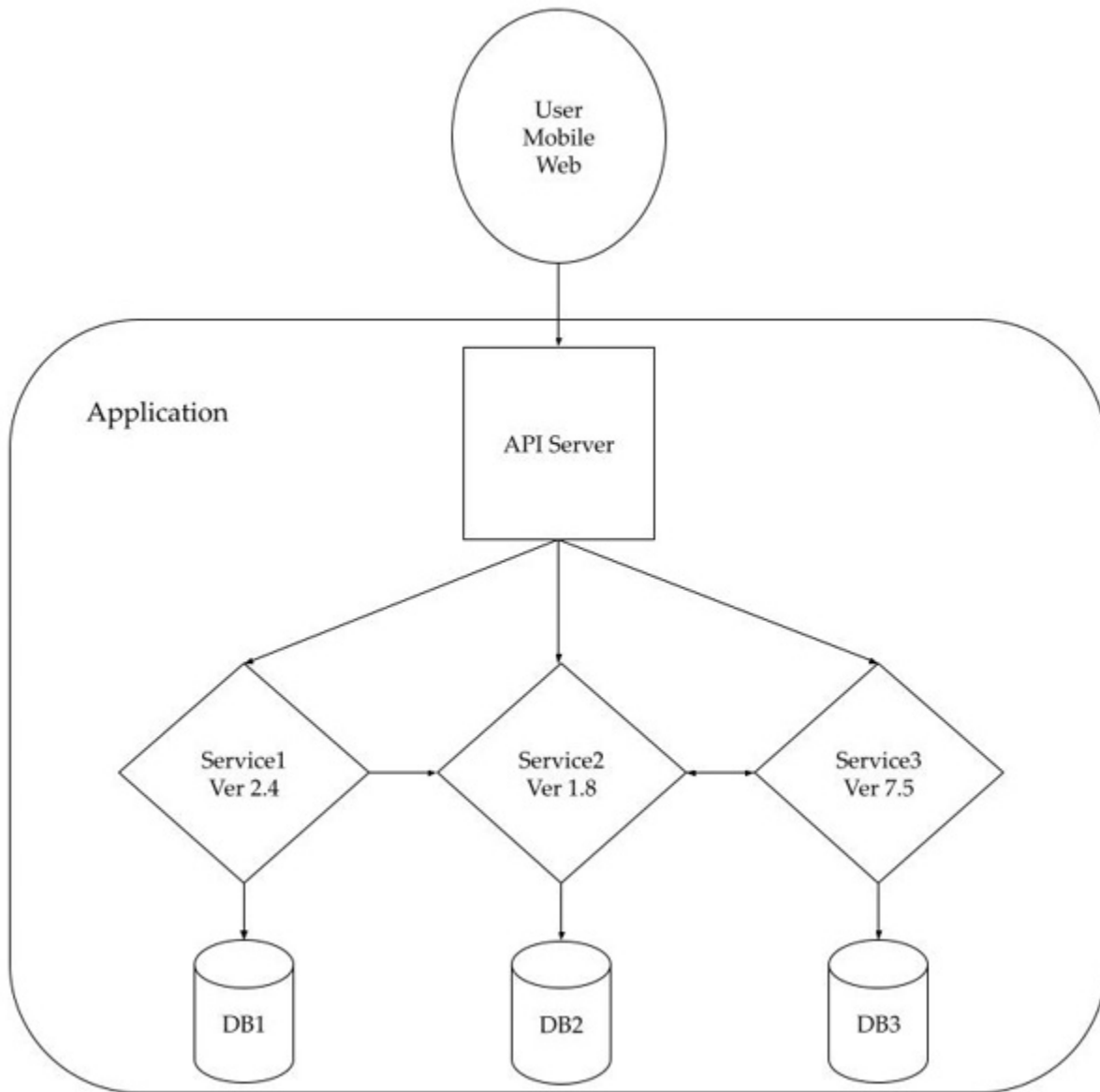


Figure 4.1: Example microservice architecture-based application

Advantages

The growing usage for this architectural approach in modern software development can be attributed to the following aspects:

- **Agility:** Development teams can work on different services simultaneously without interfering with each other, enabling faster development cycles and more frequent releases.
- **Scalability:** Microservices allow individual services to be scaled independently, which can optimize resource usage and improve

performance under load. This is particularly useful for services with varying demand.

- **Resilience:** The isolation of services means that a failure in one service is less likely to impact the entire system. This isolation also simplifies testing and debugging.
- **Technology flexibility:** Different services can be built using the most appropriate technologies, enabling teams to leverage the best tools and frameworks for their specific needs.
- **Business alignment:** By aligning services with business domains, organizations can more easily evolve their systems in response to changing business requirements.

Disadvantages

Just like any other aspect of software development, even microservices architecture has its set of challenges that point to the other side of the coin for advantages. These are mostly on the operational side, as can be observed by the keen reader:

- **Complexity:** Microservices architectures are inherently more complex than monolithic architectures due to the distributed nature of services. Managing many microservices requires additional overhead for deployment, monitoring, and troubleshooting. Additionally, communication between services introduces network latency and potential points of failure, which can complicate system design and implementation.
- **Resource overhead:** Running and managing multiple microservice instances can consume more resources than a monolithic application deployed on a single server. Each microservice requires its container runtime, networking stack, and memory footprint, leading to increased resource overhead and infrastructure costs, especially at scale. The distributed nature of microservices can also introduce network latency and overhead associated with remote calls between services.
- **Data management challenges:** Microservices architectures tend to result in distributed data management, where each service manages its

own database or data store. This can lead to data consistency issues. If consistency needs to be maintained across multiple databases, it becomes more challenging, and the application has to be aware of this. Implementing distributed transactions or eventual consistency patterns requires careful design and introduces additional complexity.

- **Operational overhead:** Operating a microservices-based system requires specialized expertise and tooling for managing container orchestration, service discovery, load balancing, distributed tracing, logging, and fault tolerance. Organizations must invest in building and maintaining robust infrastructure and DevOps practices to ensure the reliability and scalability of microservices deployments. This can increase operational costs and complexity.
- **Testing and debugging:** Testing and debugging microservices-based applications can be more challenging compared to monolithic applications. End-to-end testing becomes more complex due to the need to mock or stub dependencies, and debugging distributed systems requires specialized tools and techniques for tracking requests across multiple services. In such scenarios, the addition of logging and tracing during development phase helps ease the debugging overhead.
- **Service interdependencies:** Microservices often have interdependencies, where changes to one service can impact other dependent services. This tight coupling can introduce coordination challenges and increase the risk of cascading failures if not managed properly. Versioning, backward compatibility, and API evolution become critical considerations in microservices architectures.

Coordinating deployments and managing dependencies between services can be complex, especially in environments with frequent releases and updates. Service versioning, dependency resolution, and rolling upgrades must be carefully orchestrated to minimize downtime and avoid service disruptions.

- **Organizational alignment:** Adopting microservices often requires organizational changes to align development teams, processes, and culture with the principles of service ownership, autonomy, and accountability. Organizations must invest in cross-functional teams,

communication channels, and governance frameworks to ensure the successful adoption and operation of microservices architectures. This can add friction if teams want to migrate from monolithic to microservices, but it need not be a major issue compared to the long-term gains.

Overall, while microservices offer scalability, flexibility, and resilience, they also introduce complexity, operational challenges, and overhead that organizations must carefully consider and manage to reap the full benefits of this architectural approach.

Best practices

Some of the options for using microservices are listed below to ensure the most developmental and operational benefit:

- **API Gateway:** An API Gateway can provide a single-entry point for clients to interact with the microservices, handling request routing, composition, and protocol translation.
- **Service discovery:** Service discovery mechanisms help microservices find and communicate with each other dynamically, supporting scalability and resilience. On top of that, load balancers help the service instances to be uniformly utilized.
- **Circuit breaker:** The circuit breaker pattern helps to prevent cascading failures by detecting failures and short-circuiting requests to failing services for a certain time threshold.
- **Event-driven communication:** Services can communicate asynchronously using events, which helps to decouple services and improve scalability and reliability.
- **Containers and orchestration:** Containers (for example, Docker) and orchestration tools (for example, Kubernetes) are often used to deploy and manage microservices, providing isolation, scalability, and simplified management.

Microservices architecture offers a flexible, scalable, and resilient approach to building modern applications by breaking down complex systems into smaller, manageable services. However, it also introduces significant

complexity and operational challenges that require careful planning and sophisticated infrastructure. When implemented effectively, microservices can lead to more agile development, better alignment with business needs, and improved scalability and fault tolerance.

Service-oriented architecture versus microservices

SOA and microservices are both architectural styles that aim to build distributed systems by structuring applications into collections of services. However, there are distinct differences in their approaches, design principles, and implementations. Here is an exploration of the architectural differences between SOA and microservices:

Feature	SOA	Microservices
Granularity	Services are typically larger and coarser-grained, often encompassing entire business processes or substantial parts of an application.	Finer-grained services designed to handle a very specific piece of functionality or business capability.
Communication	Heavyweight communication protocols like Simple Object Access Protocol (SOAP) and enterprise service buses (ESBs) for message routing, transformation, and integration.	Lightweight communication protocols like HTTP/REST or messaging queues (e.g., RabbitMQ, Kafka). They avoid centralizing communication through an ESB, instead opting for direct service-to-service communication.
Data management	Services share a common database or data model, which can lead to tight coupling at the data layer.	Enforce decentralized data management, where each service manages its own database. This ensures loose coupling and service independence.
Governance	Centralized governance model, with an emphasis on standardizing services and their interfaces across the organization.	Decentralized governance, giving individual teams the autonomy to choose their own technologies, data stores, and deployment practices.
Technology stack	Often relies on more traditional, enterprise-level technologies and standards.	More modern and flexible, allowing the use of diverse and evolving technology stacks and frameworks.

Table 4.1 : Comparison of SOA and microservices features

SOA and microservices both aim to create scalable, maintainable, and reusable services but differ significantly in their approaches and philosophies. SOA's more centralized and coarse-grained model contrasts with microservices decentralized, fine-grained, and flexible approach. Microservices can be viewed as an evolution of SOA, addressing many of its challenges while aligning with modern development and deployment practices.

Virtual machines

Our next pit stop is at the compute infrastructure revolution that paved the road for modern scalable deployments. VMs are logical emulators of physical machines running just an internal OS and an application, each having its own slice of the CPU, memory, storage and network. This helps run multiple VMs in parallel on the same hardware, providing better isolation and resource utilization.

Advantages

They offer numerous salient advantages for application deployment and infrastructure management. Let us take a look at them:

- **Resource utilization:** VMs enable better utilization of physical hardware by running multiple VMs on a single physical machine, streamlining the use of CPU, memory, and storage resources across them.
- **Isolation:** VMs provide strong isolation between different environments. Each VM operates independently, so issues in one VM do not affect others. So, they can be used for testing new features so they do not affect production systems.
- **Flexibility:** VMs support various OS and applications on the same physical hardware, providing flexibility for development, testing, and deployment.
- **Scalability:** VMs can be easily created, cloned, and scaled to meet changing workload demands. This makes them ideal for dynamic and

scalable environments.

- **Disaster recovery and backup:** VMs can be backed up, snapshotted, and restored quickly, facilitating disaster recovery and business continuity.
- **Legacy application support:** VMs can run older OS and applications, allowing organizations to continue using legacy software without requiring old hardware.

Disadvantages

Overall, there are some challenges to using VMs at a larger scale. They are:

- **Resource overhead:** Running multiple VMs introduces overhead on CPU, memory and storage due to the need for virtual hardware emulation and hypervisor management. This can lead to less efficient resource utilization compared to containers or bare metal deployments.
- **Performance:** Although VMs offer good performance, the abstraction layer introduced by the hypervisor can lead to slightly lower performance compared to running applications directly on physical hardware. In multi-tenant environments or situations where multiple VMs share the same physical hardware, resource contention can occur.
- **Complexity:** Managing and maintaining a large number of VMs can be complex, requiring robust virtualization management tools and administrator expertise.
- **Licensing costs:** Licensing costs for virtualization platforms software and guest OS can be significant, especially in large-scale deployments.
- **Security risks:** Virtualization introduces additional attack vectors and security risks, as vulnerabilities in the hypervisor or guest OS could potentially compromise the entire virtualized environment. Administrators must implement robust security measures to protect against threats such as VM escape attacks and unauthorized access to virtualized resources.

VMs are a foundational technology in modern computing, providing flexible, scalable, and isolated environments for running multiple OS and

applications on a single physical machine. While they introduce some overhead and complexity, their benefits in terms of resource utilization, isolation, and flexibility have made them an essential tool in many IT infrastructures, from data centers to cloud computing platforms.

Organizations should weigh the pros and cons of VMs against their business requirements and consider alternative approaches, such as containerization, which we discuss in the next few sections.

Containers

We now look at the next step in the evolution to even more lightweight constructs that can be built and deployed independently. The evolution of containers has been influenced by several key developments in systems technology and OS. Here is a timeline highlighting the major milestones in the evolution of containers:

- **Chroot:** The concept of isolation in Unix-like OS dates back to the 1970s with the introduction of the `chroot` system call. Chroot allowed processes to run in a restricted environment with their own **root** directory, providing a basic form of process isolation.
- **FreeBSD jails:** In the early 2000s, FreeBSD introduced the concept of jails, which extended chroot to provide more robust isolation mechanisms for processes. Jails allowed administrators to partition a FreeBSD system into multiple smaller environments, each with its own file system, network stack, and process space.
- **Linux-VServer:** Developed in the early 2000s, was one of the first implementations of OS-level virtualization on Linux. It allowed multiple virtual environments (known as **VServers**) to run on a single Linux kernel, each with its own file system, network configuration, and process space.
- **Linux containers (LXC):** Introduced in 2008, built upon the concepts of chroot and **control groups (cgroups)** to provide a user-friendly interface for managing Linux containers. LXC allowed users to create and manage lightweight containers with their own isolated namespaces for processes, networking, and file systems.

- **Docker containers:** Launched in 2013, Docker played a pivotal role in popularizing container technology and making it accessible to a broader audience tools and services eg. Docker introduced a standardized container format (Docker image) and a platform for building, shipping, and running containers across different environments. Docker's simplicity, portability, and ecosystem contributed to its rapid adoption by developers and organizations.
- **Container orchestration:** As container usage grew, the need for tools to orchestrate and manage containerized applications across distributed environments became evident. Container orchestration platforms like Kubernetes, introduced by *Google* in 2014, emerged to address this need. Kubernetes provided powerful features for deploying, scaling, and managing containers in production environments, establishing itself as the de facto standard for container orchestration.
- **Serverless containers:** Building upon the success of containers, serverless computing platforms like *AWS Fargate* and *Microsoft Azure serverless* introduced serverless container services. These platforms abstract away the underlying infrastructure and management complexity, allowing developers to focus on writing and deploying containerized applications without managing servers or clusters.

Overall, the evolution of containers has been marked by a progression from basic process isolation techniques to robust, standardized container solutions that enable efficient application deployment and management across diverse environments.

Virtual machines versus containers

VMs and containers are both popular technologies used for deploying and managing applications, but they have different architectures, characteristics, and use cases. Here is a comparison of the various aspects of VMs and containers:

- **Architecture:**
 - VMs virtualize hardware resources, including CPU, memory, storage, and network interfaces, allowing multiple guest OS to run

on a single physical host. Each VM includes its own complete OS, known as the **guest OS**, along with the application and its dependencies.

- Containers virtualize the OS at the process level, allowing multiple containers to run on a single host OS kernel. Each container with matching base image shares the host OS kernel and libraries but has its own isolated file system, process space, and networking stack.

- **Isolation:**

- VMs provide strong isolation between applications by running each application within its own VM. This isolation ensures that applications cannot interfere with each other's runtime environment or access each other's data directly.
- Containers provide lightweight process isolation, where each container runs in its own isolated namespace. While containers offer strong isolation at the process level, they may not provide the same level of isolation as VMs, especially in multi-tenant or security-sensitive environments.

- **Resource overhead:**

- VMs have higher resource overhead compared to containers because each VM includes its own guest OS. This overhead includes memory, CPU, and storage space required for running multiple OS simultaneously.
- Containers have lower resource overhead compared to VMs because they share the host OS kernel and libraries. This allows for higher density and more efficient resource utilization, as containers can be started and stopped quickly with minimal overhead. Hence containers can provide better scalability.

- **Portability:**

- VMs are less portable compared to containers because they encapsulate the entire OS along with the application and its dependencies. Moving VMs between different environments or

cloud providers can be more complex and time-consuming.

- Containers are highly portable because they encapsulate only the application and its dependencies, rather than the entire OS. Container images can be easily packaged, distributed, and deployed across different environments, making them well-suited for microservices architectures and cloud-native applications.
- **Startup time:**
 - VMs typically have longer startup times compared to containers due to the overhead of booting a complete OS. This can impact the agility and responsiveness of applications, especially in resource constrained environments.
 - Containers have faster startup times compared to VMs because they do not need to boot a complete OS. Containers can be started and stopped in seconds, enabling rapid deployment and scaling of applications in response to changing demand.

In summary, VMs offer stronger isolation and encapsulation at the cost of higher resource overhead and slower startup times, while containers provide lightweight isolation and portability with lower resource overhead and faster startup times. The choice between VMs and containers depends on application requirements, resource constraints, scalability needs, and teams deployment preferences. With most modern applications, a well-orchestrated set of containers is much more efficient and flexible.

Kubernetes architecture

Once containers for each service in an application are created, the application needs to be deployed to potentially run multiple such containers. Kubernetes is a powerful container orchestration platform that automates such containerized applications deployment, scaling, and management. Its architecture is designed to provide high availability, scalability, and flexibility. This section provides an overview of the key components and architecture of Kubernetes.

Master nodes

These are dedicated nodes in a Kubernetes cluster that control the nodes running applications and process user requests. Let us take a look at them:

- **API server:** The central management component of Kubernetes, responsible for serving the Kubernetes API, validating and processing requests, and maintaining the desired state of the cluster.
- **Scheduler:** The component responsible for scheduling and assigning workloads (containers) to individual worker nodes based on resource requirements, availability, and other constraints.
- **Controller manager:** A set of controllers that continuously monitor the state of the cluster and reconcile the actual state with the desired state specified in the Kubernetes API. Controllers include the replication controller, endpoint controller, node controller, and others.
- **etcd:** A distributed key-value store that serves as the cluster's backing store for all cluster data, including configuration settings, API objects, and state information.

Worker nodes

The worker nodes get application work scheduled onto them and have light-weight system processes. Let us take a look at them:

- **Kubelet:** The primary agent running on each worker node, responsible for communicating with the API server, managing containers (pods), and reporting node status.
- **Container runtime:** The software responsible for running containers, such as Docker, container, or CRI-O.
- **Kube-proxy:** A network proxy that runs on each worker node, implementing the Kubernetes Service concept by maintaining network rules and forwarding traffic to the appropriate backend pods.
- **Pod:** The smallest deployable unit in Kubernetes, consisting of one or more containers, shared storage, and networking resources. Pods are scheduled and managed by Kubernetes and represent the basic building blocks of an application.

Networking

The logical group of pods within or across applications need a way to communicate instead of using host networks so that the resources can be shared. These abstractions provide that support:

- **Service:** An abstraction that defines a logical set of pods and a policy by which to access them. Kubernetes Services enable networking and load balancing across multiple pods.
- **Ingress:** An API object that manages external access to services in a Kubernetes cluster, typically acting as a layer 7 HTTP/HTTPS router and load balancer.
- **Container Network Interface (CNI):** A standard interface for network plugins in Kubernetes that allows various networking solutions to be plugged into the cluster, enabling pod-to-pod communication and external connectivity.

Storage

Some of the applications are stateful and need access to existing storage to be able to restart and function across restarts. Those can use the following features:

- **Persistent Volumes (PV):** Storage volumes that exist beyond the lifecycle of individual pods and can be dynamically provisioned and attached to pods as needed.
- **Persistent Volume Claims (PVC):** These are requests for storage by pods, allowing them to consume storage resources provisioned by the underlying storage infrastructure.
- **Storage classes:** Kubernetes resource objects that define different storage configurations and provisioners, enabling dynamic provisioning of storage volumes based on predefined policies.

Custom components

Based on the service's business logic, custom functionality might be needed to monitor related resource creations or updates. These options support that.

- **Custom resource definitions (CRDs):** Extensions to the Kubernetes API that allow users to define custom resource types and controllers.

- **Custom controllers:** Additional controllers that extend the functionality of Kubernetes by managing custom resources and implementing domain-specific logic.

Kubernetes architecture follows a distributed, modular design, with each component responsible for specific functions and interactions. This design allows Kubernetes to scale horizontally, handle planned upgrades or failures gracefully, and provide a resilient platform for deploying and managing containerized applications. In [Chapter 9, Containerization](#), we will discuss how to build and deploy containers for Golang services.

Conclusion

In this chapter, we explored different deployment options throughout software development history. Starting with simple local deployment of a single service, we moved on to a large-scale deployment option of microservices.

Kubernetes has evolved as an indispensable tool to achieve this end goal of running large-scale distributed services across different platforms, whether on-premises bare metal setup or public cloud infrastructure.

The next step is understanding the various dimensions of distributed system architectures that influence building a scalable software service. This will be discussed in the next chapter.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 5

Distributed Systems Overview

Introduction

Having delved into code building and deployment solutions in the previous chapters, let us look deeper at the requirements for running systems at scale. This chapter covers all the important ingredients a backend software engineer needs to understand to help build a robust and resilient distributed system. Since modern services cannot just run on a resource starved single computer, we describe how distributed setups help the problem and what are the common pitfalls that need to be avoided in a distributed system.

A distributed system is one in which the failure of a computer you did not even know existed can render your own computer unusable.

—Leslie Lamport

This quote brings out the potential unexpected impact of a distributed solution when built without careful planning. To understand distributed systems, we will first start with a well-accepted definition on the internet.

Distributed systems consist of various software components spread across different computers running as a single entity working towards a common goal.

In essence, these computers are connected and share messages, resources, and actions across their running programs to achieve a known set of features expected from that system. Hence, they need to be aware of what

work is in progress as well as related synchronization and of any failures across the system. We learn how managing these aspects leads to a more scalable, resilient, and flexible system in the long term.

Structure

To get the full picture of the various scenarios and tradeoffs of distributed systems, we will get familiar with the following aspects:

- History
- Components
- Distributed systems fallacies
- Concurrency
- Data consistency
- Fault tolerance
- Load balancing
- CAP theorem
- Managing distributed systems
- Golang for distributed systems

Objectives

The main goal of the chapter is to present an overview of the dimensions of building large-scale distributed systems. We also provide insights into the challenges these present and dive into recipes for ensuring applications can overcome these hurdles and make the most of the whole system.

By the end of this chapter, the reader will be able to understand concurrency, consistency, fault tolerance solutions, and system and load management. Understanding application availability, consistency, and network distribution will guide the application guarantees. We finally wrap it up with how Golang eases the building of software for such systems.

History

As always, it is useful to first understand how the current landscape was created. Distributed systems have evolved significantly over the decades, driven by the need for increased computational power due to increased data processing needs and system reliability and scalability requirements. Here is a brief history of distributed systems leading up to modern use cases built with the latest concepts:

- 1970s: Early glimpses:
 - **Time-sharing systems:** In the 1960s, the concept of time-sharing systems emerged, allowing multiple users to share computing resources. This was an early form of distributed computing, where terminals were connected to a central mainframe.
 - **ARPANET:** The precursor to the modern internet was developed in the early 1970s. Using two aptly named programs called **Creeper** and **Reaper**, it demonstrated packet-switching technology and connected multiple computers across universities and research institutions.
- 1980s: Birth of distributed systems:
 - The client-server architecture became popular in the 1980s. Clients (personal computers) requested services from servers (more powerful computers), allowing distributed processing, mostly on the client side.
 - The proliferation of **local area networks (LAN)** in businesses and universities facilitated the development of such distributed applications and systems within local networks.
 - The concept of distributed data storage emerged, where data is stored across multiple locations but managed as a single entity by compute servers that were connected to storage units.
- 1990s: Standardization and scale:
 - The exponential growth of the internet in the 1990s transformed distributed systems. The **World Wide Web (WWW)** became a global platform for distributed applications.
 - Middleware technologies, such as **Common Object Request**

Broker Architecture (CORBA), emerged to facilitate communication and interoperability between distributed components.

- Cluster and grid computing gained popularity, where multiple interconnected computers worked together to perform tasks, often used in scientific computing and data centers.
- 2000s: Modern distributed systems:
 - Grid computing aimed to provide a model where computing resources from multiple locations were shared and used collectively, often in scientific research within a limited geographical range.
 - **Service-oriented architecture (SOA)** became popular, emphasizing modularity and reusability of services across distributed systems. Web services and XML-based protocols (SOAP, WSDL) were widely adopted.
 - **Peer-to-Peer (P2P)** networks like Napster and BitTorrent demonstrated distributed systems for file sharing, where each node acted as both client and server.
- 2010s: Cloud computing and beyond:
 - Cloud computing revolutionized distributed systems by providing scalable, on-demand access to computing resources over the Internet. Major providers like *AWS*, *Google Cloud*, and *Azure* offered **infrastructure as a service (IaaS)**, **platform as a service (PaaS)**, and **software as a service (SaaS)**.
 - Big data technologies-built systems for processing large datasets, such as Hadoop and Apache Spark, and became essential in handling the vast amounts of data generated in various domains.
 - Microservices architecture evolved, emphasizing small, independent services that communicate over networks, improving scalability and maintainability of applications.
- 2020s: Edge computing and blockchain:

- With the rise of **Internet of Things (IoT)**, edge computing became crucial. It involves processing data closer to where it is generated or needed, reducing latency and bandwidth usage.
- Blockchain technology introduced a decentralized, distributed ledger for secure and transparent transactions. It underpins cryptocurrencies like Bitcoin and has applications in various industries.
- Serverless computing architectures has also gained traction, allowing developers to build and deploy applications without having to manually manage the underlying infrastructure.

Components

Modern distributed system architecture comprises of several key components that work together to provide a cohesive and efficient system. We will discuss the main components in this section.

Computing nodes

These are processing units that cater to user requests, data crunching, or a combination and can operate in the following modes:

- **Client nodes:** End-user devices or software that interact with the system to request and consume services.
- **Server nodes:** Machines that provide services to clients, including database servers, application servers, web servers, and any such backend cross node computing.
- **Peer nodes:** In P2P architectures, each node can act as a client and a server.

Network infrastructure

This component consists of the following aspects needed to send and receive messages across the compute nodes:

- **Communication links:** Physical or wireless connections enabling data transmission between nodes.

- **Network protocols:** Standards and rules (such as TCP/IP, HTTP, FTP) that govern data exchange and messaging packets over the network.

Messaging

The need to track the contract and format of the content being transferred across the network entails the following options:

- **Message-oriented middleware (MOM):** This technology facilitates message passing between distributed components, such as RabbitMQ, NATS and Apache Kafka.
- **Remote Procedure Call (RPC) systems:** These allow a sender program to cause a procedure to execute on another address space and return information for the current address space to use for the next steps. For example, gRPC, Apache Thrift.
- **Object Request Brokers (ORBs):** Manage communication between objects in a distributed environment. For example, CORBA.

Data storage and management

To store information needed to process remote calls or maintain current programs state and metadata, there is the need to use persistent storage systems. They are:

- **Distributed databases:** Formatted data such as relational data or JSON can use databases distributed across multiple nodes, such as *Google Spanner*, *Amazon DynamoDB*, *Mongo Atlas* or *YugabyteDB*. Data search indexes solution is provided by *Lucene* and *Amazon Elasticsearch*.
- **File systems:** Unformatted data can use distributed file systems like **Hadoop Distributed File System (HDFS)** and **Google File System (GFS)**.
- **Caching systems:** Distributed caches to improve performance when installed in front of some of the persistent datastores, e.g., Redis, memcached.

Coordination and synchronization

In a distributed system, there is an inherent need to ensure the order of processing data across nodes is agreed upon by a majority of the nodes and a leader node commits that decision. This is the common distributed model for ensuring coordination of system state across nodes. Two industry-standard mechanisms related to this are:

- **Distributed algorithms:** Algorithms for coordinating actions of multiple nodes, ensuring consistency and fault tolerance, such as consensus algorithms (for example, Paxos or Raft).
- **Locking mechanisms:** Techniques to prevent conflicting operations in a distributed environment, such as distributed locks using Zookeeper.

Security

The following security measures ensure that the data is being used by the rightful users and does not fall prey to hackers:

- **Authentication and authorization:** Ensuring only authorized users and services can access resources.
- **Encryption:** Protecting data in transit and at rest using encryption techniques.
- **Firewalls and intrusion detection systems:** Safeguarding the network and nodes from unauthorized access and attacks.

Scalability and load balancing

Ensuring that the servers are used uniformly and can be adjusted to meet user load demand is an essential part of the long-term usability of a distributed system.

- **Load balancers:** Distribute incoming network traffic across multiple servers to ensure no single server is overwhelmed, e.g., NGINX, HAProxy.
- **Auto scaling mechanisms:** Automatically adjust the number of nodes in response to load changes can be performed using tools provided by orchestration systems like Kubernetes.

Fault tolerance and reliability

Since the distributed system should not have a **single point of failure (SPOF)**, mechanisms should handle sub-system failures to ensure availability. There might be cases where performance degrades, but auto scaling should help get back to the original state or better with a reasonable SLA. They are:

- **Replication:** Creating copies of data and services to ensure availability during failures. For example, master-slave replication.
- **Failover mechanisms:** Automatic switching to a backup system in case of failure.

Service discovery

This component helps automate service registry and reduce manual configuration to setup new service configurations in the system. Its features are:

- **Registry services:** Keep track of available services and their locations. For example: Consul, etcd.
- **Dynamic configuration:** Update configurations without service disruption.

Monitoring and tracking

While monitoring is geared towards ensuring smooth operation of the system and help plan for system upgrades or adding resources, logging and tracking is needed to analyze for any errors or exceptions being hit by the servers that can be addressed to reduce system overhead.

- **Monitoring tools:** Track performance, availability, and health of the system. For example, Prometheus, Grafana.
- **Logging systems:** Collect and analyze logs for troubleshooting and auditing. For example, **Elasticsearch, Logstash, and Kibana (ELK)** stack.
- **Heartbeat mechanisms:** Regular health checks from nodes or processes to monitor their health status and add replacement nodes or VMs as needed.

These components together create a robust distributed system capable of handling complex, large-scale tasks with high availability, reliability, and performance.

Distributed system fallacies

Designing and managing distributed systems with the above components is complex and fraught with potential challenges that are listed as follows:

- **Scalability:** The ability to handle growth in users and data.
- **Reliability:** Ensuring system availability and fault tolerance.
- **Consistency:** Maintaining data accuracy and synchronization across distributed components to achieve a single source of truth and not lead to split-brain problems.
- **Latency:** To allow for a better user experience, needs to reduce delay in data processing and communication across components.
- **Security:** Protecting data and operations in a distributed/shared environment.

Due to these challenges, there are several common fallacies or misconceptions that new developers and engineers often assume, leading to issues in system reliability, performance, and maintainability. These fallacies, first outlined by *Peter Deutsch* and others at *Sun Microsystems* in the 1990s, which later got extended, are known as the **Fallacies of Distributed Computing**. The following is a list of common fallacies:

- **Network is reliable:**
 - **Misconception:** The network will always be able to deliver messages without loss or error.
 - **Reality:** Networks can fail, packets can be dropped, and suffer from latency and jitter. Systems sending or receiving messages must be designed to handle communication failures gracefully through retries, acknowledgments, and timeouts.
- **Network latency is zero:**
 - **Misconception:** Data can be transferred instantly across the

network.

- **Reality:** Network latency can vary and often introduces significant delays. Systems need to account for latency in their design, especially for time-sensitive operations.
- **Network bandwidth is infinite:**
 - **Misconception:** There is unlimited network bandwidth available for data transfer.
 - **Reality:** Bandwidth is finite and can become a bottleneck. Efficient data transfer methods, such as compression and minimizing data movement, are essential.
- **Network is secure:**
 - **Misconception:** Data transferred over the network is safe from unauthorized access.
 - **Reality:** Networks can be susceptible to attacks and unauthorized access. Security measures such as encryption, secure protocols, and firewalls are necessary.
- **Topology does not change:**
 - **Misconception:** The network topology (the arrangement of nodes and connections) remains constant.
 - **Reality:** Network topologies can change due to node failures, reconfigurations, or dynamic environments like mobile and ad hoc networks. Systems should adapt to changing topologies.
- **Singleton administrator:**
 - **Misconception:** A single entity manages the entire system, ensuring consistency and coordination.
 - **Reality:** Distributed systems often span multiple administrative domains with different policies and management practices. Coordination and consistency across these domains can be challenging and might need the right permissions to be agreed upon by admins.

- **Transport cost is zero:**
 - **Misconception:** Sending data across the network is free.
 - **Reality:** Data transfer can incur significant costs, especially over large distances or high-bandwidth connections. Optimizing data movement and considering cost implications is crucial.
- **The network is homogeneous:**
 - **Misconception:** All parts of the network have the same characteristics.
 - **Reality:** Networks can vary widely in terms of bandwidth, latency, and reliability. Systems must be designed to handle heterogeneous network environments.

Some other additional fallacies that got added to the original list are as follows:

- **Clocks are consistent:**
 - **Misconception:** Clocks on different nodes in the distributed system are synchronized.
 - **Reality:** Clock drift and synchronization issues can lead to inconsistencies. Protocols like **Network Time Protocol (NTP)** can help, but time discrepancies must still be managed.
- **Processing power is infinite:**
 - **Misconception:** Each node has unlimited processing power.
 - **Reality:** Nodes have limited CPU, memory, and storage resources. Load balancing and efficient resource management are necessary to avoid overloading nodes.
- **Change is instantaneous:**
 - **Misconception:** Changes in the system (e.g., configuration updates, deployments) take effect immediately.
 - **Reality:** Propagation of changes can take time and might not be uniform across the system. Rolling updates and eventual consistency models are strategies to handle this.

Fixing the fallacies

To address these fallacies, designers, and developers of distributed systems should adhere to the following best practices:

- **Implement redundancy:** Use replication and failover strategies to enhance reliability and be prepared to handle different aspects of system failures.
- **Optimize data transfer:** Reduce data movement, use efficient serialization, and techniques such as data compression.
- **Plan to reduce latency:** Design systems with asynchronous communication and eventual consistency in mind.
- **Enhance security:** Use encryption, secure protocols, and access controls.
- **Monitor and Adapt:** Continuously monitor the system and adapt to changes in network topology and performance.
- **Use distributed algorithms:** Employ consensus algorithms such as Raft and coordination mechanisms to manage distributed states and operations.

We will look deeper into mechanisms that address some of these challenges of distributed systems in the following few sections.

Concurrency

Concurrency requirements in distributed systems are essential for ensuring that multiple operations can be executed simultaneously without leading to inconsistent, unexpected or incorrect states for the data being processed. This section will discuss some key concurrency requirements and corresponding solutions in distributed systems.

Mutual exclusion

This aspect ensures that multiple processes or threads do not enter critical sections of code simultaneously, and if not performed, could lead to data corruption or inconsistency. Solutions include:

- **Locks and semaphores:** These are usually programming language

constructs provided to ensure only one process accesses a resource at a time.

- **Distributed locks:** Systems like Apache Zookeeper or etcd provide distributed lock services.
- **Optimistic concurrency control:** Assumes multiple transactions can complete without affecting each other, with validation of the assumption at commit time.

Atomicity

This aspect ensures that a series of operations are completed as a single unit of work. If any operation in the series fails, the entire series should be rolled back. Solutions include:

- **Two-phase commit protocol (2PC):** Ensures all nodes agree to commit a transaction before it is finalized by using two-step vote and the commit process.
- **Three-phase commit protocol (3PC):** Adds an additional phase to 2PC to reduce blocking scenarios.
- **Atomic broadcast:** Ensures all nodes receive transactions in the same order, which can be used to maintain consistency.

Isolation

This aspect ensures that concurrently executing transactions do not interfere with each other. Each transaction should execute as if it were the only one running in the system. Solutions include:

- **Snapshot isolation:** Provides each transaction with a consistent snapshot of the database.
- **Serializable isolation:** Ensures transactions produce the same result as if they were executed sequentially.

Deadlock detection and avoidance

Ensures that the system can detect and recover from deadlock situations where two or more processes are waiting indefinitely for resources held by

each other. Solutions include:

- **Deadlock detection algorithms:** Use techniques like wait-for graphs to detect and resolve deadlocks.
- **Timeouts:** Impose time limits on how long a process can hold a lock.

Synchronization

This ensures that nodes and processes in a distributed system can coordinate their actions to achieve a common goal. Options include:

- **Barriers:** Synchronization points built on top of mutexes where processes can wait until all participants reach the barrier.
- **Clock synchronization:** Using protocols like NTP to synchronize clocks across nodes.

Transactional support

Transactional support ensures that a series of operations are treated as a single unit, with guarantees on **atomicity, consistency, isolation, and durability (ACID)** properties. Potential options are:

- **Transaction managers:** Components that manage the lifecycle of transactions.
- **Distributed databases:** Build on top of databases, such as *Google Spanner*, that are designed to support distributed transactions.

Non-blocking operations

This aspect ensures that operations proceed without waiting indefinitely for other operations to complete, enhancing system responsiveness. The options are:

- **Lock-free data structures:** Data structures that allow multiple threads to operate without locks.
- **Asynchronous programming models:** This mode uses callbacks, futures, or promises to handle operations without blocking.

By addressing these concurrency requirements, software engineers can manage the complexity of concurrent operations, thus ensuring high

performance and scalability of the system.

Data consistency

Consistency in distributed systems refers to the need to ensure that all nodes in the system agree on the current state of the data. Achieving this can be challenging due to the inherent characteristics of distributed systems, such as clock skews, network partitions, latency, and node failures. Here is a quick list of consistency needs with potential solutions along with sample use cases for each:

- **Strong consistency:**

- This mode ensures that all reads return the most recent write for a given row or record. Every read operation after a write will see that write or a subsequent one. This often requires synchronization protocols like 2PC, Paxos, or Raft, which can impact performance and availability.
- **Use cases:** Financial transactions, inventory management systems where accurate, up-to-date information is crucial.

- **Eventual consistency:**

- It guarantees that, given enough time, all replicas will converge to the same value, assuming no new updates are made to the data. NoSQL systems like Cassandra and DynamoDB use eventual consistency models, leveraging techniques like gossip protocols to synchronize data over time.
- **Use cases:** Social media feeds, **Domain Name System (DNS)**, where temporary inconsistencies are acceptable.

- **Causal consistency:**

- It ensures that operations that are causally related are seen by all nodes in the same order. Operations that are not causally related may be seen in different orders. These systems require tracking causality using mechanisms like vector clocks or version vectors.
- **Use cases:** Collaborative editing applications and chat systems

where the order of messages matters.

- **Read-your-writes consistency:**

- It guarantees that after a write completes, the same client will always see that write in subsequent read operations. Typically managed by ensuring that read requests from a client are directed to the same replica where the write was acknowledged. This can also be extended to per client session-based consistency.
- **Use cases:** User profile updates in web applications where a user expects to see their recent changes immediately.

- **Linearizability:**

- This is a stronger form of consistency, where read or write operations on an object (say row or record) appear to occur instantaneously at some point between their invocation and their response. It is a single source of ordering where every operation on an object appears atomic in the same order across processes.

Note: If such guarantees are provided across objects involved in logical transactions, it will become serializable consistency. Such setups require atomic operations and often rely on consensus protocols like Raft to enforce this strict order.

- **Use cases:** Critical sections in distributed transactions, ensuring correctness in high-stakes fast-moving environments like financial systems.

By understanding and addressing these consistency requirements, system developers can ensure that the semantics needed for applications running on their distributed system behave as expected and do not cause data loss or corruption.

Fault tolerance

Fault tolerance is a critical aspect of distributed systems, ensuring that the system remains operational even in the face of failures. Since distributed systems have multiple components, as we saw in earlier sections, the requirement for designing and building such systems is to account for recovery from failures of any sub-system. This section discusses some

standard strategies and options for implementing fault tolerance in distributed systems.

Replication

Storing copies of the data on different sets of nodes is the easiest solution for using it as a backup in failures related to node or intermittent software failures. This replication can be achieved in a few ways, as follows:

- **Synchronous replication:** Ensures all copies are updated simultaneously, providing strong consistency but potentially higher latency.
- **Asynchronous replication:** Updates copies with a delay, offering better performance for user workload throughput but with a risk of temporary inconsistency.

Another option is to run multiple instances of a service replicated on different nodes to provide compute redundancy and load balancing. A couple of options include:

- **Multi-region deployment:** Deploying services and data across multiple geographic regions to protect against regional failures (for example, data center outages). Some cloud providers perform this automatically for deployed services.
- **Geo-replication:** Ensuring data is replicated across different regions to enhance availability and disaster recovery.

Combining replication with load balancing and backup node replicas provides good hardware and uniform resource utilization along with preparedness for recovery.

Erasure coding

This technique is used at disk level commonly to ensure corrupted data can be recovered:

- **Data redundancy:** Instead of replicating complete data, this splits data into chunks and encodes them with redundant data information. This reduces storage overhead while providing fault tolerance.

- **Reed-Solomon coding:** A common erasure coding technique used in distributed storage systems to track separate metadata for reconstructing bit-rot on disk media.

Checkpointing and rollback

Some of the concepts from databases can be used to reduce the overhead of recovery when there is a cascade of failures in the system. They are:

- **Periodic checkpointing:** Saving the state of a system at regular intervals to a different server can help recover from such a checkpoint. If a failure occurs, the system can roll back single node or such entity to the last checkpoint. Since this is potentially a heavy-duty operation, its frequency might need to be controlled on a daily basis, for example.
- **Transactional checkpointing:** Ensures all operations within a transaction across nodes are saved together to maintain cluster wide consistency. This might need some tooling which might be out of the business logic purview.

Consensus algorithms

When nodes fail or are removed for planned maintenance, there is a need to allow the system to progress with agreement on state across the remaining nodes. Consensus mechanisms help achieve this that could help during reduced node replicas case. These algorithms help distributed systems agree on a single value or sequence of values even in the presence of faults:

- **Paxos and Raft:** They are used to manage replication and consistency. They work by distributing data across multiple nodes in a way that each node only stores a portion of the data, and that slice is replicated. They ensure that a majority (quorum) of nodes agree on operations before they are committed by a leader node. If a node fails, only part of the data is affected, and the replication ensures availability and leadership can get transferred as well.
- **Zab protocol:** Earlier generation solution used by Apache Zookeeper to provide a distributed coordination service.

Health monitoring

Tracking the health of the system and individual components can often track the components that could potentially fail soon and might help fixing or failing over before actual failure. The mechanisms are:

- **Heartbeat mechanisms:** Regularly sending signals between nodes or processes to check their health can help detect failures quickly when the signal is missing.
- **Health monitoring systems:** Tools and services that continuously monitor the health of nodes and services, triggering alerts or failover procedures when registered issues are detected is another way to be fault tolerant.

Failover and self-healing mechanisms

While building fault tolerance is one side of the coin, the other side is to ensure that failover happens during failure scenarios for continued application availability. The two options are:

- **Active-passive failover:** One node is active, and another is in standby mode. If the active node fails, the standby node takes over.
- **Active-active failover:** Multiple nodes are active and share the load. If one fails, the others continue to operate, handling the load.

The above options would need some form of manual intervention, though the ideal solution would be automated to kick in seamlessly at scale. These techniques are:

- **Automated recovery:** Systems that can automatically detect failures and recover from them without human intervention can be built using orchestration services like Kubernetes.
- **Replica management:** Dynamically adjusting the number of replicas based on the current load and failure conditions can be configured using IaC tools.

There are also cloud-based fault tolerance service solutions that can save developer effort. Utilizing cloud provider services that offer built-in fault tolerance features such as *Amazon RDS Multi AZ*, *Google Cloud Spanner*,

and *Azure Cosmos DB* can be designed into the system to reuse existing technologies and build faster. Implementing these strategies can significantly enhance the fault tolerance of distributed systems, ensuring high availability, reliability, and recoverability, even in the presence of hardware, software, or network failures.

Load balancing

Load balancing is a critical aspect of distributed systems, ensuring that workload is evenly spread across multiple nodes to optimize resource utilization, minimize response time, and avoid overloading any single node that would potentially increase failures on that node. Here are some common techniques and strategies used to achieve load balancing in distributed systems:

- **DNS based load balancing:** Using DNS, the requests are distributed among multiple servers by mapping a single domain name to multiple IP addresses. DNS servers can return different IP addresses for the same domain name in a round-robin fashion, directing clients to different servers. This has drawback of being static in nature and does not account for current work consuming resources on the servers.
- **Hardware load balancers:** These are dedicated hardware devices that distribute incoming network traffic across multiple servers. These devices use algorithms to decide which server should handle each incoming request based on various factors such as server load, health, and response times. Sample solutions include *F5 BIG-IP*, and *Citrix ADC*.
- **Software load balancers:** This is purely software applications that perform load balancing functions, often running on standard hardware. Similar to hardware load balancers, these use current or known load-based algorithms and health checks to distribute requests. These are also part of reverse proxy load balancing mechanisms. Example solutions include HAProxy, NGINX, and Apache Traffic Server.
- **Client-side load balancing:** In this mode, the client sending requests is responsible for distributing requests across multiple servers. Clients can use standard libraries or logic to select the appropriate server based

on various factors such as server health and response times. Sample solutions include *Netflix's Ribbon* and gRPC's client-side load balancing.

- **Application-level load balancing:** The load balancing logic is implemented within the server application itself. The application can track server loads and distribute requests accordingly. Sample use case is custom load balancing logic in microservices architectures.

Load balancing algorithms

Each of the above strategies could use any of the following algorithms while distributing the actual work items:

- **Round-robin:** This distributes requests sequentially across a list of servers. This is the most basic, simple mode as well as easy to implement. Since this is static, it does not account for server load or response time while assigning additional load.
- **Least connections:** This directs requests to the server with the fewest active connections. This mode balances load more effectively in environments with varying request processing times, though it has the overhead of tracking connection counts in near real-time.
- **Least response time:** Sends requests to the server with the lowest average response time. This accounts better for current server performance. Though it requires monitoring and calculation of response times.
- **IP hash:** This mode uses a hash of the client's IP address to determine the server. This ensures the same client is directed to the same server (useful for session persistence). However, even with the best hash functions, this can lead to uneven load distribution.
- **Weighted round-robin:** Assigns a weight to each server and distributes requests based on these weights, such as CPU speed or memory size. Allows for differentiation between servers with different capacities. Does requires manual configuration of such weights.
- **Random:** This just distributes requests randomly across servers. Simple to implement yet may not result in even distribution.

Either round-robin or weighed version might be the easiest option for the initial setup of such load balancers. There are other advantages that can be derived from them. They are:

- **Auto scaling:** Once can get automatic addition or removal of servers based on current load. Monitoring metrics like CPU usage, memory usage, or request rates to trigger scaling actions can be configured. Sample systems like *AWS Auto Scaling* and *Google Cloud Auto Scaling* provide this feature.
- **Health checks:** This can regularly check the health of servers and removes unhealthy ones from the pool. One can use basic ping checks, HTTP checks, or application-specific endpoints. Some well-known health checkers are NGINX, HAProxy, and cloud load balancers.

Thus, load balancing is crucial for ensuring the high availability, efficiency, and reliability of distributed systems. The choice of load balancing strategy and tool often depends on specific system requirements, including the nature of the workload, the architecture of the system, and the desired level of resilience and scalability.

CAP theorem

The CAP Theorem, also known as **Brewer's theorem**, is a fundamental principle in distributed system design. It was proposed by *Eric Brewer* in 2000 and formally proven as a theorem by *Seth Gilbert* and *Nancy Lynch* in 2002. This theorem states that it is impossible for a distributed data store to simultaneously provide all three of the following guarantees:

- **Consistency (C):** This means that all nodes in the distributed system will return the same, most up-to-date value for a given piece of data. Essentially, every read receives the most recent data written or an error, and the system behaves like a single, consistent entity.
- **Availability (A):** Every request (read or write) receives a (non-error) response, without guaranteeing that it contains the most recent write. This means that the system is always available for reads and writes, even if some of the nodes fail. Each request gets a response, though it may not be the most current data.

- **Partition tolerance (P):** The system continues to operate despite arbitrary partitioning due to network failures. This means that the system can tolerate network failures that prevent some nodes from communicating with others. The system remains operational even if there are partitions in the network.

According to the CAP theorem, in the presence of a network partition, a distributed system can only guarantee either consistency or availability, but not both. In a distributed system, network partitions are inevitable due to various reasons like hardware failures, network issues as we discussed in the above section. With partition tolerance assumed, the trade-off is between consistency and availability. Systems must choose to prioritize one over the other based on the application requirements. Here is what systems choose in practice:

- **Consistency and partition tolerance (CP):** The system maintains consistency and tolerates network partitions but may not be available for some operations during partitions. Example: HBase is a distributed database that prioritizes consistency and partition tolerance. Some of the distributed SQL systems like *Google Spanner* or *YugabyteDB* also follow this model.
- **Availability and partition tolerance (AP):** The system remains available and tolerates network partitions but may serve stale data (not always the most recent write). Example: Cassandra, Couchbase, and DynamoDB are distributed databases that prioritize availability and partition tolerance but might return non-latest data.
- **Consistency and availability (CA):** The system ensures both consistency and availability in the absence of network partitions, though, this is not practical in a distributed system that needs to tolerate network partitions. Single-node databases like traditional RDBMS (for example, PostgreSQL, MySQL in standalone mode) can follow this model.

The CAP theorem highlights the inherent trade-offs in distributed systems. It guides architects and designers in making informed decisions about the behavior of their distributed system under different failure conditions. As an

extension, the PACELC theorem provides a more nuanced understanding of the trade-offs in distributed systems, particularly under normal conditions and in the presence of partitions. It was introduced by *Daniel J. Abadi*.

PACELC

The PACELC theorem states that:

- If there is a partition (P), the system has to choose between availability (A) and consistency (C), similar to the CAP theorem.
- Else (E), when the system is running normally without partitions, it must choose between latency (L) and consistency (C).

This implies that during network partitions (P), the distributed system needs to choose between consistency (C) and availability (a), just like CAP mode. Else (E) during normal operation, the options become:

- **Latency (EL):** The system prioritizes lower latency and faster responses, potentially allowing for some inconsistency.
- **Consistency (EC):** The system ensures strong consistency, which may lead to higher latency due to the need to synchronize data across nodes.

The sample systems either choosing latency or consistency during normal operations are:

- **AP/EL systems:** DynamoDB and Cassandra prioritize availability during partitions and low latency during normal operation. They may serve stale data to keep the system responsive and fast.
- **AP/EC systems:** Some configurations of MongoDB or CouchDB prioritize availability during partitions but ensure consistency at the cost of higher latency during normal operations.
- **CP/EC systems:** Traditional relational databases like PostgreSQL in distributed configurations, *YugabyteDB* or *Google Spanner*, prioritize consistency both during partitions and normal operation, resulting in higher latency.

The choice between these trade-offs depends heavily on the application's specific needs. For example, a financial application may prioritize consistency over availability and latency, while a social media platform might prioritize availability and low latency. Some systems try to balance

these trade-offs dynamically, adjusting their behavior based on the current state of the network and operational conditions.

Managing distributed systems

We have until now uncovered the components needed and their internals while building a distributed system. Once they are deployed, managing the distributed systems is a must from the operational standpoint to ensure user workloads can be supported. This involves addressing various aspects to ensure the system operates efficiently, reliably, and securely. These management aspects include monitoring, services and their configuration, security, and scalability dimensions, along with performance optimizations. This section provides an overview of the key management aspects of distributed systems.

Monitoring and logging

Different aspects of monitoring the system metrics can be implemented and supported to get the overall snapshot of the system state at any given point of time. These include:

- **Performance monitoring:** Continuously tracking system performance metrics such as CPU usage, memory usage, network latency, and throughput to identify bottlenecks and ensure optimal performance.
- **Health monitoring:** Regularly checking the health status of system components (e.g., servers, databases, application processes, network links) to detect and address failures promptly.
- **Logging:** Collecting and analyzing logs from various components to troubleshoot issues, understand system behavior, and audit operations.
- **Alerting:** Setting up alerts to notify administrators of critical issues such as hardware failures, security breaches, or performance degradation.
- **Cost:** Continuously monitoring and analyzing costs associated with using resources and running processes on the distributed system can help the business side. This can also help with budgeting, and forecasting the need for more resources.

Configuration management

Some options for setting up the metadata needed for process setup and upgrades include:

- **Automated deployment:** Using tools to automate the deployment of applications and configurations across multiple nodes to ensure consistency and reduce manual errors.
- **Version control:** Managing versions of configuration files and application code in an online repository to track changes and facilitate rollback in case of issues.
- **Centralized management:** Maintaining a central repository for configuration files and using tools to distribute these configurations across the system in a rolling manner.
- **IaC:** Managing and provisioning computing infrastructure through machine-readable configuration files to automate and standardize setup.

Security management

Security is front and center for data access patterns on large-scale systems. Setting up components to help manage the following aspects should be designed into the system:

- **Access control:** Implementing strict access controls to ensure that only authorized users and systems can access resources.
- **Encryption:** Encrypting data at rest and in transit to protect sensitive information from unauthorized access and breaches.
- **Authentication and authorization:** Ensuring that users and services are properly authenticated and authorized before accessing system resources.
- **Auditing and compliance:** Keeping detailed records of system and comprehensive audit trails for access and changes to comply with regulatory requirements and audit standards.
- **Policy enforcement:** Ensuring that all operations comply with organizational and regional data policies and regulatory requirements.

- **Data governance:** Implementing data governance practices to ensure data quality, security, and compliance.

Scalability management

There can also be mechanisms to perform day-two operations such as expanding or shrinking the cluster, as well as load balancing, we discussed before:

- **Horizontal scaling:** Adding more nodes to the system to handle increased load without compromising performance. Automating this gives an auto-scaling features for ease of management.
- **Vertical scaling:** Increasing the capacity of existing nodes by adding more resources (for example, CPU, memory).
- **Load balancing:** Distributing incoming requests evenly across nodes to ensure no single node becomes a bottleneck.

Service management

When there is an increase in number of services that can be deployed and upgraded in parallel by independent teams using the system, having automated service management will help ease the manual toil as well reduce human errors.

- **Service discovery:** Using service discovery mechanisms to dynamically locate and connect to services within the distributed system.
- **Service orchestration:** Coordinating the deployment, scaling, version upgrades, management of services, and inter-service communication often using container orchestration tools like Kubernetes.

Performance optimization

Once the system is running there are add-ons that could further the performance and usability of the system by efficiently allocating resources to different parts of the system to maximize performance.

- **Caching:** Implementing caching strategies (such as Redis) reduced

latency and improves response times for frequently accessed data.

- **Latency reduction:** Once can identify and mitigate sources of latency in the system by using profilers to improve service responsiveness and, hence, user experience.

Tools and technologies

Some of the standard tools used for distributed systems management along with their use case are noted in this section. They are:

- **Monitoring and logging:** Prometheus, Grafana, ELK stack, Splunk, Log4j.
- **Configuration management:** Ansible, Chef, Puppet, Terraform.
- **Security management:** Vault by HashiCorp, AWS IAM, **Open Policy Agent (OPA)**.
- **Scalability management:** Kubernetes, AWS Auto Scaling, Google Cloud Auto Scaling.
- **Service management:** Kubernetes, Docker Swarm, Istio, Linkerd, Consul.

As can be seen, some of the tools span across management aspects. Effective management of distributed systems requires a comprehensive approach that covers these aspects, utilizing appropriate tools and best practices to ensure the system's continued reliability, day-2 operations, performance, and security.

Golang for distributed systems

With the above context for building distributed systems, let us see why Go (or Golang) is regarded as a great option for building such systems. Several of Go key features and design principles mentioned below align well with the requirements:

- **Concurrency and goroutines:** Go makes concurrency a first-class citizen with its lightweight goroutines and channels, which help in structuring distributed systems that are easier to code cleanly, reduce bugs, and improve maintainability. Goroutines are extremely efficient

in terms of memory usage and can handle thousands of concurrent tasks, making it easier to build highly concurrent and scalable systems. This is crucial in distributed systems where multiple tasks often need to be executed concurrently on same node or on different nodes.

- **Efficient networking:** Go provides a powerful standard library for networking, including HTTP servers and clients, and packages like **net/http** and **net/rpc** that simplify communication between distributed components. Its built-in support for TCP/IP and UDP makes it easy to create networked applications, which is fundamental for distributed systems.
- **Scalability:** Go's design favors scalability, both in terms of handling large numbers of concurrent requests and in its ability to scale horizontally across multiple machines. Its efficient use of system resources and support for lightweight threads (goroutines) make it suitable for deploying distributed systems that need to scale effectively.
- **Deployment and maintenance:** Go's statically linked binaries make deployment straightforward and reduce dependency management issues, which is advantageous when deploying and updating distributed systems across multiple nodes or containers.
- **Static types and compilation:** Go is statically typed and compiled to machine code, which results in faster execution compared to interpreted languages. This performance advantage is beneficial in distributed systems where efficiency and speed are crucial for handling large volumes of data and complex computations across multiple nodes. It also provides the ability to create different binaries across packages which can then be imported individually in other microservices leading to flexible usage and faster development.
- **Standard library support:** Go's standard library is extensive and includes packages for cryptography, encoding/decoding, data serialization (for example, JSON, protobuf), and more. These libraries simplify common tasks in building distributed systems, such as secure communication, data serialization between nodes, and handling various data formats.

Overall, Go's combination of concurrency support, efficient networking, performance, scalability, and robust standard library make it a strong choice for developing distributed systems that require high performance, scalability, and reliability.

Conclusion

This chapter brought out the nuances of distributed systems that add-on top of previous chapters knowledge about robust software development. It drives home the caveats for keeping in mind while developing and deploying software on a large-scale. Software architects need to account for concurrency, consistency and fault tolerance in the system design and plan for operational monitoring the system in totality.

Distributed systems have become essential backbone for building modern web applications as well as support cloud infrastructure and services. The models chosen for the different components of the distributed system can be based on the application behavior that is required to be supported.

The next chapter provides more depth into how the components of distributed systems can communicate with each other to achieve the goal of application functionality.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 6

Cross Service APIs

Introduction

Equipped with the background on distributed systems overview, let us take a further look at how the services deployed on such a system can work in tandem to accomplish the goal of the entire application. Services depending on each other need a mechanism to transfer information for making progress and delivering the right functionality to the user of the application.

Nothing in life is more important than the ability to communicate effectively.

-Gerald Ford, former US President

Just like in life, effective communication is the key to software systems functioning, scaling, and performing efficiently. We discuss some of the standard options such as **REpresentational State Transfer (REST)**, **Google remote procedure calls (gPRC)** and **Graph Query Language (GraphQL)** that are used in most current enterprises to achieve this need to communicate.

Structure

The chapter covers the following topics:

- History
- REpresentational State Transfer

- Google remote procedural calls
- GraphQL
- Comparing REST, gRPC and GraphQL

Objectives

The goal of the chapter is to provide the user with a concrete understanding of most prevalent cross service and process communication options. Following industry standards in such scenarios would help deliver any cross-service features in a very robust and streamlined fashion.

By the end of the chapter, software engineers will have the tools needed to build cross service APIs using the provided sample Golang code.

History

Message passing is a crucial concept in computer science, particularly in the context of **inter-process communication (IPC)**. The history of message passing between processes involves the development of various methods and systems to enable processes and services to communicate and synchronize their actions. In this section, we will discuss a brief overview of its history.

Early days: 1950s-1960s

In the earliest computing systems, programs on mainframe machines were executed in batch mode on different sets of data, without having the need for interaction between processes. Communication was not a primary concern for such use cases. With the advent of multiprogramming on single machine, multiple programs could reside in memory simultaneously, and the need for synchronization and communication became evident to ensure information and data get updated and used in a consistent manner.

Time-sharing systems: 1960s-1970s

MIT's Compatible Time-Sharing System (CTSS) was one of the first systems to introduce time-sharing, which allowed multiple users to interact

with the computer simultaneously. Basic forms of IPC began to emerge to support this client-server interaction. The **Multiplexed Information and Computing Service (Multics)** project, started in 1965 and introduced more sophisticated IPC mechanisms, including pipes.

UNIX pipes: 1970s

Developed by *AT&T Bell Labs* in the early 1970s, UNIX introduced the concept of pipes, which allowed for simple linear communication between processes running on the same computer. Pipes became a fundamental IPC mechanism in UNIX-like systems. UNIX also introduced other IPC mechanisms, such as signals, which allowed processes to send simple notifications to each other to allow semantics such as waiting for an event, getting notified about such event completion, and continuing execution.

Distributed networking: 1980s

The concept of **remote procedure call (RPC)** was developed, where a process can cause a procedure to execute in another address space. This was a significant building block for distributed systems where processes ran on different machines.

Systems like *IBM's MQSeries* (later known as *IBM MQ*, *Message Queue*) were also being developed, providing robust message queuing systems for reliable communication between distributed processes.

Parallel and distributed computing: 1990s

Message Passing Interface (MPI) was standardized in the early 1990s and became the dominant model for message passing in **high-performance computing (HPC)**. It provided a standard API for communication in parallel computing environments.

Common Object Request Broker Architecture (CORBA), developed by the **Object Management Group (OMG)** in the 1990s, allowed for communication using standard objects in a distributed system, regardless of the programming language or platform, allowing further flexibility for building messaging infrastructure.

Modern IPC mechanisms: 2000s-present

Message-oriented middleware (MOM) technologies like Apache Kafka, RabbitMQ, and ActiveMQ emerged, providing asynchronous, robust, scalable messaging solutions for modern distributed applications.

With the rise of web services, synchronous REST (RESTful) APIs became a common way for processes to communicate over HTTP for quick state data transfer. WebSockets provided a full-duplex communication channel over a single TCP connection as well.

Microservice mechanisms: 2010s-present

Modern microservices architectures rely heavily on message passing for communication between loosely coupled distributed services. Technologies like gRPC, a high-performance RPC framework, are often used to maintain a scalable messaging framework.

Similarly, cloud platforms and container orchestration systems like Kubernetes have integrated sophisticated messaging and service discovery mechanisms to facilitate IPC directly into cloud-native applications.

The evolution of message passing has paralleled the development of computing systems from single, isolated programs to complex, distributed systems. From the early days of batch processing to the sophisticated, high-performance, and scalable systems of today, IPC mechanisms have continuously evolved to meet the needs of ever-more complex applications and architectures.

Representational State Transfer

The **REpresentational State Transfer (REST)** protocol is a set of principles for designing information transfer across applications connected over a network. It was introduced by *Roy Fielding* in his 2000 doctoral dissertation, which aimed to describe an architectural style that could guide the design of modern web services with ease of use, extensibility, and scalability as the cornerstones. This section provides a detailed look at the evolution of the REST protocol.

Origins: 1990s

During the mid 1990s, the early web used HTTP/1.0, which was a simple protocol for transferring hypertext documents. HTTP/1.1 introduced improvements like persistent connections and chunked transfer encoding, setting the stage for more sophisticated interactions between clients and servers.

In late 1999s, *Roy Fielding*, one of the principal authors of the HTTP specification, introduced REST as an architectural style in his doctoral dissertation. REST was designed to guide the development of the web and web services by emphasizing scalability, statelessness, and a uniform interface.

Early adoption: 2000s

During early 2000s, **Simple Object Access Protocol (SOAP)** was the dominant protocol for web services, supported by major industry players. Developers began to recognize the simplicity and flexibility of REST compared to SOAP. Early adopters included *Amazon*, which introduced its S3 storage service using a RESTful API in 2006.

Early RESTful frameworks and tools began to emerge, simplifying the development of RESTful APIs. Examples include Ruby on Rails (introduced in 2004) and Django (introduced in 2005).

Mainstream adoption: 2010s

As it gained popularity, **JavaScript Object Notation (JSON)** became the de facto standard for data interchange in RESTful APIs, replacing XML due to its simplicity and compatibility with JavaScript. Best practices for designing RESTful APIs became widely accepted. These include using HTTP methods (GET, POST, PUT, DELETE), proper use of status codes, and designing resource-oriented APIs. Tools like Swagger (now OpenAPI), Postman, and API Gateway solutions (such as **Amazon Web Service (AWS) API Gateway**) emerged to help developers design, document, and manage RESTful APIs.

Current trends: 2020s

An often-underutilized aspect of REST, the addition of **Hypermedia as the Engine of Application State (HATEOAS)** enables a client to interact entirely with an application through hypermedia provided dynamically by application servers. This principle aims to make APIs more self-descriptive and discoverable.

Strategies for versioning RESTful APIs have evolved, with common approaches including URI versioning (e.g., /v1/resource), query parameter versioning, and header versioning. OAuth 2.0 and **JSON Web Tokens (JWT)** have become standard methods for securing RESTful APIs, ensuring secure and stateless authentication and authorization.

Serverless computing platforms like AWS Lambda and Azure Functions allow developers to build RESTful APIs without managing servers, leading to more scalable and cost-efficient applications, building on top of microservices as well. RESTful APIs are being used to expose machine learning models and AI services, making it easier to integrate intelligent capabilities into applications.

The REST protocol has evolved from a theoretical concept to a fundamental aspect of modern web and mobile applications. Its simplicity, scalability, and flexibility have made it the preferred choice for designing APIs. As technology continues to evolve, REST remains a crucial component of the digital ecosystem, adapting to new paradigms and technologies.

REST standards

The usage of REST APIs follows several established standards and best practices. These guidelines ensure consistency, reliability, and ease of use to quickly build and deliver software across different services within any business unit or company. This section provides the key standards and best practices for REST usage.

Resource identification

There should be a clear, consistent, and resource-oriented **Uniform Resource Identifier (URI)**, one of the target access points clients can call

using HTTP (example, curl) based commands. Resources that are tracked by the server should be identified using nouns rather than verbs for clients to call. The examples are as follows:

- **Ideal:** `/users/123`
- **Not recommended:** `/getUser?id=123` or `/getUser/123`

Resource relationships can also be used via nested resources. Nested URIs show relationships between specific resource types, as shown:

- **Example:** `/users/123/orders/456`

Here order number **256** is related to user **123** resource.

HTTP methods

Accessing most of the following endpoints can be supported by the server for REST resources of interest:

- **POST:** Create a new resource.
- **GET:** Retrieve a resource with a unique identifier or input filters.
- **PUT:** Update an existing resource or create a resource if it does not exist.
- **DELETE:** Remove the given resource with a unique identifier.
- **PATCH:** Apply partial modifications to a resource.

HTTP status codes

For each of the above method types, there could be different HTTP status codes that indicate the result of that API request. Some of the most common ones include:

- **200 OK:** The request was successful.
- **201 Created:** A new resource has been created.
- **204 No Content:** The request was valid and successful, but there is no representation to return (for example, after a DELETE).
- **400 Bad Request:** The request was malformed or invalid.
- **401 Unauthorized:** Authentication is required, and given one has failed validation, or it has not yet been provided.

- **403 Forbidden:** The request was valid, but the server refuses action due to internal rules.
- **404 Not Found:** The requested resource could not be found.
- **500 Internal Server Error:** A generic error message for server-side issues. The client cannot act on these but could potentially log a bug with service or follow-up.

The API should ideally also return a meaningful error message in the response body with appropriate HTTP status codes. For example:

```
1. {  
2.   "error": {  
3.     "code": 400,  
4.     "message": "Invalid request",  
5.     "details": "The 'email' field is required."  
6.   }  
7. }
```

Payload format

The best option is to use JSON for data interchange rather than XML. JSON is lightweight, easy to read, and widely supported, and it allows for better typesetting and faster serialization/deserialization during transfer. One can also use the Accept header to allow clients to specify the desired response format (for example, application/json) and also Content-Type header can be set in response with the same JSON value.

Statelessness

Each request from a client to the server must contain all the information the server needs to fulfil the request. The server should not store any session information about the client. If unable to parse or missing information in the input payload, the server can return a 400 error code, as stated above.

Versioning

One can also implement versioning to ensure backward compatibility when making changes to the API. This can be a special format like **vX** in the URI

format where **X** indicates a monotonically increasing version number. For example, **/v2/users/123** which might return different info than the **v1** version.

Security

The authentication mechanism for the API call should use OAuth 2.0 or JWT to secure APIs. The call should always use HTTPS to encrypt the data transmitted between the client and server.

Documentation

The API should provide clear and comprehensive documentation for the API using tools like Swagger (OpenAPI) or API Blueprint. Developers should also include examples for different endpoints, methods, request/response formats, and error codes.

Hypermedia as the engine of application state

API return payload can include hyperlinks in responses to provide clients with information on available actions and related resources and potential pagination with next/previous paging options. In this example, the user can use **orders** as the next API to explore or get the next set of records from the current API call:

```
1. {  
2.   "id": 123,  
3.   "name": "John Doe",  
4.   "links": [  
5.     {  
6.       "rel": "self",  
7.       "href": "/users/123"  
8.     },  
9.     {  
10.      "rel": "orders",  
11.      "href": "/users/123/orders"  
12.    },
```

```
13. {
14.   "next":
15.   {
16.     "href": "/users/123/orders"
17.   }
18. }
19. ]
20. }
```

More considerations

These are some of the more advanced options that API developers can enforce:

- Implement rate limiting to control the requests a client can make to your API for a given time period. API can return **429 Too Many Requests** when the rate limit is exceeded.
- Configure **cross-origin resource sharing (CORS)** to control which domains can access your API. This is particularly important for APIs accessed from web browsers.
- Ensure that PUT and DELETE operations are idempotent. This means that making the same request multiple times should have the same effect as making it once. Returning any errors for such noop operations usually does not provide much gain.
- Consider making POST operations idempotent when creating resources, using unique identifiers for the given payload.

Adhering to these standards and best practices can help you create robust, user—and developer-friendly RESTful APIs that are easy to maintain in the long run.

REST using Golang

Using REST in Go (Golang) programs involves creating a web server that can handle HTTP requests and respond appropriately. Go has a powerful standard library that includes packages like **net/http** for building web

servers and handling HTTP requests and responses. Here is a step-by-step guide to creating a simple REST API in Go:

1. Setup Go project:

First, create a directory for your project and initialize a Go module, as shown:

1. `mkdir first-rest-api`
2. `cd first-rest-api`
3. `go mod init first-rest-api`

2. Install dependencies:

While Go's standard library is powerful, you might want to use some external libraries to handle JSON and routing more efficiently. For this example, we will use **gorilla/mux** for routing:

1. `go get -u github.com/gorilla/mux`

3. Create REST APIs:

Add the following in a **main.go** file in the project directory above:

1. `package main`
- 2.
3. `import (`
4. `"encoding/json"`
5. `"log"`
6. `"net/http"`
7. `"github.com/gorilla/mux"`
8. `)`
- 9.
10. `type Book struct {`
11. `ID string `json:"id"``
12. `Title string `json:"title"``
13. `Content string `json:"content"``
14. `}`
- 15.

```
16. var books []Book
17.
18. func getBooks(w http.ResponseWriter, r *http.Request) {
19.     w.Header().Set("Content-Type", "application/json")
20.     json.NewEncoder(w).Encode(books)
21. }
22.
23. func getBook(w http.ResponseWriter, r *http.Request) {
24.     w.Header().Set("Content-Type", "application/json")
25.     params := mux.Vars(r)
26.     for _, item := range books {
27.         if item.ID == params["id"] {
28.             json.NewEncoder(w).Encode(item)
29.             return
30.         }
31.     }
32.     http.Error(w, "Book not found", http.StatusNotFound)
33. }
34.
35.
36. func createBook(w http.ResponseWriter, r *http.Request) {
37.     w.Header().Set("Content-Type", "application/json")
38.     var book Book
39.     _ = json.NewDecoder(r.Body).Decode(&book)
40.     books = append(books, book)
41.     json.NewEncoder(w).Encode(book)
42. }
43.
44. func updateBook(w http.ResponseWriter, r *http.Request) {
```

```
45. w.Header().Set("Content-Type", "application/json")
46. params := mux.Vars(r)
47. for index, item := range books {
48.     if item.ID == params["id"] {
49.         books = append(books[:index], books[index+1:]...)
50.         var book Book
51.         _ = json.NewDecoder(r.Body).Decode(&book)
52.         book.ID = params["id"]
53.         books = append(books, book)
54.         json.NewEncoder(w).Encode(book)
55.         return
56.     }
57. }
58. http.Error(w, "Book not found", http.StatusNotFound)
59. }
60.
61. func deleteBook(w http.ResponseWriter, r *http.Request) {
62.     w.Header().Set("Content-Type", "application/json")
63.     params := mux.Vars(r)
64.     for index, item := range books {
65.         if item.ID == params["id"] {
66.             books = append(books[:index], books[index+1:]...)
67.             break
68.         }
69.     }
70.     json.NewEncoder(w).Encode(books)
71. }
72.
73. func main() {
```

```

74. r := mux.NewRouter()
75. books = append(books, Book{ID: "1", Title: "Book One",
76.                  Content: "Content of book one"})
77. books = append(books, Book{ID: "2", Title: "Book Two",
78.                  Content: "Content of book two"})
79.
80.
81. r.HandleFunc("/books", getBooks).Methods("GET")
82. r.HandleFunc("/books/{id}", getBook).Methods("GET")
83. r.HandleFunc("/books", createBook).Methods("POST")
84. r.HandleFunc("/books/{id}", updateBook).Methods("PUT")
85. r.HandleFunc("/books/{id}", deleteBook).Methods("DELETE")
86.
87. log.Fatal(http.ListenAndServe(":8000", r))
88. }

```

One can take up the exercise to enhance the returned objects to include links.

4. Run the Go REST API:

Run your Go program to start the server in the top-level directory of your project directory:

```
1. go run main.go
```

The REST server will be running at **http://localhost:8000** endpoint.

Accessing the API

To test the above API endpoints, one can use tools like curl or Postman application UI. The following are some curl-based command-line examples:

- **Get all stored books:**

```
1. curl -X GET http://localhost:8000/books
```

- **Get a specific book:**

```
1. curl -X GET http://localhost:8000/books/1
```


- **Insert/create a new book:**

```
1. curl -X POST http://localhost:8000/books -H "Content-Type: application/json" -d '{"id": "3", "title": "Book Three", "content": "Content of book number three"}'
```

- **Update a book:**

```
1. curl -X PUT http://localhost:8000/books/1 -H "Content-Type: application/json" -d '{"title": "Updated Article", "content": "Updated content for book one"}'
```

- **Delete an existing book:**

```
1. curl -X DELETE http://localhost:8000/book/1
```

This basic setup demonstrates how to create a simple REST API using Go and the **gorilla/mux** package. For production applications, you may need to consider additional factors such as error handling, logging, security (for example, authentication, HTTPS), and scalability.

Google remote procedure calls

gRPC is a high-performance, open-source framework developed by *Google*. It enables communication between client and server applications by providing an efficient and robust framework for RPC. Just like REST, the evolution of gRPC can be traced through several key milestones and advancements in the field of distributed systems and communication protocols.

Origins and development at Google: 2000s

Before gRPC, *Google* developed and used an internal RPC framework called **Stubby**, which provided the foundation for gRPC. Stubby was designed to meet *Google* requirements for high performance, scalability, and low latency in their distributed systems. Around the same time, *Google* developed **Protocol Buffers (protobuf)** as a language-neutral, platform-neutral, extensible mechanism for serializing structured data. protobuf became a core part of gRPC, allowing efficient data serialization and

deserialization, which is desired for network transfer and receive performance.

Launch of gRPC: 2010-2015

In February 2015, *Google* open-sourced gRPC, releasing it to the public as version 1.0. gRPC was built on top of HTTP/2, which provided several key features such as multiplexing, flow control, header compression, and bi-directional communication. The initial release of gRPC included features like support for multiple programming languages (including C++, Java, Go, and Python), code generation from protobuf files, and support for bi-directional streaming and asynchronous communication.

Community adoption and growth: 2016-2018

gRPC community quickly expanded, with contributors adding support for additional programming languages, including Node.js, Ruby, PHP, and more. This broad language support made gRPC an attractive choice for polyglot environments. Major cloud providers, including *Google Cloud*, *Microsoft Azure*, and *AWS*, started offering native support for gRPC in their services, further driving its adoption in cloud-native applications.

Projects like Istio and Linkerd began integrating gRPC into their service mesh architectures, enabling advanced features like load balancing, service discovery, and observability for gRPC services.

Advanced features: 2019-2020

The gRPC-Web project was introduced to enable gRPC communication from web browsers. This allows developers to use gRPC in client-side applications, expanding its reach to web-based frontends.

gRPC-Gateway project provided a way to generate RESTful HTTP APIs from gRPC services, allowing seamless interoperability between REST and gRPC clients.

The gRPC team and community continued to optimize performance and scalability, making gRPC suitable for high-throughput, low-latency applications. Features like client-side and server-side load balancing,

advanced flow control, and improved connection management were introduced.

Widespread adoption: 2020-present

By this time, gRPC had established a mature ecosystem with extensive tooling, libraries, and frameworks for various use cases, including observability (for example, OpenTelemetry), security (for example, mTLS), and service discovery. gRPC also became popular for microservices architectures due to its efficiency, strong typing, and support for streaming and bi-directional communication. It is used by companies like *Netflix*, *Square*, *LinkedIn* and *Dropbox* in their microservices architectures.

The gRPC community continues to evolve and enhance the framework, with ongoing discussions and developments around features like service mesh integration, support for newer versions of HTTP, and improvements in tooling and developer experience.

To summarize, gRPC has evolved from *Google's* internal RPC framework to a widely adopted, open-source, high-performance communication framework. Its use of HTTP/2 and protobuf, broad language support, and advanced features like streaming, load balancing, and integration with cloud-native tools have made it a popular choice for modern distributed systems. The continued growth of its ecosystem and community ensures that gRPC remains at the forefront of RPC technologies.

gRPC using Golang

Using gRPC in Go (Golang) involves several steps, from defining the service using protobufs to implementing the server and using the service calls in a client. This section provides a comprehensive guide to getting started with gRPC in Go.

Install prerequisites

You need to install the **protobuf compiler (protoc)** and the Go plugin for using gRPC:

- **Install protoc:** Download the appropriate version for your OS from the [protobuf releases](#) page at

<https://github.com/protocolbuffers/protobuf/releases>. Unzip and move the protoc binary to a directory in your **PATH**. Install Go plugins for protobuf:

1. `go install google.golang.org/protobuf/cmd/protoc-gen-go@latest`
2. `go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest`
- Add the binaries to your **PATH** (if they are not already):
 1. `export PATH="$PATH:$(go env GOPATH)/bin"`

Define protocol buffers

Create a **.proto** file to define your gRPC service and messages in a chosen Go repository. For example, create a file named **helloworld.proto** with the following content:

```
1. syntax = "proto3"
2.
3. option go_package = "example.com/helloworld"
4.
5. package helloworld
6.
7. // The greeting service definition.
8. service Greeter {
9.   // Sends a greeting
10.  rpc SayHello (HelloRequest) returns (HelloReply)
11. }
12.
13. // The request message containing the user's name.
14. message HelloRequest {
15.   string name = 1
16. }
17.
18. // The response message containing the greeting.
19. message HelloReply {
```

```
20. string message = 1
21. }
```

Go code from the .proto file

Generate the Go code using the protoc with the gRPC plugin:

```
1. protoc --go_out=. --go-grpc_out=. helloworld.proto
```

This command generates **helloworld.pb.go** and **helloworld_grpc.pb.go** files.

Server implementation

We will use the above generated proto Go content to create a **server.go** to implement the server logic for the service:

```
1. package main
2.
3. import (
4.     "context"
5.     "log"
6.     "net"
7.     "google.golang.org/grpc"
8.     pb "example.com/helloworld"
9. )
10.
11. const (
12.     port = ":50051"
13. )
14.
15. // server is used to implement helloworld.GreeterServer.
16. type server struct {
17.     pb.UnimplementedGreeterServer
18. }
19.
```

```

20. // SayHello implements helloworld.GreeterServer
21. func (s *server) SayHello(ctx context.Context,
22.   in *pb.HelloRequest) (*pb.HelloReply, error) {
23.   log.Printf("Received: %v", in.GetName())
24.   return &pb.HelloReply{Message: "Hello " + in.GetName()}, nil
25. }
26.
27. func main() {
28.   lis, err := net.Listen("tcp", port)
29.   if err != nil {
30.     log.Fatalf("failed to listen: %v", err)
31.   }
32.   s := grpc.NewServer()
33.   pb.RegisterGreeterServer(s, &server{})
34.   log.Printf("server listening at %v", lis.Addr())
35.   if err := s.Serve(lis); err != nil {
36.     log.Fatalf("failed to serve: %v", err)
37.   }
38. }

```

Emulate client calls

Create a Go file named **client.go** to implement the calls from a typical client using the above service:

```

1. package main
2.
3. import (
4.   "context"
5.   "log"
6.   "os"
7.   "time"
8.   "google.golang.org/grpc"

```

```
9.  pb "example.com/helloworld"
10. )
11.
12. const (
13.     address    = "localhost:50051"
14.     defaultName = "world"
15. )
16.
17. func main() {
18.     // Set up a connection to the server.
19.     conn, err := grpc.Dial(address, grpc.WithInsecure(),
20.                             grpc.WithBlock())
21.     if err != nil {
22.         log.Fatalf("did not connect: %v", err)
23.     }
24.     defer conn.Close()
25.     c := pb.NewGreeterClient(conn)
26.
27.     // Contact the server and print out its response.
28.     name := defaultName
29.     if len(os.Args) > 1 {
30.         name = os.Args[1]
31.     }
32.     ctx, cancel := context.WithTimeout(context.Background(),
33.                                         time.Second)
34.     defer cancel()
35.     resp, err := c.SayHello(ctx, &pb.HelloRequest{Name: name})
36.     if err != nil {
37.         log.Fatalf("could not greet: %v", err)
38.     }
39.     log.Printf("Greeting: %s", resp.GetMessage())
```

```
40. }
```

Run the server and client

To run the server:

```
1. go run server.go
```

To run the client:

```
1. go run client.go
```

You should see the client sending a greeting request to the server and receiving a response via the log messages.

This guide provides a basic introduction to using gRPC in Go. By defining a service in a **.proto** file, generating the corresponding Go code, and implementing the server and client, you can create a powerful and efficient RPC framework. gRPC supports advanced features such as streaming, authentication, and load balancing, which you can explore further in the official gRPC documentation at <https://grpc.io/docs/>.

GraphQL

GraphQL is a query language for APIs and a runtime for executing those queries by using a type system defined on the data. GraphQL was developed by *Facebook* in 2012 and released as an open-source project in 2015. It has become a popular alternative to REST for designing web APIs. This section provides an overview of the evolution of GraphQL.

Origins at Facebook: 2012-2015

GraphQL was developed internally at *Facebook* in 2012. The goal was to address the limitations of REST in mobile applications, where the over-fetching and under-fetching of data often led to performance issues. *Facebook* aimed to improve the efficiency and performance of their mobile applications by allowing clients to request exactly the data they needed and no more. This led to the creation of GraphQL, which allows clients to specify the shape and structure of the response.

Community growth: 2015-2016

In 2015, *Facebook* open-sourced GraphQL. The release included a specification, reference implementation in JavaScript, and documentation. The developer community quickly embraced GraphQL due to its flexibility, efficiency, and ease of use. Several early adopters contributed to the growing ecosystem by creating tools and libraries for various programming languages.

Ecosystem expansion: 2017-2018

Key tools like *Apollo Client* and *Relay* (both for JavaScript) were developed to simplify the integration of GraphQL in frontend applications. These tools provide powerful features like caching, pagination, and query batching. The GraphQL ecosystem expanded to include libraries and tools for many programming languages, such as GraphQL.js (JavaScript), Graphene (Python), graphql-java (Java), Sangria (Scala), and more. Companies like *GitHub*, *Shopify*, and *Twitter* began adopting GraphQL for their public APIs, demonstrating its viability for large-scale applications.

Advancements and enterprise adoption: 2018-2021

In 2018, the GraphQL Foundation was established under the *Linux Foundation* to foster the growth and adoption of GraphQL. This move aimed to ensure the open governance and sustainability of GraphQL.

The GraphQL specification continued to evolve with new features and improvements. For example, the addition of subscriptions enabled real-time updates, enhancing GraphQL's capabilities for real-time applications. Tools like GraphQL Playground and GraphiQL (interactive in-browser GraphQL IDEs) made it easier for developers to explore and test their APIs.

Apollo introduced *Apollo Federation*, a solution for building a distributed GraphQL architecture. This allowed companies to break down monolithic GraphQL schemas into manageable services while still presenting a unified API to clients. Cloud providers like *AWS*, *Azure*, and *Google Cloud* started offering managed GraphQL services (e.g., *AWS AppSync*, *Azure Static Web Apps* with GraphQL support). These services made it easier for developers to deploy, monitor, and scale GraphQL APIs.

Latest innovations: 2021-present

GraphQL has become a mainstream technology used in production by many companies, ranging from startups to large enterprises. Its flexibility and efficiency make it suitable for various use cases, from content delivery networks to real-time applications. The GraphQL community has also developed best practices and patterns for schema design, error handling, authentication, and performance optimization. These practices help ensure the stability and scalability of GraphQL APIs.

Tools like GraphQL Mesh enable developers to unify multiple data sources (which use REST, SOAP, gRPC, or databases) under a single GraphQL API, making it easier to integrate and query diverse data sources.

GraphQL has evolved from an internal solution at *Facebook* to a widely adopted technology for API design and development. Its ability to allow clients to request exactly the data they need, combined with powerful developer tools and a growing ecosystem, has made it a popular choice for modern web and mobile applications. The ongoing development and support from the GraphQL Foundation and the wider community ensure that GraphQL will continue to innovate and address the needs of developers and enterprises alike.

GraphQL using Golang

Using GraphQL in Go involves several steps, including setting up a GraphQL server, defining your schema, and implementing resolvers. Here is a comprehensive guide to getting started with GraphQL in Go:

1. **Initialize a Go project:** Ensure you have Go installed. You can download it from <https://golang.org/dl/>.

1. `mkdir my-graphql-app`
2. `cd my-graphql-app`
3. `go mod init example.com/my-graphql-app`

2. **Install GraphQL-Go library:**

1. `go get github.com/graph-gophers/graphql-go`

3. **Define schema:** Create a file named `schema.graphql` to define sample

GraphQL schema as below:

```
1. type Query {  
2.   hello: String!  
3. }
```

4. **Create resolvers:** Implement resolvers for your schema in Go. Create a file named **main.go**:

```
1. package main  
2.  
3. import (  
4.   "context"  
5.   "log"  
6.   "net/http"  
7.  
8.   "github.com/graph-gophers/graphql-go"  
9.   "github.com/graph-gophers/graphql-go/relay"  
10. )  
11.  
12. // Schema definition  
13. const schemaString = `  
14.   type Query {  
15.     hello: String!  
16.   }  
17. `  
18.  
19. // Resolver  
20. type resolver struct{}  
21.  
22. func (r *resolver) Hello() string {  
23.   return "Hello, world!"  
24. }  
25.
```

```

26. func main() {
27.     // Parse schema
28.     schema := graphql.MustParseSchema(schemaString, &resolver{
29.     })
30.     // Set up HTTP handler
31.     http.Handle("/graphql", &relay.Handler{Schema: schema})
32.
33.     // Start HTTP server
34.     log.Println("Now server is running on port 8080")
35.     log.Fatal(http.ListenAndServe(":8080", nil))
36. }

```

5. **Run the server:** Run your Go application to start the GraphQL server:

```
1. go run main.go
```

Your GraphQL server is now running on **http://localhost:8080/graphql**.

6. **Query GraphQL API:** You can use a tool like GraphiQL (at <https://github.com/graphql/graphiql>) or Postman (at <https://www.postman.com/>) to send queries to your GraphQL server.

For example, you can send the following query:

```

1. {
2.   hello
3. }

```

This will return the current value for just that **hello** field.

Advanced features

For a more complex application, you might want to include more advanced features like mutations, input types, and complex queries. Here is an example of adding a mutation, which allows updating a field and querying back the value in the same request:

- Update the schema in **schema.graphql**:

```
1. type Query {
2.   hello: String!
3. }
4.
5. type Mutation {
6.   createMessage(input: MessageInput!): Message!
7. }
8.
9. input MessageInput {
10.  text: String!
11.  user: String!
12. }
13.
14. type Message {
15.  id: ID!
16.  text: String!
17.  user: String!
18. }
```

- Update resolvers in **main.go**:

```
1. package main
2.
3. import (
4.   "context"
5.   "log"
6.   "net/http"
7.   "sync"
8.
9.   "github.com/graph-gophers/graphql-go"
10.  "github.com/graph-gophers/graphql-go/relay"
11. )
```

```
12.
13. type message struct {
14.     ID string
15.     Text string
16.     User string
17. }
18.
19. // Schema definition
20. const schemaString = `
21.     type Query {
22.         hello: String!
23.     }
24.
25.     type Mutation {
26.         createMessage(input: MessageInput!): Message!
27.     }
28.
29.     input MessageInput {
30.         text: String!
31.         user: String!
32.     }
33.
34.
35.     type Message {
36.         id: ID!
37.         text: String!
38.         user: String!
39.     }
40. `
41.
42. // Resolver
```

```
43. type resolver struct {
44.     messages []*message
45.     mu       sync.Mutex
46. }
47.
48. func (r *resolver) Hello() string {
49.     return "Hello, world!"
50. }
51.
52. func (r *resolver) CreateMessage(ctx context.Context,
53.     args struct{ Input *message }) *message {
54.     r.mu.Lock()
55.     defer r.mu.Unlock()
56.
57.     newMessage := &message{
58.         ID: "1",
59.         Text: args.Input.Text,
60.         User: args.Input.User,
61.     }
62.     r.messages = append(r.messages, newMessage)
63.     return newMessage
64. }
65.
66. func main() {
67.     // Parse schema
68.     schema := graphql.MustParseSchema(schemaString, &resolver{
69.     })
70.     // Set up HTTP handler
71.     http.Handle("/graphql", &relay.Handler{ Schema: schema})
72.
```

```
73. // Start HTTP server
74. log.Println("Now server is running on port 8080")
75. log.Fatal(http.ListenAndServe(":8080", nil))
76. }
```

Using GraphQL in Go involves defining your schema using GraphQL syntax, implementing resolvers in Go, and setting up an HTTP server to handle GraphQL requests. The **graphql-go** library provides the necessary tools to parse the schema and handle queries and mutations. With this setup, you can build powerful and flexible APIs that allow clients to request exactly the data they need.

Comparing REST, gRPC and GraphQL

Choosing between REST, gRPC, and GraphQL depends on the specific requirements of your project. Each technology has its strengths and weaknesses and is suited to different applications and use cases. Here is a detailed comparison of REST, gRPC, and GraphQL:

Protocol and data format

In the case of protocol and data format:

- **REST:**
 - Uses HTTP/1.1 as the transport protocol.
 - Typically uses JSON for data interchange.
 - Human-readable and easy to debug.
- **gRPC:**
 - Uses HTTP/2 as the transport protocol, which provides benefits like multiplexing, flow control, header compression, and bi-directional streaming.
 - Uses protobuf for data serialization, which is compact and efficient.
 - Binary format is more efficient but less human-readable.

- **GraphQL:**
 - Uses HTTP (typically HTTP/1.1) as the transport protocol.
 - Uses JSON for data interchange.
 - Human-readable and provides flexible queries, on par with REST offerings.

Performance

In the case of performance:

- **REST:**
 - Generally slower due to the verbosity of JSON and the overhead of HTTP/1.1. Though Go ecosystem provides libraries for fast JSON conversion.
 - Limited to request-response communication.
 - Allows use of **Data Transfer Objects (DTOs)** pattern to reduce data transfer.
- **gRPC:**
 - Typically, faster due to the efficiency of HTTP/2 and ProtoBuf.
 - Supports multiple communication patterns: unary (request-response), server streaming, client streaming, and bi-directional streaming.
- **GraphQL:**
 - Can be slower for simple queries due to query parsing and execution overhead.
 - Allows clients to request only the data they need, potentially reducing the amount of data transferred.

Development and tooling

In the case of development and tooling:

- **REST:**

- Easier to implement and understand due to its simplicity and widespread use.
- Rich ecosystem with many tools and libraries for all major programming languages as it is a more mature product.
- Well-supported by web browsers.
- **gRPC:**
 - Requires learning protobuf and understanding HTTP/2.
 - Excellent tooling support, but primarily targeted at backend services rather than web browsers.
 - Tools like protoc generate client and server code for multiple languages, ensuring consistency.
- **GraphQL:**
 - Requires understanding of GraphQL schema and query language.
 - Tools like *Apollo Client* and *GraphiQL* make it easier to explore and test GraphQL APIs.
 - Rich ecosystem with libraries and tools for various languages.

Compatibility and interoperability

In the case of compatibility and interoperability:

- **REST:**
 - Highly compatible with web technologies.
 - Easily consumed by web clients (browsers, JavaScript frameworks).
 - JSON is widely supported and easily understood.
- **gRPC:**
 - Requires gRPC-Web for browser support, which is less mature compared to native REST support.
 - Best suited for microservices and internal communications within data centers.

- **GraphQL:**
 - Highly compatible with web technologies, same as REST.
 - Easily consumed by web clients.
 - JSON is widely supported and easily understood.

Streaming and real-time communication

In the case of streaming and real-time communication:

- **REST:**
 - Primarily supports request-response model.
 - Real-time communication requires minor additional setups like WebSockets or **server-sent events (SSE)**.
- **gRPC:**
 - Natively supports streaming, which makes it suitable for real-time communication scenarios.
 - Supports server streaming, client streaming, and bi-directional streaming out of the box.
- **GraphQL:**
 - Supports real-time updates with subscriptions, which are typically implemented using WebSockets.

Error handling

In the case of error handling:

- **REST:**
 - Uses HTTP status codes to indicate success or failure (e.g., 200 OK, 404 Not Found, 500 Internal Server Error).
 - Error details are often included in the response body in JSON format.
- **gRPC:**
 - Uses its own status codes, which map to HTTP/2 status codes.

- Error details can be included in the response using protobuf.
- **GraphQL:**
 - Errors are included in the response JSON under an errors field.
 - Allows for partial responses, even with errors.

Security

In the case of security:

- **REST:**
 - Uses standard HTTP security mechanisms (e.g., TLS/SSL for encryption, OAuth for authentication).
- **gRPC:**
 - Supports TLS/SSL for encryption and can be integrated with various authentication mechanisms.
 - Benefits from the built-in security features of HTTP/2.
- **GraphQL:**
 - Uses standard HTTP security mechanisms (e.g., TLS/SSL for encryption, OAuth for authentication).
 - Requires additional measures to protect against complex queries and introspection.

Types of use cases

In the scenario of types of use cases:

- **REST:**
 - Ideal for public APIs, where interoperability and ease of use are critical.
 - Suitable for applications where human readability and ease of testing/debugging are important.
- **gRPC:**
 - Ideal for microservices architectures, where performance and

efficiency are critical, especially for larger payloads.

- Suitable for internal APIs within a data center, where binary formats and streaming are beneficial.
- Real-time services that require streaming data (e.g., chat applications, live data feeds).
- **GraphQL:**
 - Ideal for APIs where clients need flexibility in querying data across data sources.
 - Suitable for applications with complex data requirements and relationships.
 - Real-time applications that benefit from subscriptions.

Summary

In conclusion, based on the comparisons across these three options, developers can use the following heuristics to pick the right RPC type for their requirements:

- Choose REST if you need a simple, widely compatible API that is easy to implement and debug. It is ideal for web applications and public APIs.
- Choose gRPC if you need high performance and efficiency, especially for internal microservices communication. It is well-suited for real-time and low-latency applications.
- Choose GraphQL if you need flexible querying capabilities and efficient data fetching, particularly for applications with complex data relationships and requirements.

Conclusion

In this chapter, we discussed the main options for cross-service APIs to help build scalable services and applications. Each technology has its place in modern API design, and the best choice depends on your application's specific needs and context. It is also possible to use a combination of them across services with very large-scale applications.

Communication mechanisms are essential, and the content being transferred also needs to be semantically sound. With the background gained on the mechanisms used for data transfer, let us discover in the next chapter how that data can be stored on the server and used for various access patterns.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 7

Data Modeling

Introduction

Data is an integral part of distributed systems that need to store and transfer information across services for end-user application consumption or offline processing. The volume and velocity of new data generated across systems have increased tremendously in the past two decades. With the advent of real world applications using AI, data growth and usage are bound to increase.

Those who rule data will rule the entire world.

—Masayoshi Son, CEO SoftBank

Given data's importance in Web 3.0, it is necessary to make it efficiently and securely ready for use by any application. We will discuss various persistence stores and the trade-offs between traditional relational databases and streaming services and the newer general NewSQL/Distributed SQL options.

Structure

The chapter covers the following topics:

- History
- Data entity relationship
- Using data modeling
- NoSQL data modeling

- Golang database connectors
- Golang data modeling support
- Data streaming solutions

Objectives

Within this chapter, the reader will first get an overview of the various data modeling evolution over the past five decades.

Relational modeling will lay the foundation for creating data models with stringent data format requirements. Then, we see how this evolved into more semi-structured data from Web 2.0 apps needing big data handling and streaming platforms that allow real-time analytics.

We will conclude the chapter with sample data connectors that can be used in Go to work with various data stores at scale.

History

Let us start with the usual historical perspective. Several key phases have marked the evolution of data modeling, each reflecting advancements in technology, data management needs, and business requirements. Here is an overview of the major stages:

- **Hierarchical model (1960s):** One of the first data models, exemplified by *IBM Information Management System (IMS)*. Data is organized in a tree-like structure with parent-child relationships.
 - **Pros:** Fast data retrieval for hierarchical data.
 - **Cons:** Not flexible and difficult in representing many-to-many relationships.
- **Network model (1970s):** Developed by the **Conference on Data Systems Languages (CODASYL)**. More flexible than the hierarchical model, allowing many-to-many relationships. Data is represented using graphs with nodes (records) and edges (links).
 - **Pros:** Better handling of complex relationships.
 - **Cons:** Complexity in design and maintenance.

- **Relational model (1970s-1980s):** This was introduced by *E.F. Codd* in 1970. Data is organized in tables (relations) with rows (tuples) and columns (attributes). Uses **Structured Query Language (SQL)** for data manipulation. Became the dominant model for databases, leading to the development of **relational database management systems (RDBMS)** like *Oracle*, *MySQL*, and *SQL Server*.
 - **Pros:** Simplicity, flexibility, and powerful query capabilities.
 - **Cons:** Less scalable and costly.
- **Object-oriented model:** Integrates object-oriented programming concepts with databases. Data is represented as objects, similar to objects in programming languages like Java and C++.
 - **Pros:** Better alignment with application code, supporting complex data types and inheritance.
 - **Cons:** Less mature than relational databases and less support from industry standards.
- **Object-relational model:** Combines features of both relational and object-oriented models. Extends RDBMS to support complex data types and object-oriented concepts. This offers the robustness of RDBMS with additional flexibility for complex data types.
- **NoSQL databases (mid-2000s):** Developed to handle the vast amounts of unstructured and semi-structured data generated by web applications, social media, and big data analytics. Includes various types such as document stores (for example, MongoDB), key-value stores (for example, Redis), column-family stores (for example, Cassandra), and graph databases (for example, Neo4j).
 - **Pros:** Scalability, flexibility, and performance for specific use cases.
 - **Cons:** Lack of standardization and limited support for complex queries compared to RDBMS.
- **Big data technologies (2010s):** Tools and frameworks designed to process and analyze large volumes of data. Examples include Hadoop (for distributed storage and processing), Spark (for in-memory data

processing), and various ecosystem tools like Hive, Pig, and HBase.

- **Pros:** Ability to handle massive datasets and perform complex analytics.
- **Cons:** Complexity in setup and management, and the need for specialized skills.
- **Graph databases (2010s):** Specialized databases optimized for handling graph data structures. Suitable for applications like social networks, recommendation engines, and fraud detection.
 - **Pros:** Efficiently handles complex relationships and graph traversals.
 - **Cons:** Niche use cases and less mature than traditional RDBMS.
- **Cloud-based data solutions (mid 2010s):** Data storage and processing solutions provided as cloud services (for example, *AWS*, *Azure*, *Google Cloud*). Offers scalability, flexibility, and managed services.
 - **Pros:** Reduces infrastructure management overhead and supports elastic scaling.
 - **Cons:** Concerns about data security and cost management.

The evolution of data modeling reflects the changing landscape of technology and business needs. From hierarchical and network models to relational, object-oriented, NoSQL, and big data solutions, each phase has brought advancements that address specific challenges in data management. Modern trends continue to push the boundaries, emphasizing scalability, flexibility, and real-time processing capabilities.

Data entity relationship

Data **entity relationship (ER)** is a conceptual representation of the data structures and their relationships within a system or database. It is a crucial aspect of data modeling used to design and visualize how data is interconnected in relational database use cases. The following are the key components of entity-relationship models:

- **Entities:** These are objects or things in the real world that have distinct

existence. Each entity has a set of attributes. In a hospital database, entities might include doctors, hospital rooms and patients.

- **Attributes:** Characteristics or properties of an entity. Each entity can have multiple attributes. A doctor entity might have attributes like **DoctorID**, **Name**, **DateOfBirth**, and **Specialty**.
- **Relationships:** Associations or interactions between entities. They represent how entities are related to one another. Some of types of Relationships include:
 - **One-to-One (1:1):** Each instance of entity *A* is associated with only one instance of entity *B*.
 - **One-to-Many (1:N):** Each instance of entity *A* is associated with multiple instances of entity *B*.
 - **Many-to-Many (M:N):** Instances of entity *A* can be associated with multiple instances of entity *B* and vice versa.

Entity-relationship diagram

The primary tool for the representation of ER is the **entity-relationship diagram (ERD)**. An ERD is a visual representation of the data entities, their attributes, and the relationships between them. It helps in understanding and designing the structure of a relational database. The main components of an ERD are usually represented as follows:

- **Entities:** Represented by rectangles. Labeled with the entity name.
- **Attributes:** Represented by ovals connected to their respective entities. Labeled with the attribute name.
- **Relationships:** Represented by diamonds. Connected to the entities involved in the relationship. Labeled with the relationship name.
- **Cardinality:** Indicates the number of instances of one entity related to one instance of another entity. Represented by notation like 1:1, 1:N, or M:N near the relationship lines.

Figure 7.1 shows the sample ERD example for a hospital database covering the following entities and relationships:

- **Entities:**

- Doctor (**DoctorID**, Name, Specialty, Salary)
- Patient (**PatientID**, Name, DateOfBirth, Allergies, Medications)
- Rooms (**RoomID**, Building, Floor)
- **Relationships:**
 - Visits (Patient—Doctor, M:N)
 - Manages (Doctor—Rooms, 1:N)

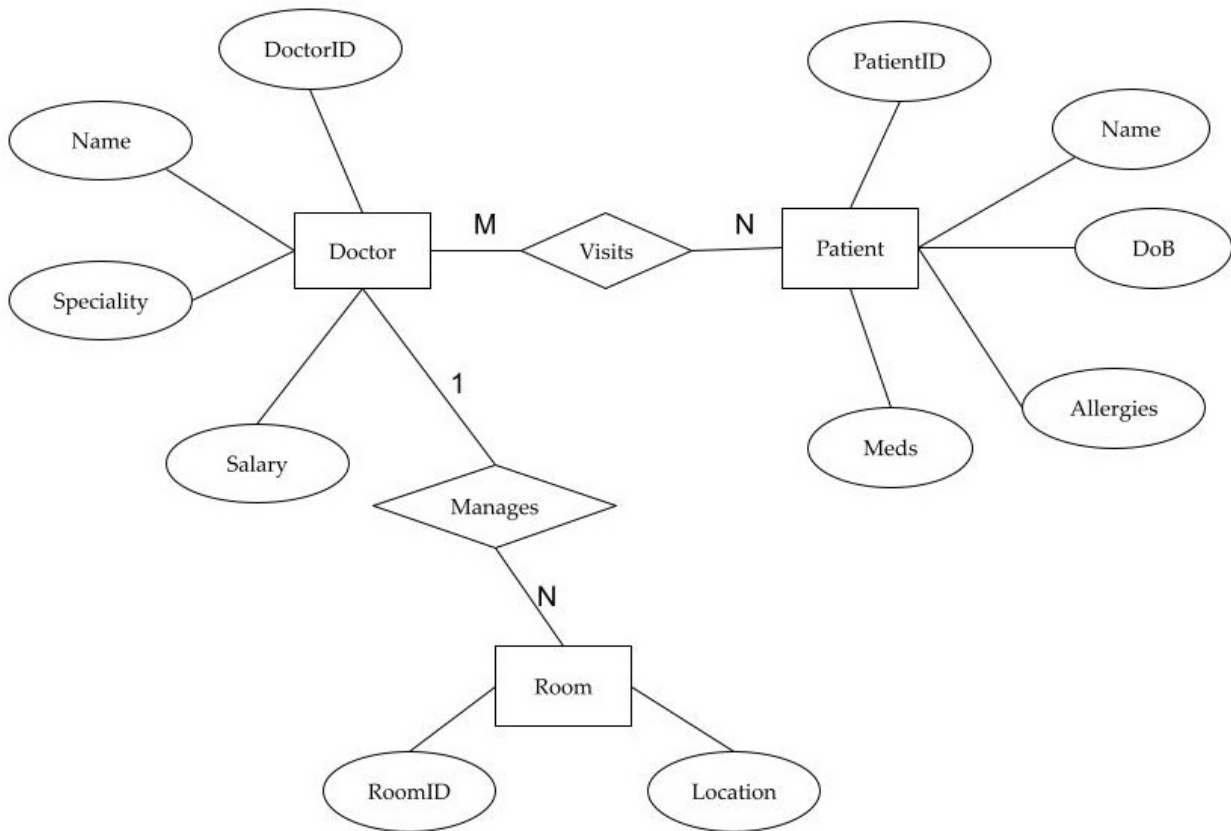


Figure 7.1: ERD visualization of a hospital database

Benefits of using ERDs

ERDs provide the following benefits:

- **Clarity:** Provides a clear visualization of the data structure and relationships, making it easier to understand and communicate.
- **Design:** Helps in the design phase of database development, ensuring a well-structured and efficient database.

- **Consistency:** Aids in maintaining data integrity and consistency by clearly defining relationships and constraints.
- **Documentation:** Acts as a reference for developers and stakeholders throughout the development and maintenance of the system.

ERD modeling is a foundational aspect of database usage, providing a structured approach to understanding and visualizing the relationships between different data entities. ERDs are essential tools for database architects, developers, and analysts to ensure that databases are well-organized, easy to understand, scalable, and aligned with business requirements.

Using data modeling

Applications use data modeling as a fundamental process for defining and organizing the data structures required by the application. Effective data modeling ensures that the data is accurate, consistent, and can be efficiently retrieved and manipulated. The following are the key steps and considerations for how software engineers should proceed with data modeling aspects:

- **Requirements analysis:** The first step involves gathering requirements from stakeholders to understand the business needs and data usage. Then the data designer determines the main entities (objects) and their relationships within the application. These needs engaging with business analysts, developers, and end-users throughout the data modeling process to ensure the model meets all requirements.
- **Conceptual data modeling:** The next step is to develop an ERD (as seen in the previous section) to visualize the high-level structure of the data, including entities, attributes, and relationships. One can identify all entities and their attributes, ensuring they reflect the business requirements. Then, define the relationships between entities, including cardinality and participation constraints.
- **Logical data modeling:** The next stage is to go one level deeper to ensure the data model eliminates redundancy and dependency by following normalization rules. One can assign primary keys to uniquely identify each entity and establish foreign keys for relationships.

Following industry standards for creating detailed description of each entity, attribute, and relationship, including data types and constraints would also be part of this step.

- **Physical data modeling:** Translating the logical model into a physical schema by specifying how data will be stored in the database is the next phase of data modeling. Indexing, partitioning, and denormalization strategies should also be considered to optimize performance based on the application's usage patterns. Define access controls and data encryption to ensure data security is built in during the next phase.
- **Implementation:** As part of actually developing the software solution, using a **database management system (DBMS)** to create and manage the physical database based on the schema is the main aspect. Connect the application to the database, ensuring seamless data access and manipulation. Create APIs for **Create, Read, Update, and Delete (CRUD)** operations, allowing the application to interact with the database and its entities. One can utilize data modeling tools such as erwin, PowerDesigner, or open-source tools like MySQL Workbench to create and manage such data models efficiently.
- **Testing, validation and documentation:** Once build, data validation ensures the data model accurately represents business requirements and that the data is valid and consistent. Testing the database performance under various loads can help optimize the system as necessary. Verifying that security measures protect the data from unauthorized access is also a crucial part of this step. And take steps to maintain comprehensive documentation of the data model, including diagrams, data dictionaries, and any assumptions or decisions made during the modeling process.
- **Maintenance and evolution:** As with any software system, continuously monitoring the database for performance issues, data integrity, and security vulnerabilities is key to long term success. As business needs evolve, update the data model, ensuring the application remains aligned with organizational needs. And design the data model with scalability in mind, considering future growth in data volume and complexity. Implement constraints, triggers, and validation rules to

maintain data integrity and consistency as part of such evolutions.

Data modeling is a critical aspect of application development that ensures data is structured, stored, and accessed efficiently and accurately. By following a systematic approach to data modeling, applications can achieve better data integrity, performance, and scalability, ultimately leading to more robust and reliable software systems.

NoSQL data modeling

NoSQL data modeling is an approach to designing schemas for non-relational databases, which have become an integral part of the data industry for the past fifteen years. Unlike traditional relational databases, NoSQL databases are designed to handle large volumes of unstructured or semi-structured data, offering flexibility, scalability, and high performance. There are several types of NoSQL databases, each with its data modeling principles: document stores, key-value stores, column-family stores, and graph databases. We will describe the different types of NoSQL databases and their corresponding data modeling solutions in this section.

Document stores

Databases such as MongoDB and CouchDB use JSON-like documents containing keyed data with potentially nested sub-structures to allow for semi-structured format. Some common modeling approaches to these include:

- Grouping related data into a single document.
- Use nested documents (maps) and arrays to represent 1:N relationships.
- Design document formats to support various application query patterns.

The following is an example entry of a user document with multiple orders from an ecommerce site:

```
1. {  
2.   "user_id": "user123",  
3.   "name": "John Doe",  
4.   "email": "john.doe@example.com",  
5.   "orders": [
```

```
6.  {
7.    "order_id": "order1",
8.    "date": "2023-07-12",
9.    "items": [
10.     {"product_id": "product1", "quantity": 2, "price": 10.00},
11.     {"product_id": "product2", "quantity": 1, "price": 15.00}
12.   ]
13. },
14. {
15.   "order_id": "order2",
16.   "date": "2023-07-13",
17.   "items": [
18.    {"product_id": "product3", "quantity": 1, "price": 25.00}
19.   ]
20. }
21. ]
22. }
```

Key-value stores

Some data stores such as Redis, DynamoDB, or RocksDB provide key value storage either in-memory or on persistent media (such as disk or flash drives). These simple key-value pair take the following modeling approach:

- Store individual objects as values with unique keys. Keys are generally strings or bytes.
- Suitable for caching and session management for access to transition or temporary information.

One example of a user sessions cached row can be as follows:

```
1. "session:user123": {
2.   "user_id": "user123",
3.   "login_time": "2023-07-12T10:00:00Z",
4.   "cart": ["product1", "product2"]
5. }
```


5. }

Column-family stores

As part of the big data revolution of the 2010s, data stores such as *Amazon DynamoDB*, *Cassandra*, and *HBase* extended simple key-value rows to rows with columns grouped into families. It is used for handling structured as well as semi-structured data across distributed servers. This modeling approach proposed:

- Grouping related columns into families for efficient write operations. Reads might take longer but get amortized over time.
- It used the **log structured merge (LSM)** trees and **write ahead logs (WAL)** to gain increased write speeds.

One example is event logging from an IoT device, which sends regular activity signals:

```
1. CREATE TABLE device_signal (  
2.   device_id TEXT,  
3.   send_time TIMESTAMP,  
4.   signal_type TEXT,  
5.   signal_details TEXT,  
6.   PRIMARY KEY (device_id, send_time)  
7. );
```

Graph databases

Graph databases are an extension to NoSQL that store data as a networked graph, emphasizing the relationship between data entities. For example, *Neo4j* and *Amazon Neptune* use nodes and edges to represent entities and relationships, respectively. This is ideal for social networks, recommendation engines, and fraud detection use cases.

Here is an example of Cypher QL for creating users with friend relationships in a social network application. This can be queried with a MATCH getting all friends of **user1**:

```
1. CREATE (user1:User {user_id: 'user123', name: 'John Doe'})
```

2. **CREATE** (user2:User {user_id: 'user456', name: 'Jane Smith'})
3. **CREATE** (user1)-[:FRIENDS_WITH]->(user2)
- 4.
5. **MATCH** (user1 {name: 'John Doe'})—(friend)
6. **RETURN** friend.name

Takeaways

These are the takeaways for NoSQL modeling principles, building on top of the basic examples above:

- **Data model flexibility:** Since there is no predefined schema is required; NoSQL allows for dynamic addition of new attributes and data types.
- **Denormalization:** Data is often denormalized to improve read performance by storing related data together to minimize the need for joins. Data redundancy is accepted to reduce the need for complex joins during read operations. One can use references to data that is accessed independently or changes frequently.
- **Query patterns:** Data schema and models are designed based on specific query requirements and access patterns. Emphasis is on optimizing for read or write performance, depending on use cases for which is a more frequent operation.
- **Design for scalability:** Data can be partitioned to distribute load evenly across the database cluster. One can use consistent hashing and sharding to manage large datasets.
- **Performance:** Adding indexes for frequently queried fields to speed up read operations is also possible with such data model. And caching strategies can also reduce database load.
- **Data consistency:** Developer can choose the appropriate consistency model (e.g., eventual consistency, strong consistency) based on application requirements. Implementing data validation and integrity checks within the application logic would also be possible.

NoSQL data modeling requires a different approach than relational databases, focusing on flexibility, scalability, and performance. By understanding the application's specific needs and the strengths of each type

of NoSQL database, developers can design efficient data models that meet their requirements.

Golang database connectors

In Go (Golang), connecting to databases requires the use of database drivers and connectors. There are many options available for different types of databases, including SQL, NoSQL, and graph databases. Here is a comprehensive overview of the steps for using some of the popular Go database connectors.

PostgreSQL

Installation: `go get github.com/jackc/pgx/v4:`

```
1. package main
2.
3. import (
4.     "context"
5.     "github.com/jackc/pgx/v4"
6.     "log"
7. )
8.
9. func main() {
10.     conn, err := pgx.Connect(context.Background(),
11.         "postgresql://username:password@localhost:5432/database_name")
12.     if err != nil {
13.         log.Fatal(err)
14.     }
15.     defer conn.Close(context.Background())
16. }
```

MySQL

Installation: `go get github.com/go-sql-driver/mysql:`

```
1. package main
2.
3. import (
4.     "database/sql"
5.     "log"
6.     "github.com/go-sql-driver/mysql"
7. )
8.
9. func main() {
10.     db, err := sql.Open("mysql",
11.         "username:password@tcp(localhost:3306)/database_name")
12.     if err != nil {
13.         log.Fatal(err)
14.     }
15.     defer db.Close()
16. }
```

MongoDB

Installation: `go get go.mongodb.org/mongo-driver/mongo:`

```
1. package main
2.
3. import (
4.     "context"
5.     "log"
6.     "go.mongodb.org/mongo-driver/mongo"
7.     "go.mongodb.org/mongo-driver/mongo/options"
8. )
9.
10. func main() {
11.     client, err := mongo.Connect(context.TODO(),
12.         options.Client().ApplyURI("mongodb://localhost:27017"))
```

```
13. if err != nil {
14.     log.Fatal(err)
15. }
16.
17. defer client.Disconnect(context.TODO())
18. }
```

Redis

Installation: `go get github.com/go-redis/redis/v8:`

```
1. package main
2.
3. import (
4.     "context"
5.     "github.com/go-redis/redis/v8"
6.     "log"
7. )
8.
9. func main() {
10.     rdb := redis.NewClient(&redis.Options{
11.         Addr:     "localhost:6379",
12.         Password: "", // no password set
13.         DB:       0, // use default DB
14.     })
15.
16.     _, err := rdb.Ping(context.Background()).Result()
17.     if err != nil {
18.         log.Fatal(err)
19.     }
20. }
```

Neo4j

Installation: `go get github.com/neo4j/neo4j-go-driver/v4/neo4j`:

```
1. package main
2.
3. import (
4.     "github.com/neo4j/neo4j-go-driver/v4/neo4j"
5.     "log"
6. )
7.
8. func main() {
9.     driver, err := neo4j.NewDriver("bolt://localhost:7687",
10.         neo4j.BasicAuth("username", "password", ""))
11.     if err != nil {
12.         log.Fatal(err)
13.     }
14.     defer driver.Close()
15. }
```

Golang provides a variety of connectors for SQL, NoSQL, and graph databases, each suited for different types of data and use cases. Whether you are building a small project or a large-scale application, choosing the right database and connector is crucial for performance, scalability, and maintainability. Make sure to follow best practices for connection management, error handling, and security when integrating with any database.

Golang data modeling support

In Golang, data modeling is often handled through the use of structs, which define the shape of the data and its properties. For more advanced data modeling, especially when working with databases, several libraries and tools are available to facilitate the process. This section provides an overview of data modeling support in Golang.

Golang object-relational mapping

Golang object-relational mapping (GORM) is a popular ORM library for Go that provides features for data modeling, migrations, and query building.

Installation: `go get -u gorm.io/gorm:`

```
1. package main
2.
3. import (
4.     "gorm.io/driver/sqlite"
5.     "gorm.io/gorm"
6.     "log"
7. )
8.
9. type User struct {
10.     gorm.Model
11.     Name    string
12.     Email   string
13.     BirthDate string
14. }
15.
16. func main() {
17.     db, err := gorm.Open(sqlite.Open("test.db"), &gorm.Config{})
18.     if err != nil {
19.         log.Fatal(err)
20.     }
21.
22.     // Migrate the schema
23.     db.AutoMigrate(&User{})
24.
25.     // Create
26.     db.Create(&User{Name: "John Doe",
27.         Email: "john.doe@example.com",
28.         BirthDate: "1990-01-01"})
```

```

29.
30. // Read
31. var user User
32. db.First(&user, 1) // find user with integer primary key
33. log.Println(user)
34.
35. // Update
36. db.Model(&user).Update("Name", "Jane Doe")
37.
38. // Delete
39. db.Delete(&user, 1)
40. }

```

Ent

Ent is an entity framework for Go that provides a high-level API for data modeling and query building, designed to work well with SQL databases.

Installation: `go get -u entgo.io/ent/cmd/ent:`

```

1. // Generate the models using the Ent code generator
2. //go:generate go run -
   mod=mod entgo.io/ent/cmd/ent generate ./schema
3.
4. package main
5.
6. import (
7.     "context"
8.     "log"
9.     "entgo.io/ent/dialect/sql"
10.    "github.com/mattn/go-sqlite3"
11.    "github.com/your_project/ent" // replace with your project
12. )
13.

```



```

14. func main() {
15.     client, err := ent.Open("sqlite3",
16.         "file:ent?mode=memory&cache=shared&_fk=1")
17.     if err != nil {
18.         log.Fatalf("failed opening connection to sqlite: %v", err)
19.     }
20.     defer client.Close()
21.     ctx := context.Background()
22.
23.     // Run the auto migration tool.
24.     if err := client.Schema.Create(ctx); err != nil {
25.         log.Fatalf("failed creating schema resources: %v", err)
26.     }
27.
28.     // Create a new user.
29.     user, err := client.User.
30.         Create().
31.         SetName("John Doe").
32.         SetEmail("john.doe@example.com").
33.         SetBirthDate("1990-01-01").
34.         Save(ctx)
35.     if err != nil {
36.         log.Fatalf("failed creating user: %v", err)
37.     }
38.     log.Println("user was created:", user)
39. }

```

SQLX

SQLX is a library that provides a set of extensions on top of the standard **database/sql** package, making it easier to work with databases in a more idiomatic way.

Installation: go get github.com/jmoiron/sqlx:

```
1. package main
2.
3. import (
4.     "database/sql"
5.     "log"
6.     "github.com/jmoiron/sqlx"
7.     _ "github.com/lib/pq" // Postgres driver
8. )
9.
10. type User struct {
11.     ID      int    `db:"id"`
12.     Name     string `db:"name"`
13.     Email    string `db:"email"`
14.     BirthDate string `db:"birth_date"`
15. }
16.
17. func main() {
18.     db, err := sqlx.Connect("postgres",
19.         "user=username dbname=mydb sslmode=disable")
20.     if err != nil {
21.         log.Fatalln(err)
22.     }
23.
24.     user := User{}
25.     err = db.Get(&user, "SELECT * FROM users WHERE id=$1", 1)
26.     if err != nil {
27.         log.Fatalln(err)
28.     }
29.     log.Println(user)
30. }
```

GQLGen

GQLGen is a Go library for building GraphQL servers, providing a strong type-safe way to build APIs and manage data models.

Installation: `go get github.com/99designs/gqlgen:`

```
1. // Define your GraphQL schema in schema.graphql
2. // Generate the models and resolvers using GQLGen
3. // go:generate go run -mod=mod github.com/99designs/gqlgen
4.
5. package main
6.
7. import (
8.     "github.com/99designs/gqlgen/handler"
9.     "net/http"
10.)
11.
12. func main() {
13.     http.Handle("/", handler.Playground("GraphQL playground",
14.         "/query"))
15.     http.Handle("/query", handler.GraphQL(
16.         NewExecutableSchema(Config{Resolvers: &Resolver{}})))
17.
18.     log.Fatal(http.ListenAndServe(":8080", nil))
19. }
```

Some of the considerations for deciding the right connector could include the following:

- **Concurrency:** Since Go provides high concurrency support, using a ORM connector that allows for this feature to be utilized using connection pooling would be a great performance advantage. Having an adjustable configuration to allow changing the connection pool size would improve application scalability, too.
- **Compatibility:** Check that the requirements of needed data types,

including nested and array data types, are provided by the chosen connector. Also, ensure complex query support and error handling semantics match the application requirements.

- **Security:** One can pick connectors with built-in support for encrypted TLS connections to ensure secure communication with the database. Confirm that connector can also provide authentication and support mechanisms like OAuth, Kerberos, or role-based access control if your database uses them.
- **Ease of use:** Consider how easy it is to configure the connector, especially if you need to set options for different environments (for example, dev, staging, production). Ensure the connector offers good documentation, clear error messages, and logging options to simplify debugging.

Golang provides robust support for data modeling through structs and a variety of libraries and tools. Whether you are working with relational databases, NoSQL databases, or GraphQL APIs, there are solutions available to fit your needs. By leveraging these libraries and best practices, you can efficiently manage and manipulate data in your Go applications.

Data streaming solutions

Data streaming solutions are essential for real-time data processing, enabling applications to ingest, process, and analyze data as it is generated in near real-time. Data modeling for data streams involves designing the structure of data as it flows through a system in real-time. Unlike traditional batch processing, streaming data is continuous, and its schema needs to accommodate this dynamic nature. First, we will define some of the common terms:

- **Stream:** A continuous flow of data records into the system.
- **Event:** A single unit of data in the stream, often representing a change or an occurrence in the system.
- **Window:** A logical grouping of events within a specific time frame. The starting point in time and size of the window could be varied.
- **State:** Data that accumulates over time, often required for aggregation

or pattern detection.

Now let us get an overview of designing and modeling strategies for data streams.

Define the event schema

An event schema describes the structure of individual events within the stream. It typically includes fields that represent the data attributes. An example of an event with its payload can look as follows:

```
1. {  
2.   "event_id": "string",  
3.   "timestamp": "ISO 8601 string",  
4.   "type": "string",  
5.   "payload": {  
6.     "user_id": "string",  
7.     "action": "string",  
8.     "value": "number"  
9.   }  
10. }
```

The corresponding Go struct used to generate events for the above schema would be as follows, with JSON names mapped to the fields:

```
1. type Event struct {  
2.   EventID string `json:"event_id"`  
3.   Timestamp time.Time `json:"timestamp"`  
4.   Type string `json:"type"`  
5.   Payload Payload `json:"payload"`  
6. }  
7.  
8. type Payload struct {  
9.   UserID string `json:"user_id"`  
10.  Action string `json:"action"`  
11.  Value float64 `json:"value"`
```

12. }

Event and processing times

The following two need to be accounted for ensuring smooth event processing:

- **Event time:** The time at which the event actually occurred.
- **Processing time:** The time at which the event is processed by the system.

Using event time for time-based operations (like windowing) ensures more accurate and consistent results, especially in distributed systems.

Design for schema evolution

Streaming systems often need to handle changes in the event schema over time. Adopt strategies like:

- **Backward compatibility:** New consumers can process old data.
- **Forward compatibility:** Old consumers can process new data.
- **Schema registry:** Use a schema registry (for example, *Confluent Schema Registry*) to manage and version schemas.

Use of metadata

Include metadata such as source, partition, and offset to manage the stream effectively:

```
1. type EventMetadata struct {  
2.     Source string `json:"source"`  
3.     Partition int `json:"partition"`  
4.     Offset int64 `json:"offset"`  
5. }
```

Aggregation and state management

Model stateful operations for aggregations, such as counting events, calculating averages, or maintaining running totals:

```
1. type AggregatedEvent struct {  
2.     UserID string `json:"user_id"`  
3.     Action string `json:"action"`  
4.     Count int `json:"count"`  
5.     TotalValue float64 `json:"total_value"`  
6. }
```

Data modeling for data streams in Golang involves defining flexible, evolving schemas for events, handling metadata, and managing state for real-time processing. Using the right tools and libraries, you can build robust streaming data pipelines that accommodate the dynamic nature of real-time data.

In Golang, several libraries and frameworks can be used to implement data streaming solutions, each catering to different needs such as data ingestion, processing, and delivery. We will give an overview of popular data streaming solutions and how to use them with Golang.

Apache Kafka

Kafka is an open-source distributed event streaming platform capable of handling trillions of daily events. Its use cases include large-scale data pipelines, real-time analytics, and mission-critical applications. A few options for client libraries can be used, including **Sarma**, **Confluent**, **Goka**, and **segmentation**. Let us take Sarma to demonstrate a setup for Kafka processing using Golang.

Once the library is installed using **go get github.com/IBM/sarama**, the following is the sample code to use, produce, and consume Kafka messages:

```
1. package main  
2.  
3. import (  
4.     "log"  
5.     "github.com/IBM/sarama"  
6. )  
7.
```

```
8. func main() {
9.     // Kafka producer
10.    producer, err :=
11.        sarama.NewSyncProducer([]string{"localhost:9092"}, nil)
12.    if err != nil {
13.        log.Fatal("Failed to start Sarama producer:", err)
14.    }
15.    defer producer.Close()
16.
17.    msg := &sarama.ProducerMessage{
18.        Topic: "example_topic",
19.        Value: sarama.StringEncoder("This is a test message"),
20.    }
21.    _, _, err = producer.SendMessage(msg)
22.    if err != nil {
23.        log.Fatal("Failed to send message:", err)
24.    }
25.
26.    // Kafka consumer
27.    consumer, err :=
28.        sarama.NewConsumer([]string{"localhost:9092"}, nil)
29.    if err != nil {
30.        log.Fatal("Failed to start Sarama consumer:", err)
31.    }
32.    defer consumer.Close()
33.
34.    partitionConsumer, err := consumer.ConsumePartition(
35.        "example_topic", 0, sarama.OffsetNewest)
36.    if err != nil {
37.        log.Fatal("Failed to start partition consumer:", err)
38.    }
```



```

39. defer partitionConsumer.Close()
40.
41. for message := range partitionConsumer.Messages() {
42.     log.Printf("Consumed message offset %d: %s",
43.         message.Offset, string(message.Value))
44. }
45. }

```

Note: One can enhance the consumer to be a pool of worker goroutines that process a disjoint set of the produced messages and can potentially use channels to accumulate any results if needed.

Apache Pulsar

Pulsar is a distributed pub-sub messaging platform with a flexible messaging model and an intuitive client API.

Installation: `go get github.com/apache/pulsar-client-go/pulsar:`

```

1. package main
2.
3. import (
4.     "context"
5.     "log"
6.     "github.com/apache/pulsar-client-go/pulsar"
7. )
8.
9. func main() {
10.     client, err := pulsar.NewClient(pulsar.ClientOptions{
11.         URL: "pulsar://localhost:6650",
12.     })
13.     if err != nil {
14.         log.Fatal(err)
15.     }
16.     defer client.Close()

```

```
17.
18. // Producer
19. producer, err := client.CreateProducer(pulsar.ProducerOptions{
20.     Topic: "my-topic",
21. })
22. if err != nil {
23.     log.Fatal(err)
24. }
25. defer producer.Close()
26.
27. _, err = producer.Send(context.Background(),
28.     &pulsar.ProducerMessage{
29.         Payload: []byte("hello"),
30.     })
31. if err != nil {
32.     log.Fatal(err)
33. }
34.
35. // Consumer
36. consumer, err := client.Subscribe(pulsar.ConsumerOptions{
37.     Topic: "my-topic",
38.     SubscriptionName: "my-subscription",
39.     Type: pulsar.Shared,
40. })
41. if err != nil {
42.     log.Fatal(err)
43. }
44. defer consumer.Close()
45.
46. msg, err := consumer.Receive(context.Background())
47. if err != nil {
```

```

48.     log.Fatal(err)
49. }
50. log.Printf("Received message msgId: %v -- content: '%s'",
51.     msg.ID(), string(msg.Payload()))
52. consumer.Ack(msg)
53. }

```

Redis Streams

Redis Streams is a data structure for managing data streams and time series data in Redis.

Installation: `go get github.com/go-redis/redis/v8:`

```

1. package main
2.
3. import (
4.     "context"
5.     "github.com/go-redis/redis/v8"
6.     "log"
7. )
8.
9. var ctx = context.Background()
10.
11. func main() {
12.     rdb := redis.NewClient(&redis.Options{
13.         Addr: "localhost:6379",
14.     })
15.
16.     // Producer
17.     _, err := rdb.XAdd(ctx, &redis.XAddArgs{
18.         Stream: "mystream",
19.         Values: map[string]interface{}{"message": "hello"},
20.     }).Result()

```

```

21.  if err != nil {
22.      log.Fatal(err)
23.  }
24.
25.  // Consumer
26.  result, err := rdb.XRead(ctx, &redis.XReadArgs{
27.      Streams: []string{"mystream", "0"},
28.      Count: 1,
29.      Block: 0,
30.  }).Result()
31.  if err != nil {
32.      log.Fatal(err)
33.  }
34.  for _, stream := range result {
35.      for _, message := range stream.Messages {
36.          log.Printf("Received message: %v", message.Values)
37.      }
38.  }
39. }

```

Apache Flink

Flink is an open-source framework and distributed processing engine for stateful computations over unbounded and bounded data streams. Currently, Flink does not have an official Go SDK, but it can interact with Go applications via REST APIs or by integrating with Kafka.

Let us now summarize the best practices for data streaming in Go:

- **Concurrency and goroutines:** Use Go's lightweight goroutines to handle multiple streaming operations concurrently. Manage goroutines effectively to avoid memory leaks and deadlocks.
- **Error handling:** Implement robust error handling to ensure the system can recover from transient errors. Log errors for monitoring and

debugging.

- **Graceful shutdown:** Implement signal handling to gracefully shut down streaming operations. Ensure resources like connections and goroutines are properly cleaned up.
- **Flow control:** Use buffering and rate-limiting to handle backpressure. Ensure the system can handle varying loads without crashing.
- **Monitoring and metrics:** Integrate with monitoring tools to track the performance of your streaming application. Use metrics to gain insights into throughput, latency, and error rates.

Golang offers robust support for implementing data streaming solutions with various libraries and frameworks. Whether you're dealing with Kafka, Pulsar, Redis Streams, or integrating with Flink, Go provides the tools needed to build scalable, high-performance streaming applications. By following best practices and leveraging Go's concurrency features, you can effectively manage and process real-time data streams in your applications.

Conclusion

In this chapter, we delved into the different data modeling options for modern distributed applications. Golang provides a comprehensive set of solutions for use cases ranging from relational data access to NoSQL and to streaming data providers as well. This helps software developers pick and choose the right tool for the application being developed and also maintains a clear path for enhancing data scale or adding more data models for new use cases.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 8

Scalability, Availability and Other-ilities

Introduction

Now that we have an overview of distributed system service creation, deployment, and data transfer, we will need to get familiar with how to operate such deployments to maintain application requirements. This chapter digs deeper into the various aspects of ensuring the backend service will be available for users and can adapt to changing workloads. We identify what are common bottlenecks at various layers of architecture and how to monitor such systems to ensure robust real-world solutions.

Distributed systems deploy as nodes and fail as graphs.

-John Lambert

There could be many errors that could happen at any given instant, ranging from software errors to hardware breakdowns. The application should be tolerant of a cross-section of these failures. In fact, having a way to detect some of the failures via metrics or automated alerts would also be a true north for a robust system to perform its functionality and maintain **service level agreements (SLAs)**.

Structure

The chapter covers the following topics:

- Overview
- Availability
- Scalability
- Reliability
- Monitoring
- Maintainability
- Usability
- Golang-ilities

Objectives

Keeping a system running needs the same kind of attention and potential repairs as to keeping a car in good running condition. The goal of this chapter is to gain expertise on such operational requirements of running such systems and how to perform changes on such a dynamic system while it is up and taking on user workload.

We will start from the aspects on the development side that provide some of these abilities and drive into topics of operational best practices to get the most out of the system. Then, we will stop to discuss how some of the abilities needed have related requirements across the board.

We will conclude by providing an overview of how Golang helps provide native or library support for some of these requirements.

Overview

In the context of distributed systems, **ilities** refer to various non-functional requirements that ensure the system meets quality standards. Here are some key ilities that can be tracked for any distributed software system:

- **Availability:** This is the degree to which the system is operational and accessible when required. Normally measured as a percentage, the goal is to be as close to 100% as possible, with notions like five-nine availability implying 99.999%. This metric is achieved via service

redundancy, application failover mechanisms, robust distributed architectures, and automated recovery. The ability of the system to recover quickly from failures and continue operations via self-healing mechanisms will increase the availability.

- **Scalability:** This is the ability of the system to handle increased load by adding resources. As part of this, horizontal scaling (adding more servers) or vertical scaling (adding more CPU/disk/memory/network resources per server) can be performed along with load balancing.
- **Reliability:** The ability of the system to function correctly and consistently over time is also part of system quality. Clean error handling, along with fault tolerance and consistent data replication/messaging, helps increase system resilience.
- **Maintainability:** The ease with which the system can be maintained, including bug fixes, updates, and setup improvements, is another aspect of ease of managing the system. This includes modular design, complimented by clean coding standards and automated testing, along with comprehensive documentation. The use of existing components and systems to build new functionalities on top of those helps build and maintain code as well.
- **Performance:** The efficiency with which the system processes requests and handles tasks, paired with the optimal use of system resources, can help achieve consistent and desired performance. Using efficient algorithms that ensure low request latency, high throughput, and optimized resource usage can get this to the optimal level.
- **Security and compliance:** The protection of the system from unauthorized access and attacks is never understated. Using techniques such as encryption, authentication, authorization, and secure communication protocols further the security. Adherence to regulatory and organizational standards and policies is also part of the system governance. The ability to trace system activities for compliance and troubleshooting is also a must for better accountability.
- **Usability:** The ease with which users can interact with the system and achieve their goals. The system designer should aim to provide

intuitive interfaces, responsive functionality, accessibility features, as well as up to date user documentation.

- **Flexibility:** This indicates the ability to adapt to changing requirements and environments along with the ability to add new features and functionalities with minimal impact on existing components. The ease with which changes can be made to the system includes reusable code libraries, modular components, and standardized interfaces using open or industry standards.

By being cognizant and addressing these ilities in the system design, software developers can design and build distributed systems that are robust, efficient, and capable of meeting diverse and evolving requirements.

Availability

The true north of a highly available system is to cater to user workload at all times. Ensuring the availability of a software system involves a combination of design principles, best practices, and ongoing enhancements to operational strategies. Here are the key aspects and techniques to enhance system availability:

- **Redundancy:** There is a need to duplicate critical components (servers, databases, services etc.) to eliminate single points of failure across them. Using geographic distribution one can deploy resources across multiple data centers or regions to mitigate the risk of localized failures or disasters.
- **Failover mechanisms:** Setting up automated failover mechanisms can help systems to transparently switch to a backup system or component when a failure is detected in the corresponding primary (or current) component. There could be three tiers of standby systems - hot, warm, or cold. Use different types of standby systems based on **recovery time objectives (RTO)**, hot being the fastest to be ready for serving user traffic and cold being the slowest, needing extra warmup time before serving traffic.
- **Load balancing:** Using standard load balancers to evenly distribute traffic across multiple servers ensures no single server becomes a

bottleneck. This could also account for continuously monitoring the heartbeat of servers and directing traffic only to the healthy ones so that the failed ones can be swapped out with newer servers.

- **Monitoring and alerting:** Along with server heartbeat, one can use industry-standard monitoring tools to track system performance, resource utilization, and application-level health. Admins could also set up alerts for anomalies or failures to enable rapid response while traffic is diverted. One can also establish clear SLAs with service providers to ensure they meet the required availability standards for responding to alerts.
- **Scaling:** Modern services implement auto scaling to adjust resources dynamically based on current demand, which would potentially improve the latency and throughput metrics. Designing the system to handle varying loads by scaling up or out during peak times and down or in during off-peak times reduces chances of degraded performance and reduces failure possibilities, respectively.
- **Data recovery:** The system should be designed to regularly back up data (in an incremental manner) and ensure backups are stored in geographically diverse locations, including via cloud services. The operational aspect of developing and regularly testing disaster recovery plans to ensure quick recovery from catastrophic failures could reduce application downtime when actual failures occur.
- **Architecture:** As discussed before, designing the system using microservices helps isolate failures and reduces the blast radius of impact on the entire system. One can also design the fault tolerant aspect in the system to continue operating with reduced functionality or performance in the event of partial failures. At the code level, implementing retry mechanisms for transient errors will enhance the resilience of the system, with the trade-off for increased latency.
- **System maintenance:** Regularly applying OS and security patches and updates to keep software components secure and stable will also improve the availability of the software running on top of that infrastructure. Similarly, assessing and adjusting resources to meet current and future demands will keep the system running as expected in

the long term.

By incorporating these strategies, organizations can significantly enhance the availability of their software systems, ensuring they remain operational and reliable even in the face of various system challenges and failures.

Scalability

Scalability in a software system refers to its ability to handle increasing amounts of workload or its potential to be increased in size to accommodate that growth. There are several key aspects of scalability that need to be considered when designing and maintaining a software system. We will be taking a look at them in this section.

Horizontal and vertical scaling

The following two modes for system scaling are used widely:

- **Horizontal (scaling out):** Adding more instances of servers or nodes to distribute the load across multiple machines. This is often used in cloud environments where additional resources can be provisioned easily. It is expected that the application is capable of using the newly added resources to improve the throughput of the entire system.
- **Vertical (scaling up):** Increasing the capacity of a single server or node by adding more CPU, RAM, or storage. This is limited by the maximum capacity of the hardware on that server and can also get increasingly costly as the number of cores, memory, or storage is increased.

The table below summarizes the trade-offs and considerations for choosing between these two options:

Dimension	Horizontal scaling	Vertical scaling
Cost	Cost-effective at scale, but initial setup can be higher.	Can be expensive due to high-end hardware.
Scalability	Virtually unlimited with proper application architecture.	Limited by hardware capacity.

Dimension	Horizontal scaling	Vertical scaling
Complexity	High due to distributed systems.	Low, as it involves a single machine.
Fault tolerance	High, supports redundancy and failover.	Low, single point of failure.
Performance overhead	Cross node traffic can add latency and networking costs.	No networking overhead.
Downtime	No downtime for scaling if load balancers are used.	Downtime likely during upgrades.
Data consistency	Challenging in distributed systems.	Simple, as data is on one machine.
Setup time	Longer due to distributed nature.	Faster setup for single machines.
Use cases	Large scale applications needing fault tolerance.	Monolithic applications with low traffic.

Table 8.1 : *Comparison of the various features across frameworks*

In most modern systems, a hybrid combination of the two can be used to achieve the desired improvement.

Load balancing

Using load balancers like Thunder ADC or NGINX will evenly distribute incoming traffic across multiple servers to ensure no single server becomes a bottleneck. Continuously monitoring the health of servers and directing traffic only to healthy ones to maintain performance and reliability is also provided.

Caching

Using caches to store frequently or recently accessed data in memory to reduce system load and improve response times. Examples of third-party caches include Redis and Memcached. Another example is a **content delivery network (CDN)**, which caches static assets like images, videos, and scripts closer to the end users to reduce latency and improve load times.

Asynchronous processing

Using message queues (like RabbitMQ or Kafka) to decouple components and handle tasks asynchronously can improve performance and reliability. Building services that offload long-running or resource-intensive tasks to background workers keep the main application responsive to user interactions. Applications using async design and execution scale by allowing parallel execution of tasks without getting serialized waiting on completion of each task completion. This allows improving compute resource efficiency which need not wait on network or I/O bound work items. On the other side, these systems tend not to provide strong consistency guarantees and could also be harder to debug as the execution state could be spread across different components.

Microservices

Break down a monolithic application into smaller, independent services that can be developed, deployed, and scaled individually. Using API Gateways to manage and route requests to appropriate microservices can help scale them independently.

Elasticity

Implementing auto scaling mechanisms to dynamically adjust the number of running instances based on current demand, ensuring optimal resource usage and cost-efficiency. Automatically provisioning and de-provisioning resources based on predefined policies and thresholds can help reduce manual toil for the scalability mechanisms.

Database scalability

For database usage, there are a few techniques that help with spreading the load and improving scalability. They are:

- **Read replicas:** Creating read-only replicas of the database to distribute read traffic and reduce the load on the primary database.
- **Write scaling:** Implementing strategies to handle high write loads, such as write partitioning or using specialized databases designed for high write throughput.
- **Sharding:** Splitting a large database into smaller, more manageable

pieces called **shards**. Each shard holds a subset of the data, which helps in distributing the load and improving performance.

Monitoring and analytics

Another key aspect is to continuously monitor system performance and resource usage to identify bottlenecks and scale resources appropriately. Using analytics to predict traffic patterns and plan for future scalability needs is also recommended.

By addressing these scalability aspects, a software system can be designed to handle increasing workloads efficiently and reliably, ensuring it can grow and adapt to meet the needs of users and business growth.

Reliability

Reliability in software systems refers to the ability of the system to operate correctly and consistently over time, providing the expected functionality without failure. This ility tracks failure counts and their impact whereas availability tracks for how long system is unusable, so some of the aspects are related and affect each other. Achieving high reliability involves several aspects and best practices that have been discussed in this section.

Redundancy

Using replication, one can duplicate critical components (servers, databases, services, etc.) to avoid single points of failure. On top of that, geographic distribution by deploying resources across multiple data centers or physical regions will reduce the impact of localized failures. To reduce full application failure, systems automatically switch to a backup system or component when a failure is detected in the primary system.

Fault tolerance

Since prevention is the best cure, having options for graceful degradation instead of immediate service termination is a suggested best practice. Hence, designing the system to continue operating with reduced functionality or performance in the event of partial failures is followed in

many systems. Implementing retries related code mechanisms for transient errors also enhances resilience and improves reliability. Distributing traffic evenly prevents any single component from becoming a hot spot for system bottleneck.

Monitoring and alerting

As with availability, using tools to continuously track system performance, resource utilization, and application health also helps with tracking potential failure or reliability aspects. Setting up alerts for anomalies or failures to enable rapid response also helps.

Error handling and logging

Comprehensive error-handling logic in code ensures that all potential failures are anticipated and properly handled. Specifically, intermittent ones are retried to keep the system running uninterrupted for longer. Maintaining detailed logs also aids in diagnosing and resolving any error-related issues quickly.

Data integrity

Since data is the gold of any service, ensuring it is properly stored and protected is a central aspect to ensuring a reliable system. Some of the techniques for the same include:

- **Checksums and hashing:** Storing checksums and hashes to detect and prevent data corruption can help validate that data is usable.
- **Consistency checks:** Regularly performing consistency checks in the background to ensure data integrity is maintained across dependent records is also an option.
- **Regular backups:** Performing regular backups to ensure data can be restored in case of failure can help undo any human induced or software induced bugs.
- **Disaster recovery planning:** Developing and regularly testing plans to recover from catastrophic failures to remove or reduce data loss should be part of the team charter.

Software management

At the outset, designing with microservices to isolate failures and reduce the impact on the entire system is a good starting point for a basic layer of built-in reliability. After deployment, applying software updates to include new fully tested features, enhancements, and bug fixes across the stack helps keep the stack running without hiccups. Ensuring the system can handle increased loads without failure by testing scaling operations is also a requirement.

Security

Implementing robust access control mechanisms to prevent unauthorized access and potential disruptions would help keep system reliable against external threats. This would include implementing measures to protect against **distributed denial of service (DDoS)** attacks. On a software updates-related note, ensuring the timely application of security patches to keep software components secure and stable is also essential.

Documentation

Maintaining detailed documentation to aid in system maintenance and troubleshooting, thus reducing the potential for human errors or incorrect updates to the systems. Another aspect is ensuring that knowledge about the system and assumptions are shared among team members to avoid single points of failure in expertise. This will help with operational clarity for the on-call team as well.

By focusing on these reliability aspects, organizations can build software systems that are robust, resilient, and capable of operating correctly and consistently over time, ensuring a high level of service for users.

Monitoring

Monitoring is a critical aspect of managing and maintaining distributed software systems, providing the necessary visibility to ensure functionality, performance, reliability, and security. Let us discuss the key aspects of monitoring in this section.

Performance monitoring

Some of the dimensions of metrics to be tracked for complete system performance include:

- **Operating system metrics:** Tracking CPU usage, memory consumption, disk I/O, network bandwidth, and other system-level metrics could point out hardware or infrastructure issues.
- **Application metrics:** Monitoring application-specific metrics such as response times, request rates, error rates, and throughput across all servers in the application provides insights into potential fixes needed.
- **Database performance:** Being the central source of truth for most systems, databases deserve their own monitoring metrics. Observing query performance, connection counts, cache hits/misses, and other database-related metrics can help prevent bottlenecks.

Health monitoring

Application health can be monitored using:

- **Service health checks:** Regularly checking the availability and responsiveness of services and endpoints. This can be provided as a REST endpoint per service that can be queried on a regular basis by a monitoring service.
- **Heartbeat monitoring:** Extending the above, verifying that all critical components across the system stack are alive and functioning correctly through periodic checks can be helpful.
- **Resource utilization:** Monitoring resource usage to ensure efficient utilization and avoid over-provisioning or under-provisioning can prevent wastage and unexpected behaviors in the system.
- **Forecasting:** Using historical data and other resource trends to predict future resource needs can help plan capacity accordingly.

Security monitoring

Security related monitoring can help prevent fraud and crippling intrusions to the system, these can be subdivided into:

- **Intrusion detection:** Identifying potential security breaches or unauthorized access attempts via scanning of system logs.
- **Vulnerability scanning:** Regularly scanning for security vulnerabilities in software setup and configurations.
- **Compliance monitoring:** Ensuring adherence to security policies and regulatory requirements based on location.

Network monitoring

Cross node traffic efficiency is needed in distributed system to reduce **single point of failures (SPOF)** and ensure uniform bandwidth usage across the nodes in the cluster. One can setup the following:

- **Traffic analysis:** Monitoring network traffic to identify bottlenecks, unusual patterns, and potential DDoS attacks can be performed by tools like the *Cisco Network Analysis Module* or the *ManageEngine NetFlow analyzer*.
- **Latency monitoring:** Measuring network latency to ensure fast and reliable communication between components or services can also be added as a metric.

Alerting

Automated alerts or notifications to the team for any unexpected values of the metrics also help ensure timely remediation. Some types of alerting include:

- **Threshold-based alerts:** Setting thresholds for critical metrics and triggering alerts when the current values are in breach of contract can be enabled.
- **Anomaly detection:** Using machine learning or statistical methods to detect unusual patterns over a longer period to trigger alerts can also be set up.
- **Notification channels:** Sending alerts through various channels like email, SMS, phone calls, chat apps (*Microsoft Teams* or *Slack*), or other incident management systems that are in place.

Visualization

A concise and centralized dashboard with charts and timeseries metrics panel is the holy grail of tracking all aspects of the system monitoring in one place. This can help detect patterns and pinpoint root causes faster. One can create the following:

- **Real-time dashboards:** Creating dashboards to display key metrics and performance indicators in real-time can help track the current state of the system.
- **Historical data analysis:** Providing tools to analyze historical data for trend analysis and capacity planning is also part of the system.

Tools such as Tableau or Zoho provide flexible options along with zooming and AI integration as well.

Logging and tracing

Last but not least, adding the right amount of logging and having the ability to track workflows can help get insight into the root cause of complicated issues. These can be categorized into:

- **Centralized logging:** Aggregating logs from different components and services into a centralized system for easier analysis.
- **Log analysis:** Automatically parsing and analyzing logs to identify patterns, errors, and anomalies.
- **Request tracing:** Tracking the flow of requests across different services to identify bottlenecks and failures.
- **Latency breakdown:** Analyzing the time spent in each component or service to pinpoint performance issues.

Identifying incidents quickly through effective monitoring and alerting is a must for any large-scale distributed system. Monitoring data can be used to diagnose and understand the root causes of incidents. By covering these aspects of monitoring, organizations can achieve comprehensive visibility into their systems, enabling proactive management, quick detection of issues, and effective troubleshooting to maintain high levels of performance, reliability, and security.

Maintainability

Maintainability in software systems refers to the ease with which a system can be modified, extended, and corrected. Ensuring maintainability involves following best practices in design, coding, documentation, and testing. This section discusses the key aspects and practices to ensure maintainability.

Modular design

Designing from the get-go to separate the system into distinct modules or components, each with a specific responsibility, keeps services independently developed and deployed. Applying proven design patterns to solve common problems in a structured way should be part of this step. Designing the system to scale easily with increasing loads and ensuring the system can accommodate new features and changes with minimal impact on production can help reduce bumps during system maintenance operations.

The goal should be to keep services or components small and focused on a single responsibility to make them easier to understand and maintain. Hiding the internal details of modules and exposing only necessary interfaces at service boundaries or external functions will also reduce code churn from dependent service implementation updates.

Code quality

Maintaining the code quality during implementation and enforcing it as part of the review process is a must have for maintainable code at the end of the day. Writing readable, understandable, and well-structured code can be achieved with shared knowledge. Following consistent coding conventions and style guides like *Google* style guides will help set industry level quality and ease of building on top of it for any person working in the future on that project. Using tools to perform static code analysis and identify potential issues would benefit each logical code change.

Documentation

Three aspects of documentation help different types of persona gain system

knowledge as per their needs. These documentation types include:

- **Design documentation:** Documenting system architecture, design decisions, and data flow diagrams is the first aspect that helps architects and stakeholders get the big picture.
- **Code comments:** Adding meaningful comments to explain the purpose and logic of the code. Focusing on the why is recommended compared to the what, the actual code. This will help the developer team be in the know.
- **API documentation:** Providing comprehensive documentation for APIs, including usage examples, helps users build and use these quickly.

Automated testing

Various testing strategies discussed before can help save the following:

- **Unit testing:** Writing tests for individual units of code to ensure they function correctly. The goal should be 100% function-level coverage and a high threshold for line coverage (say 95%).
- **Integration testing:** Testing the interactions between different components and ensuring intended functionality is delivered end-to-end.
- **Regression testing:** Ensuring that new changes do not introduce new bugs or regressions.
- **Performance and stress testing:** Running tests at large scale input to the system and measuring metrics are acceptable is also a good goal to target.

Version control

Versioning of different components of the system can provide ease in rolling back to the last known good version for restoration purposes. These are the common techniques used to achieve the same:

- **Source control management (SCM):** Teams should use version control systems like Git to track changes and collaborate effectively.

- **Branching and merging:** Implementing strategies for branching for major and minor versions, merging branches, or deleting old branches can help manage the history of the code changes.
- **External configuration:** Keeping deployment configuration settings separate from the code makes the system easier to manage and deploy using **infrastructure as code (IaC)** tools.
- **Versioned configuration:** Managing configuration changes using version control.

Continuous integration and continuous deployment

As discussed before, **continuous integration and continuous deployment (CI/CD)** helps with code maintainability in these three aspects:

- **Automated builds:** Setting up automated build processes to compile, package and version the software.
- **Automated testing:** Running automated tests as part of the build process to catch regression issues early.
- **Continuous deployment:** Automating the deployment process for each version produced to ensure smooth, consistent, and continuous releases.

Knowledge sharing

An often-overlooked aspect for raising the bar for coding and operational excellence is the need for team collaboration. Encouraging collaboration and knowledge sharing among team members along with documenting access steps or useful scripts can go a long way in systems maintenance. This includes providing training and mentorship to keep the team updated with the latest tools and processes and how to apply them practically.

By focusing on these aspects, organizations can build and maintain software systems that are easy to understand, modify, and extend, ensuring long term maintainability and reducing the overall cost of maintenance.

Usability

Ensuring the usability of a software system involves making it easy to learn for developers and stakeholders, efficient to use, and satisfying for the end users. We will discuss the key principles and practices to ensure usability in this section.

User-centered design

Front and center for this is to ensure users can easily use the functionality and workflows are clear, concise, and intuitive. The following steps help get there:

- **User research:** Conducting research to understand the needs, preferences, and behaviors of the target users.
- **Personas:** Creating personas representing different user types to guide design decisions with stakeholders.
- **User stories:** Writing user stories to define and understand user requirements and expectations helps team be on the same page for the intended usage.

User interface design

The actual design should provide these capabilities for the end user to be able to start using the solutions quickly:

- **Consistency:** Using consistent design elements, colors, fonts, and layouts throughout the application.
- **Simplicity:** Keeping the interface simple and uncluttered to avoid overwhelming users.
- **Affordances:** Designing elements to suggest their function, like buttons that look clickable or shapes or colors that are logical for the operation.

Information architecture

The flow of information should be logically similar to what is expected in the real world. Some aspects to bear in mind to achieve this are as follows:

- **Clear navigation:** Design intuitive navigation structures so users can

easily find what they need. Organizing information in a logical hierarchy to facilitate understanding top-down. Ensure that interactions, such as clicking buttons and navigating pages, are smooth and responsive.

- **Search functionality:** Implementing effective search capabilities to help users find information quickly would also improve usage.
- **Fast load times:** Optimize the system to load quickly, as slow performance can frustrate users. This could mean keeping the front end lightweight and calling the back end with compact queries on-demand.

Accessibility

The ability to provide an accessible **user interface (UI)** is a must-have goal of any good UI design. This includes:

- **WCAG compliance:** Adhering to the **Web Content Accessibility Guidelines (WCAG)** ensures the system is accessible to users with disabilities.
- **Keyboard navigation:** Ensure that all interactive elements can be accessed and operated via keyboard.
- **Screen reader support:** Provide support for screen readers to aid visually impaired users.

Responsiveness

In the modern world, there are various devices across a multitude of operating systems that can be accessed across the globe at any given time. Such applications need UI to deliver:

- **Cross-device compatibility:** Ensuring the system works well on different devices and screen sizes, including desktops, tablets, and smartphones.
- **Fluid layouts:** Use fluid grids and flexible images to create a responsive layout that adapts to various screen sizes.
- **Consistency:** Maintain a consistent tone and style throughout the application to keep uniformity across usage or workflows.

Error handling

Users could run into expected cases for which UI should show a path forward and clear any stale state. They are:

- **Clear error messages:** Display clear and helpful error messages that explain what went wrong and how to fix it.
- **Immediate feedback:** Providing instant feedback for user actions, such as button clicks and form submissions.
- **Undo functionality:** Allowing users to undo actions to recover from mistakes easily.

Documentation

Three levels of help can be provided at the minimum to the users for any further questions or clarity needed. They are:

- **Inline help:** Provide contextual help and tooltips within the application.
- **User manuals:** Create comprehensive user manuals and guides that can be accessed from the UI page in one click.
- **Support channels:** Offer multiple support channels, such as FAQs, chat/slack support, and user forums.

Workflow design

Workflows of users can be streamlined to get the smoothest experience using:

- **Task analysis:** Analyze common user tasks to streamline and simplify workflows.
- **Automation:** Automate repetitive tasks to reduce user effort.
- **Shortcuts:** Providing keyboard shortcuts and other mechanisms to speed up frequent actions.
- **Visual appeal:** Create an aesthetically pleasing design to enhance user satisfaction.
- **Engaging content:** Use engaging and relevant content to keep users

interested.

Localization

Since UI is the entry point for users, they should convey expectations clearly no matter the locale from which the user is accessing it. This is covered by:

- **Clarity:** Use clear and straightforward language in all communications.
- **Language support:** Support multiple languages and regional formats.
- **Cultural sensitivity:** Ensure the design is culturally appropriate for different user groups.

By focusing on these principles and practices, organizations can design and develop software systems that are user-friendly, efficient, and satisfying, leading to higher user adoption and satisfaction.

Golang-ilities

As discussed before, Golang (Go) is a programming language designed with simplicity, performance, and ease of development in mind. Its features and characteristics make it well-suited to addressing many of the ilities of distributed systems. Here is how Go helps with them:

- **Availability:** Go programs compile to single binaries with minimal dependencies, ensuring quick startup times, which is beneficial for rapid recovery and availability. Go's defer, panic, and recover mechanisms help manage errors and ensure graceful degradation and recovery.
- **Scalability:** Go's lightweight goroutines and channels provide a simple and efficient way to handle concurrency, making it easier to scale applications horizontally. Go's runtime efficiently manages goroutines, allowing high concurrency without significant resource overhead.
- **Reliability:** Go's static typing and compile-time checks catch many errors early, enhancing code reliability. The extensive standard library provides reliable and well-tested tools for building robust applications without reinventing the wheel for such functionality. Go's simple and

consistent syntax also reduces the likelihood of inconsistencies in the codebase. Commands like **gofmt** and **go vet** ensure consistent formatting and static analysis across the codebase to provide a reliable codebase at each commit. Libraries like **go-kit** or **go-micro** provide frameworks that are pluggable and can be used to solve distributed system requirements.

- **Maintainability:** Go emphasizes simplicity and readability, making it easier for developers to understand, maintain, and extend code. Go's built-in formatting command **gofmt** enforces a consistent code style across the codebase, improving maintainability even if a different developer modifies the code. Go modules **go.mod** manage dependencies efficiently, ensuring that the correct versions are used consistently.
- **Usability:** Go's simple and clear syntax makes it easy to learn and use. It has built-in tools for formatting, testing, and documentation that enhance the developer experience and usability. The package system encourages modular design, making it easier to build flexible and extensible applications. Golang's performance characteristics and concurrency model make it a popular choice for quickly building highly usable microservices architectures.
- **Efficiency:** Go compiles to native machine code, which provides high performance and efficient execution. Its fast compilation times enhance development efficiency, allowing for rapid iteration and testing. Go's runtime is optimized for performance, with efficient garbage collection and low-latency execution suitable for performance-critical applications with efficient memory management.
- **Security:** Go's design and ease of implementation avoid common security pitfalls like null pointer dereferencing and buffer overflows. The standard library includes secure implementations of cryptographic functions and other security features like authentication. Go promotes best practices and standard coding conventions, helping ensure compliance with coding standards. The inclusion of security-focused libraries helps ensure compliance with security standards and regulations.

- **Resilience:** Go's explicit error handling encourages developers to handle errors appropriately, improving resilience. Go's concurrency model makes it easier to design applications that can recover from failures and continue operating. There are patterns such as circuit breakers, retries and graceful degradation that can help application continuity in the face of unexpected conditions:
 - **Circuit breaker:** This mechanism provides a way for the system to skip retrying operations that can lead to cascading failures. The **go-resiliency/circuit breaker** library provides this capability.
 - **Retries:** A standard pattern across programming languages, this option helps automatically retry a failed operation a limited number of times, using exponential back-off each time, thus reducing the system load on failures.
 - **Graceful degradation:** As the name indicates, this pattern helps run the system in a partial functionality mode and lets it get into a complete service collapse state.

By leveraging these features and practices, Go helps developers build distributed systems that are scalable, reliable, maintainable, and performant, thus addressing many of the critical ilities effectively.

Conclusion

In this chapter we reasoned about some of the operational aspects of distributed systems and how to keep it running efficiently, with high availability and automated monitoring of it around the clock.

One should be able to understand the ilities needed to keep the system always running, which can help push the envelope to build and run robust applications. Some of the inherent aspects of distributed systems could impact latency due to data consistency requirements, error handling retry logic and could impact the user experience too.

With this background in distributed systems, let us look at the real-world application deployment and monitoring options using the latest

containerization techniques in the next chapter.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 9

Containerization

Introduction

With the advent of microservice architectures in the mid-2010s, the rise of containerized web and modern applications has taken off in the enterprise software world. It has been evolving for easier deployment streamlining, better post-deployment management, and observability. This chapter is an extension of [Chapter 4, Microservices](#), where we discussed containerization components.

Treating containers like a black box will eventually leave you in the dark.
-Kelsey Hightower

Now, we will shed further light on real-world steps and deep dive into how containers can be created and deployed securely for Golang built software packages along with the various options available for container runtime monitoring and management.

Structure

The chapter covers the following topics:

- Creating containers
- Deploying containers
- Container management tools

- Container security
- Health checks
- Container APIs in Golang

Objectives

This chapter provides a practical hands-on solution to use Kubernetes containers to run and monitor a Golang application. The reader can apply the theoretical knowledge gained from the previous chapters to develop and deploy an industry standard application.

Security and application integrity needed for configuration, data, and resource protection are also tied into the deployment to keep the system hack-free and stable.

We will end the chapter with the necessary operational information about monitoring such distributed applications. We will use existing frameworks that seamlessly track application behavior and can alert and predict changes in access patterns.

Creating containers

Container is a software package of a logical solution, such as a single service, that contains all components needed to run on any platform (or runtime environment). Creating containers typically refers to using Docker to package and run any application such as a containerized environment. This section provides a step-by-step guide on how to do this with a sample Golang server program.

Creating a Go application

First, we write a sample Golang application that creates a server listening on a known port. For example, let us create a simple web server which listens on port **8080**:

1. `package main`
- 2.
3. `import (`


```

4.  "fmt"
5.  "net/http"
6. )
7.
8. func handler(w http.ResponseWriter, r *http.Request) {
9.     fmt.Fprintf(w, "Hello, World!")
10. }
11.
12. func main() {
13.     http.HandleFunc("/", handler)
14.     http.ListenAndServe(":8080", nil)
15. }

```

Save this as **server.go** to be used in the following step.

Creating a Dockerfile

Next, we will create a **Dockerfile** that defines how your Go application should be built and run inside a container, as shown:

```

1. # First, use latest official Golang docker image as the base
2. FROM golang:1.23
3.
4. # Set the Current Working Directory inside the container
5. WORKDIR /app
6.
7. # Copy the source from the current directory to the working
8. # directory inside the container
9. COPY . .
10.
11. # Build the Go app
12. RUN go build -o server .
13.
14. # Expose port 8080 to the outside world

```

15. **EXPOSE** 8080
- 16.
17. *# Command to run the executable*
18. **CMD** ["/server"]

Building the Docker image

Next, go to a terminal in the directory containing the above **Dockerfile** and **server.go**, and build the Docker image:

1. `docker build -t my-go-server .`

This command tells the Docker engine to build an image using the **Dockerfile** in the current directory (.) and to tag it as **my-go-server**. Note that the Docker daemon should be installed and running on your operating system for this to run successfully.

Running the Docker container

Once the image is built, you can run it as a container:

1. `docker run -p 8080:8080 my-go-server`

This command maps port **8080** on your local machine to port **8080** on the container, allowing you to access the Go application via **http://localhost:8080**.

Application testing

Open a web browser or use **curl** to test your application:

1. `curl http://localhost:8080`

You should see the output **Hello, World!** on the browser or command line.

Docker image registry

This is an optional step if you want to share your containerized application. One can push the Docker image to a registry like Docker Hub for public access:

1. `docker tag my-go-server username/my-go-server`

2. `docker push username/my-go-server`

Deploying the container

You can now deploy this container on any Docker-compatible infrastructure, such as Kubernetes, AWS ECS, or even on your local machine. In the next section, we will discuss this topic in detail.

This is a basic setup, and depending on the use cases, more advanced options are available for optimization, multi-stage builds, environment variables, and other configurations.

Deploying containers

Deploying an application container using Kubernetes involves creating and applying a series of Kubernetes resources, such as Pods, Deployments, and Services. This section provides a step-by-step guide to deploy a containerized application on Kubernetes.

Prerequisites

Ensure you have access to a Kubernetes cluster that is running and logged into it before issuing the commands below. You can use a local setup like minikube, Docker Desktop, or your own cluster in an on-premise data center or just repurpose cloud provider options like **Google Kubernetes Engine (GKE)**, Amazon EKS, or Azure AKS.

The Kubernetes command-line tool, **kubectl**, should be installed and configured to interact with the chosen cluster.

Creating a deployment YAML file

A Kubernetes deployment is responsible for managing a set of identical Pods running your application. Here is a basic **deployment.yaml** file that captures the main aspects of such a setup:

1. `apiVersion: apps/v1`
2. `kind: Deployment`
3. `metadata:`

```
4.  name: my-go-server-deployment
5.  spec:
6.    replicas: 3
7.    selector:
8.      matchLabels:
9.        app: my-go-server
10.   template:
11.     metadata:
12.       labels:
13.         app: my-go-server
14.     spec:
15.       containers:
16.         - name: my-go-server-container
17.           image: username/my-go-server:latest # Replace with your Docker image
18.           ports:
19.             - containerPort: 8080
```

Following are some of the customizable fields in the above spec:

- **replicas**: Specifies the number of copies (or Pods) to run.
- **image**: Replace **username/my-go-server:latest** with the Docker image of your Go application as needed.
- **containerPort**: This is the port your application listens on within the container.

Once this file is present, we also need a way to set up access to that port from clients/users.

Creating a service YAML file

A Kubernetes service exposes your application to the user network, allowing external traffic to reach it. Here is a sample **service.yaml** file:

```
1. apiVersion: v1
```

2. **kind:** Service
3. **metadata:**
4. **name:** my-go-server-service
5. **spec:**
6. **selector:**
7. **app:** my-go-server
8. **ports:**
9. - **protocol:** TCP
10. **port:** 80
11. **targetPort:** 8080
12. **type:** LoadBalancer

Salient fields include:

- **selector:** Ensures the service targets the Pods with the label **app: my-go-server**.
- **port:** The external port exposed by this service (for example, **80**).
- **targetPort:** The port that the application listens on inside the container (for example, **8080**).
- **type:** This type exposes the Service externally using a cloud provider's load balancer (so the value is **LoadBalancer**). For local setups like minikube, you can use NodePort.

Deploying to Kubernetes

Use **kubectl** to deploy your application onto the current Kubernetes cluster in the following two steps:

Apply the deployment file:

1. **kubectl apply -f deployment.yaml**

Deploy the service:

1. **kubectl apply -f service.yaml**

Verifying the deployment

You can verify that your application is running as expected using the following commands that check various aspects of the deployment:

- Pods status:
 1. `kubectl get pods`
- Service status:
 1. `kubectl get services`
- External IP for LoadBalancer:
 1. `kubectl get svc my-go-server-service`

If you are using a cloud provider, the external IP field will show the public IP. You can access your application by navigating to this IP on the browser or via other secure channels.

For local setups like minikube, use the following to check the service state:

1. `minikube service my-go-server-service`

Accessing the application

Once deployed, access the application via the external IP or through the command provided by minikube test bed.

These steps are a basic guide, but Kubernetes offers powerful tools for scaling, updating, and managing applications in production environments. Reading the documentation provided by Kubernetes via manual help or online docs will provide the developer with more clarity and different options to understand the deployment states and steps to fix them as needed.

Container management tools

After the initial deployment, Kubernetes provides several built-in tools and features for scaling, updating, and managing applications and their resources for day-two operations. These tools help ensure that your applications are resilient, scalable, and easy to maintain. This section lists some of the key Kubernetes tools and concepts used for these purposes.

Horizontal Pod Autoscaler

Horizontal Pod Autoscaler (HPA) helps automatically scale the number of Pod replicas in a deployment or replication controller based on observed CPU utilization or other custom resource metrics. The HPA tooling system continually monitors Pod resource usage and adjusts the number of replicas to maintain a desired level of utilization (say, 50% CPU).

```
1. kubectl autoscale deployment my-deployment --cpu-percent=50  
   --min=1 --max=10
```

This command helps increase the number of Pods by a maximum of 10 when CPU usage is above 50% on the average across all existing nodes. One can also use custom metrics for scaling by integrating Kubernetes with a monitoring system like Prometheus.

Another Pod level scaling option is via the **Vertical Pod Autoscaler (VPA)**, which adjusts the resource requests and limits within a container.

Cluster Autoscaler

For the case where the cluster is running out of free nodes, this **Cluster Autoscaler (CA)** automatically adjusts the size of the Kubernetes cluster by adding or removing nodes based on the node requirements for the existing workloads. When the application Pods cannot be scheduled due to insufficient cluster resources, the CA increases the cluster size by adding existing free nodes in the data center (or cloud) into the cluster as usable nodes. CA is a deployment type resource which does not look at the Pods CPU or memory usage, but only at count of Pods in un-schedulable (or pending) state.

Conversely, it scales down the cluster when resources are underutilized. This tool is typically integrated with cloud provider services like AWS Auto Scaling Groups, GCP Managed instance groups or Azure Virtual Machine Scale Sets.

Rolling updates

For large-scale applications, the system's availability needs to be maintained even during software update cycles. Kubernetes provides the option to gradually update the Pods in a deployment with new versions of

your application without users seeing any downtime. Kubernetes replaces old Pods with new ones incrementally, ensuring that some application instances remain available during the update process.

One can add the following to the deployment YAML spec:

1. `minReadySeconds: 5`
2. `strategy:`
3. `type: RollingUpdate`
4. `rollingUpdate:`
5. `maxSurge: 2`
6. `maxUnavailable: 1`

Here are the main aspects specified in that sample YAML:

- `minReadySeconds`: Tells Kubernetes how long it should wait until it creates the next Pod. This property ensures that all application Pods are ready during the update.
- `maxSurge`: Specifies the maximum number (or percentage) of Pods above the specified number of replicas. In the example above, the maximum number of Pods will be 5 since 3 replicas are specified in the `.yaml` file.
- `maxUnavailable`: Suggests the maximum number (or percentage) of unavailable Pods during the update. If **`maxSurge`** is set to 0, this field cannot be 0.

These values are honored when commands like the one below are run to update Pods to a newer image:

1. `kubectl set image deployment/my-deployment my-container=myimage:2.0`

One can also configure the rollout strategy, such as **`maxSurge`** and **`maxUnavailable`**, to control the speed and availability during the update.

Canary deployments

Instead of a dedicated testing cluster with a simulated workload that could double the cost of operations, there is another way for containers to be

tested. Canary deployments gradually shift traffic to a new version of the application to test it under real-world conditions before fully rolling it out.

A small percentage of traffic is automatically routed to the new version Pods. Based on its performance and metrics, developers can increase the traffic in incremental steps or roll back changes fully. This can be done using Kubernetes native tools by creating a different deployment YAML file with the new image version and fewer replicas and then slowly reducing the primary/stable replicas and increasing these after ensuring the system is always fully functional.

Rollout management

Another useful tool provided, rollout management, helps manage and track the rollout status of Kubernetes deployments. **kubectl rollout** commands allow you to check the status of a deployment, pause, resume, or roll back changes if necessary. Some sample options include:

- **Check status:** **kubectl rollout status deployment/my-deployment**
- **Roll back:** **kubectl rollout undo deployment/my-deployment**
- **Pause:** **kubectl rollout pause deployment/my-deployment**
- **Resume:** **kubectl rollout resume deployment/my-deployment**

Helm

Helm charts allow you to package, manage, and deploy complex Kubernetes applications. They comprise a set of packages containing pre-configured Kubernetes resources. Helm also allows you to easily manage dependencies and version deployments. The usual steps are to create and then install the custom charts are:

1. `helm create test-helm`

This creates a directory that contains deployment, service; ingress manifests in the templates folder. They can be customized and then installed using commands like:

1. `helm install --name test-helm .`

Kustomize

This tool provides customized Kubernetes YAML configurations without modifying the original files. Kustomize allows you to define overlays on top of base configurations, making it easier to manage different environments (for example, dev, staging, production). Some of the main steps for using Kustomize are as follows:

- Create, customize, and install a helm chart as shown in the previous section.
- Create a base Kustomize configuration in a **kustomize/base** directory containing deployment and service **.yaml** files, for example.
- Create **kustomization.yaml** in that **base** folder with those two files are resources:

1. resources:
2. - **deployment.yaml**
3. - **service.yaml**

- Create an overlays related sub directory like **kustomize/overlays/dev** with a **kustomization.yaml** in that directory having contents:

1. resources:
2. - **../..base**
- 3.
4. patchesStrategicMerge:
5. - **deployment-patch.yaml**

The **deployment-patch.yaml** file will contain the difference from the base deployment, like number of replicas or other specs.

- Finally, just apply that overlays subdir like:

1. **kubectl** apply -k kustomize/overlays/dev

Kustomize is natively integrated with **kubectl**, so you can use it directly without additional installations.

Prometheus and Grafana

These industry standard tools help monitor and visualize the performance and health of aspects of the Kubernetes clusters and applications.

Prometheus scrapes metrics from registered applications and cluster

components, and Grafana visualizes this data in dashboards. One can deploy Prometheus and Grafana using Helm charts or YAML files. It can integrate Prometheus with Kubernetes to collect metrics such as CPU, memory usage, and Pod status. Prometheus can be used to provide custom metrics for HPA.

Service mesh

Another aspect of container deployments is to secure and monitor traffic between microservices in the Kubernetes cluster. Service meshes like Istio and Linkerd inject proxies alongside deployed services, providing capabilities like traffic management, security policies, and observability. One can deploy these tools and define traffic routing, load balancing, retries count, and fault injection using custom resources. These tools provide traffic control (e.g., canary releases and A/B testing), secure service-to-service communication, and detailed metrics and tracing.

All the above tools and operations allow Kubernetes to provide robust, scalable, and manageable environments for deploying and operating modern applications.

Container security

As alluded to in the previous section, establishing security practices is essential for ensuring containerized applications run without interruptions. Security and **role-based access control (RBAC)** are critical aspects of containerized application management, particularly in Kubernetes environments. In this section, we outline how security is handled both at the container level and within the Kubernetes orchestrator and how RBAC is implemented to control access.

Container image security

Using tools like Clair, Trivy or Anchore one can statically scan container images for vulnerabilities. This should ideally be integrated into the CI/CD pipeline to ensure that only secure images are deployed in production. Use standard minimal base images like Alpine Linux can reduce the number of potential vulnerabilities. Lastly, tools like Notary or Cosign can be used to

sign container images, ensuring their integrity and authenticity before deployment.

Container runtime security

Runtime security tools like Falco or Sysdig Secure can monitor running containers for unusual behavior. These tools can detect and alert on anomalies such as unauthorized network connections or file modifications. Also, using Kubernetes resource requests and limits to constrain CPU and memory usage prevents any single container from monopolizing resources.

One can also configure the security context for containers to enforce security policies. For example, running containers as non-root users, dropping unnecessary Linux capabilities, and mounting filesystems as read-only. Below, we set read only file access in the Pod deployment YAML specification:

1. `securityContext:`
2. `runAsUser: 1000`
3. `runAsGroup: 3000`
4. `fsGroup: 2000`
5. `capabilities:`
6. `drop:`
7. `- ALL`
8. `readOnlyRootFilesystem: true`

Network security

Kubernetes networking is an integral part allowing communication across containerized applications. It connects Pods, services, and external users, enabling communication within and outside the Kubernetes cluster. Pods within the same cluster communicate without the need for **network address translation (NAT)** as each Pod gets a unique IP address. Services provide stable networking endpoints on top of Pods, suitable for load balancing and discovery. Ingress resource can be used to expose HTTP(S) routes from outside the cluster to services in the cluster, and egress option allows Pods to send traffic to targets outside the cluster.

Using Kubernetes network policies to control the traffic between Pods is recommended. This ensures that Pods can only communicate with those they are supposed to, reducing the risk of traffic spread impact in case of a breach:

```
1.  kind: NetworkPolicy
2.  apiVersion: networking.k8s.io/v1
3.  metadata:
4.    name: allow-specific
5.  spec:
6.    podSelector:
7.      matchLabels:
8.        app: my-app
9.    policyTypes:
10.   - Ingress
11.   - Egress
12.    ingress:
13.   - from:
14.     - podSelector:
15.       matchLabels:
16.         app: allowed-app
17.    egress:
18.   - to:
19.     - podSelector:
20.       matchLabels:
21.         app: allowed-app
```

Implementing a service mesh (for example, Istio, Linkerd) as mentioned before can enforce **mutual TLS (mTLS)** between services, securing the traffic within the cluster.

Secrets management

Storing sensitive information like API keys, passwords, and certificates can be performed using Kubernetes Secrets resource types. Secrets are base64-encoded but not encrypted by default, so use encryption at rest and ensure that secrets are mounted as files or environment variables in Pods:

1. `apiVersion: v1`
2. `kind: Secret`
3. `metadata:`
4. `name: my-secret`
5. `type: Opaque`
6. `data:`
7. `username: mylogin`
8. `password: needbetterpassword`

Integrating Kubernetes with external secret management tools like Hashicorp Vault, AWS Secrets Manager, or Azure Key Vault can be quicker and robust secret management providing auditing capabilities.

Role-based access control

RBAC in Kubernetes controls access to the Kubernetes API, enabling one to define who (persona) can perform what actions (verbs) on which entities (resources) within the cluster. The main components of this include:

- **Roles:** Define a set of permissions within a specific namespace.
- **ClusterRoles:** Similar to Roles but applies to the whole Kubernetes cluster scope.
- **RoleBinding:** Assign a Role to a user, group, or service account within a namespace.
- **ClusterRoleBinding:** Assign a ClusterRole to a user, group, or service account at the cluster level.

Following is the sample YAML for setting the role and its binding. Here the user **janedoe** is allowed only read access to Pods using Pod-reader role:

1. `apiVersion: rbac.authorization.k8s.io/v1`
2. `kind: Role`
3. `metadata:`

```
4. namespace: default
5. name: pod-reader
6. rules:
7. - apiGroups: [""]
8.   resources: ["pods"]
9.   verbs: ["get", "watch", "list"]
10.
11. ---
12. apiVersion: rbac.authorization.k8s.io/v1
13. kind: RoleBinding
14. metadata:
15.   name: read-pods
16.   namespace: default
17. subjects:
18. - kind: User
19.   name: janedoe
20.   apiGroup: rbac.authorization.k8s.io
21. roleRef:
22.   kind: Role
23.   name: pod-reader
24.   apiGroup: rbac.authorization.k8s.io
```

Pod Security Standards

Kubernetes provides predefined **Pod Security Standards (PSS)** to help operators apply consistent security controls based on best practices. Three levels of isolation are possible:

- **Privileged:** Least restrictive, suitable for trusted workloads.
- **Baseline:** Default policy with reasonable defaults for most workloads.
- **Restricted:** Most restrictive, suitable for untrusted or critical workloads.

These levels can be chosen based on the application requirements and are enforced via Pod Security Admission or other mechanisms like OPA

discussed next.

Admission controllers

Admission controllers are plugins that govern and enforce how the containers in the cluster are set up. They can be used to enforce policies like preventing the deployment of containers running as root or requiring that images are pulled only from trusted registries. Some of the common controllers include:

- **Pod Security Admission:** Enforces security-related policy standards for restricting the creation or access to Pods. Prior to Kubernetes v1.21, this was managed by the **Pod Security Policy (PSP)**, feature.
- **OPA/Gatekeeper:** Enforces custom policies using the **Open Policy Agent (OPA)**. And Gatekeeper integrates OPA with Kubernetes admission control to enforce policies across the cluster.
- **ImagePolicyWebhook:** This check using webhooks allows for dynamic admission control of images to be used in the cluster.

Service accounts

Trusted entities can be set up to prevent unauthorized users from using the Kubernetes cluster. These service accounts provide a way to run Pods with specific permissions within the cluster. When a Pod starts to run, it can authenticate to the Kubernetes API using a service account token. Permissions are granted to service accounts using **RoleBinding** or **ClusterRoleBinding**, as shown:

1. `apiVersion: v1`
2. `kind: ServiceAccount`
3. `metadata:`
4. `name: my-service-account`
5. `namespace: default`
- 6.
7. `---`
8. `apiVersion: rbac.authorization.k8s.io/v1`

9. **kind:** RoleBinding
10. **metadata:**
11. **name:** bind-service-account
12. **namespace:** default
13. **subjects:**
14. - **kind:** ServiceAccount
15. **name:** my-service-account
16. **namespace:** default
17. **roleRef:**
18. **kind:** Role
19. **name:** pod-reader
20. **apiGroup:** rbac.authorization.k8s.io

Encryption and certificates

Kubernetes supports data encryption in etcd, including secrets and other sensitive data. Enabling encryption providers in the cluster API server can configure this. Secure communication between Kubernetes components can also be established using TLS certificates. You can manage these certificates using tools like **cert-manager** or integrate with external certificate authorities.

Auditing

Along with preemption, tracking and checking for security lapses helps catch gaps in setup or new features needed against hackers. Kubernetes provides auditing capabilities to track and record API requests made to the cluster. This can be configured to monitor, log, and analyze access patterns and potential security incidents. One can define an audit policy to specify which events are logged:

1. **apiVersion:** audit.k8s.io/v1
2. **kind:** Policy
3. **rules:**
4. - **level:** Metadata

5. `verbs: ["create", "update", "delete"]`
6. `resources:`
7. `- group: ""`
8. `resources: ["pods"]`

In summary, the best practices include regularly scanning and updating images, using least privilege principles, enforcing network and Pod security policies, and monitoring your cluster for security incidents. Implementing these security practices helps ensure that your containerized applications are protected and that access is tightly controlled for use by known personas.

Health checks

At runtime, Kubernetes provides various types of status checks (also known as **probes**) to monitor the health and readiness of applications running within containers. These checks help ensure that your applications are functioning correctly and can handle traffic—and corrective action can be taken for any red flags from these checks. The main types of status checks are discussed in this section.

Liveness probe

A liveness probe checks whether a container is running and responsive. If the liveness probe fails, Kubernetes will restart the container to restore it to a healthy state. These checks help detect and recover from situations where an application is stuck or deadlocked. There are three configuration options to set this in the Pod template:

- **HTTP GET:** Sends an HTTP GET request to the container.
- **TCP Socket:** Checks if a TCP connection can be established to the container.
- **Exec Probe:** Runs a command inside the container and checks the exit status.

Here is an example to set up a liveness probe using the http get endpoint, with a check every 10 seconds:

1. `livenessProbe:`

2. `httpGet:`
3. `path: /healthz`
4. `port: 8080`
5. `initialDelaySeconds: 3`
6. `periodSeconds: 10`

Readiness probe

A readiness probe checks whether the application process inside a container is ready to serve traffic. If a readiness probe fails, the Pod will be marked as not ready, and it will be removed from the service's load balancers. This helps ensure that the application has finished initializing and is ready to accept user requests. The configuration options are the same as the liveness probe, and here is an example of an HTTP GET based periodic check every five seconds:

1. `readinessProbe:`
2. `httpGet:`
3. `path: /ready`
4. `port: 8080`
5. `initialDelaySeconds: 5`
6. `periodSeconds: 5`

Startup probe

A startup probe is used to determine if the application within the container has been spawned. It is useful for containers that take a long time to start up during too many setup steps or dependencies. If the startup probe fails after the time threshold, the container will be killed and restarted. This is useful for applications with long initialization times, ensuring that liveness and readiness probes are only applied after the startup probe succeeds. Using similar configuration options as earlier probes one can http end point check which fails after 30 seconds threshold, as shown:

1. `startupProbe:`
2. `httpGet:`
3. `path: /startup`

4. `port: 8080`
5. `initialDelaySeconds: 10`
6. `periodSeconds: 5`
7. `failureThreshold: 30`

Customized checks

The above checks are generic and apply to most applications. However, sometimes basic HTTP, TCP, or exec probes may not be sufficient, and one might want to use custom application metrics to determine health status. Such applications might need to monitor internal states like memory usage, queue depth, resource limit, or custom business metrics. These requirements can typically be achieved by integrating with a monitoring system like Prometheus and using the HPA or a custom controller to scale or manage Pods based on these metrics.

Event-based checks

Kubernetes events can also be monitored to detect specific conditions, such as Pod crashes, network failures, or similar events of interest. These can be used to trigger alerts, scale up or down the replicas, or initiate other recovery procedures based on events. To achieve this, one can use tools like Fluentd or other custom controllers that watch Kubernetes events and react accordingly.

Using these probes and checks, Kubernetes can automatically manage applications health and availability, ensuring that they remain responsive and resilient in production environments.

Container APIs in Golang

Container APIs in Go (Golang) are libraries and interfaces that allow developers to interact with and manage containers programmatically. These APIs enable the creation, management, and orchestration of containers, typically using tools like Docker or Kubernetes. This section will discuss some of the key container APIs available in Go.

Docker APIs

There could be cases where the application container might need to perform Docker operations. There are APIs that allow such Go programs to interact with Docker, providing functionality to manage containers, images, networks, volumes, and more. There are a couple of official ways to interact with docker from Go applications:

- **Docker:** The official Docker Go SDK. This package provides complete access to the Docker API from within a Go application.
- **go-dockerclient:** A lightweight and more idiomatic Go client for Docker, offering a simplified interface to interact with Docker's REST API.

Here is an example for creating a Docker container using the Docker API in Go:

```
1. package main
2.
3. import (
4.     "context"
5.     "github.com/docker/docker/api/types"
6.     "github.com/docker/docker/api/types/container"
7.     "github.com/docker/docker/client"
8.     "fmt"
9. )
10.
11. func main() {
12.     ctx := context.Background()
13.     cli, err := client.NewClientWithOpts(client.FromEnv,
14.         client.WithAPIVersionNegotiation())
15.     if err != nil {
16.         panic(err) // or throw error
17.     }
18.
```

```

19. resp, err := cli.ContainerCreate(ctx, &container.Config{
20.     Image: "myimage-name",
21.     Cmd: []string{"echo", "hello world"},
22. }, nil, nil, nil, "")
23. if err != nil {
24.     panic(err) // or throw error
25. }
26.
27. if err := cli.ContainerStart(ctx, resp.ID,
28.     types.ContainerStartOptions{}); err != nil {
29.     panic(err) // or throw error
30. }
31.
32. fmt.Println("Container started with ID:", resp.ID)
33. }

```

Kubernetes API

The Kubernetes API enables Go developers to interact with Kubernetes clusters. This can include creating and managing Pods, services, deployments, and Kubernetes resources. A couple of standard libraries that can be used in this context include:

- **client-go:** The official Go client for Kubernetes, which allows for managing Kubernetes resources from within a Go application.
- **controller-runtime:** A higher-level framework for building Kubernetes controllers and operators using Go.

Following is an example for creating a Kubernetes Pod using the **client-go** library:

```

1. package main
2.
3. import (
4.     "context"

```

```
5.  "fmt"
6.  "k8s.io/client-go/kubernetes"
7.  "k8s.io/client-go/tools/clientcmd"
8.  v1 "k8s.io/api/core/v1"
9.  metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
10.)
11.
12. func main() {
13.     // Need to have kube config setup
14.     config, err := clientcmd.BuildConfigFromFlags("",
15.         clientcmd.RecommendedHomeFile)
16.     if err != nil {
17.         panic(err)
18.     }
19.
20.     clientset, err := kubernetes.NewForConfig(config)
21.     if err != nil {
22.         panic(err)
23.     }
24.
25.     pod := &v1.Pod {
26.         ObjectMeta: metav1.ObjectMeta{
27.             Name: "child-pod",
28.         },
29.         Spec: v1.PodSpec{
30.             Containers: []v1.Container{
31.                 {
32.                     Name: "childContainer",
33.                     Image: "child-1.10.2",
34.                     Command: []string{
35.                         "sleep", "3600",
```

```
36.         },
37.     },
38.     },
39. },
40. }
41.
42. pod, err = clientset.CoreV1().Pods("default").
43.     Create(context.Background(), pod, metav1.CreateOptions{})
44. if err != nil {
45.     panic(err)
46. }
47.
48. fmt.Printf("Pod created: %s\n", pod.ObjectMeta.Name)
49.
50. // List all the pods in that namespace.
51. pods, err := clientset.CoreV1().Pods("default").List(
52.     context.TODO(), metav1.ListOptions{})
53. if err != nil {
54.     panic(err)
55. }
56.
57. for _, pod := range pods.Items {
58.     // Check pod.Name to confirm above pod creation and status.
59.     fmt.Printf("Pod Name: %s, Status: %s\n",
60.         pod.Name, pod.Status.Phase)
61. }
62.
63. logOptions := &v1.PodLogOptions {
64.     Container: "one-of-the-container-names",
65.     Follow:    true,
66. }
```



```

67. // Get the logs for a given pod.
68. req := clientset.CoreV1().Pods("default")
69.     .GetLogs("child-pod", logOptions)
70. podLogs, err := req.Stream(context.TODO())
71. if err != nil {
72.     panic(err)
73. }
74. defer podLogs.Close()
75.
76. buf := new(bytes.Buffer)
77. _, err = io.Copy(buf, podLogs)
78. if err != nil {
79.     panic(err)
80. }
81.
82. fmt.Println(«Logs:»)
83. fmt.Println(buf.String())
84. }

```

In the above example, we created a Pod with a container in the default namespace. Then, we fetch all the Pods in that namespace, which can be used to confirm the new Pod creation. Also highlighted is the fetching of the logs for the new Pod, which can help dynamic debugging and other use cases. One can also perform multiple Pod management operations using similar APIs to perform updates and deletions of the Pods and other resources as well.

Open Container Initiative APIs

Open Container Initiative (OCI) APIs are designed to interact with containers that adhere to the OCI standards. These APIs focus on container runtime and image specifications. These are potential libraries to use in such cases:

- **opencontainers/runc**: A CLI tool and library for spawning and

running containers according to the OCI specification.

- **containers/image and containers/storage:** Libraries for working with container images and storage according to OCI specifications.

These are typically used in scenarios where compliance with OCI standards is required or when developing custom container runtimes or image management tools.

These container APIs in Go allow developers to programmatically manage containers and orchestrate containerized environments, whether they are working with Docker, Kubernetes, or other container runtimes.

Conclusion

In this chapter, we discussed the steps to create and deploy applications using standard container frameworks and how Kubernetes has become a *de facto standard* to use for containerized application deployment and monitoring.

The basic steps outlined in this chapter provide core foundations to developers for building and maintaining large-scale applications reliably using industry standard tools.

With this knowledge, in the next chapter, we will discuss how this containerized code can be deployed and how cloud technologies provide building blocks for easing such operations.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 10

Code, CI/CD and Cloud

Introduction

Building the code using the right algorithms to ensure it works in large-scale systems is the first step to setting the service up for success. Once the basic aspects of the server functionality are in place, there are more steps of the software lifecycle that make the deployments more robust and easier to maintain on-premises server infrastructure or on public clouds. There are some common development and monitoring tools that work with web-scale services, which we will explore here.

The cloud services companies of all sizes. The cloud is for everyone. The cloud is a democracy.

–Marc Benioff, CEO, Salesforce

Starting in the early 2010s, cloud infrastructure has reduced the need for reinventing the wheel for setting up servers, storage, and networking for any business model. This is especially useful for non-technical industry verticals or for startups to help them focus on their problem and solution space instead of becoming a technical powerhouse. We will investigate how these building blocks help companies build faster across any industry.

Structure

The chapter covers the following topics:

- Code quality
- DevOps
- Continuous integration/continuous deployment
- Day 2 operations
- Public cloud services
- Golang for cloud

Objectives

Software development is more than writing code. It involves writing maintainable code and ensuring it runs well on the servers and provides the right service availability. We will start by seeing the best practices and software industry standards for writing and maintaining high quality Golang code.

Following that we will discuss the recommended steps to ensure the code runs in production smoothly and adheres to the norms of operational efficiency. Since cloud services have become part of the tech stack of many companies, we will learn some of the most common services that systems developers need to be aware of to build systems on top of them.

Code quality

Code quality is an essential aspect of software development that ensures the codebase remains reliable, efficient, and easy to work with over time. A well-maintained codebase is less likely to have bugs and more likely to perform correctly under different conditions. Let us go over an overview of the concepts involved, why they are important, and some best practices to achieve high code quality to ensure long life and easy maintainability. High quality code reduces the time and effort needed to debug, extend, or modify the codebase.

Code is considered high quality if it is:

- **Functional:** It works as expected, without bugs, corner cases or unintended behavior.
- **Readable:** Easy for humans to read, understand and quickly made

changes. This includes following consistent naming conventions, formatting, and style guides.

- **Efficient:** Performs well in terms of run time, latency and throughput. Memory or other resource usage is also not overused or wasted.
- **Maintainable:** Easy to modify, extend, and refactor. This includes having a clear structure, avoiding complex dependencies, and adhering to modular design principles.
- **Testable:** Structured in a way that makes it easy to write tests and verify that the code works as intended.

Clear, readable code with good documentation helps teams work together more effectively and onboard new members more easily. High quality code is easier to refactor and adapt when requirements change, making the codebase more resilient to new demands. Now let us go through the best practices for ensuring high code quality. We use Golang as the example programming language, though the concepts can apply to any other server-side option.

Readable code

Write clear and simple code that is easy to understand and maintain. Avoid unnecessary abstractions or complex patterns unless they provide clear benefits. Functions should do one thing and do it well. This makes them easier to test, understand, and reuse. Follow naming conventions that reflect the purpose of packages, classes, functions, variables and constants.

One should strive to adhere to a consistent coding style, which can be enforced with tools like Prettier for JavaScript or Black for Python. Go has a very opinionated style guide, and `gofmt` is the standard for formatting. It ensures all Go code looks the same, which helps with readability and reduces cognitive load when switching between different repositories.

Follow coding standards

From idiomatic Go, follow naming conventions that reflect the purpose of the entities. Follow the Go community's naming conventions, such as `camelCase` for variable names and `PascalCase` for exported function names, and avoid unnecessary repeated boilerplate code.

Error cases also bear mentioning as they always needed to be handled in code. Go emphasizes explicit error handling from functions, which improves robustness and clarity. When handling errors, return as early as possible to avoid nested code and improve readability. Use **fmt.Errorf** or the **errors** package to wrap errors with additional context during logging, making them easier to understand, as shown in the example below.

```
1. package main
2.
3. import (
4.     "fmt"
5.     "strconv"
6. )
7.
8. func createIdentity(id int) (string, error) {
9.     if id <= 0 {
10.         return "", fmt.Errorf("invalid input value for id: %d",
11.                                id)
12.     }
13.     return "Name " + strconv.Itoa(id), nil
14. }
15.
16. func main() {
17.     user, err := createIdentity(-1)
18.     if err != nil {
19.         fmt.Println("Error:", err)
20.         return
21.     }
22.     fmt.Println("User:", user)
23. }
```

Use comments to document the why of your code, especially for exported functions, types, and packages. Use **godoc** comments (`//` or `/* */`) to describe the purpose and usage of code elements. Good documentation for the use

cases and assumptions is critical for usability and maintenance in the long run.

Use standard library

Most mature programming languages come with built-in standard libraries for very common operations. Go's standard library is powerful and covers a wide range of functionality for data structures, from compression to HTTP connections. Use it whenever possible instead of third-party libraries, as native ones are usually well-tested and maintained. Also, using Go tools such as **go vet**, **golint**, **staticcheck**, **golangci-lint**, and **gocyclo** can help catch common mistakes, enforce style, and detect potential issues in the code before merging into the master branch.

Follow standard secure coding practices to prevent common vulnerabilities such as SQL injection, **cross-site scripting (XSS)**, and buffer overflows. Regularly audit the codebase for security vulnerabilities using tools like Snyk or OWASP dependency-check to check for known vulnerabilities in dependencies.

Testing

As discussed before, all round testing is the key to speeding up production robustness and reducing deployment hiccups. Automated testing is the true north and can be achieved by adding these to the changeset:

- **Unit tests:** Cover any newly added code with unit tests to ensure that each function behaves as expected. Go has a built-in testing framework (**testing** package) that is easy to use, and **JUnit** for Java, **pytest** for Python
- **Test edge cases:** Make sure to test for edge cases and potential error conditions. This helps prevent bugs in unexpected scenarios and handles corner cases correctly up the stack.
- **Use table-driven tests:** In Go, it is common to write table-driven tests, where a slice of test cases is defined, and a single loop iterates over them to run the tests and matches the corresponding expected result.
- **Integration tests:** Ensure that components work together correctly by writing integration tests that test multiple parts of the application

together.

Continuous integration (CI) tools like Jenkins, Travis CI, or GitHub Actions can be used to run tests automatically on every commit.

Documentation

One should make writing meaningful code comments part of muscle memory to explain the why behind any non-obvious code. Always document any public APIs, libraries, and externally imported modules clearly, describing the inputs, outputs, behavior, and errors of each function or method. At a high level, keeping documentation in **README.md** files or wikis to provide an overview of the project, setup instructions, and usage examples is a standard practice.

Code reviews

Some of the main technical learnings for any new software engineer are from reviewing code from experienced team members. So, these options for code review help raise the team bar:

- **Peer reviews:** Have other developers review your code to catch potential issues, ensure adherence to standards, and share knowledge. One can conduct regular code reviews to build expertise among team members. Reviews help maintain code quality and foster a culture of continuous improvement. There are a lot of existing websites that can be used to read guidelines for both creating changeset and the review process itself.
- **Automated code reviews:** Use tools like SonarQube, Codacy, or Code Climate to automatically review your code for potential quality issues, security vulnerabilities, and technical debt.
- **Pair programming:** Pair programming can help improve code quality by providing immediate feedback and fostering collaboration.

Refactoring

As a codebase grows and evolves, there will be potential for simplification or enhancements. One can regularly revisit and refactor code to improve its

structure and readability. Follow the **Don't Repeat Yourself (DRY)** principle to avoid code duplication. Reducing complexity by simplifying logic, using simpler algorithms, or breaking down large functions and classes into smaller, more manageable pieces could help the team maintain and reuse the code more easily.

The goal should be to improve readability, maintainability, and code performance. Reducing technical debt can also be addressed in cases where some to-dos were not skipped earlier due to time constraints. But try to purely keep cosmetic or format refactoring of code in a different change than a more logic-oriented one.

Performance optimization

Some common optimization techniques include:

- **Profiling:** Use profiling tools to identify and optimize performance bottlenecks. Use Go's profiling tools (**pprof**, **trace**) changeset during the testing phase. Here is a sample code for enabling CPU profiling during runtime:

```
1. package main
2.
3. import (
4.     "log"
5.     "os"
6.     "runtime/pprof"
7. )
8.
9. func main() {
10.     f, err := os.Create("cpu_pprof.txt")
11.     if err != nil {
12.         log.Fatal(err)
13.     }
14.     defer f.Close()
15.
```

```
16. // Start CPU profiling
17. pprof.StartCPUProfile(f)
18. defer pprof.StopCPUProfile()
19.
20. // Code to be tested runs here
21. }
```

After that, the Go tool **pprof cpu_pprof.txt** can be used for offline analytics of the profile. A similar setup can be made to capture profiling for heap or thread/goroutine or all those resources. Based on the output of **top**, **web**, and other **pprof** commands, one can narrow down the root cause bottleneck in the given code that was run. One can fix the issues based on the root cause type using these potential, most common options:

- **CPU:** Use faster algorithms when possible and reduce synchronization points in the code. Using goroutines and channels helps reduce dependency on long held locks.
- **Memory/Heap:** Reuse memory allocations where possible or pre-allocate slices to prevent resizing overheads. Ideally can also avoid long hold on memory references or potential memory leaks.
- **Input/Output:** Optimize database queries or other read/write operations to disk or network that reduce the amount of data to be processed in line within the main thread or move it to a goroutine.
- **Resource management:** Be mindful of memory usage and avoid leaks. Optimize for memory usage where possible, as it can impact performance. Reuse slices and buffers where possible. Also, check if data is written to logs, storage, or network can be compressed (with CPU overhead trade-off) to reduce resource usage.
- **Efficient data structures:** Choose the right data structures for the job. For example, use slices instead of arrays for dynamic sizes and maps when you need fast lookups.

Versioning and dependencies

Use a version control system like Git to track changes, collaborate with others, and maintain a history of your codebase. Adopt a branching strategy

(for example, Git flow, GitHub flow) that suits your team's workflow for managing feature development, bug fixes, and releases.

Limiting the number of dependencies reduces security risks and potential conflicts as well. Ensure that dependencies are necessary and actively maintained by regularly updating third-party libraries and frameworks to benefit from bug fixes, security patches, and new features. Use Go modules to manage dependencies effectively. This ensures reproducible builds and avoids dependency conflicts.

By integrating these best practices and tools into the development workflow, one can significantly enhance code quality and maintainability, leading to a more robust, efficient, and adaptable software product. Leveraging Go's tools and conventions, one can ensure the Golang codebase is up to the state-of-the-art quality and performs well.

DevOps

After the development practices above come the deployment steps. DevOps is the name for the combined facets of **development (Dev)** and **operations (Ops)** to streamline and automate the processes involved in building, testing, and deploying software. It aims to improve collaboration between development and operations teams, enhance deployment frequency, reduce failures, and achieve faster time-to-market. DevOps encompasses a broad range of practices, tools, and team cultures and philosophies. This section gives the breakdown of the key components of DevOps.

Continuous integration

CI is the practice of regularly merging code changes into a shared repository. Each merge triggers an automated build and test process, ensuring the code is always working. This reduces potential integration issues when done with more delay, detects problems early, and encourages collaborative development practices.

Continuous delivery

Continuous delivery (CD) extends CI by automatically deploying code changes to a staging environment or production environment after passing

the build and test phases. Sometimes called CD also, this process helps reduce manual toil via a fully automated pipeline. The goal is to always keep the software in a deployable state. This reduces deploying bad quality code risk, increases deployment frequency, and helps quickly release new features for users.

Infrastructure as code

Infrastructure as code (IaC) is the practice of managing and provisioning computing infrastructure using machine-readable definition files, rather than through physical hardware configuration or manual interactive configuration tools. This automates infrastructure management, reduces errors, increases consistency across teams, and improves scalability.

Configuration management

Configuration management involves maintaining computer systems, servers, and software in a desired, consistent state. It includes automation tools that configure and manage these environments. It allows for consistency across environments, reduces configuration drift, and simplifies system management.

Continuous monitoring

Continuous monitoring involves constant monitoring of the application's performance, infrastructure, and security in real-time. It helps identify issues before they affect end-users, find performance bottlenecks, identify security threats, ensure system reliability, and provide insights into user behavior.

Security and compliance

DevSecOps integrates security practices within the DevOps process to ensure that security is a shared responsibility across stakeholder teams throughout the software lifecycle. This step proactively identifies vulnerabilities, ensures compliance with security standards, and reduces the risk of attacks that could cripple the system. We will look more into this in the next chapter.

Collaboration

DevOps emphasizes a culture of collaboration and communication between development, operations, and other teams involved in the software delivery lifecycle. It also involves collecting feedback continuously from automated systems, users, and other stakeholders to improve the software product and development process. This improves team productivity, fosters a shared responsibility culture, reduces silos of assumptions while ensuring quick identification and resolution of issues, aligns the product with user needs, and provides continuous improvement.

Release management

Release management involves planning, scheduling, and controlling the movement of releases to test and live environments. It ensures that releases are delivered smoothly and effectively. Deployment risk, release quality, and ensuring better coordination among teams.

Observability

Observability involves understanding the internal states of a system by examining its outputs, such as logs, metrics, and traces. It allows teams to monitor, debug, and improve system performance and reliability. Provides deep visibility into application performance, helps diagnose issues quickly, and enables proactive performance optimization.

Incident management

Site reliability engineering (SRE) is an integral part of DevOps and is involved in observing the metrics and dashboards. This helps incident management by identifying, analyzing, and responding to incidents correlating to metric pattern changes that may cause disruptions in services. It includes automating the response to incidents to ensure quick recovery. The goal is to minimize downtime, reduce the impact radius of incidents, and improve the reliability and stability of systems.

DevOps is a comprehensive approach that covers every aspect of the software development lifecycle, from coding and building to testing,

deployment, monitoring, and feedback. Following is a pictorial depiction of the steps:

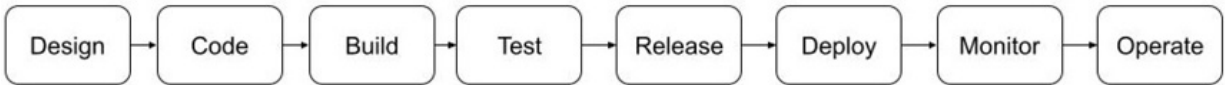


Figure 10.1: Common steps for DevOps teams

DevOps fosters a culture of collaboration, automation, and continuous improvement, enabling teams to deliver high quality software faster and more reliably. By leveraging the best DevOps practices and tools effectively, organizations can achieve significant improvements in resource efficiency, application quality, and user satisfaction.

Continuous integration/continuous deployment

Let us discuss the tools for CI/CD automation that can be used by DevOps team. This integrates and streamlines the software development process with the build, test, and deploy process of applications, ensuring high quality and efficiency. These tools help create workflows and ensure that code changes are continuously integrated, tested, and deployed on *day one*.

Some of the salient features that standard CI/CD tools should include:

- **Automation:** Automate repetitive tasks such as testing, building, and deploying applications. Flag any failures clearly that need fixes.
- **Integration:** Integrate with various development tools like version control systems, issue trackers, and cloud providers.
- **Pipeline management:** Define and manage pipelines that describe the steps for building, testing, and deploying software.
- **Monitoring and reporting:** Provide feedback on build status and times, test results, code quality, and deployment status.
- **Scalability:** Handle multiple projects and large teams with ease, scaling as needed to accommodate future growth.
- **Security:** Integrate security checks to identify vulnerabilities early in the development process.

Here are some of the most widely used CI/CD tools, each with its strengths and typical use cases:

- **Jenkins:** This is one of the most popular open-source CI/CD tools. It is highly extensible, with a vast plugin ecosystem that supports a wide range of development, testing, and deployment tools. It supports a wide range of languages and technologies and is highly customizable with a variety of settings. It allows for complex builds, multi-stage pipelines with Jenkins Pipeline (declarative and scripted pipelines). This is ideal for teams needing a customizable and extensible solution, especially when managing complex builds and deployments.
- **GitHub Actions:** This CI/CD tool is integrated directly into GitHub. It allows developers to automate workflows directly from their GitHub repositories. It supports a variety of containerized build workflows using YAML configuration files and provides a marketplace with pre-built actions to extend functionality. It is best suited for teams already using GitHub for version control and want to streamline their workflows within the GitHub ecosystem. It just needs a **.yaml** file created under the **.github/workflows/** directory, which contains a sequence of steps needed as part of each commit. These could include building the application, running tests, creating and publishing a new docker image, and deploying the application to a Kubernetes cluster.
- **GitLab CI/CD:** This is a powerful, built-in feature of GitLab, a web-based DevOps lifecycle tool. It offers a comprehensive set of features for CI/CD. It supports Kubernetes and Docker, making it ideal for cloud-native applications. It also allows for pipeline visualization and dependency management. This makes it very suitable for teams using GitLab for repository management and seeking an all-in-one solution for DevOps.
- **CircleCI:** This is a cloud-based CI/CD tool that emphasizes ease of use and performance. It integrates well with popular version control systems and offers a fast, scalable platform for CI/CD. It not only provides fast build performance with caching and parallelism features but also allows for custom environments and multiple programming languages. Ideal for teams looking for a cloud-based CI/CD solution with strong Docker

support and fast build times.

- **Travis CI:** Another cloud-based CI/CD tool known for its simplicity and integration with GitHub. It offers both free and paid plans and is popular in the open-source community. It supports multiple programming languages and environments and offers a free tier for open-source projects. Well-suited for open-source projects and when teams want a straightforward CI/CD tool with strong GitHub integration.
- **Azure DevOps:** Coming to cloud solutions, Azure DevOps is a comprehensive suite of DevOps tools provided by *Microsoft*, including Azure Pipelines for CI/CD. It also supports multiple languages and platforms, including .NET, Java, Node.js, Python, and more with built-in support for Kubernetes and containerized deployments. Providing integrated test management and artifact storage it is ideal for teams using *Microsoft* technologies or those who want a comprehensive set of DevOps tools in one platform.
- **Amazon Web Services (AWS) CodePipeline:** This is a fully managed CI/CD service that integrates with other AWS services to automate release pipelines for fast and reliable application and infrastructure updates. Having deep integration with other AWS services like AWS Lambda, EC2, S3, and CodeBuild it also provides a visual editor to build pipelines with drag-and-drop functionality. Tight out-of-the-box support for AWS services makes it perfect for teams heavily invested in the AWS ecosystem looking for a native CI/CD solution.

To summarize, when selecting a CI/CD tool, consider the following factors:

- **Integration with existing tools:** Choose a tool that integrates well with your version control system, issue tracker, and other tools your team uses.
- **Scalability:** Consider the size of your team and projects. Some tools are better suited for small teams and projects, while others can scale to handle large, complex builds and allow for growth easily.
- **Ease of use:** Look for tools that match your team's expertise level and offer user-friendly interfaces or configuration options.

- **Cost:** Consider the pricing model of the tool, including any hidden costs associated with scaling, plugins, or additional features.
- **Platform support:** Ensure the tool supports the languages and platforms you are using in your projects.
- **Cloud vs. on-premises:** Decide whether you need a cloud-based solution for easy setup and scalability or an on-premises solution for more control and security.

By considering these factors and understanding the features and use cases of different CI/CD tools, you can choose the best tool for your team's needs and workflow.

Day 2 operations

The above sections covered the main aspects of getting the service up and running, normally referred to as **initial deployment** or **Day 1** operations of a system. In the context of distributed systems, **Day 2** operations refer to the ongoing management and operational tasks that occur after that. So, Day 2 operations involve the continuous maintenance and optimization required to keep the system running smoothly over time. Let us look at the key aspects of Day 2 operations that need to be driven by DevOps teams:

- **Monitoring:** Implementing tools like Graphana and putting in place processes to continuously monitor system performance, health, and resource utilization helps monitor the system. Setting up alerts for any issues that might arise, such as performance degradation, downtime, or failures can help address issues quickly.
- **Scaling:** Adjusting system resources (for example, adding or removing servers, storage, or network capacity) to handle changing workloads and traffic patterns is a crucial aspect of running any large-scale system. Implementing auto scaling mechanisms to dynamically adjust these resources based on traffic demand or threshold of CPU usage increase can be set in place to adjust to user load.
- **Backup and disaster recovery:** Ensuring that data is regularly backed up and that recovery procedures are in place and tested is a must have. This planning and implementation of strategies for disaster recovery

helps minimize downtime and data loss in case of catastrophic events.

- **Security management:** Continuously apply security patches and updates to keep the system secure. This can be achieved by monitoring and responding to security threats and vulnerabilities, as we will see in [Chapter 11](#), *Securing Your Server*.
- **Performance optimization:** Analyzing system performance metrics to identify bottlenecks and areas for improvement can help tune the system components and configurations to optimize performance.
- **Software updates:** Regularly update software components to include new features, enhancements, and bug fixes from needed external software products. This manages dependencies and ensures compatibility between different parts of the system.
- **Incident management:** Developing and following procedures for identifying, diagnosing, and resolving incidents is also part of ensuring good user experience. Performing root cause analysis and post mortems can help prevent future occurrences of similar issues.
- **Configuration management:** One can maintain and manage system configurations to ensure consistency and reliability of usage of lower-level infrastructure needed by the system. Using tools and practices like IaC can automate and version control the configurations.
- **Compliance and auditing:** One should ensure that the system complies with relevant regulations and standards, in all the geographic regions where the software is deployed. Conducting regular audits and assessments should help verify compliance and identify areas for improvement.
- **User and access management:** Managing user accounts, permissions, and roles to ensure appropriate access controls is essential, as we will see in [Chapter 11](#), *Securing Your Server*. Monitoring and auditing access also detects and prevents unauthorized access.

Overall, Day 2 operations are critical for ensuring the reliability, performance, security, and scalability of distributed systems in production environments.

Public cloud services

Public cloud usage has become a cornerstone of modern IT infrastructure, providing businesses and individuals with flexible, scalable, and cost-effective computing resources. A public cloud is a cloud computing model in which third-party service providers deliver computing resources—such as servers, storage, applications, and networking—over the internet. These resources are shared among multiple customers, but each customer's data and services are isolated and secure via multi-tenancy options. Let us discuss the key aspects of public clouds that help get the most out of such systems.

Service types

Public cloud services are broadly categorized into three main types:

- **Infrastructure as a service (IaaS):** IaaS provides virtualized computing resources over the internet, including **virtual machines (VMs)**, storage, and networking. This is the lowest layer of infrastructure upon which website hosting, development, and testing environments, big data processing, and disaster recovery can be implemented by users.
- **Platform as a service (PaaS):** PaaS is a service platform allowing customers to develop, run, and manage applications without worrying about the underlying infrastructure. This is useful for application development and deployment, automated scaling, and management of web applications.
- **Software as a service (SaaS):** In this mode, SaaS delivers software applications over the internet on a subscription basis. The cloud provider manages the infrastructure, application software, and data. This is a completely hosted solution and useful for **customer relationship management (CRM)** and **enterprise resource planning (ERP)** applications.

Benefits of public cloud

Popularity of the cloud services stays high due to the following advantages:

- **Scalability:** Public cloud providers offer virtually unlimited scalability,

allowing businesses to quickly adjust their computing resources based on demand. This flexibility is crucial for handling peak loads, such as during a product launch or holiday season.

- **Cost efficiency:** Clouds operate on a pay-as-you-go model, meaning businesses only pay for the resources they use. This can significantly reduce capital expenditures on hardware and software and lower the cost of IT operations by not operating at maximum capacity.
- **Global reach:** The large-scale public cloud providers have a global network of data centers, enabling businesses to deploy applications and services closer to their customers and ensure low-latency, high-performance experiences. This also provides geographic disaster recoverability.
- **High availability:** Most cloud providers offer built-in redundancy and disaster recovery solutions, ensuring high availability and reliability for hosted applications and services.
- **Managed services:** Many public cloud providers offer managed services, such as databases, **machine learning (ML)**, and analytics, allowing businesses to focus on their core competencies while leveraging the cloud provider's expertise.
- **Security and compliance:** Public cloud providers invest heavily in security and compliance, offering features such as encryption, **identity and access management (IAM)**, and compliance with regulations like GDPR, HIPAA, and PCI DSS.

Sample use cases

Some use cases that are commonly supported on public cloud include:

- **Application development:** Public clouds provide a flexible environment for developing, testing, and deploying applications. Developers can quickly provision resources, experiment with new technologies, and scale environments as needed.
- **Data storage:** All cloud services offer scalable storage solutions that can be used for storing large amounts of data and creating periodic backups at certain times. This is ideal for companies that need to store

unstructured data, such as media files, or require a reliable backup solution.

- **Disaster recovery:** Public clouds offer cost-effective disaster recovery solutions that ensure business continuity in the event of data loss or hardware failure. Businesses can replicate their data and applications to different geographic locations to protect against regional outages or catastrophes.
- **Big data and analytics:** The computational power and storage needed to analyze large datasets and perform complex calculations is the corner stone of most cloud providers. This is essential for businesses looking to gain insights from big data, conduct ML experiments, or run high-performance computing tasks.
- **Content delivery networks (CDNs):** CDNs can cache content in multiple geographic locations to ensure fast delivery to users worldwide. This is particularly useful for media companies, e-commerce platforms, and any business with a global audience.
- **IoT and edge computing:** Infrastructure necessary to support **Internet of Things (IoT)** applications include enabling the collection, processing, and analysis of data from connected devices in real-time. Cloud can also provide edge computing capabilities that allow data processing closer to the source, reducing latency.
- **Artificial intelligence (AI) and ML:** Since the early 2020s, clouds have been offering a range of AI and ML services, such as natural language processing, image recognition, and predictive analytics, enabling businesses to integrate advanced analytics and automation into their products.

Major cloud providers

The top players in the cloud space include:

- **AWS:** AWS is the first, largest, and most comprehensive public cloud provider, offering a wide range of services across IaaS, PaaS, and SaaS. As the longest-running cloud, it provides extensive global infrastructure, a broad range of services, strong support for enterprise use cases, and a

robust ecosystem of partners. Some of the most used services include EC2, S3, Lambda, RDS, DynamoDB, SageMaker, ECS, EKS, and Redshift.

- **Microsoft Azure:** Azure is a leading public cloud provider that offers integrated services across computing, analytics, storage, and networking. Strong integration with *Microsoft* products, integration with AI products, hybrid cloud capabilities, comprehensive compliance offerings, and a focus on enterprise and developer productivity make it a desirable solution. The most used services include Azure VMs, Azure SQL Database, **Azure Kubernetes Service (AKS)**, Azure DevOps, and Azure AI.
- **Google Cloud Platform (GCP):** GCP provides a suite of cloud computing services with a focus on data analytics, ML, and open-source technologies. Main offering is ML capabilities, strong open-source support, global network infrastructure, and competitive pricing. Some key services here are *Google Compute Engine*, *Google Kubernetes Engine*, *BigQuery*, *TensorFlow*, and *Google Cloud AI*.
- **Oracle Cloud Infrastructure (OCI):** OCI is designed for enterprise workloads, offering services optimized for performance, security, and reliability. It is highly optimized for *Oracle* applications, has strong performance and security, and has hybrid and multi-cloud capabilities. Solutions include *Oracle Databases*, *Oracle Cloud Compute*, *Autonomous Database*, and *VMware Solution*.
- **Alibaba Cloud:** This is a leading public cloud provider in *Asia*, offering a wide range of services for computing, storage, networking, and analytics. It has an extensive global network, competitive pricing, and a focus on e-commerce and AI. The main services are **Elastic Compute Service (ECS)**, **Object Storage Service (OSS)**, *Alibaba Cloud Kubernetes*, and *Alibaba Cloud AI*.

Challenges

With the above advantages covered, let us also look at the flip side of the trade-offs of using the cloud:

- **Cost management:** Public cloud costs can escalate quickly without

proper cost management. It is essential to monitor usage and optimize resources to avoid unexpected expenses.

- **Security and privacy:** Although public cloud providers offer robust security measures, organizations are still responsible for securing their data and applications. Concerns about data breaches, compliance, and privacy remain.
- **Vendor lock-in:** Reliance on a single cloud provider's tools and services can lead to vendor lock-in, making it difficult to switch providers or move workloads to another platform. There are some solutions for multi-cloud setup but need application changes.
- **Compliance and data sovereignty:** Organizations in regulated industries must ensure that their cloud usage complies with data protection laws and industry standards. Additionally, data sovereignty concerns arise when data is stored in different jurisdictions or geographical locations.
- **Performance and latency:** While public clouds offer global reach, latency and performance issues can arise depending on the geographic location of the cloud resources and the end-users.
- **Complexity and skills:** Managing cloud infrastructure requires specialized skills and knowledge. Organizations may face challenges in hiring and training personnel with the necessary expertise.

Future trends

Looking forward, the clouds seem here to stay and expand their reach for simplifying application development. Some trends include:

- **Multi-cloud solutions:** Many organizations adopt multi-cloud strategies to avoid vendor lock-in, improve resilience, and optimize costs by leveraging the best services from different providers.
- **Hybrid cloud solutions:** Organizations increasingly use hybrid cloud solutions that combine on-premises infrastructure with public cloud services to meet specific business needs, such as compliance, latency, or legacy system integration.
- **Serverless computing:** Serverless computing models, such as AWS Lambda and Azure Functions, are gaining popularity due to their ability

to automatically scale and manage resources without server management overhead for the end-users.

- **Edge computing:** Public cloud providers are extending their services to the edge, enabling low-latency, real-time processing for IoT applications, gaming, and other latency sensitive use cases.
- **AI and ML integration:** Expanding AI and ML capabilities using GPUs and other sophisticated resources can include offering pre-trained models, automated ML, and infrastructure for deep learning, making it easier for businesses to incorporate AI into their operations as a plug and play model.

Public cloud usage continues to grow as organizations of all sizes leverage its flexibility, scalability, and cost-efficiency. By understanding the key aspects, benefits, use cases, providers, and challenges of public cloud usage, businesses can make informed decisions and effectively harness the power of the cloud to drive innovation and growth.

Golang for cloud

As we have discussed, Golang (or Go) is a cloud ready, statically typed, compiled programming language designed by *Google* that is particularly well-suited for cloud computing environments. Golang's simplicity, concurrency support, performance, and efficiency make it a popular choice for developing cloud-native applications. This section discusses how Golang can further leverage various cloud features.

Cloud APIs

Golang has robust **software development kits (SDKs)** support for interacting with various cloud services. For example:

- **AWS SDK for Go:** Allows developers to interact with AWS services like EC2, S3, DynamoDB, Lambda, and more via their corresponding API calls.
- **Azure SDK for Go:** Enables integration with Azure services like Azure Blob Storage, Cosmos DB, and AKS.
- **Google Cloud SDK for Go:** Provides libraries to access *Google Cloud*

services such as BigQuery, Cloud Storage, Pub/Sub, and Compute Engine.

- **Object-relational model (ORM):** Go can connect to cloud-managed databases (such as *Amazon RDS*, *Google Cloud SQL*, and *Microsoft Azure SQL Database*) using native drivers or ORM libraries like **GORM** or **sqlx**.
- **REST APIs:** Go's built-in **net/http** package and third-party libraries like **gorilla/mux** make it easy to build HTTP clients and servers for interacting with other cloud services through REST APIs.

Microservices orchestration

Golang applications can be easily containerized using Docker. Go's statically linked binaries eliminate runtime dependencies, simplifying the creation of small Docker images, which are essential for cloud deployment. Golang is often used to build microservices due to its lightweight with small memory footprint and fast startup time, that make it ideal for containerized applications in a microservices architecture.

Go's in-built concurrency model, including goroutines and channels, makes it easy to develop highly concurrent services. This is especially useful for microservices that need to handle multiple requests simultaneously or perform asynchronous tasks. So Golang is the language of choice for developing application for Kubernetes environments.

Pub/Sub messaging

Go applications can interact with cloud-based message queuing services like *AWS SQS*, *Azure Service Bus*, and *Google Pub/Sub*. Libraries like **confluent-kafka-go** and **kafka-go** enable integration with Apache Kafka-based cloud services. Go's asynchronous concurrency model allows efficient processing of messages and events from cloud Pub/Sub systems, making it suitable for real-time data processing applications.

Serverless computing

Golang is supported by major cloud serverless platforms such as *AWS Lambda*, *Microsoft Azure Functions*, and *Google Cloud Functions*. Go's fast

startup times and very low memory usage make it suitable for serverless architectures where resources are dynamically allocated based on demand. Also, with Golang, developers can build event-driven serverless functions that respond to events like storage uploads, database changes, or HTTP requests, integrating seamlessly with serverless platforms from various cloud providers.

Cloud deployments

In the first step of deployment, Golang provides testing packages that make it easy to write unit and integration tests. These tests can be automatically run as part of CI/CD workflows to ensure application reliability and quality before deployment. Such applications are well-suited for integration into CI/CD pipelines due to their fast compile times and ability to produce single static binaries.

Tools like Jenkins, GitHub Actions, GitLab CI/CD, and CircleCI can easily build, test, and deploy Go applications to any of the public clouds. IaC settings can also be used to write custom Terraform providers using Golang. Since Terraform is written in Go, Go developers can easily extend it by developing custom resources and data sources.

Logging and monitoring

Go supports structured logging, which can be integrated with cloud-native logging and monitoring tools like *AWS CloudWatch*, *Azure Monitor*, or *Google Cloud Logging*. Libraries like **logrus** or **zap** provide enhanced logging capabilities in Go applications, which can be analyzed for errors or specific patterns. One can also use instrument-distributed tracing tools like OpenTelemetry or Jaeger in Go services. This is essential for monitoring the performance of microservices and root causing issues in a cloud environment.

Security and compliance

All applications written in Golang can interact and set up security aspects needed to set the privilege levels for resource access. Go can be used to interact with cloud IAM services, allowing developers to programmatically manage permissions, roles, and access controls for cloud resources. Go also

supports data encryption and decryption using libraries like **crypto** (built-in) and third-party libraries, enabling secure communication with cloud services and ensuring compliance with data protection standards.

Golang is well-equipped to leverage a wide array of cloud features due to its simplicity, performance, and powerful concurrency model. It is particularly effective in cloud-native applications, serverless computing, microservices, and environments where efficiency, scalability, and rapid deployment are crucial. By using Go, developers can build efficient and scalable cloud applications that take full advantage of modern cloud infrastructure and platform services.

Conclusion

As part of this chapter, the reader gained insights into the steps going from software development to deployment tools. We got a good overview of the operational challenges and tools to help run a large-scale distributed system smoothly. Finally, we took a tour in the cloud realm for all the infrastructure and platform services from various public cloud providers that help with the software development lifecycle.

In the next chapter, we will take a pivot to get a grasp on the security aspects of distributed systems.

CHAPTER 11

Securing Your Server

Introduction

Software and server security is a crucial aspect needed in any software system to prevent data breaches or misuse of information and other malware. We will investigate different possibilities of potential security holes and how to prevent such vulnerabilities in this chapter.

It takes 20 years to build a reputation and few minutes of cyber-incident to ruin it. Technology trust is a good thing, but control is a better one.

-Stephane Nappo, CISO

Hackers tend to exploit known gaps in software systems by building and using tools that can quickly crawl the web for victim systems. These tools try to attack different web servers or software end points to gain unchecked or backdoor access into server backends. Taking control of the data via encryption to render a site or system useless is the main target of such dark entities for getting paid ransom to release and decrypt the data. Let us delve into the security world to deter such activity and keep control of the server in the right hands.

Structure

The chapter covers the following topics:

- Security concepts

- Open Web Application Security Project
- Data leaks
- Authentication
- Authorization
- Transport Layer Security
- Security measures
- Golang software security

Objectives

Security cannot be neglected in modern software services and systems. This chapter provides insights into the various components that need to be aligned to secure servers and services.

This chapter's main goal is to give an overview of the possible security loopholes that can be exploited at every level of the backend software stack. We will investigate prevention mechanisms needed for all known issues and detection tools to reduce the blast radius for any system hack or data breach.

The focus will be on streamlining security mechanisms and processes, starting with the development discipline and moving up to operational rigor. We will also learn about the community source for tracking security updates, which keeps organizations ahead of bad actors.

Security concepts

Security is a critical aspect of software systems, as these systems are often targets for cyberattacks. Addressing security in software involves protecting the system against threats such as unauthorized access, data breaches, malware, and other vulnerabilities. The impacts of providing software security include ensuring data integrity and user confidentiality in an ever-evolving digital world.

Let us first summarize some of the key security concepts to consider when designing, building, and maintaining software systems:

- **Authentication and authorization:** Authentication ensures that users or entities are who they claim to be and keeps bad actors outside the

system. Authorization controls what authenticated users are allowed to do, allowing only read or write access to the right persona.

- **Encryption:** Encryption protects data confidentiality by converting it into a coded format that can only be read by someone with the decryption key. This applies to data both at rest (stored data) and in transit (data being transmitted across networks).
- **Secure coding:** Following secure coding guidelines reduces vulnerabilities such as SQL injection, **cross-site scripting (XSS)**, and buffer overflows. During the development phase, there should be a plan to mitigate all known possible vulnerabilities.
- **Parameter validation:** During the coding phase, one can ensure that all input is validated and sanitized to prevent injection attacks and other malicious input exploits. This includes validating input type, length, format, corner case, and range checks.
- **Security audits:** Regular security audits involve reviewing the software and its codebase which can help identify and mitigate security vulnerabilities before making to production.
- **Patch updates:** Keeping software and dependencies up to date to protect against known vulnerabilities can be automated to reduce known unexpected gaps across the stack.
- **Logging and alerting:** Implementing comprehensive logging can help monitor access and actions at the entry points to the software system. Alerting for any unusual activities or potential security incidents in real-time will enable quick response.
- **Backup and recovery:** Regularly backing up data and having a robust disaster recovery plan in place to restore data and system functionality in case of a security breach or data loss. Ensuring that backups are stored securely and are protected from unauthorized access in an environment secluded from the primary is ideal.
- **Access control:** Implementing the principle of least privilege, where users and systems have only the minimum level of access necessary to perform their functions is a good gating design.
- **Third-party risks:** Assess the security practices of third-party vendors

and ensuring they meet your security standards, especially if they handle sensitive data or integrate with your software system. Contracts that include security requirements and expectations for third-party vendors should be locked in.

- **Incident response plan:** Defining security policies and procedures for handling sensitive data, responding to incidents, and managing user access should be part of regular training. Ensuring that these policies are well-documented and communicated to all stakeholders. Developing and maintaining a clear incident response plan to quickly and effectively respond to security incidents. Conducting regular drills and updates to the incident response plan to ensure readiness.
- **Regulatory requirements:** Ensuring that software systems comply with relevant regulations and standards (for example, GDPR, HIPAA, PCI DSS) to avoid legal repercussions and build trust with users. Providing regular security awareness training to all users, developers, and stakeholders to help them understand security best practices and recognize potential threats.
- **Security in development and operations (DevSecOps):** Integrating security practices into every phase of the **software development lifecycle (SDLC)**, from design to deployment and maintenance. Automating security checks and using **continuous integration/continuous deployment (CI/CD)** pipelines to ensure ongoing security.

By incorporating these aspects into software development and maintenance, organizations can better protect their software systems from a wide range of security threats. [Figure 11.1](#) captures the above components of software security:



Figure 11.1: Components of software system security

Next, we will learn how such security vulnerabilities are unearthed and how developers can use that to stay ahead of the hackers.

Open Web Application Security Project

Open Web Application Security Project (OWASP) is a global non-profit organization focused on improving the security of software. Founded in 2001, OWASP has been providing resources, tools, documentation, and community-driven projects to help organizations and developers create secure applications and avoid missing out on ongoing gaps that bad actors try to take advantage.

OWASP Top 10

One of OWASP's most well-known projects is the OWASP Top 10. This is a regularly updated list of web applications Top 10 most critical security risks. The list helps developers and organizations understand the most common vulnerabilities and how to mitigate them. Currently, the OWASP Top 10 has not been officially released after the last update of the 2021 version. However, OWASP is planning to release the 2025 list based on data collected through end of 2024.

Here is a summary of the OWASP Top 10 list from 2021, which still holds relevance today for understanding common web vulnerabilities:

1. **Broken access control:** This most common ongoing flaw allows unauthorized users to access restricted resources, such as sensitive data or administrative functions. This is one of the most frequently encountered issues in security breaches.
2. **Cryptographic failures:** Previously known as **sensitive data exposure**, this category focuses on weaknesses in cryptographic implementations that could lead to data being exposed or tampered with, often due to using outdated or weak encryption methods.
3. **Injection:** These occur when untrusted data is sent to an interpreter as part of a command or query. Examples include SQL, NoSQL, OS, and LDAP injection, which can lead to unauthorized data access or manipulation.
4. **Insecure design:** This new category for 2021 emphasizes the need for secure design patterns and principles from the start of the software development process to prevent vulnerabilities like XSS, broken authentication, and more.
5. **Security misconfigurations:** This refers to the improper configuration of security settings in applications, frameworks, and servers, which can lead to vulnerabilities such as leaving unnecessary services enabled, default accounts active, or failing to set proper permissions.
6. **Stale components:** Using libraries, frameworks, and other software modules that contain known vulnerabilities can lead to exploitation. Regular updates and patches are crucial to mitigate these risks.
7. **Authentication failures:** Issues in this category include weak password policies, lack of **multi-factor authentication (MFA)**, and improper

session management, which can allow attackers to assume the identities of legitimate users.

8. **Data integrity failures:** This includes assumptions about software and data integrity that can be exploited, such as not verifying the integrity of software updates or using unsecured CI/CD pipelines.
9. **Logging and monitoring failures:** This category covers the lack of sufficient logging and monitoring mechanisms, which can delay or prevent the detection of breaches, leading to extended periods of undetected malicious activity.
10. **Server-side request forgery (SSRF):** This vulnerability occurs when a web application is tricked into making a request to an unintended website or URL. SSRF can lead to unauthorized access to internal systems, often bypassing firewall rules.

These correspond to the list of potential vulnerabilities in software applications, and OWASP aims to periodically update its list to address new and emerging threats in the cybersecurity landscape. For more detailed information on each category and the latest updates, you can visit the OWASP Top 10 website: <https://www.owasptop10.org>. In the future sections we provide some of the solutions to prevent such vulnerabilities while building software systems.

OWASP projects

OWASP maintains several projects that offer tools, guidelines, and documentation to improve software security. Some of the notable projects include:

- **Zed Attack Proxy (ZAP):** A popular open-source web application security scanner used for finding vulnerabilities in web applications.
- **Dependency-check:** A software composition analysis tool that identifies project dependencies and checks if there are any known, publicly disclosed vulnerabilities.
- **Application Security Verification Standard (ASVS):** A framework for testing the security of web applications and determining the level of confidence in the security posture.

- **Software Assurance Maturity Model (SAMM):** A framework to help organizations formulate and implement a strategy for software security that is tailored to the specific risks facing the organization.

Community and guidance

OWASP is a community-driven organization with local chapters around the world. These chapters organize meetups, workshops, and conferences, facilitating networking and collaboration among developers, security experts, and organizations interested in web security. They also provide numerous educational resources, including books, tutorials, and online courses, aimed at raising awareness and educating developers and security professionals about application security best practices.

Hosting events and conferences, such as the OWASP Global AppSec events, provide forums for the security community to share knowledge and collaborate on improving security practices. The organization relies heavily on volunteers and contributions from the global security community, making it a collaborative and open resource. OWASP produces comprehensive guides and documentation on various aspects of application security, such as the *Testing Guide*, which provides a framework for testing the security of web applications, and the *Proactive Controls*, which lists security controls that developers should include in every software project.

By promoting a security-first approach to software development, OWASP helps reduce vulnerabilities in web applications, contributing to a safer internet for everyone. For more information, you can visit the official OWASP website at <https://owasp.org>. Now let us look into some of the common solutions to prevent the above vulnerabilities.

Data leaks

A data leak refers to the unauthorized or unintentional exposure, sharing, or loss of sensitive data. This data can include personal information, financial records, intellectual property, and other confidential information, which is very commonly found within most companies. Data leaks can occur due to various reasons, such as weak security practices, human error, or malicious attacks. Unlike a data breach, which typically involves deliberate intrusion

into a system by hackers, data leaks often result from inadequate or preventive data protection measures.

Common causes of data leaks include:

- **Human error:** Misconfigurations, sending sensitive information to the wrong recipients, or losing physical devices like USB drives and laptops where data is not password locked.
- **Weak security practices:** Poor password management, lack of encryption, and inadequate access controls for critical data.
- **Phishing attacks:** Trick users into divulging sensitive information or credentials, leading to unauthorized access to systems and data.
- **Unsecured networks or devices:** Using unsecured Wi-Fi or connecting devices without proper security controls.
- **Vulnerabilities in system or dependencies:** Exploiting known vulnerabilities in outdated or unpatched software stack.

Detecting data leaks

Detecting a data leak involves monitoring, analyzing, and responding to unusual activities that might indicate unauthorized data access or network transmission. Here are some methods to detect data leaks:

- **Traffic monitoring:** Using tools like **intrusion detection systems (IDS)** and **intrusion prevention systems (IPS)** one can monitor and analyze network traffic for unusual or unauthorized data transmissions. Unexpected patterns, such as large data transfers to unknown external IP addresses or repeated access attempts from unusual geographic locations could be signs of data leak.
- **Data Loss Prevention (DLP):** DLP software monitors and protects sensitive data by identifying, categorizing, and enforcing policies to prevent unauthorized data transmission or storage. DLP tools can be deployed on endpoints, networks, and cloud environments to detect and block attempts to send sensitive information outside the organization.
- **User behavior analytics (UBA):** UBA tools analyze user behavior patterns to detect anomalies that could indicate potential data leaks. For example, an employee downloading unusually large amounts of data

may trigger an alert. By understanding normal user behavior, UBA systems can flag deviations that might indicate malicious activity or insider threats.

- **File Integrity Monitoring (FIM):** FIM tools monitor files and configurations for unauthorized changes. If sensitive files are accessed, modified, or moved without authorization, an alert is generated. This is particularly useful for monitoring sensitive data repositories and critical system files.
- **Security information and event management (SIEM):** SIEM systems aggregate and analyze logs from various sources (firewalls, servers, applications) to detect potential security incidents, including data leaks. By correlating data from multiple sources, SIEM systems can provide a more comprehensive view of potential leaks and other security threats.

Preventing data leaks

Preventing data leaks involves implementing a combination of technical controls, policies, and best practices to protect sensitive information:

- **Strong access controls:** Use the principle of least privilege, granting users only the access necessary to perform their job functions. Regularly review and update user permissions to ensure they remain appropriate as roles change.
- **Encryption:** Encrypt sensitive data both at rest (stored data) and in transit (data being transmitted across networks) to prevent unauthorized ability to view the data content even if data is intercepted or accessed.
- **MFA:** MFA adds an additional layer of security by requiring users to provide two or more verification factors to gain access to resources, reducing the risk of unauthorized access due to single compromised credentials.
- **Regular patching:** Keep all applications and dependencies on third-party software and systems up to date with the latest security patches to protect against known vulnerabilities that could be exploited in a data leak.
- **Security awareness training:** Educate employees on data security best

practices, including recognizing phishing attempts, proper data handling procedures, and the importance of reporting suspicious activities.

- **Data loss prevention (DLP):** Use DLP tools to monitor, detect, and block unauthorized attempts to transfer or store sensitive data outside the organization.
- **Regular security audits:** Conduct regular security audits and risk assessments to identify and address potential vulnerabilities and gaps in data protection measures.
- **Incident response plan:** Develop and maintain an incident response plan to quickly and effectively respond to data leaks and minimize damage. This includes identifying a response team, defining response procedures, and conducting regular drills.

By implementing these detection and prevention strategies, organizations can significantly reduce the risk of data leaks and protect sensitive information from unauthorized access or exposure.

Authentication

Authentication is the process of verifying the identity of a user, device, or system to ensure that the entity attempting to access a service or resource is who or what it claims to be. It is a critical aspect of cybersecurity and access control that helps prevent unauthorized access to sensitive data and storage systems.

Let us first look at the various options for authentication:

- **Single-factor authentication (SFA):** The most basic form of authentication, typically involving a username and password. SFA relies on a single credential, making it less secure because passwords can be easily guessed, stolen, or hacked.
- **MFA:** Enhanced version which involves two or more independent credentials for verification. The bottom two of the following categories are generally preferred along with the first (SFA):
 - **Something you know or created:** Passwords or PINs.
 - **Something you have:** Physical tokens, smart cards, or mobile

devices.

- **Something you are:** Biometrics like fingerprints, facial recognition, or retina scans.
- **Certificate-based authentication:** Relies on digital certificates, which are electronic documents that use a digital signature to bind a public key with an identity. It is commonly used in scenarios like **Secure Socket Layer/Transport Layer Security (SSL/TLS)** for websites or client-side certificates for **virtual private networks (VPNs)**.
- **Token-based authentication:** Users authenticate initially with credentials, and then a token is generated to access other resources without re-entering passwords. OAuth is a popular token-based authentication framework used in web applications.

Detecting authentication issues

Detecting authentication issues involves identifying potential threats that compromise authentication mechanisms, such as unauthorized access, credential stuffing, or brute force attacks. Here are some ways to detect authentication-related issues:

- **Monitoring logins:** Track the number of failed and successful login attempts. A high number of failed attempts from a single IP address or account could indicate a brute force attack or credential stuffing attempt. System can track logins from unusual geo locations or devices, which could signify compromised credentials.
- **Anomaly detection:** Anomaly detection systems can learn normal user behaviors and flag deviations, such as logging in from different geographic locations within a short period. Analyze user behavior patterns like access time, frequency, and duration to detect suspicious activities.
- **Reviewing security logs:** Regularly review security logs for signs of potential breaches or misuse. Look for patterns like repeated access to sensitive resources, attempts to bypass authentication or frequent use of administrative privileges.
- **SIEM:** SIEM systems collect and analyze data from various sources

(for example, firewalls, servers, applications) to detect and respond to potential security incidents in real-time, including authentication failures.

Preventing authentication issues

Preventing authentication issues involves implementing robust security measures to ensure that only authorized users gain access. Key prevention strategies include:

- **Zero trust model:** Implement a zero-trust security model where no device or user is automatically trusted, regardless of whether they are inside or outside the network perimeter. Authentication and verification are required at every stage of access.
- **Strong password policies:** Require complex passwords that include a mix of letters, numbers, and special characters. Implement regular password expiration policies to reduce the risk of compromised credentials. Use password managers to help users manage complex passwords securely.
- **MFA:** MFA significantly increases security by requiring multiple forms of verification. Even if one factor (for example, a password) is compromised, the additional layers (for example, a fingerprint or a code sent to a mobile device) help prevent unauthorized access. Using biometric authentication for high-security environments makes it harder for attackers to impersonate authorized users.
- **Authentication tokens:** Protecting tokens (used in OAuth or SSO) with strong encryption can ensure they are securely stored as well. Using short-lived tokens and refresh tokens helps limit exposure in case of token theft.
- **Account lockout:** Temporarily locking accounts after a predefined number of failed login attempts can thwart brute force attacks. Ensure the lockout mechanism is appropriately tuned to balance security and user convenience.
- **Regular security audits:** Regularly auditing and testing authentication mechanisms can identify and fix vulnerabilities. Penetration testing can

also help simulate attacks and assess the robustness of authentication controls.

- **Role-based access control (RBAC):** Restricting access based on user roles and responsibilities, ensures that users have only the access necessary for their job functions. This minimizes potential damage to compromised accounts.

Authentication is a foundational aspect of cybersecurity, essential for protecting sensitive data and systems from unauthorized access. By combining robust detection mechanisms and preventive measures, organizations can enhance their authentication security posture and mitigate the risks associated with authentication failures.

For further details on authentication best practices and tools, you can explore resources like the *National Institute of Standards and Technology (NIST) Digital Identity Guidelines* at <https://pages.nist.gov/800-63-3/>.

Authorization

Authorization is the process of determining whether a user, device, or system has the right to access specific resources or perform certain actions. Unlike authentication, which verifies the identity of a user, authorization controls what an authenticated user is allowed to do within a system.

These are the options for this critical aspect of cybersecurity to enforce access policies and protect sensitive data from unauthorized users:

- **RBAC:** RBAC restricts access based on the roles assigned to users within an organization. Each role has a set of permissions associated with it, defining what actions a user in that role can perform. Common in environments where user roles are well-defined, such as in corporate IT systems (e.g., admin, manager, employee).
- **Attribute-based access control (ABAC):** ABAC allows access decisions based on attributes (characteristics) of the user, the resource, and the environment. Attributes can include user department, resource sensitivity, or time of access. More flexible than RBAC, allowing for fine-grained access control policies.
- **Discretionary access control (DAC):** DAC gives data owners the

discretion to decide who can access their resources. Owners can grant or revoke access to others. Commonly used in collaborative environments where users need to share files or resources freely.

- **Mandatory access control:** Mandatory access control enforces access policies based on fixed rules determined by a central authority, often using classifications and security clearances. Users cannot change permissions. Commonly used in government and military applications where security is paramount.
- **Policy-based access control (PBAC):** PBAC uses policies, often written in languages like XACML, to govern access decisions. Such secured policies are evaluated against requests to access resources. Useful in complex environments where access decisions need to be highly customizable and dynamic.

Detecting authorization issues

Detecting authorization issues involves monitoring and analyzing user activities and access patterns to identify unauthorized access attempts, misuse of privileges, or policy violations.

Here are some common methods to detect authorization issues:

- **Access logs monitoring:** Regularly review access logs to identify patterns of unauthorized access, such as attempts to access resources beyond a user's role or permissions. Monitor for unusual access patterns, such as access to sensitive resources outside of normal business hours or from unrecognized devices or locations.
- **Real-time alerts:** Implement real-time alerting systems that notify administrators of potential authorization breaches or policy violations, such as access to restricted files by unauthorized users. Configure alerts to trigger on activities that deviate from predefined access policies.
- **UBA:** Use UBA tools to analyze user behaviors and detect anomalies that could indicate misuse of privileges or unauthorized access. For example, a user accessing a large volume of sensitive files might indicate an insider threat. Compare current user activities against a baseline of typical behavior to identify potential issues.

- **SIEM:** As noted before, SIEM systems aggregate and analyze data from various sources to detect and respond to potential security incidents, including unauthorized access attempts and privilege escalations across systems and identify potential breaches that exploit authorization weaknesses.
- **Audits:** Regular access reviews and audits ensure that user permissions align with their roles and responsibilities. It identifies and remediates any unauthorized access or privilege creep beyond what is needed for a user's role.

Preventing authorization issues

Preventing authorization issues involves implementing strong access controls, continuously monitoring for potential breaches, and maintaining a robust policy framework. Here are some strategies to prevent authorization-related issues:

- **Principle of least privilege (PoLP):** Grant users the minimum level of access necessary to perform their job functions. Regularly review and adjust permissions as roles and responsibilities change, implementing the finer granularity of control where needed. Use RBAC or ABAC to enforce PoLP effectively.
- **Strong access controls:** Use RBAC, ABAC, or mandatory access control depending on your organization's needs to enforce consistent and robust access control policies. Ensure access controls are integrated across all systems and applications to prevent gaps in enforcement.
- **Regular access reviews:** As with other aspects before, conducting periodic access reviews to ensure that users have appropriate access based on their current roles is imperative. One can also implement automated tools to identify and remediate excessive permissions or dormant accounts. Automated auditing tools help regularly check for compliance with access policies and can be used for auditing and forensic purposes.
- **Automated policy enforcement:** Use automated tools to enforce access control policies consistently across all systems and applications. Ensure that these tools are regularly updated to reflect changes in policy or

organizational structure. Automate the detection and remediation of policy violations to reduce the risk of unauthorized access.

- **Segregation of duties (SoD):** Ensure that critical tasks are divided among multiple users to prevent a single user from having too much control, reducing the risk of fraud or misuse. Use automated tools to enforce SoD policies and detect any violations.

Authorization is a crucial aspect of securing access to resources and ensuring that only authorized users can perform specific actions on service. By implementing robust detection and prevention mechanisms, organizations can mitigate the risks associated with unauthorized access and maintain the integrity and confidentiality of their data and storage systems.

Transport Layer Security

Based on the past two sections, once the data at rest is secured in storage systems, applications need to ensure that data moved across services over the network is also hackproof. TLS is a cryptographic protocol designed to provide such secure communication over computer networks. TLS is widely used to secure web browsing, email, instant messaging, and other forms of internet communications by encrypting data transmitted between clients and servers, ensuring data privacy and integrity.

Let us look at the key points of TLS functionality:

- **Encryption:** TLS encrypts data to ensure that it remains private and cannot be read by unauthorized parties during transmission. This encryption protects sensitive information, such as login credentials, payment details, and personal data.
- **Authentication:** TLS provides mechanisms to authenticate the parties involved in a communication. For example, when you visit a website using HTTPS, TLS verifies the server's identity with a digital certificate issued by a trusted *Certificate Authority (CA)*.
- **Data integrity:** TLS ensures the integrity of data transmitted over a network by using **Message Authentication Codes (MACs)**. This prevents attackers from tampering with data in transit without being detected.

- **Forward secrecy:** Modern versions of TLS support forward secrecy, a feature that ensures that session keys cannot be compromised even if the server's private key is later compromised. This is achieved by generating unique session keys for each connection using ephemeral Diffie-Hellman key exchanges.

TLS operates in two main phases: The handshake and the record protocol. The handshake establishes a secure connection between a client and a server using digital certificate to the client for authentication. After the handshake, the TLS protocol ensures secure data transmission. It divides the application data into manageable chunks, encrypts them, and attaches a MAC for integrity before transmitting them over the network.

TLS is extensively used in various applications to provide secure communication. Some common uses include:

- **Web browsing (HTTPS):** TLS is most widely known for securing HTTP connections, where it is referred to as **HTTPS**. When a user connects to a website using HTTPS, TLS encrypts all data exchanged between the user's browser and the server, protecting sensitive information such as login credentials, payment details, and personal data from snoopers.
- **Email:** Email protocols such as SMTP, IMAP, and POP3 can be secured with TLS to protect emails during transmission. This ensures that emails cannot be intercepted or read by unauthorized parties as they are sent from sender to receiver.
- **VPNs:** TLS can secure VPN connections, allowing users to create encrypted tunnels over public networks. TLS-based VPNs, such as those using OpenVPN, provide secure remote access to private networks.
- **Voice over IP (VoIP):** TLS is used to secure VoIP communications and instant messaging applications to prevent eavesdropping and ensure message integrity.
- **File transfer protocols (FTP):** Secure FTP uses TLS to encrypt FTP connections, protecting files during transfer over the internet.
- **Application programming interfaces (APIs):** Many web APIs use TLS to secure data exchanged between applications, ensuring that

sensitive information, such as user data or financial transactions, is protected.

As a best practice for TLS, always use the latest version of TLS (currently TLS 1.3) as it offers stronger security features and better performance. TLS 1.3 eliminates outdated and less secure cryptographic algorithms and handshake steps. The digital certificates need to be issued by trusted CAs with strong key sizes (for example, 2048-bit RSA or 256-bit ECDSA) also need regular updates. One should regularly monitor and audit TLS configurations to ensure they adhere to best practices and compliance requirements using automated tools to check for vulnerabilities and misconfigurations.

TLS is an essential technology for securing internet communications and protecting data from interception, tampering, and unauthorized access.

Security measures

Since prevention is the best cure for almost any problem, software security measures are essential for identifying, managing, and preemptively mitigating vulnerabilities in software applications and systems. We have seen a few options for data at rest and in motion in the past two sections. There are more tools that cover static code analysis, dynamic analysis, penetration testing, network monitoring, and can be enabled for more scenarios.

Following is an overview of various types of software security measures functionality, along with examples:

- **Static Application Security Testing (SAST):** SAST tools analyze source code or binaries for vulnerabilities without executing the program. They help identify issues like SQL injection, XSS, and buffer overflows early in the development lifecycle. Some commonly used ones include:
 - **SonarQube:** An open-source platform for continuous inspection of code quality, supporting various programming languages and integrates with CI/CD pipelines.
 - **Checkmarx:** A commercial tool that provides comprehensive code

scanning and integrates with development environments and CI/CD tools.

- **Fortify analyzer:** A powerful tool that identifies vulnerabilities and provides detailed guidance on remediation.
- **Dynamic Application Security Testing (DAST):** DAST tools test running applications by simulating attacks to identify vulnerabilities that could be exploited in runtime environments. They focus on finding issues such as authentication flaws, server configuration problems, and runtime security vulnerabilities. Some commonly used ones include:
 - **OWASP ZAP:** An open-source DAST tool designed to find security vulnerabilities and assessments of web applications.
 - **Burp Suite:** A popular tool among security professionals for web application security testing, offering both automated and manual testing capabilities.
 - **Acunetix:** A commercial tool that performs thorough scanning of web applications to detect vulnerabilities like SQL injection and XSS.
- **Software composition analysis (SCA):** SCA tools analyze the open-source components and libraries used in software projects to identify known vulnerabilities and licensing issues. Some commonly used ones include:
 - **Dependabot:** A GitHub integrated tool that automatically checks for vulnerabilities in dependencies and suggests updates.
 - **Snyka:** A developer-friendly tool that integrates with development workflows to monitor and manage vulnerabilities in open-source libraries and containers.
 - **Black Duck:** A comprehensive tool that helps manage open-source risks across the entire SDLC.
- **Interactive application security testing (IAST):** IAST tools analyze running applications in real-time by integrating with the application server, providing a hybrid approach between SAST and DAST. Some commonly used ones include:

- **Contrast security:** A tool that continuously monitors applications in real-time, providing detailed insights into vulnerabilities and their specific locations in the codebase.
- **Seeker by Synopsys:** CI/CD pipeline integrated tool that provides real-time vulnerability analysis and remediation guidance.
- **Runtime Application Self-Protection (RASP):** RASP tools integrate into applications to provide security controls during runtime, helping to detect and block attacks in real-time. Some commonly used ones include:
 - **AppDynamics RASP:** Integrates with applications to protect against threats by monitoring and controlling application behavior.
 - **Imperva RASP:** Provides in-depth visibility and protection against application vulnerabilities in real-time.
- **Penetration testing:** Referred to as **pen test** for short, these tools simulate attacks on a system to identify vulnerabilities that could be exploited by malicious actors. These tools are often used by security professionals to assess the security behavior of applications and networks. Some commonly used ones include:
 - **Metasploit framework:** A powerful open-source tool used for penetration testing, enabling security professionals to develop and execute exploit code against a target system.
 - **Kali Linux:** A Linux distribution specifically designed for penetration testing and security auditing, containing a wide range of security tools.
 - **Nmap:** A network scanning tool used to discover hosts and services on a network, useful for identifying open ports and potential vulnerabilities.
- **Network security:** These tools help monitor, analyze, and protect network traffic and infrastructure. Some commonly used ones include:
 - **Wireshark:** An open-source network protocol analyzer that provides detailed insights into network traffic for troubleshooting and security analysis.

- **Snort:** An open-source network **intrusion detection and prevention system (IDS/IPS)** that analyzes traffic in real-time to detect and respond to threats.
- **Suricata:** A high-performance network IDS/IPS and network security monitoring engine.
- **Vulnerability management:** These tools scan systems and applications for known vulnerabilities and provide remediation guidance. Some commonly used ones include:
 - **Nessus:** A popular vulnerability scanner that assesses systems for a wide range of security vulnerabilities and configuration issues.
 - **QualysGuard:** A cloud-based platform that offers continuous monitoring and scanning for vulnerabilities across the entire IT environment.
 - **OpenVAS:** An open-source vulnerability scanner that provides comprehensive coverage of security vulnerabilities in networked systems.
- **SIEM:** As briefly mentioned before, SIEM tools aggregate and analyze security events and logs from multiple sources to provide real-time monitoring and alerting. Some commonly used ones include:
 - **Splunk:** A powerful data analysis tool that provides real-time visibility and monitoring for security events, supporting automated response and incident investigation.
 - **IBM QRadar:** A SIEM solution that provides comprehensive visibility into network activities, helping detect and respond to potential security threats.
 - **ArcSight:** A platform that collects and analyzes logs from various sources to detect and respond to security incidents.
- **Container and cloud security:** These tools help secure containerized applications and cloud environments, ensuring that configurations are robust, and vulnerabilities are addressed. Some commonly used ones include:
 - **Aqua Security:** A container security platform that provides

runtime protection, vulnerability scanning, and compliance checks for containerized environments.

- **Falco:** An open-source runtime security tool for monitoring and detecting anomalous behavior in containers and cloud-native environments.
- **Twistlock:** A cloud-native security platform that offers comprehensive security for containerized applications, serverless functions, and cloud infrastructure.
- **Backup and disaster recovery:** Implementing frequent and secure backups of critical data is essential in the unfortunate case of data getting compromised despite all the above tooling being deployed. Ensure backups are stored offline or in separate locations to protect against ransomware or data corruption. This helps air gap the primary data stores from the backed-up storage systems. Equally important is to have a robust disaster recovery plan that outlines procedures for restoring systems and data in the event of a cyberattack or other catastrophic events. Failing to plan will be planning to fail in such emergency cases. Some commonly used ones include:
 - **Veeam:** A comprehensive backup solution for virtualized, physical, and cloud environments, their platform provides features like continuous data protection, instant VM recovery, and automation. It is ideal for complex IT organizations that need cloud native backup solution.
 - **Acronis:** Backup solution with integrated cybersecurity features like anti-malware protection. Their software covers cloud, hybrid, and on-premises backup. They cover both physical and virtual servers with Integrated ransomware protection.
 - **Rubrik:** This offers a cloud-based data management platform focused on backup, recovery, and data archiving. Their solution is known for automation and simplicity and has instant recovery for virtualized environments, and is ideal for enterprises looking for modern, cloud-integrated data management solutions.

Choosing the right security tools depends on your organization's specific needs, technology stack, and security requirements. A layered approach that includes a combination of these tools will provide a comprehensive security posture, helping to detect, prevent, and mitigate security threats across all stages of the SDLC.

Golang software security

Golang (Go) is a statically typed, compiled programming language designed for simplicity, reliability, and efficiency. While Go is known for its concurrency support and performance, it is also important to consider security aspects when developing software in Go. Incorporate security throughout the SDLC. This includes threat modeling, secure code reviews, and regular security testing.

Here are some best practices during the development phase for writing secure Go code:

- **Input validation:** Unsensitized inputs can lead to injection attacks (for example, SQL injection, command injection) or other vulnerabilities. Always validate and sanitize user inputs. Use Go's standard library packages such as **net/http** for HTTP requests and escape special characters to prevent injections. Also, avoid exposing unnecessary functions, services, or endpoints to minimize the code radius that is exposed to untrusted inputs and hackers.
- **Error handling:** Inadequate error handling can expose sensitive information and allow attackers to deduce application behavior or state. Ensure that errors do not leak sensitive information. Use **error** package properly and log errors without exposing implementation details or identifiable information such as bank account numbers.
- **Clean coding practices:** Poor coding practices can introduce vulnerabilities such as race conditions, buffer overflows, and improper error handling. Follow secure coding guidelines such as avoiding the use of unsafe packages like **unsafe** or **reflect** unless necessary. Use Go's concurrency features like goroutines and channels cautiously to avoid race conditions.
- **Dependency management:** Using outdated or vulnerable third-party

packages can introduce security risks. Regularly update dependencies using Go modules (**go.mod**). Use tools like govulncheck to identify known vulnerabilities in dependencies. Regularly audit the Go code base and conduct penetration testing to identify and remediate security vulnerabilities.

- **Concurrency safety:** Go's concurrency model makes it easy to write concurrent programs, but it also creates the risk of race conditions and deadlocks, which might cause non-trivial data consistency issues. Use Go's race detector (**go run-race**) to identify race conditions during development. Properly synchronize access to shared resources using channels or sync primitives like **sync.Mutex**.
- **Memory safety:** Go is generally memory-safe, but improper use of pointers or slices can lead to vulnerabilities such as nil pointer dereference or buffer overflows. Avoid direct manipulation of memory. Use Go's built-in data structures like slices and maps, which handle memory allocation and bounds checking.
- **Logging and monitoring:** As always, use structured logging to capture security-relevant events without leaking sensitive information. Implement monitoring to detect and respond to security incidents promptly.

Now let us get a quick overview of Go's security specific tooling:

- **Security libraries:** Standard **crypto** Package provides a robust **crypto** package with sub-packages for encryption **crypto/aes**, hashing **crypto/sha256**, and random number generation **crypto/rand**. Always prefer using secure hash algorithms like SHA-256 over older ones like MD5. **net/http** package provides secure HTTP client and server implementations by supporting HTTPS out of the box with TLS configuration.
- **Zero Trust:** One can secure communication between microservices by requiring authentication using **mutual TLS (mTLS)**. Go's **crypto/tls** package supports mTLS to ensure that both client and server authenticate each other. Also use **JSON Web Tokens (JWT)** or API keys for securing RESTful APIs. Libraries like **jwt-go** can help for

implementing JWT in Go.

- **Code scanning tools:** GoSec is a static analysis tool that scans Go source code for common security issues, such as hard-coded credentials and weak cryptographic algorithms. Staticcheck is a comprehensive static analysis tool for Go that helps identify code smells, bugs, and potential security issues.
- **Vulnerability management:** Govulncheck, a tool for checking vulnerabilities in Go code and its dependencies, helps identify known issues using Go's checklist database.

Go provides several built-in security features and libraries that help developers write secure code. However, security is an ongoing process that requires awareness, proper coding practices, and regular monitoring. By following these guidelines and leveraging Go's tools, developers can significantly reduce the risk of security vulnerabilities in their applications.

For further information on secure Go development, refer to the *Go wiki on security* (<https://github.com/golang/go/wiki/Security>) and the *Go Vulnerability Management* (<https://go.dev/security/vuln>).

Conclusion

This chapter shone more light on why security is front and center across all phases of the software lifecycle. The reader gained knowledge on how to secure the system from well-known vulnerabilities and install tooling and processes for detecting and preventing future gaps. Following the core principles of securing large-scale systems will help reduce data and revenue loss for any company that relies on its server software to be resilient.

In the following chapter, we will pivot towards techniques for ensuring the detailed aspects of deployment management, including security and upgrades.

CHAPTER 12

Upgrades and Maintenance

Introduction

Version changes are inevitable in an enterprise software system, as these are integral aspects of **software development lifecycle (SDLC)**. We will provide some insights options for rolling out of services and related maintenance processes in this chapter.

If, at first, you do not succeed, call it version 1.0.

-Khayri Woulfe

For any software system or team, going from zero to releasing the first version is a hard climb. As we have touched upon before, deploying changes to software after that is a continuous process to provide more and improved functionalities or fixes to the users of the application.

There are some industry-standard processes that allow distributed systems developers to keep track of how to maintain their services and keep up with all the changes needed to improve them.

Structure

The chapter covers the following topics:

- Day 2 operations
- Upgrades overview

- Upgrades models
- Golang support

Objectives

Within this chapter, the reader will gain information on the types of system operations needed to keep it running smoothly and routine steps to keep it updated with the latest changes.

At the end of the chapter, one should be able to determine how to set up automated upgrades and ensure testing and metrics are in place to ensure continued availability of the entire system.

Let us start by getting an overview of all the usual responsibilities of a **site reliability engineer (SRE)** whose team is normally in charge of ensuring deployments are up to the mark as per the operating requirements.

Day 2 operations

In the context of distributed systems, **Day 2 operations** refer to the ongoing management, maintenance, and operational tasks that occur after the initial deployment of a system. While **Day 1 operations** focus on the installation, configuration, and initial setup, Day 2 operations involve the continuous maintenance and optimization required to keep the system running smoothly over time. We will discuss the key aspects of Day 2 operations in this section.

Monitoring

Possibly the most important first step is to implement tools and processes to continuously monitor system performance via metrics, health, and resource utilization. On top of that, setting up alerts for any issues that might arise, such as performance degradation of metrics and downtime of services or failures, would help set up operational excellence.

Software upgrades

Besides monitoring, the team must regularly update software components to include new features, enhancements, and bug fixes. Managing dependencies and ensuring compatibility between different system parts is essential for robust deployment. These are potentially the most regular operations for which any team must build a smooth process for fast operational velocity. We will look into some models for performing such operations in the following sections.

Scaling

There will be a need for adjusting system resources (for example, adding or removing servers, storage, or network capacity) to handle changing workloads and traffic patterns, as there could be user growth for the service or could be a time of year demand. Implementing automated scaling mechanisms to dynamically adjust resources based on the demand for threshold changes for resources would help optimal usage of the system. Tools like kubernetes allow this to be a quick configuration change, which can also be automated based on resource usage thresholds for making the changes.

Backup and disaster recovery

Ensuring that data is regularly backed up and that recovery procedures are in place and tested is another dimension to ensure data availability to be used by the service at any time. Planning and implementing strategies for disaster recovery to minimize downtime and data loss in case of catastrophic events by setting up geographically remote backup servers are a common best practice for application availability.

Security management

Continuously applying security patches and updates to keep the system secure. Monitoring for and responding to security threats and vulnerabilities. Developing and following procedures for identifying, diagnosing, and resolving incidents. Performing root cause analysis to prevent future occurrences of similar issues.

Performance optimization

Analyzing system performance metrics to identify bottlenecks and areas for improvement can be tracked using metrics or graphs (using tools like Graphana). Tuning system components and configurations to optimize performance for the underlying issue can be handled by the development team as needed.

Configuration management

Maintaining and managing system configurations ensures the consistency and reliability of large-scale systems. Using tools and practices like **infrastructure as code (IaC)** can improve automation and version control configuration update steps.

Compliance and auditing

This involves ensuring that the system complies with relevant regional regulations and standards. Conducting regular audits and assessments to verify compliance and identify areas for improvement help maintain this aspect. Checking and managing user accounts, permissions, and roles ensures appropriate access controls. Automated monitoring and auditing system-access on top of manual checking allows to detect and prevent unauthorized access.

Overall, Day 2 operations are critical for ensuring the reliability, performance, security, and scalability of distributed systems in production environments. [Table 12.1](#) below summarizes what each type of maintenance operation entails and the goals they achieve:

Operation	Focus	Goal
Corrective maintenance	Fixing bugs, errors, and defects in the current release version.	To correct known issues and restore expected functionality.
Perfective maintenance	Enhancing performance, features, or usability.	To improve the software response based on user feedback or technological advances.
Preventive maintenance	Proactively addressing potential issues or resource related inefficiencies.	To prevent future problems and maintain reliability.

Emergency maintenance	Responding to unplanned critical failures or urgent loss of service.	To restore service quickly and mitigate the immediate impact. Plan for prevention of root causes.
Security maintenance	Addressing vulnerabilities and securing the software.	To maintain security and protect against evolving threats.
Technical debt	Refactoring code and addressing suboptimal solutions.	To reduce long-term maintenance costs and complexity.
User support	Helping users resolve issues and providing advanced documentation and training.	Maintain up-to-date resources and ensure a positive user experience.
Monitoring and logging	Continuously tracking system performance and availability.	To detect and resolve issues before they impact system stability.
Configuration management	Managing and automating system configurations and deployment settings.	To ensure consistency and stability across runtime environments.
Compliance maintenance	Ensuring software adheres to legal and regulatory standards.	To avoid legal risks and maintain compliance with industry standards.

Table 12.1 : *Summary of software maintenance operational aspects*

We will focus on the upgrade aspects next, as that is the most common and repetitive task that will create the most value when planned and performed as per industry standards.

Upgrades overview

Software upgrades are essential for maintaining, improving, and securing applications or systems over time. They involve changes to the code, features, dependencies, security, and underlying infrastructure of any software system. In this section, we will discuss the main reasons for performing software upgrades.

Bug fixes

The most common reason for regular upgrades is to correct known bugs or glitches reported by users or discovered by developers. These fixes ensure that the software runs more reliably with fewer crashes or errors. Such

upgrades may also address suggestions or feedback from users, improving usability, fixing existing features settings, or common pain points.

Security enhancements

Upgrades often address vulnerabilities discovered in previous versions by providing patches and security fixes either for the deployed code or its dependencies. Newer versions may incorporate the latest changes, such as stronger encryption algorithms or better security protocols to protect data.

Performance improvements

Using better or latest coding solutions, developers tend to refine the existing code to make it more efficient, leading to faster processing times and better resource management. Improved handling of larger workloads or more users, as well as enhancing the software's scalability, could be another reason to upgrade the system to take advantage of these enhancements.

New functionality

The functionality could be in any of the following layers of the technical stack that is part of the end-to-end software system:

- **Code features:** Upgrades to introduce new capabilities or expand the software's functionality can be requested by the development team.
- **UI/UX enhancements:** Updates can improve the **user interface (UI)** and **user experience (UX)** by making the software easier and more intuitive to use.
- **Hardware/software compatibility:** As hardware or other software (for example, operating systems) evolve, upgrades ensure that the software continues to work seamlessly and can take advantage of lower layer functionality.
- **Integration with new standards:** Incorporating newer standards, APIs, or technologies to stay relevant could be part of upgrade cycles.
- **Regulations:** Upgrades may ensure compliance with new laws, industry standards, or data protection regulations like DMA or GDPR.

Infrastructure changes

Sometimes, an upgrade involves moving to a newer version of an underlying framework or platform (for example, migrating from AngularJS to Angular 2+) or from one framework to another (for example, from REST to gRPC). Modern software might shift toward cloud-based architectures or microservices for greater flexibility and scalability, which would necessitate a system update.

Data migration

Upgrades often might involve migrating data from an older version to a newer version for a schema or data storage change to ensure data integrity during the process. This provides a backward compatible way for the software to still interact with older components, systems, or versions where necessary.

End of life management

Certain functions or features might be removed or marked as deprecated in dependent software packages, and users are advised to migrate to newer functionalities triggering the need for an upgrade. So, major newer versions might signify the end of official support for previous versions, encouraging users to upgrade.

In summary, software upgrades help keep systems secure, performant, and aligned with modern standards while ensuring users benefit from new features and improvements. Let us now see how these upgrades can be performed, along with the trade-offs for each option.

Upgrades models

There are several models for handling software upgrades, each with distinct approaches to delivering updates based on team expertise, system architecture dependencies, and business or user expectations. The choice of a software upgrade model depends on the system's complexity, the criticality of uptime, the user base, and how changes should be managed. In this section, we will discuss some common software upgrade models.

Monolithic upgrades

This model involves upgrading the entire system in one shot, often during scheduled downtime. It applies the new version of the software all at once. This mode implies users will not be able to use the system for a reasonably long period of downtime. Hence, it is best for smaller systems or non-critical applications where the cost of downtime is low.

This is normally simple to plan and execute and gets all planned features, dependency changes, and bug fixes deployed at once. On the flip side, it can disrupt users and create high risk if something goes wrong, as rollback can be complicated. It is not a practical option for large-scale distributed systems.

Rolling upgrades

A rolling upgrade involves upgrading software components gradually across different parts of the infrastructure without taking the entire system offline. This model is commonly used in distributed systems. This can also be termed as an incremental upgrade, where parts of the system are upgraded at a time. This is best for cloud-based or microservices architectures where different components can be updated independently.

This mode provides zero or very minimal downtime, as there is a part of the service that is always online for continuous availability. On any upgrade failure, it is easier to isolate rollbacks to individual components, thus keeping the blast radius to a minimum. Conversely, it can be complex to manage, especially for highly interdependent components or modules. This also requires backward data compatibility between old and new component versions during the transition.

Phased rollout

In a phased rollout, upgrades are released to specific users, or subsets of users, in phases. After observing how the upgrade performs in the first phase, the next phase is gradually introduced to a larger group of users. This option is best for systems with a large user base or critical applications where testing on a small group is necessary before a wider release.

This mode reduces the risk of widespread failures or issues and allows real-world testing and feedback before scaling. This is easier to revert in case of issues but could take a prolonged upgrade period, requiring the maintenance of multiple versions.

Canary releases

This mode is similar to phased rollout but more focused on deploying the new version to a small subset of servers (often called **canaries**) to test the stability and performance in a real-world environment before rolling out to the entire server fleet. This is best for mission-critical systems where it is important to minimize the risk of deployment errors.

This mode has very low risk, as issues are detected early in a small subset and rollback can be quickly done if problems are detected. This also helps collect real-world data for performance analysis. This mode also requires the ability to manage and analyze multiple versions simultaneously, and the complete upgrade process can take quite a bit of time.

There are further options called **dark canary**, sometimes referred to as **shadow testing**, which involves deploying a new version of the software in a production environment without exposing it to end-users. Instead, the system runs the new version on a different set of servers to monitor and test how the new version behaves under live traffic, but without any user interaction or side effects. Dark canary is better suited for testing infrastructure changes, backend updates, or ensuring performance and stability before exposing real users to the new version. It is a way to safely test the technical aspects of a new version without risking user impact.

Blue-green deployment

In this model, two identical production environments (blue and green) are maintained. While one (for example, the blue environment) is live, the other (for example, green) is used for testing the new version. After successful testing, traffic is switched to the green environment, and the old blue environment becomes the fallback.

These help in environments where high availability and rapid rollback capabilities are crucial. This enables quick rollback in case of failure with

no downtime for users during the switch.

Both environments can be tested in isolation but require double the infrastructure during deployment, which can be resource-intensive and costly. To ensure the best outcome, this mode needs careful traffic management and monitoring.

Feature flags

Feature toggles allow one to deploy new code to production without immediately making the new features available or enabled to all users. Specific features can be dynamically toggled on or off based on conditions, allowing you to control the timing of when users experience the new functionality. This is the best option for continuous delivery pipelines where new features are regularly pushed to production, but activation timing is flexible.

This mode allows for continuous delivery without affecting users and can be quick and easy to enable/disable features without redeployment. This enables A/B testing and gradual rollouts of new features but can add complexity to the codebase if not managed properly. Since this requires additional overhead in managing toggles and feature states, this has the potential for technical debt if old toggles are not cleaned up.

Hot fixes

In hot-swapping, minor upgrades or fixes (such as hotfixes) are applied to the running system without needing to shut it down or restart. This is typically used for small patches or updates where the process overhead should not come in the way of a highly desired immediate super critical fix.

This is for systems that need constant uptime and where downtime for fixes is unacceptable. Care should be taken to ensure it is a small, isolated fix only and is tested thoroughly before deployment.

In summary, the selection of an upgrade model depends on:

- **System complexity:** Larger and more complex systems typically require phased or rolling upgrade models.
- **Risk tolerance:** Mission-critical systems may require low-risk models

like canary releases or blue-green deployments.

- **Downtime tolerance:** If downtime is a major concern, models like rolling upgrades or feature toggles are preferable.
- **Infrastructure:** Cloud-based and microservices systems are better suited for rolling upgrades and blue-green deployments, while on-premises systems might need incremental or big-bang approaches.

The right upgrade model will ensure minimal disruption, reduced risk, and a seamless user experience.

Golang support

Golang offers various approaches and tools to support software upgrades, especially in systems where high performance, concurrency, and minimal downtime are critical. Whether it is upgrading a monolithic application, microservices, or distributed systems, Go provides several mechanisms and patterns for handling software upgrades in various scenarios.

In this section we will discuss key approaches and patterns provided by Go for software upgrades.

Graceful shutdowns and restarts

Go supports graceful shutdowns and restarts natively, which are crucial for minimizing downtime during software upgrades. Graceful shutdowns allow an application to complete in-flight requests and cleanly shut down and release resources (such as database connections, open sockets, etc.) before terminating, ensuring a smooth transition during upgrades.

Before deploying a new version of the application, you want to ensure that no active requests are interrupted and complete their current action and leave data in a consistent state. **http.Server.Shutdown()** in Go's **net/http** package has this method allows graceful shutdown of HTTP servers. Signal handling with **os/signal** allows capturing system signals (e.g., `SIGINT`, `SIGTERM`) to trigger a graceful shutdown.

The following code sample shows how to cleanly terminate a server process once the signal to terminate is detected:

```
1. package main
2.
3. import (
4.     "context"
5.     "log"
6.     "net/http"
7.     "os"
8.     "os/signal"
9.     "time"
10. )
11.
12. func main() {
13.     srv := &http.Server{Addr: ":8080"}
14.
15.     go func() {
16.         if err := srv.ListenAndServe(); err != nil &&
17.             err != http.ErrServerClosed {
18.             log.Fatalf("listen: %s\n", err)
19.         }
20.     }()
21.
22.     // Wait for an interrupt signal
23.     stop := make(chan os.Signal, 1)
24.     signal.Notify(stop, os.Interrupt)
25.
26.     <-stop // Block until signal is received
27.
28.     // Waiting for random time to shutdown in a controlled test.
29.     // Note: One can take up as an exercise adding a graceful
30.     // shutdown which waits for in progress tasks to complete.
31.     ctx, cancel := context.WithTimeout(context.Background(),
```

```
32.     5*time.Second)
33.     defer cancel()
34.     if err := srv.Shutdown(ctx); err != nil {
35.         log.Fatal("Server forced to shutdown:", err)
36.     }
37.
38.     log.Println("Server exiting")
39. }
40.
```

Hot reloading

Zero downtime deployment, where the application continues to run during the upgrade process, can be achieved through hot reloading or replacing binaries on the fly.

The main tool for hot reloading is <https://github.com/air-verse/air>. It is a live reloading tool that watches Go files and automatically recompiles and reloads the server during development. **goreload** is another library for live reloading Go applications during development. During development or production, when one needs to upgrade a running application without stopping it, just running `air` in the Go project directory. This will automatically recompile and restart your Go application whenever a file is saved.

Blue-green deployment in Go

As discussed before, blue-green deployment is a strategy where two production environments (blue and green) exist, and one can switch between them during an upgrade. Go applications, especially when deployed in containerized environments like Docker or Kubernetes, can easily support blue-green deployments.

One can deploy a new version without downtime by routing traffic to a new instance of the application (the green environment) while keeping the old instance (the blue environment) as a fallback. Go applications can be containerized and deployed using orchestration tools like Kubernetes,

Docker Swarm, or Consul, which inherently support blue-green or rolling deployments.

Canary deployment and feature flags in Go

Canary deployments involve releasing a new version to a small subset of users before a full-scale rollout. Go's lightweight concurrency model makes it ideal for managing canary releases in a microservices architecture using Kubernetes.

Libraries like **go-feature-flag**, **flagd**, and **Unleash** help control feature rollouts, allowing one to enable or disable features in the application for selected users or environments. Canary deployment patterns are often managed at the infrastructure level, using service mesh tools like Envoy or Istio in conjunction with Go microservices.

Here is a feature flag example where a setting for a feature flag is checked:

```
1. package main
2.
3. import (
4.     "fmt"
5.     ffclient "github.com/thomaspoignant/go-feature-flag/"
6.     "github.com/thomaspoignant/go-feature-flag/retriever"
7.     "github.com/thomaspoignant/go-feature-flag/retriever/fileretriever"
8.     "time"
9. )
10.
11. func main() {
12.     client, err := ffclient.New(ffclient.Config{
13.         PollingInterval: 10 * time.Second,
14.         Retrievers: []retriever.Retriever{
15.             &fileretriever.Retriever{Path: "./flags.yaml"},
16.         },
17.     })
18.     if err != nil {
```

```
19.     panic(err)
20. }
21. defer client.Close()
22. user := ffcontext.NewEvaluationContext("new-feature-key")
23. enabled, err := client.BoolVariationDetails("new-feature",
24.     user, false)
25. if enabled.Value {
26.     fmt.Println("The new feature is enabled!")
27. } else {
28.     fmt.Println("The new feature is disabled!")
29. }
30. }
```

The same flag configuration in the **flags.yaml** in the code above is as follows:

```
1. new-feature:
2.   variations:
3.     Default: false
4.     False: false
5.     True: true
6.   targeting:
7.     - name: rule1
8.       query: key eq "new-feature-key"
9.     percentage:
10.      False: 0
11.      True: 100
12.   defaultRule:
13.     name: defaultRule
14.     variation: Default
15.   metadata:
16.     description: this is a simple feature flag
```

One can also check the latest values of the flag file in other code paths of interest to make the flag value fetching and usage more dynamic.

Rolling deployments with Kubernetes

As discussed before, Go applications are commonly deployed in containerized environments (for example, Docker and Kubernetes). Kubernetes supports rolling deployments, allowing you to update a Go service incrementally without downtime. For distributed Go applications deployed as microservices, rolling deployments allow for smooth transitions between versions by gradually replacing old instances with new ones.

Following code is a sample Kubernetes deployment configuration file that sets up the rolling upgrade option for spawning new pods. The **maxSurge** with a value of 1 says that only one pod with a new version should be spawned at a time, and after it is successfully running only, an existing/older version of the pod can be terminated:

```
1. apiVersion: apps/v1
2. kind: Deployment
3. metadata:
4.   name: go-app
5. spec:
6.   replicas: 3
7.   selector:
8.     matchLabels:
9.       app: go-app
10.  template:
11.    metadata:
12.      labels:
13.        app: go-app
14.    spec:
15.      containers:
16.        - name: go-app-container
```

17. `image: go-app:latest`
18. `strategy:`
19. `type: RollingUpdate`
20. `rollingUpdate:`
21. `maxSurge: 1`
22. `maxUnavailable: 0`

Database schema migrations

During software upgrades, database schema changes are common. Go has several libraries for managing database migrations, ensuring that schema updates during upgrades happen smoothly without breaking the application. **golang-migrate** or **pressly/goose** is a popular tool for database migrations in Go applications. It supports multiple databases (PostgreSQL, MySQL, SQLite, etc.) and is commonly used for schema upgrades. The following is a migration example:

1. `migrate -database postgres://user:password@localhost:5432/dbname -path ./migrations-file up`

The sample **migrations-file** used in the above command could look like this:

1. `CREATE TABLE users (`
2. `id SERIAL PRIMARY KEY,`
3. `name VARCHAR(100),`
4. `email VARCHAR(100)`
5. `);`

Versioning and compatibility

Go encourages the use of backward compatibility, especially when upgrading libraries, APIs, or services. When upgrading Go applications, versioning ensures that new releases do not break existing functionality or dependencies. Go modules use semantic versioning (v0.0.1, v1.0.0, etc.) to manage upgrades. Ensure backward compatibility by adhering to semantic versioning rules in Go projects. Go modules example:

1. `go mod tidy`
2. `go get example.com/newlib@v2.0.0`

Go provides a strong foundation for handling various types of software upgrades, offering robust tools for managing upgrades with minimal downtime and risk.

Conclusion

There is a vast amount of maintenance and upkeep work that constitutes running a large-scale software system. We covered most of the important aspects in this chapter that would ensure a reliable and robust running system catering to the user-required functionality.

Now that we have the full technical view of running a large-scale distributed system using Golang as the base language, there are a lot of soft skills that need to be understood and mastered to evolve with the fast-changing software landscape. The condensed lessons that relate to software craft from all the past chapters will be summarized in the next chapter, along with the next steps for the ever-learning readers.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 13

Summary and Conclusion

Introduction

All the earlier chapters set the stage for the why, what, and how aspects that help build and maintain distributed systems in the modern internet era. We built a deep understanding of using Golang and the ecosystem around it to help develop, securely deploy, and perform Day 2 operations at scale for applications running on a large-scale backend system.

Software development is both challenging and rewarding. It's creative like an art-form, but (unlike art) it provides concrete, measurable value.

-Chad Fowler

While knowledge and execution of software systems are considered hard skills and can be mastered with practice, there is also the other side of the coin related to soft skills. These skills apply mostly to the who, when, and where aspects of delivering a fully functional system. Most soft skills apply to other industries and potentially to day-to-day lives as well. These can be made part of one's mind-muscle memory by learning from the right sources and putting them into practice continuously.

Structure

The chapter covers the following topics:

- Software team building

- Twenty-one lessons
- Soft skills
- Planning for the unknown
- Next steps

Objectives

Within this chapter, we first build a quick overview of the team-building aspects of a software project and provide insights into how to plan and deliver large features involving the right stakeholders and development organization. We will also look at ways to reduce uncertainty in feature delivery by using the right set of planning tools and frameworks.

The reader will also be able to gain insights into how they can learn and improve their skill set and abilities to handle the scaling of the team and continue improving at various software development contributor levels. By the end of the chapter, there should also be a good overview gained of all non-technical aspects of software development and team management.

Software team building

The first and foremost priority for any new project or company is to build the right team with the right personnel aligned to perform the right set of tasks for the planned timelines. Starting and continuously building a backend software engineering team requires strategic planning, strong leadership, and technical expertise. Following are the main aspects to consider while building teams:

- **Defining the team's goals:** All personnel need to understand the product needs and how the backend system will serve those goals. There could be a vision and mission statement and a related strategy document for how to achieve the same. Choosing the appropriate technologies (for example, programming languages, building API, databases, cloud platforms, etc.) can be the next step to aligning it with the project's needs and scalability plans.
- **Hiring:** It is desirable to hire engineers with varied skill sets, including

database management, API development, cloud infrastructure, microservices architecture, and DevOps. Hiring full stack engineers can also provide flexibility. If there is any domain expertise needed for the project, those could have some technical leads hired as well, along with the product management team. Ensure candidates fit well with the team's culture, are good communicators, and can collaborate across departments like frontend, product, and design.

- **Technical leadership:** Appoint experienced technical leaders to guide architectural decisions, mentor junior developers, and make crucial decisions on scalability, security, and performance. Foster a culture of collaboration with other departments (for example, frontend, product management) to ensure seamless integration of services. Some of the traits that top leadership skills include emotional intelligence, vision, and adaptability, along with the ability to delegate and empower.
- **Establishing processes:** Implement Agile methodologies (Scrum, Kanban) for clear communication, iterative development, and flexibility. Set up version control (for example, Git), **continuous integration (CI)**, and **continuous development (CD)** pipelines to automate deployments and reduce manual errors. Set up rigorous code review processes to ensure code quality, maintainability, and shared knowledge within the team.
- **Infrastructure and architecture:** Right from choosing a cloud provider or on-premises setup and securing access controls to deciding whether to use a monolithic approach or break the backend into microservices and up to designing your databases, all these aspects will affect the kind of team can seamlessly work together to get the right product delivered to users. Backend technologies evolve quickly, so keeping up with the latest trends is essential.
- **Team culture and growth:** Having an open culture of celebrating successes to learning together in failures help build trust in each other and can serve as the cornerstone for a one-team mentality. Encouraging learning through mentorship, training programs, or conferences will aid the team's growth further. Leadership can encourage team members to take ownership of their projects, giving teams autonomy while holding

them accountable for results. Promoting a culture of cross-functional collaboration between backend engineers, product managers, DevOps, and other departments will lead to constructive growth.

- **Communication and collaboration:** As the number of teams grows, the need for cross team collaboration can be achieved by establishing clear communication channels with frontend teams, backend developers, designers, and product managers along with stakeholders to ensure alignment. One should also ensure thorough documentation of APIs, system architecture, and processes so that the team and future hires can quickly onboard and contribute. Keeping non-technical stakeholders informed about progress, potential risks, and timelines helps align expectations.

In summary, starting and building an engineering team is not just about hiring the right people, but also about creating the right structure, setting up scalable processes, and fostering a culture of technical excellence and collaboration.

Twenty-one lessons

Based on software industry experiences and as one goes through their career progression, one can come across a variety of challenges that bring along new learnings. Following are the twenty-one key lessons to bear in mind that can help software engineers succeed and grow in their careers, drawn from both technical and professional perspectives:

- **Master the basics:** Master data structures, algorithms, operating systems, and computer architecture. These fundamentals of computer science remain relevant regardless of the technologies or languages you use.
- **Do not reinvent the wheel:** Once the basics are known, one will have enough context to leverage existing libraries, frameworks, and tools where appropriate. It is often better to use battle-hardened solutions than to build everything needed from scratch. Of course, one should use them with the right software licenses. This saves time and helps develop the needed logic for business faster.

- **Industry standard code:** Write clean, maintainable, and well-documented code. Other developers, including your future self, will need to read and modify it. Prioritize code readability and long-term maintainability over early optimization. Follow standard coding practices and well-established guidelines for the chosen language.
- **Focus on logic:** While syntax is important, focusing on solving the problem in a logical way is a must have. Strong problem-solving skills are what set great engineers apart, regardless of the language or framework being used. Simplicity is key in software engineering, and it uses clear logic that can be understood readily.
- **Testing and coverage:** Always write automated tests, whether unit tests, integration tests, or end-to-end tests. Testing saves time in the long run and ensures the reliability of your code. Keeping the coverage above a certain high threshold also gives confidence that there are no major breaks in production.
- **Embrace code reviews:** Code reviews are a valuable way to learn from others, catch errors, and ensure quality. Be open to feedback based potentially on the vast experience of a senior person. Use reviews as a collaborative learning process, not a criticism session.
- **Learn to debug:** Debugging is an essential skill. Learn how to use debugging tools (for example, breakpoints, logs) and read error messages effectively. Be methodical and patient when troubleshooting.
- **Master version control:** This might seem like a simple thing, but understanding Git (or other version control systems) is indispensable. Learn how to use it effectively—branching, merging, rebasing, and resolving conflicts are crucial skills for collaboration. This will save a lot of time and energy when iterating quickly on code reviews feedback or hot patch deployments.
- **Reduce tech debt:** There are many cases when faced with tight deadlines, some cases are not handled or marked as TODOs to deliver a feature. When a dependency gets updated, an update might be needed in the current codebase, too. Such cases can add up as technical debt that could cause future planning or production issues. Refactoring and

fixing up such code are essential to keep the codebase healthy and easy to maintain.

- **Learn design patterns:** Design patterns like singleton, factory, or observer provide tried-and-tested solutions to common problems. Know when and how to use them but avoid overcomplicating simple problems with patterns.
- **End-to-end architecture:** Learn the full stack of technologies used for the feature, from frontend to backend to infrastructure. Understanding how components like databases, servers, and APIs interact makes debugging and optimization easier. Understand how to structure software applications—whether it is monolithic, microservices, or serverless. Good architecture makes systems scalable, maintainable, and easier to extend.
- **Communication is key:** Being a good engineer is not just about writing code—it is about clearly communicating your ideas, requirements, and issues. Whether in code comments, documentation, or discussions, clarity is key. Understand who the target audience for a document or write-up is fine-tuned to give the right amount of context to grasp and respond to the presented material.
- **Keep learning:** Technology evolves quickly. Keep learning new languages, tools, and frameworks. Develop a habit of continuous learning through books, blogs, courses, and open-source contributions.
- **Embrace failure:** One will make mistakes, and things will break. Instead of fearing failure, treat it as an opportunity to learn and improve. Debugging and problem-solving skills often improve the most after failures. Using this learning in future to prevent issues and spreading the lessons to junior team members helps scale the team.
- **Understand business priorities:** Learn to align the right set of work items with the immediate business goals. Technical decisions should be guided by the value they bring to the business and its users. Engineering is not just about technical excellence but delivering value at the right time.
- **Be a team player:** Software development is often a team effort. Learn

to collaborate, share knowledge, and mentor others. Build good relationships with other developers, designers, product managers, and stakeholders. Also be a leader in the team even when there is no title. Once you are already signing up and performing tasks for the next level of impact, growth will be inevitable. Do unto others what you would expect.

- **Conflict resolution:** There will always be potential differences of opinion between team members or questions about priorities or technical directions. One should focus on the problem rather than the other person or team and get the right data points to the table to discuss the pros and cons of both sides. If that does not still provide the right direction, the next option could be to include the right stakeholders to provide the final decision so that all can commit and proceed. Keeping calm and making progress will get one further.
- **Knowledge sharing:** Sharing your knowledge builds your reputation and contributes to the broader developer community. Write blog posts and tutorials or record screencasts to share what you have learned. Give talks at meetups, conferences, or within your company. Participate in forums or **question and answer (Q&A)** platforms like *Stack Overflow*. One can become the go-to person by showing a deep understanding of focus areas in a project.
- **Feedback and mentorship:** Regular feedback and mentorship accelerate your growth by exposing you to different perspectives and best practices. Ask peers and managers for regular feedback on your work. Participate in pair programming sessions to learn from more experienced developers. Find mentors for different dimensions of your career development. They could be within or outside your company and could provide career advice, technical guidance, or soft skill development.
- **Width and depth:** For any given technical project and its users and ecosystem there can be various levels of details across a breadth of focus areas. Some folks tend to thrive on the details aspects for one and become the go-to subject expert in that area and continue investigating and enhancing it. Other folks lean more towards building together

portions of the areas across each other and have a bigger picture view. One can figure out which is more appealing as both these are essential personas that can coexist in a team and might complement each other.

- **Offloading:** To expand one's scope and impact at work, one can proactively start offloading items being tracked by senior technical leads or management onto their own plate. Once delivered, this will set confidence in stakeholders to assign more impactful work items going forward, thus enabling one to improve and increase their radius of influence.

By internalizing these lessons, software engineers can improve both their technical expertise and professional skills, leading to stronger code, more effective teamwork, and overall career success.

Soft skills

Soft skills are just as important as technical abilities for software engineers and industry. These skills enhance collaboration, problem-solving, communication, and adaptability, all of which are essential in dynamic work environments. They apply to many aspects of life as well as many other industries. Following are some essential soft skills for software engineers and the tools that can help improve them:

- **Communication:** Software engineers need to communicate complex technical concepts clearly to both technical and non-technical team members. This includes discussing features, requirements, bugs, and solutions. *Slack* or *Microsoft Teams* are good for quick and effective communication across teams. *Grammarly* helps improve writing quality in emails, documentation, or reports. For synchronous meetings to converge quickly, there are *Zoom* or *Google Meet* to have video communication, especially in remote teams.
- **Collaboration and teamwork:** Software projects often involve cross-functional teams, including frontend engineers, designers, product managers, and Q&A. Being a good collaborator makes team workflows smoother and more productive. *Jira* or *Trello* help as project management and planning tools that help engineers collaborate on

tasks, track progress, and manage sprints.

- **Problem-solving:** Software engineers constantly solve problems, whether debugging code, optimizing performance, or figuring out how to implement a feature. Structured problem-solving is key to being effective. Tools like Lucidchart or Miro help visualize workflows and architecture diagrams and break down complex problems into manageable components. *ChatGPT* and *Stack Overflow* provide platforms for learning how others have solved similar problems and to ask further questions.
- **Time management and prioritization:** Engineering tasks can range from feature development to bug fixes to code refactoring. Knowing how to prioritize tasks based on deadlines, business needs, and effort is crucial. Tools like Toggl or RescueTime help you understand how your time is spent and identify areas for better focus. Todoist or Notion are task management tools that organize and prioritize work effectively.
- **Adaptability:** Technology evolves quickly. Engineers must be adaptable to learn new languages, frameworks, or approaches and also deal with unexpected changes in requirements. *LinkedIn*, *Coursera*, or *Udemy* provide online learning platforms for continuous learning and adaptation to new technologies. Feedly or Pocket are tools for staying updated with industry trends and news.
- **Attention to detail:** A small error in code can lead to big problems. Attention to detail is essential for writing bug-free, optimized, and secure code. Linters (e.g., ESLint, Prettier) can automatically check code for errors and enforce coding standards. Code review tools help ensure attention to detail through peer reviews. Checklists or wiki steps ensures key steps are not missed in workflows, deployments, or testing processes.
- **Empathy:** Engineers must understand user needs, their team's concerns, and the broader business goals. Empathy fosters better problem-solving and enhances collaboration. User research tools (for example, SurveyMonkey, UserTesting) help engineers and stakeholders understand user feedback and make empathetic decisions.

- **Leadership and mentorship:** Engineers often grow into senior roles where they mentor junior team members, lead projects, or make architectural decisions. Mentorship facilitates mentoring relationships by sharing knowledge, best practices, and guidelines with the team.
- **Emotional Quotient (EQ):** Being able to manage your emotions and understand others emotions is key to working effectively in high-pressure environments, managing conflicts, and fostering positive team dynamics. Meditation apps help improve emotional self-regulation and manage stress. EQ assessments (for example, six seconds) tools help assess and improve EQ.
- **Negotiation and conflict resolution:** Engineers often need to negotiate deadlines, technical trade-offs, or priorities with stakeholders and other teams. Negotiation Books like *Getting to Yes* by *Roger Fisher* and *William Ury* provide reading material to develop negotiation skills. Conflict resolution tools like <https://mediation.com/> provide structured resolution techniques and frameworks to mediate and resolve conflicts effectively.
- **Critical thinking:** This helps in evaluating the pros and cons of different technical solutions and making decisions that align with both short-term and long-term goals. Use tools like charts and design drawings to help visualize problems, analyze possible solutions, and connect ideas logically. Reading and discussing on online forums also helps sharpen critical thinking by exploring diverse opinions on technical topics.
- **Feedback handling:** Receiving feedback, both positive and negative, is critical for personal and professional growth. On the other side, learning to give constructive feedback to peers is equally important. Platforms like Lattice and 360 encourage continuous feedback loops within teams.
- **Self-motivation:** Great engineers are naturally curious, always asking why and how things work. This curiosity leads to innovation and creative problem-solving. Especially in remote or autonomous roles, engineers need to stay motivated and disciplined without constant supervision. Online research sites help engineers explore cutting-edge

technology and concepts and are good starting point for gaining new knowledge.

These tools, combined with soft skills, help software engineers navigate the non-technical challenges of their profession, enhancing their overall effectiveness and career growth.

Planning for the unknown

There can be many variables for some of the complex and long-term projects. Planning for such unknowns in software engineering is challenging but necessary, as software projects frequently encounter changing requirements, unexpected technical issues, personnel changes, and evolving stakeholder or client needs. Here are strategies and best practices to effectively manage and mitigate these unknowns:

- **Prioritize a minimum viable product (MVP):** For any new project, starting with an MVP goal ensures that you deliver the core functionality of your product while leaving room for adjustments based on feedback or changing requirements. One can identify the critical features and aim to deliver them first and reduce the amount of throw away work. Anything non-essential can be deferred to later phases or sprints.
- **Embrace Agile:** Agile methodologies, like Scrum and Kanban, are designed for projects where requirements may change over time. Working in sprints allows teams to adapt to new information and requirements. Break down work items down into small, manageable sprints, focusing on delivering increments that provide value early. Retrospectives can be useful when done at a larger granularity (like a quarter) but ensure that the agile processes do not come in the way of technical or focused progress.
- **Plan for design flexibility:** One should design their systems to ideally be modular and loosely coupled so that changes in one area will not require major overhauls in others. Flexible, scalable architectures are better able to adapt to unknown or evolving requirements. Using design patterns that promote flexibility (for example, microservices) and considering scalability from the beginning are standard practices.

Documenting the design rationale to aid future adaptations is a great way to set the source of truth.

- **Plan for prototyping:** Building prototypes and spikes (short-term, exploratory solutions) can help reduce uncertainty by testing approaches and getting data points before committing to them fully. For any feature that involves new or unknown technology, build a quick prototype. This helps assess feasibility, identify potential blockers, and refine the approach.
- **Estimation techniques:** Techniques like range-based estimation and story points allow for more flexible time predictions. They also build in buffer time for unknowns. Instead of rigid estimates in terms of number of days, use ranges (for example, 3-5 days) or use T-shirt sizing (S, M, L) to estimate effort. This can also be used to regularly update estimates based on what is learned in previous sprints for similar tasks and teams velocity. One can identify potential risks and assign probability and impact ratings to them. Regularly review these risks and prepare mitigation strategies during estimation process.
- **Incorporate continuous testing:** As discussed before CI and continuous testing along with canary testing ensures that new code is regularly integrated and tested, minimizing the risk of discovering issues late in the development cycle or production. One can use tools like Jenkins, GitHub Actions, or GitLab CI/CD to automate testing and integration processes. Frequent automated testing provides quick feedback on changes and helps detect issues early. Automation reduces the workload and frees up team members to handle unknowns as they arise.
- **Strong communication channels:** Unknowns are often clarified or managed through communication. Make sure all team members, especially stakeholders, are on the same page regarding expectations, goals, and the current status, along with blockers. Hold regular meetings (for example, stand-ups, sprint reviews, demos) with the right set of people and document decisions and assumptions to keep the entire team up to speed on the final decisions. Encourage stakeholders to provide feedback early and often to reduce divergence due to

assumptions.

- **Experimentation culture:** Encouraging experimentation helps teams become more comfortable with uncertainty, as experimentation often yields unexpected insights and creative solutions. There are quite a few companies that allow time for developers to try out new ideas or experiment with emerging technologies that could benefit their related projects or focus areas. Hackathons, R&D days, tech-debt or innovation time can support this further and set the goal for better solutions.
- **Metrics for success:** Metrics provide objective data on project progress, enabling one to detect potential issues or roadblocks before they become critical. Tracking metrics like code quality, test coverage, performance benchmarks, and user satisfaction improvement, along with a reduction in a number of production incidents, can help build a dashboard for driving overall success. And these metrics can be used to make more data-driven decisions, especially when unknowns arise.
- **Postmortem:** Despite best efforts, one might still run into unexpected production issues. Conducting postmortems on such incidents, as well as on projects that hit delays or failures, helps identify what went well and what did not, creating a feedback loop that helps the team anticipate future unknowns. The expectation is not finger-pointing but a way to improve the team operations skillset. After each project or major incident, hold a retrospective to review successes, failures, and surprises. Document learnings to improve planning, testing, and deployment processes that will help handle such potential unknowns in future projects.
- **Prepare for failures:** There are variety of reasons software systems can fail. Most of the stress testing and integration might not be able to simulate a series of failures in a specific order to generate issue at production system scale. To prevent widespread issues, keep the services independent as much as possible in order to keep the blast radius of failures.
- **Regular documentation:** Documenting requirements, designs, and decisions as you go helps minimize knowledge gaps when new

unknowns emerge. Create and maintain documentation on system architecture, design decisions, and key workflows, including from the user's point of view. This helps the team quickly onboard to changes and avoid re-evaluating past decisions.

- **Knowledge sharing:** Having a cross-functional team with overlapping knowledge makes it easier to handle the unknowns that affect multiple domains. Conduct regular knowledge-sharing sessions, such as lunch-and-learns or tech talks, where team members can share insights on topics outside their primary expertise.

By implementing these strategies, software engineering teams can build flexibility, resilience, and adaptability into their processes, making it easier to handle the inevitable unknowns that come with complex projects.

Next steps

Improving in the software industry requires a combination of technical growth, soft skill development, and a deep understanding of how to adapt to industry trends and business needs. Especially with AI assisting in a lot of tasks, making it an ally is the right direction at any stage of the career. Here are some suggested ways to continuously grow and thrive in the software engineering field:

- **Always be learning:** Stay up to date with technology trends, The software industry evolves quickly, so regularly learning new languages, frameworks, and tools is crucial. Take online courses and follow reputable tech blogs, *YouTube* channels, or newsletters Attend tech conferences (in person or virtual), meetups, and workshops on emerging technologies like AI, blockchain, and DevOps.
- **Contribute to open source:** Contributing to open-source projects helps improve coding skills, collaboration, and exposes you to large-scale, real-world projects and builds your portfolio. One can start by working on fixing bugs, adding features, or improving documentation for projects you use or are interested in. This will help engage with the community via submitting pull requests and discussing solutions with project maintainers.

- **Side projects:** One can also build side projects to help you experiment with new technologies, languages, or paradigms and allow you to showcase your practical experience. One can contribute to real-world problems, create your own tools, or solve common challenges and share those projects on platforms like GitHub, Dev.to, or Medium.
- **Master algorithms:** Strong problem-solving skills make you a better engineer. Many technical interviews also emphasize algorithms and data structures. Practice coding challenges on platforms like LeetCode or HackerRank. Study algorithmic approaches and data structures (trees, graphs, heaps, dynamic programming, etc.). Engage in competitive programming or participate in coding competitions (for example, *Google Code Jam* or *TopCoder*).
- **Improve soft skills:** Communication, teamwork, leadership, and emotional intelligence are as important as technical skills, especially as you grow into more senior roles. Practice clear and concise communication, especially in code reviews and technical discussions. Engage in leadership activities like mentoring junior engineers or leading small projects. Read books to develop interpersonal skills.
- **Business understanding:** How your work impacts the business helps you align technical decisions with company goals and add more value. Message and engage with non-technical stakeholders like product managers, designers, or business analysts to understand business needs. Read about the intersection of software engineering and business in books like *The Lean Startup* by *Eric Ries* or *Measure What Matters* by *John Doerr*.
- **Improve execution:** These can be broken down into three main aspects:
 - Writing clean, efficient, and maintainable code is vital for both individual and team productivity. Adopt code quality tools like linters, formatters, and static analysis tools (for example, ESLint, SonarQube). Participate in thorough code reviews and incorporate feedback.
 - Refactor legacy code to improve readability, performance, and

scalability. Follow best practices like **Don't Repeat Yourself (DRY)**, and **Keep It Simple, Stupid (KISS)** principles.

- DevOps skills like CI/CD, **infrastructure as code (IaC)**, and containerization are increasingly important for developers, even those who did not work in operations. Experiment with containerization using Docker and orchestration with Kubernetes.
- **Work in different domains:** Exposure to different industries (for example, fintech, healthcare, e-commerce) or different types of engineering (for example, frontend, backend, mobile) makes you a more versatile engineer. Volunteer for projects outside your immediate expertise or comfort zone when possible. Explore consulting or freelance opportunities to gain exposure to a variety of projects. Intern or transition into other roles, such as mobile development, AI/ML, data engineering, or DevOps, to build diverse experiences.
- **Master a technology stack:** Being proficient in one or more complete technology stacks (frontend, backend, database, cloud) allows you to work effectively across a range of projects. Choose a stack that aligns with your interests or the industry you want to work in (for example, MERN, LAMP, MEAN, or Java Spring Boot + React). Build projects that involve both frontend and backend elements to understand the full development lifecycle. Get hands-on experience with cloud technologies (for example, *AWS*, *Microsoft Azure*, *Google Cloud*).
- **Participate in hackathons:** Hackathons encourage creativity, rapid prototyping, and collaboration with new team members under tight time constraints. Join hackathons (virtual or in-person) to work on innovative projects and new technologies. Collaborate with other engineers and designers to solve real-world problems. Use these opportunities to learn new skills, meet peers, and expand your professional network.
- **Gain certifications:** Certifications validate your skills and knowledge, especially in cloud platforms, cybersecurity, or specific programming languages. Earn certifications in cloud platforms (for example, *AWS Certified Solutions Architect*, *Google Cloud Professional*). Look for certifications in security (for example, **Certified Ethical Hacker**

(CEH)) or Agile (for example, Certified ScrumMaster). Get language-specific certifications (for example, *Oracle Certified Professional: Java*, *Microsoft Certified: Azure Developer Associate*).

- **Develop cross-functional skills:** Being able to communicate and collaborate effectively with designers, marketers, and product managers strengthens your ability to deliver valuable products. Learn basic design skills (for example, Figma, Sketch) to better communicate with designers. Understand how to write user stories and break down tasks into smaller units from a product manager's or stakeholder perspective. Participate in product ideation and brainstorming sessions.
- **Remote collaboration:** Many companies are shifting to remote or hybrid work. Mastering remote collaboration tools and strategies is essential. Get comfortable with remote communication tools like *Zoom*, *Slack*, and *Microsoft Teams*. Set up regular virtual catch-ups with team members along with managers and stakeholders.

Conclusion

In this final chapter we rounded up the soft skills aspects of software engineering, which are surely applicable to other industries and general civilized life. One can use all the tools and tips to enhance their craft and get a step up in improving their chances of making faster career progression. We also touched upon some of the next set of items outside the scope of this book that can further the impact and touch points for a well-rounded software engineer.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

Index

A

- acceptance testing [13](#)
- ACID properties [98](#)
- Active Server Pages (ASP) [4](#)
- Advanced Package Tool (APT) [69](#)
- Alibaba Cloud [229](#)
- Amazon Web Services (AWS) [15](#), [228](#)
- API endpoints
 - accessing [122](#), [123](#)
- application
 - launching [48](#)
- application-level load balancing [103](#)
- application programming interfaces (APIs) [11](#)
 - GraphQL [12](#)
 - Remote Procedure Calls (RPC) APIs [11](#)
 - REST APIs [12](#)
 - SOAP APIs [11](#)
 - WebSocket APIs [12](#)
- Application Security Verification Standard (ASVS) [238](#)
- ARPANET [89](#)
- arrays [19](#), [20](#)
- authentication [241](#), [242](#)
 - issues, detecting [242](#)
 - issues, preventing [242](#), [243](#)
- authorization [243](#), [244](#)
 - issues, detecting [244](#), [245](#)
 - issues, preventing [245](#)
- availability [171-173](#)
- AWS CodePipeline [224](#)
- Azure DevOps [224](#)
- Azure Kubernetes Service (AKS) [228](#)

B

- backend programming languages [3](#)
 - history [3-5](#)
 - selecting [5](#)
- backend systems components
 - APIs [11](#)
 - cloud services [15](#), [16](#)

- data modeling [8](#)
- deployment and monitoring [14](#)
- monitoring and maintenance [14](#), [15](#)
- post-deployment tasks [14](#)
- scaling [15](#)
- security [10](#), [11](#)
- server [3](#)
- source code version control [6](#)
- system testing [12](#)
- verification [14](#)

Beego [57](#)

- features [57](#), [58](#)
- working with [58-60](#)

Brewer's theorem [104](#)

C

canary-based deployments [15](#)

CAP Theorem [104](#), [105](#)

Certified Ethical Hacker (CEH) [281](#)

channels [31](#)

- buffered channels [31](#), [32](#)

Chroot [81](#)

CI/CD automation tools [222-224](#)

CI/CD pipelines [14](#), [46](#)

- containerization [47](#)
- deployment [47](#)
- Docker images, pushing [47](#)

CircleCI [223](#)

client [3](#), [11](#)

client-side load balancing [103](#)

cloud providers

- Alibaba Cloud [229](#)
- AWS [228](#)
- Azure [228](#)
- GCP [228](#)
- OCI [228](#), [229](#)

Cluster Autoscaler (CA) [196](#)

code quality [214](#), [215](#)

- code reviews [217](#), [218](#)
- coding standards [215](#), [216](#)
- documentation [217](#)
- performance optimization [218](#), [219](#)
- readable code [215](#)
- refactoring [218](#)
- standard library [216](#)
- testing [217](#)
- versioning and dependencies [219](#), [220](#)

Common Business-Oriented Language (COBOL) [3](#)

- Common Object Request Broker Architecture (CORBA) [4](#)
- compatibility testing [13](#)
- Compatible Time-Sharing System (CTSS) [112](#)
- concurrency [30](#)
- concurrency, distributed systems
 - atomicity [97](#)
 - deadlock detection and avoidance [97](#)
 - isolation [97](#)
 - mutual exclusion [96](#)
 - non-blocking operations [98](#)
 - synchronization [97](#)
 - transactional support [98](#)
- cond variable [37](#)
- Conference on Data Systems Languages (CODASYL) [142](#)
- Container APIs in Go [207](#)
 - Docker APIs [207](#)
 - Kubernetes API [208-211](#)
 - Open Container Initiative APIs [211](#)
- container deployment [192](#)
 - application, accessing [195](#)
 - deployment YAML file, creating [193](#)
 - prerequisites [192](#)
 - service YAML file, creating [194](#)
 - to Kubernetes [194](#)
 - verifying [195](#)
- container management tools [195](#)
 - canary deployments [197](#)
 - Cluster Autoscaler [196](#)
 - Grafana [199](#)
 - Helm [197, 198](#)
 - Horizontal Pod Autoscaler [195, 196](#)
 - Kustomize [198](#)
 - Prometheus [199](#)
 - rolling updates [196, 197](#)
 - rollout management [197](#)
 - service mesh [199](#)
- Container Network Interface (CNI) [85](#)
- container orchestration [82](#)
- containers [81, 82](#)
 - application testing [192](#)
 - creating [190](#)
 - deploying [192](#)
 - Docker container, running [192](#)
 - Dockerfile, creating [191](#)
 - Docker image, building [191](#)
 - Docker image registry [192](#)
 - Go application, creating [190, 191](#)
- container security [199](#)

- admission controllers [203](#)
- auditing [204](#), [205](#)
- container image security [199](#)
- container runtime security [199](#), [200](#)
- encryption and certificates [204](#)
- network security [200](#), [201](#)
- Pod Security Standards (PSS) [203](#)
- role-based access control [202](#)
- secrets management [201](#)
- service accounts [203](#)
- content delivery network (CDN) [174](#), [228](#)
- Create, Read, Update, and Delete (CRUD) operation [39](#)
- Creeper [89](#)
- Cross-Site Scripting (XSS) [11](#)

D

- dark canary [260](#)
- data [141](#)
- database management system (DBMS) [147](#)
- data entity relationship [144](#)
 - entity-relationship diagram (ERD) [144](#), [145](#)
- data leaks [239](#)
 - causes [239](#)
 - detecting [240](#)
 - preventing [240](#), [241](#)
- Data Loss Prevention (DLP) [240](#)
- data modeling [8](#)
 - entities [8](#)
 - history [142-144](#)
 - NoSQL data modeling [147](#), [148](#)
 - relationships [8-10](#)
 - using [146](#), [147](#)
- data streaming solutions [159](#)
 - aggregation and state management [161](#)
 - Apache Flink [166](#)
 - Apache Kafka [162](#)
 - Apache Pulsar [163](#)
 - best practices [166](#), [167](#)
 - design, for schema evolution [161](#)
 - event and processing times [160](#), [161](#)
 - event schema, defining [160](#)
 - metadata, using [161](#)
 - Redis Streams [165](#)
- data structures [19](#)
 - arrays [19](#), [20](#)
 - maps [21](#)
 - slices [20](#)
 - structs [21](#), [22](#)

- Day 2 operations [225](#), [226](#), [254](#)
 - backup and disaster recovery [255](#)
 - compliance and auditing [255](#), [256](#)
 - configuration management [255](#)
 - monitoring [254](#)
 - performance optimization [255](#)
 - scaling [255](#)
 - security management [255](#)
 - software upgrades [254](#)
- denial of service (DoS) attacks [10](#)
- dependency-check [238](#)
- DevOps [220](#)
 - collaboration [221](#)
 - configuration management [221](#)
 - continuous delivery [220](#)
 - continuous integration [220](#)
 - continuous monitoring [221](#)
 - incident management [222](#)
 - Infrastructure as code (IaC) [220](#)
 - observability [221](#)
 - release management [221](#)
 - security and compliance [221](#)
- Distributed Component Object Model (DCOM) [4](#)
- distributed denial of service (DDoS) attacks [11](#), [177](#)
- distributed systems
 - concurrency [96](#)
 - data consistency [98](#), [99](#)
 - fallacies [93-95](#)
 - fallacies, fixing [96](#)
 - fault tolerance [99](#)
 - history [88-90](#)
 - overview [87](#)
- distributed systems, components
 - computing nodes [90](#)
 - coordination and synchronization [92](#)
 - data storage and management [91](#)
 - fault tolerance and reliability [92](#)
 - messaging [91](#)
 - monitoring and tracking [93](#)
 - network infrastructure [91](#)
 - scalability and load balancing [92](#)
 - security [92](#)
 - service discovery [93](#)
- distributed systems, managing [106](#)
 - configuration management [107](#)
 - monitoring and logging [106](#)
 - performance optimization [108](#)
 - scalability management [107](#), [108](#)

- security management [107](#)
- service management [108](#)
- tools and technologies [108](#)

DNS based load balancing [102](#)

Docker APIs [207](#)

Docker containers [82](#)

Domain Name System (DNS) [99](#)

Don't Repeat Yourself (DRY) principle [218](#)

Dynamic Application Security Testing (DAST) [248](#)

E

Echo [54](#)

- features [54](#), [55](#)
- working with [55-57](#)

Elastic Compute Service (ECS) [229](#)

Elasticsearch, Logstash, and Kibana (ELK) stack [48](#)

empty interface [24](#)

endpoints [11](#)

Ent [41](#), [156](#)

entity-relationship diagram (ERD) [144](#)

- benefits [146](#)

event-based checks [206](#)

F

Fallacies of Distributed Computing [94](#)

FastHTTP [63-65](#)

- features [63](#)
- working with [63-65](#)

fault tolerance, distributed systems [99](#)

- checkpointing and rollback [100](#)
- consensus algorithms [101](#)
- erasure coding [100](#)
- failover and self-healing mechanisms [101](#), [102](#)
- health monitoring [101](#)
- replication [100](#)

File Integrity Monitoring (FIM) [240](#)

Flink [166](#)

FreeBSD [81](#)

G

Gin [51](#)

- advantages [52](#)
- working with [52-54](#)

Git [7](#)

GitHub Actions [223](#)

GitLab CI/CD [223](#)

- Golang [17](#)
 - for distributed systems [109](#)
 - installation [18](#), [19](#)
- Golang database connectors [151](#)
 - MongoDB [152](#)
 - MySQL [152](#)
 - Neo4j [153](#), [154](#)
 - PostgreSQL [151](#)
 - Redis [153](#)
- Golang data modeling [154](#)
 - Ent [156](#)
 - Golang object-relational mapping (GORM) [154](#)
 - GQLGen [158](#)
 - SQLX [157](#)
- Golang for cloud [230](#)
 - Cloud APIs [230](#), [231](#)
 - cloud deployments [231](#)
 - logging and monitoring [232](#)
 - microservices orchestration [231](#)
 - Pub/Sub messaging [231](#)
 - security and compliance [232](#)
 - serverless computing [231](#)
- Golang-ilities [185-187](#)
- Golang software security [251](#)
 - best practices [251](#)
 - overview [252](#)
- Golang support [261](#)
 - blue-green deployment [263](#)
 - canary deployment [263-265](#)
 - database schema migrations [266](#)
 - feature flags [263-265](#)
 - graceful shutdowns and restarts [261](#), [262](#)
 - hot reloading [263](#)
 - rolling deployments, with Kubernetes [265](#)
 - versioning and compatibility [267](#)
- Go modules [42](#), [43](#)
 - using [43](#), [44](#)
- Google Cloud Platform (GCP) [228](#)
- Google File System (GFS) [91](#)
- Google Kubernetes Engine (GKE) [192](#)
- GORM [39](#), [154](#)
- goroutine [30](#)
- GQLGen [158](#)
- Grafana [199](#)
- GraphQL [12](#), [128](#)
 - advanced features [132-135](#)
 - Golang, using [130-132](#)
 - history [129](#), [130](#)

gRPC [123](#)

history [123](#), [124](#)

gRPC in Go

client calls, emulating [127](#)

Go code, from .proto file [125](#)

prerequisites, installing [124](#), [125](#)

protocol buffers, defining [125](#)

server and client, running [128](#)

server implementation [126](#)

using [124](#)

H

Hadoop Distributed File System (HDFS) [91](#)

hardware load balancers [102](#)

health checks [205](#)

customized checks [206](#)

event-based checks [206](#)

liveness probe [205](#)

readiness probe [205](#)

startup probe [206](#)

Helm [197](#), [198](#)

Horizontal Pod Autoscaler (HPA) [195](#)

horizontal scaling [173](#)

Hypermedia as the Engine of Application State (HATEOAS) [115](#)

I

identity and access management (IAM) [227](#)

ilities

overview [170](#), [171](#)

Infrastructure as a service (IaaS) [47](#), [226](#)

Infrastructure as code (IaC) [220](#)

tools [182](#)

integration testing [13](#)

Interactive application security testing (IAST) [248](#)

interfaces

definition [23](#)

empty interface [24](#), [25](#)

implementing [23](#), [24](#)

using [24](#)

Internet of Things (IoT) applications [228](#)

inter-process communication (IPC) [112](#)

intrusion detection systems (IDS) [240](#)

intrusion prevention systems (IPS) [240](#)

J

JavaScript Object Notation (JSON) [115](#)

Jenkins [223](#)
JSON Web Tokens (JWT) [11](#), [115](#)

K

Kafka [162](#)
Kubelet [84](#)
Kube-proxy [85](#)
Kubernetes API [208-211](#)
Kubernetes architecture [84](#)

- master nodes [84](#)
- networking [85](#)
- storage [85](#)
- worker nodes [84-86](#)

Kustomize [198](#)

L

Linux containers (LXC) [82](#)
Linux-VServer [81](#)
liveness probe checks [205](#)
load balancing [102](#)
load balancing algorithms [103](#), [104](#)
local area network (LAN) [69](#)
log structured merge (LSM) trees [149](#)

M

machine learning (ML) [227](#)
maintainability [180](#)

- automated testing [181](#)
- CI/CD [182](#)
- code quality [181](#)
- documentation [181](#)
- knowledge sharing [182](#)
- modular design [180](#), [181](#)
- version control [182](#)

map [21](#)
message-oriented middleware (MOM) [91](#)
message passing [112](#)

- history [112-114](#)

microservices

- advantages [76](#)
- best practices [78](#), [79](#)
- disadvantages [77](#), [78](#)
- overview [75](#), [76](#)
- versus, SOA [79](#)

Microsoft Azure [228](#)
mock based testing [28](#), [29](#)

- monitoring [178](#)
 - alerting [179](#)
 - health monitoring [178](#)
 - logging and tracing [180](#)
 - network monitoring [179](#)
 - performance monitoring [178](#)
 - security monitoring [178](#), [179](#)
 - visualization [179](#), [180](#)
- monitoring and logging solutions [48](#)
- monolithic architecture [70](#)
 - advantages [71](#), [72](#)
 - drawbacks [72](#), [73](#)
- multi-factor authentication (MFA) [238](#)
- Multiplexed Information and Computing Service (Multics) project [112](#)
- mutex [34](#)
- mutual TLS (mTLS) [201](#)

N

- network address translation (NAT) [200](#)
- Network Time Protocol (NTP) [95](#)
- NoSQL data modeling [147](#)
 - advantages [150](#), [151](#)
 - column-family stores [149](#)
 - document stores [148](#)
 - graph databases [150](#)
 - key-value stores [149](#)

O

- object relational mapping (ORM) [39](#)
 - Ent [41](#), [42](#)
 - GORM [39](#)
 - SQLBoiler [41](#)
- Object Request Brokers (ORBs) [91](#)
- Object Storage Service (OSS) [229](#)
- Open Container Initiative APIs [211](#)
- Open Policy Agent (OPA) [203](#)
- Open Web Application Security Project (OWASP) [237](#)
 - community and guidance [239](#)
 - OWASP projects [238](#)
 - OWASP Top 10 [237](#), [238](#)
- Oracle Cloud Infrastructure (OCI) [228](#)

P

- PACELC theorem [105](#)
- Paxos [101](#)
- performance testing [13](#)

- Persistent Volume Claims (PVC) [85](#)
- Persistent Volumes (PV) [85](#)
- planning for unknown [276-279](#)
- Platform as a service (PaaS) [47](#), [226](#)
- Pod [85](#)
- Pod Security Policy (PSP) [203](#)
- Pod Security Standards (PSS) [203](#)
- PostgreSQL [151](#)
- Principle of least privilege (PoLP) [245](#)
- probes [205](#)
- Prometheus [199](#)
- protobuf compiler (protoc) [124](#)
- public cloud services [226](#)
 - benefits [227](#)
 - challenges [229](#)
 - cloud providers [228](#)
 - future trends [229](#), [230](#)
 - sample use cases [227](#), [228](#)
 - service types [226](#)
- Pulsar [163](#)

R

- Raft [101](#)
- readiness probe checks [205](#)
- Reaper [89](#)
- recovery time objectives (RTO) [172](#)
- Redis Streams [165](#)
- regressions [12](#)
- regression testing [13](#)
- relational database management systems (RDBMS) [143](#)
- reliability [176](#)
 - data integrity [176](#), [177](#)
 - documentation [177](#)
 - error handling and logging [176](#)
 - fault tolerance [176](#)
 - monitoring and alerting [176](#)
 - redundancy [176](#)
 - security [177](#)
 - software management [177](#)
- Remote Procedure Calls (RPC) [11](#), [91](#)
- REpresentational State Transfer (REST) protocol [114](#)
 - history [114](#), [115](#)
- REST
 - Golang, using [119-122](#)
- REST APIs [12](#)
- REST, gRPC and GraphQL comparision
 - compatibility and interoperability [137](#)
 - development and tooling [136](#), [137](#)

- error handling [138](#)
- performance [136](#)
- protocol and data format [135](#), [136](#)
- security [138](#)
- streaming and real-time communication [137](#), [138](#)
- REST standards [115](#)
 - considerations [118](#), [119](#)
 - documentation [117](#)
 - HTTP methods [116](#)
 - HTTP status codes [116](#), [117](#)
 - hypermedia as the engine of application state [118](#)
 - payload format [117](#)
 - resource identification [115](#), [116](#)
 - security [117](#)
 - statelessness [117](#)
 - versioning [117](#)
- Revel [60](#)
 - features [60](#), [61](#)
 - URL [62](#)
 - working with [61](#), [62](#)
- Runtime Application Self-Protection (RASP) [248](#)
- RWMutex [36](#)

S

- scalability [173](#)
 - asynchronous processing [174](#), [175](#)
 - caching [174](#)
 - database scalability [175](#)
 - elasticity [175](#)
 - horizontal scaling [173](#)
 - load balancing [174](#)
 - microservices [175](#)
 - monitoring and analytics [175](#)
 - vertical scaling [173](#), [174](#)
- scale-out [15](#)
- scale-up [15](#)
- security concepts [234-236](#)
- security information and event management (SIEM) [240](#)
- security measures [247-250](#)
- security testing [13](#)
- segregation of duties (SoD) [245](#)
- select statement [32](#), [33](#)
- semantic versioning [46](#)
 - benefits [46](#)
- sensitive data exposure [237](#)
- server [3](#), [11](#)
- serverless containers [82](#)
- server-side request forgery (SSRF) [238](#)

- service-oriented architecture (SOA) [73](#)
 - advantages [73](#), [74](#)
 - disadvantages [74](#)
 - versus, microservices [79](#), [80](#)
- shadow testing [260](#)
- Simple Object Access Protocol (SOAP) [11](#), [114](#)
- single point of failure (SPOF) [92](#), [179](#)
- site reliability engineering (SRE) [222](#)
- slice [20](#)
- soft skills [274-276](#)
- software as a service (SaaS) [4](#), [69](#), [226](#)
- Software Assurance Maturity Model (SAMM) [238](#)
- software composition analysis (SCA) [248](#)
- software delivery [44](#)
 - process [44](#), [45](#)
- software deployment
 - history [68-70](#)
- software design approach [70](#)
- software load balancers [102](#)
- software team building [270](#), [271](#)
- software upgrades
 - bug fixes [257](#)
 - data migration [258](#)
 - end of life management [258](#)
 - infrastructure changes [258](#)
 - new functionality [257](#), [258](#)
 - overview [257](#)
 - performance improvements [257](#)
 - security enhancements [257](#)
- software upgrades models [258](#)
 - blue-green deployment [260](#)
 - canary releases [259](#), [260](#)
 - feature flags [260](#)
 - hot fixes [261](#)
 - monolithic upgrades [259](#)
 - phased rollout [259](#)
 - rolling upgrades [259](#)
- source code version control [6](#)
- SQLBoiler [41](#)
- SQLX [157](#)
- startup probe checks [206](#)
- Static Application Security Testing (SAST) [247](#)
- structs [21](#), [22](#)
- Structured Query Language (SQL) [142](#)
- subversion (SVN) [7](#)
- sync package [33](#)
 - Cond/sync.Cond [37-39](#)
 - Mutex/sync.Mutex [34](#)

- RWMutex /sync.RWMutex [36](#)
- Wait group/sync.WaitGroup [35](#), [36](#)
- system testing [12](#)
 - acceptance testing [13](#)
 - compatibility testing [13](#)
 - integration testing [13](#)
 - performance testing [13](#)
 - regression testing [13](#)
 - security testing [13](#)
 - unit testing [13](#)

T

- traffic monitoring [240](#)
- Transport Layer Security (TLS) [246](#), [247](#)
- Travis CI [224](#)
- twenty-one key lessons [271-274](#)
- type assertion [25](#)
- type switch [25](#), [26](#)

U

- unit testing [13](#), [26](#)
 - basic tests [27](#)
- usability [183](#)
 - accessibility [184](#)
 - documentation [185](#)
 - error handling [184](#)
 - information architecture [183](#)
 - localization [185](#)
 - responsiveness [184](#)
 - user-centered design [183](#)
 - user interface design [183](#)
 - workflow design [185](#)
- user acceptance testing (UAT) [13](#)
- user behavior analytics (UBA) [240](#)

V

- version control system (VCS) [6](#)
 - branches [6](#)
 - commits [6](#)
 - conflict resolution [7](#)
 - Git [7](#)
 - mercurial [7](#)
 - merging [6](#)
 - performce [8](#)
 - pull requests [7](#)
 - repository [6](#)

- subversion [7](#)
- Vertical Pod Autoscaler (VPA) [196](#)
- vertical scaling [173](#)
- virtual machines (VMs) [47](#), [80](#)
 - advantages [80](#)
 - disadvantages [80](#), [81](#)
 - versus, containers [82-84](#)

W

- wait group [35](#)
- Web Content Accessibility Guidelines (WCAG) [184](#)
- web frameworks
 - Beego [57](#), [58](#)
 - Echo [54](#), [55](#)
 - FastHTTP [63](#)
 - Gin [51](#), [52](#)
 - overview [50](#), [51](#)
 - Revel [60-62](#)
- WebSocket API [12](#)
- World Wide Web (WWW) [4](#)
- write ahead logs (WAL) [149](#)

Y

- Yellowdog Updater, Modified (YUM) [69](#)

Z

- Zab protocol [101](#)
- Zed Attack Proxy (ZAP) [238](#)